

TUGboat

Volume 46, Number 1 / 2025

General Delivery
3 From the president / *Arthur Rosendahl*
4 Editorial comments / *Barbara Beeton*
Seeking information about Peter R. Wilson; Leslie Lamport retires from Microsoft; Errata for *TUGboat* 45:3 Abstracts; (LA)TEX scheduling practices, and how you can help; Chemical/pharmaceutical terms and the *TUGboat* hyphenation exceptions list; Online GUT resources

Typography
7 Face/Interface 2025 conference: Global type design and human-computer interaction / *Boris Veytsman*
10 Typographers' Inn / *Peter Flynn*

Humanities
12 OpBible — a tool for creating study Bibles / *Šimon Konečný, Petr Olšák*

Tutorials
19 The DuckBoat — Beginners' Pond: A pilates . . . oops, PyLaTeX course / *Carla Maggi*
28 The `glossaries` package: Getting started / *Nicola Talbot*

LaTeX
35 LaTeX news, issue 40, November 2024 / *LaTeX Project Team*
40 Experiences in typesetting a complex book: Consistency, graphics, languages, notes / *Alan J. Cain*
49 `marginalia` at work: Running heads, float captions, citations, and small figures in the margins / *Alan J. Cain*
54 Typesetting code with syntax highlighting: Introducing `minted` version 3 / *Geoffrey M. Poore*
60 New tools for LaTeX package authors: Buffering, hashing, caching, serializing, and executing / *Geoffrey M. Poore*
69 Piping YAML files into LaTeX3 key–values / *Vít Starý Novotný, Marei Peischl*
71 `dtxmacrofont`: Resizable `macrocode` in `doc` implementations / *John E. Beach*
74 `datatool` v3: Performance, localisation, databases and more / *Nicola Talbot*

Accessibility
79 SpeLaTeX — Speech-enabled LaTeX / *Walter Daems, Paul Levrie*

Bibliographies
86 BibBreeze: A LaTeX package to automate bibliography data reconstruction / *Amir Mohammad Tahsiri*

Software & Tools
94 Efficient input file processing with TEX: A case study / *Martin Ruckert*
101 A DVI driver for R / *Paul Murrell*
104 A new take on paragraphs / *Hans Hagen, Mikael P. Sundqvist*

Macros
124 Externalization with Memoize / *Sašo Živanović*
132 Processing data from CSV and JSON files with OpTEX / *Petr Olšák*
136 Output routines extending `\plainoutput` / *Udo Wermuth*
145 Is a given input a valid TEX ⟨*glue*⟩/⟨*muglue*⟩? / *Udo Wermuth*
147 Comparisons for glue and muglue / *Udo Wermuth*

Graphics
151 Cookie cutters: An example workflow of how to generate simple 3D-printable objects from LaTeX graphics / *samcarter, Ulrike Fischer*
156 Styling Microsoft Power BI dashboards with TEX / *Travis Stenborg*

Abstracts
157 *Die TEXnische Komödie*: Contents of issues 4/2024–1/2025
159 *Zpravodaj*: Contents of issue 2024/1–4
159 *La Lettre GUTenberg*: Contents of issue 54 (2024)

Hints & Tricks
160 The treasure chest / *Karl Berry*

TUG Business
2 *TUGboat* editorial information
2 TUG institutional members
162 TUG 2025 election
165 TUG financial statements for 2024 / *Karl Berry*

Advertisements
166 TEX consulting and production services
167 TEXFolio

News
168 Calendar

TEX Users Group

TUGboat (ISSN 0896-3207) is published by the TEX Users Group. Web: tug.org/TUGboat.

Individual TUG memberships

2025 dues for individual members are as follows:

- Trial rate for new members: $35.
- Regular members: $115.
- Special rate: $85.

The special rate is available to students, seniors, and citizens of countries with modest economies, as detailed on our web site. Members may also choose to receive *TUGboat* and other benefits electronically, at a discount. All membership options are described at tug.org/join.

Membership in the TEX Users Group is for the calendar year, and includes all issues of *TUGboat* for the year in which membership begins or is renewed, as well as software distributions and other benefits. Individual membership carries with it such rights and responsibilities as voting in TUG elections. All the details are on the TUG web site.

Journal subscriptions

TUGboat subscriptions (non-voting) are available to libraries and other organizations or individuals for whom memberships are either not appropriate or desired. Subscriptions are delivered on a calendar year basis. The subscription rate for 2025 is $130.

Institutional memberships

Institutional membership is primarily a means of showing continuing interest in and support for TEX and TUG. Patron memberships to provide extraordinary support are also available. These also provide a discounted rate for individual members, site-wide electronic access, and other benefits. For details, see tug.org/instmem or contact the TUG office.

About the cover

The cover graphic was created by Alan Cain using Asymptote, for his extraordinary book on mathematical beauty, *Form & Number*. Alan's article on pages 40–48 has more information.

Trademarks

Many trademarked names appear in the pages of *TUGboat*. If there is any question about whether a name is or is not a trademark, prudence dictates that it should be treated as if it is.

TUGboat vol. 46, no. 1

Printed in U.S.A. [printing date: April 2025]

Board of Directors

Donald Knuth, *Ur Wizard of TEX-arcana*†
Arthur Rosendahl, *President**
Boris Veytsman*, *Vice President*
Karl Berry*, *Treasurer*
Jim Hefferon*, *Secretary*
Barbara Beeton*
Johannes Braams
Max Chernoff
Kaja Christiansen
Ulrike Fischer
Klaus Höppner
Tom Hejda
Jérémy Just
Frank Mittelbach
Ross Moore
Norbert Preining
Raymond Goucher (1937–2019), *Founding Executive Director*†
Hermann Zapf (1918–2015), *Wizard of Fonts*†

* *member of executive committee*
† *honorary*

See tug.org/board for a roster of all past and present board members, and other official positions.

Addresses

TEX Users Group
P. O. Box 2311
Portland, OR 97208-2311
U.S.A.

Telephone
+1 503 223-9994

Fax
+1 815 301-3568

Web
tug.org
tug.org/TUGboat

Electronic mail

General correspondence, membership, subscriptions:
office@tug.org

Submissions to *TUGboat*, letters to the Editor:
TUGboat@tug.org

Volunteer TEXnical support, public mailing list (a.k.a. texhax):
support@tug.org

Contact the Board of Directors:
board@tug.org

Copyright © 2025 TEX Users Group.

Copyright to individual articles within this publication remains with their authors, so the articles may not be reproduced, distributed or translated without the authors' permission.

For the editorial and other material not ascribed to a particular author, permission is granted to make and distribute verbatim copies without royalty, in any medium, provided the copyright notice and this permission notice are preserved.

Permission is also granted to make, copy and distribute translations of such editorial material into another language, except that the TEX Users Group must approve translations of this permission notice itself. Lacking such approval, the original English permission notice must be included. An information notice to the *TUGboat* editors regarding such redistribution is appreciated.

A child born today is no more accustomed to the Roman type forms than was a child born in the Paleolithic era. A new form would not be strange to people growing up with it.

Marshall Lee

TUGBOAT

COMMUNICATIONS OF THE TEX USERS GROUP
Editor Barbara Beeton

Volume 46, Number 1, 2025
Portland, Oregon, U.S.A.

TUGboat editorial information

This regular issue (Vol. 46, No. 1) is the first issue of the 2025 volume year. The deadline for the second issue in Vol. 46 (the TUG'25 conference proceedings) is July 27, 2025, and for the third (regular) issue, October 3, 2025. The deadline for the first issue of next year is March 20, 2026. Contributions are requested.

TUGboat is distributed as a benefit of membership to all current TUG members. It is also available in printed form through the TUG store (`tug.org/store`), and online at the *TUGboat* web site (`tug.org/TUGboat`). Online publication to non-members is delayed for one issue, to give members the benefit of early access.

Submissions to *TUGboat* are checked by volunteers and the Editor before publication. However, the authors are assumed to be the experts. Questions regarding content or accuracy should therefore be directed to the authors, with an information copy to the Editor.

TUGboat editorial board

Barbara Beeton, *Editor-in-Chief*
Karl Berry, *Production Editor*
Sophia Laakso, *Office Manager*
Max Chernoff, *(Lua)TEXnician*
Boris Veytsman, *Associate Editor, Book Reviews*

TUGboat advertising

For advertising rates and information, including consultant listings, contact the TUG office, or see:
`tug.org/TUGboat/advertising.html`
`tug.org/consultants.html`

Submitting items for publication

Proposals and requests for *TUGboat* articles are gratefully received. Please submit contributions by electronic mail to `TUGboat@tug.org`.

The *TUGboat* style files, for use with `plain` TEX and LATEX, are available from CTAN and the *TUGboat* web site, and are included in TEX distributions. We also accept submissions using ConTEXt. For deadlines, templates, author tips, and more, see `tug.org/TUGboat`.

Effective with the 2005 volume year, submission of a new manuscript implies permission to publish the article, if accepted, on the *TUGboat* web site, as well as in print. As of fall 2023, submission also implies permission to be indexed in EBSCO's (a large subscription management company) databases. We made this agreement with EBSCO to provide more visibility to *TUGboat* articles. See `tug.org/TUGboat/tubperm.html` for more.

If you have any concerns about these permissions, please notify the editors at the time of submission and we will do our best to make suitable arrangements.

Other TUG publications

TUG is interested in considering additional manuscripts for publication, such as manuals, instructional materials, documentation, or works on any other topic that might be useful to the TEX community in general.

If you have such items or know of any that you would like considered for publication, please contact the Publications Committee at `tug-pub@tug.org`.

TUG Institutional Members

TUG institutional members receive a discount on multiple memberships, site-wide electronic access, and other benefits:
`tug.org/instmem`
Thanks to all for your support!

Silver patron

TEXFolio, *Trivandrum*, *India.* `texfolio.org`

American Mathematical Society, *Providence*, *Rhode Island.* `ams.org`

Association for Computing Machinery, *New York City*, *New York.* `acm.org`

Aware Software, *Newark*, *Delaware.* `awaresw.com`

Center for Computing Sciences, *Bowie*, *Maryland.*

CSTUG, *Praha*, *Czech Republic.* `cstug.cz`

CTAN. `ctan.org`

Duke University Press, *Durham*, *North Carolina.* `dukeupress.edu`

Hindawi Foundation, *London*, *UK.* `hindawi.org`

Institute for Defense Analyses, Center for Communications Research, *Princeton*, *New Jersey.*

L3Harris, *Melbourne*, *Florida.* `l3harris.com`

LATEX Project. `latex-project.org`

MacTEX. `tug.org/mactex`

Maluhy & Co., *São Paulo*, *Brazil.* `maluhy.com.br`

Marquette University, *Milwaukee*, *Wisconsin.* `marquette.edu`

Masaryk University, Faculty of Informatics, *Brno*, *Czech Republic.* `fi.muni.cz`

Modular Font Editor K. `mfek.org`

Nagwa Limited, *Windsor*, *UK.* `nagwa.com`

NASA. `nasa.gov`

National Security Agency. `nsa.gov`

Ontario Tech University, *Oshawa*, *Ontario*, *Canada.* `ontariotechu.ca`

Overleaf, *London*, *UK.* `overleaf.com`

StackExchange, *New York City*, *New York.* `tex.stackexchange.com`

Tailor Swift Bot, *College Station*, *Texas.* `tailorswiftbot.com`

Université Laval, *Ste-Foy*, *Québec*, *Canada.* `bibl.ulaval.ca`

University of Oslo, Institute of Informatics, *Blindern*, *Oslo*, *Norway.* `uio.no`

VTEX UAB, *Vilnius*, *Lithuania.* `vtex.lt`

Xerdi, *Leeuwarden*, *The Netherlands.* `xerdi.com`

From the president

Arthur Rosendahl

"There are not enough talks! But that may not matter, there are not enough attendees either."

When I wrote these words to the executive committee, one week to the deadline for submitting abstracts to the TUG 2024 conference in Prague, we had 3 talk proposals and 15 registrations from prospective participants. To say that we didn't have enough talks for a whole conference would be an understatement: this wasn't even enough for a brief morning session.

From experience, I expected most submissions to come close to the deadline, or even after it. I had been part of the organising committee since the first online TUG conference in 2020, and had observed that many submissions came at the last minute — something not uncommon in other areas of life. (That might be another understatement.) I couldn't remember, however, such a low number of proposals that close to the deadline, especially as the number of applications for the bursary was also at a record low. There was real cause for concern that this conference, the second in-person one after the COVID pandemic had forced us to go online for three years, would have to be cancelled for lack of interest, compounded with organisational problems reported by the local committee. In addition, our contract with the venue specified that we would be billed for a minimum of 60 participants, so that even if we went ahead with the conference in the hope of getting more talks in the meantime, we were almost certainly looking at a considerable loss, in the tens of thousands of dollars. If, on the other hand, we decided to voluntarily cancel the event, there was of course a cancellation fee to be paid to the venue.

Having taken the matter to the whole board, we started crunching numbers. At this stage, in mid-April 2024, the most economical solution, by far, would have been to cancel forthwith; on the other hand, there was another deadline looming, where we were to make an additional, non-refundable payment to the venue: after that, all the payments already made were a sunk cost and there was no point cancelling any more. We looked at these two scenarios, as well as several other ones between these two extremes, making best-guess estimates about the number of registrations we could reasonably expect. We of course looked at ways to cut costs too.

Cancelling the conference would have been particularly poignant for me, the newly elected president, but that didn't carry any weight in the decision. We did, however, need to take into account our members and supporters who had already registered and presumably started making travel plans. This was also to a large extent quantifiable, generating more numbers to crunch to help us make a choice.

And yet ...

In spite of all the hard data, the overwhelming majority of the board was leaning towards keeping conference preparations going — at all costs (or almost), so to speak. We of course acknowledged the seriousness of the financial loss, but were also quite confident, in spite of the unprecedented low numbers, that more registrations would come, bringing not only conference fees but actual human beings to Prague to make the conference a success.

There is a "build it and they will come" aspect to TEX itself, after all: how else to explain its continued success and constant renewal? There is no logical reason why a program written to typeset a *summa* of computer science would be so widely adopted and still alive after almost half a century. No doubt the same spirit prevailed between the lines of the emails we exchanged in those tense discussions one year ago.

In the end, we went ahead with the conference, as readers will be aware. The organisational issues were ironed out, there were record numbers of last-minute submissions and registrations, sufficient to more than break even financially, and more importantly, the conference was a success. I even heard subdued complaints that there were *too many* talks!

And now the deadline for submitting talks to TUG 2025 in Trivandrum (`tug.org/tug2025`) is upon us: GO FORTH and send your contributions!

◇ Arthur Rosendahl
Uppsala University
TUG president
president (at) tug (dot) org



doi.org/10.47397/tb/46-1/tb142pres

Editorial comments

Barbara Beeton

Seeking information about Peter R. Wilson

Peter Wilson, proprietor of Herries Press in Kenilworth, UK, and author of the *TUGboat* column "Glisterings" from 2001–2017, has disappeared from all locations known to us. The last contact that anyone at TUG knows of was in early 2022. If anyone has more recent information about Peter, please provide it via email to `office@tug.org`.

Peter delivered the keynote talk at TUG 2007 in San Diego, where he gave an account of his introduction to LaTeX, the development of the memoir document class, and the history of written text.[1] A highlight of that presentation, to me, was the circulation to the audience of a number of artifacts, in particular a Sumerian clay tablet containing an inscription in cuneiform script, demonstrating both that paper is not the only surface used to record text, and that there is a medium much more permanent than paper, as clay becomes more durable when it goes through fire.

The "Glisterings" columns were collected by Peter into a book, published by TUG.[2]

Peter also, for many years, prepared a calendar that TUG members could retrieve and print for use and enjoyment. The last, for 2021, was illustrated with traditional Japanese block prints. These prints exemplified his avocation, letterpress printing, which came to the foreground after retirement and his move back to the UK.[3]

We do, of course, hope that Peter can be contacted, but if he is no longer among us, he deserves an obituary in this journal.

Leslie Lamport retires from Microsoft

Leslie Lamport is best known to the TeX community as the author of LaTeX, the most prevalent user interface to TeX.

After receiving his Ph.D. in mathematics from Brandeis in 1972, where his thesis advisor was Richard Palais,[4] Leslie left math for computer science. He has been employed by a number of computer-related companies, including Digital Equipment Corporation, Compaq, and finally Microsoft in 2001, from which he retired in January 2025. We'll take advantage of the occasion to congratulate him and very briefly review his work.

The author of many papers, primarily on distributed computing (for which he is best known), Leslie was undoubtedly interested in the process of recording one's findings accurately for publication. In 1979, he was at Stanford Research Institute (SRI International), just across the road from Stanford University, where Don Knuth had developed TeX. Ideas travel, and by 1983, Leslie had created his first version of the macro set that would become LaTeX. The core principle was to provide a mechanism by which an author could easily create a structured document, interleaving text with markup. (This idea had been developed earlier at IBM with GML, the Generalized Markup Language, later standardized as SGML, and the progenitor of HTML, XML, and MathML.) The structured approach enabled automatic numbering and cross-referencing to both text elements and bibliographical items, features essential to scholarly publication.

LaTeX 2.09 became widely available in 1986 with the publication of the first user manual, *LaTeX: A Document Preparation System*. (During a talk Leslie gave at a DECUS meeting in San Francisco, he was asked about the proper pronunciation of the system's name. The answer: "Lay-TeX or Lah-TeX, anything but Ell-Ay-TeX.") But with its growing audience, the system needed some extensions and cleanup. On 21 August 1989, at a TUG meeting at Stanford, Leslie agreed to turn over maintenance and development to Frank Mittelbach and a few others who formed the nucleus of the LaTeX3 team. (The next release was not LaTeX3, however, but LaTeX 2_ε.)

Leslie's work on distributed computing continued, with increasing recognition that included election to several Academies, honorary doctorates from five European universities, and culminated in the receipt of the 2013 Turing Award for "fundamental contributions to the theory and practice of distributed and concurrent systems, notably the invention of concepts such as causality and logical clocks, safety and liveness, replicated state machines, and sequential consistency".

On his website,[5] Leslie states

> If I have something new to say, I may give a few talks after I retire. I may also be willing to present an occasional question and answer session.

This presents a possible opportunity: Leslie may be receptive to an invitation to lead a Q&A session at a future TUG meeting.

[1] `tug.org/TUGboat/tb28-3/tb90wilson.pdf`
[2] `tug.org/store/glisterings`
[3] `tug.org/TUGboat/tb36-1/tb112wilson.pdf`
[4] Dick Palais was responsible for bringing TeX to the American Mathematical Society, and from there, founding TUG; he was the first TUG Chairman.
[5] `lamport.azurewebsites.net`

Barbara Beeton
doi.org/10.47397/tb/46-1/tb142beet

Errata for *TUGboat* 45:3 Abstracts

Incorrect page numbers were given for the articles in *Les Cahiers GUTenberg* 59 (page 405), and the order of the last two articles was reversed. In addition, several items were omitted. Page numbers have been corrected in the online PDF file. The following list provides the correct page numbers for the relevant items, and the full text for the omitted items.

Jean-Michel Hufflen, Éditorial; pp. 3–4.

Laurent Bloch, LaTeX ou Word : qui l'eût cru ?; pp. 5–12.

Flora Vern, LaTeX pour l'écriture juridique; pp. 13–31.

Maïeul Rouquette, Ce que LaTeX peut apporter à des travaux historiques et philologiques; pp. 33–47.

Thomas Savary, «Nénufar»; pp. 49–80.

Raymond Juillerat, Les spécificités du style eFrench; pp. 81–92.

Johannes Hielscher, Une réponse à *Imprimer ou ne pas imprimer*; pp. 93–96. Translation of "Leserbrief zu *Druck oder Nichtdruck*", *Die TEXnische Komödie* 3/2022; original by Peter Flynn, "To print or not to print" in Typographers' Inn, *TUGboat* **41** (2020), No. 3, pp. 265–268.

Yevhen Strakhov, Pensées et impressions ukrainiennes à BachoTEX 2019; pp. 97–105. Translation of "Ukrainian thoughts and impressions at BachoTEX 2019".

(La)TeX scheduling practices, and how you can help

All the software comprising the TEX ecosystem is maintained by volunteers and, although the main elements are released on regular schedules, the schedules for different elements do not necessarily coincide. The discussion here will deal mainly with the principal archive (CTAN), TEX Live, and the work of the LaTeX Project.

CTAN Developers, when they are ready to release a new package or update an existing one, submit it to CTAN, where it is vetted for adherence to version identification, licensing, and an intelligible description and release notes. When it passes muster, it is added to the repository and an announcement is made via the mailing list `ctan-ann@ctan.org`.[6] Approved submissions are added as received, and mirrors are synched at least daily. Retrieval of a package from CTAN should be made from a mirror; a list of mirrors and additional information can be found at `ctan.org/mirrors`. In almost all cases it is preferable to obtain updates via a MiKTEX or TEX Live update rather than directly downloading from CTAN, as locally installed packages will not be automatically updated if further releases are made.

TEX Live Most, but not all, packages on CTAN are included in TEX Live. For the requirements and recommendations for inclusion in TEX Live, see `tug.org/texlive/pkgcontrib.html`.

TEX Live is packaged for release once a year, generally in early March. The main purpose is to rebuild the TEX engines for various hardware; notification is sent out via the list `tex-live@tug.org` that the new edition is ready for pretesting. This year's activity can be reviewed by visiting the archive of the list. Before and after the release, updates to TEX Live are continuously available over the net, as new uploads are accepted on CTAN.

Before 2024, TEX Live and other software was burned to DVD and distributed to members of several user groups. As of the 2024 edition, the collection became too large to fit on that medium, and most users were retrieving it from CTAN mirrors or otherwise over the net, so it was agreed by the affected groups to discontinue the physical distribution to all members. DVDs are still available upon request; volunteers will burn and mail them (and more volunteers are always welcome). Visit `tug.org/dvd` for information and a request form.

Detailed instructions for obtaining and installing TEX Live are given at `tug.org/texlive`.

LaTeX The LaTeX system is officially updated twice a year, in June and November. However, work proceeds throughout the year, both to address bug reports[7] and to continue work on accessibility and other features. In order to facilitate testing in as many user environments as possible, two versions of the distribution are available in parallel from CTAN: the pre-release version, generically called `latex-dev`, and the production version. This convention was instituted in 2019, and the initial announcement still provides a good description of its goals.[8] Thus, all users of LaTeX, and especially package developers, are encouraged to use the `-dev` version,[9] and report any problems, so that they can be fixed before the next production version is released.

[6] The list is archived at `ctan.org/ctan-ann`

[7] `www.latex-project.org/bugs/`

[8] `www.latex-project.org/news/2019/09/01/LaTeX-dev-format/`

[9] The development version is updated and distributed several times between main releases and is automatically available in all major TEX distributions, e.g., TEX Live, MiKTEX, and MacTEX.

You can invoke the `-dev` version simply by using `pdflatex-dev`, `lualatex-dev`, etc., in your invocation of LaTeX; source documents need not be changed.

Chemical/pharmaceutical terms and the *TUGboat* hyphenation exceptions list

The hyphenation exceptions list in *TUGboat* has been accumulating more and more words used in chemistry and related fields — pharmacology, medicine, etc. In these disciplines, terminology is heavy with words compounded from multiple parts that have distinct meanings on their own. The large population of exceptions has resulted in the separation of this technical terminology into a separate sublist, a division that is not universally appreciated, but which can be justified on several grounds.

The major problem is the compounding, where proper hyphenation should choose a break between, rather than within, elements, although the latter choice should be available when necessary for even spacing in justified text. The problem is magnified for US English, which is normally hyphenated according to pronunciation rather than etymology. Presumably the situation would not be as serious for UK English, where etymology is the preferred rule governing word division.

Although the *TUGboat* list is directed toward US English, a similar problem exists in other languages as well. However, since the words for the same concepts or meanings may follow different spelling rules, the list of words used in US English documents is not suitable for use with other languages.

Like these technical terms, German is heavily compounded, and the German hyphenation patterns cope well with the situation, although I have no idea how that is accomplished. The hyphenation dictionary used to develop the patterns for UK English is (allegedly) based on etymology, rather than on pronunciation, and shows more than one level for choice of breakpoints, a scheme that supports etymological compounding. Enabling multilevel breaking would serve the technical vocabulary well.

As an example, I've chosen a small collection of terms (not yet in the *TUGboat* list), checked the recommended hyphenation in the authoritative US dictionary, and compared that with the result of using `\showhyphens` with the US patterns. The results are listed below; the preferred breakpoints are indicated by "=".

It's well past time to update the *TUGboat* list, and that is under construction. The list of "chemical terms" will continue to be a separately maintained sublist.

— TeX (US) —	— correct —
al-dos-terone	al=do-ste-rone
cat-e-cholamine	cat-e=chol=amine
dysau-tono-mia	dys=au-to=no-mia
glu-co-neo-ge-n-e-sis	glu-co=neo=gen-e-sis
glycogenol-y-sis	gly-co=gen-ol-y-sis
hy-per-v-olemia	hy-per=vol-emia
hy-per-v-olemic	hy-per=vol-emic
hy-potha-la-mic	hy-po=tha-lam-ic
hy-potha-la-mus	hy-po=thal-a-mus
hy-po-v-olemia	hy-po=vo-le-mia
hy-po-v-olemic	hy-po=vo-le-mic
neu-roen-docrine	neu-ro=en-do-crine
neu-ro-hy-pophysial	neu-ro=hy-po=phy-si-al
neu-ro-hy-poph-ysis	neu-ro=hy-poph-y-sis
neu-ro-pathic	neur-o=pa-thic
neu-ropa-thy(ies)	neu-rop-a-thy(ies)
os-more-cep-tor	os-mo=re-cep-tor
parasym-pa-thetic	para=sym-pa-thet-ic
pheochro-mo-cy-toma	pheo=chro-mo=cy-to-ma

If anyone has an idea for implementing multilevel hyphenation, especially as an extension to the method provided by the pattern mechanism, please let us know.

Online GUTenberg resources

In April 2023, the board of GUTenberg (the Groupe francophone des Utilisateurs de TeX, LaTeX et logiciels compagnons) proposed creation of a monthly series of talks on topics of interest to (La)TeX users. The talks would be at most an hour in length, and would be recorded by video for later access. The proposal was accepted, and the first talk, *une introduction à LuaLaTeX*, was presented on June 22 by Denis Bitouzé.

The talks (in French), now numbering more than a dozen, can be accessed via a list at the website `gutenberg-asso.fr/-Exposes-mensuels-`. The videos are collected on GUTenberg's YouTube channel[10] and also on the TubEdu website;[11] the TubEdu site also holds videos of presentations from regular GUTenberg meetings.

Another GUTenberg undertaking was to rebuild the French LaTeX FAQ, which has been joined by an XML FAQ. Both are accessible from the Publications page: `gutenberg-asso.fr/-Publications-`.

◇ Barbara Beeton
`https://tug.org/TUGboat`

[10] `youtube.com/@associationgutenberg5336`
[11] `tubedu.org/c/gutenberg`

Face/Interface 2025 conference: Global type design and human-computer interaction

Boris Veytsman

The Face/Interface conference at Stanford seems to have become a regular event. The previous one [1] (also reviewed in these pages [2]) took place in December 2023. The current one was a year and some weeks later, in January 2025. This certainly reflects the energy and persistence of its main organizer, Thomas S. Mullaney of Stanford.

In 2023 the conference had several main topics: the history of and new challenges for digital typesetting, creation of typefaces for historical scripts, and the digitization of scripts for underserved languages. This year it was, in my view, much more focused on the latter theme, with more than sixty percent of the talks being about the scripts of indigenous communities.

The importance of this task is gaining international recognition. Thus it is not surprising that the conference was attended by many dignitaries, including Tawfik Jelassi, UNESCO Assistant Director-General for Communication and Information (I sincerely hope that at the time you read these lines USA is still a member of UNESCO), Sarmad Hussain, a Senior Director at ICANN, and Toral Cowieson, CEO of the Unicode Consortium. The Stanford Humanities Center, which hosted the event in 2023, turned out to be too small, and the first day of the conference was held at the larger Bechtel Center. As Tawfik Jelassi quipped, one could easily recognize him and other diplomats in the crowd of scientists, engineers and artists there: diplomats wore ties. DEK also attended this conference. I regret not photographing the placard on a chair that read, "Reserved for Professor Knuth".

Thomas Mullaney opened the conference, stressing its main topic. There are more than six thousand languages left behind by market forces. It is our task to preserve and document them. SILICON, the Stanford Initiative on Language Inclusion and Conservation in Old and New Media (`silicon.stanford.edu`) is devoted to this endeavor. Mullaney described several new initiatives of SILICON, including an internship program (together with UNESCO and the Unicode Consortium) and Type Design Incubator program (together with Adobe, Letterform Archive, and Words of Type).

The talk by Bridget Chase from Typotheque described the work creating scripts for *indigenous North American communities*. It involves many different aspects: besides typographic and artistic ones, one must be a true diplomat: different members of the language community may have quite different opinions about scripts.

Rafael Dietzsch told about the activity of Typotheque in another part of the Americas: *Brazil*. There are about 274 languages used by 305 ethnic groups there. Rafael Dietzsch's doctoral study dealt with cataloging and typographic design for these languages, mostly based on the Latin 1 code block.

The Mexican language *Mixe* was discussed by Manuel López Rocha. An interesting detail about this language is the presence of rather long words. There are no hyphenation patterns for this language yet, which makes typesetting rather challenging.

The talk by Leonardo Vázquez Conde described *Wikitype*: a new multilingual classification system for type. The "wiki" in the name is for the collaborative aspect of the project: the system is intended to be extended and edited by users. The digital tool based on this system is available at `atypi.org/presentation/wikitype-a-prototype-for-type-classification`.

Lisa Huang discussed another multilingual collaborative project, *Words of Type* (available at `wiki.wordsoftype.com`). The terminology of typography in different countries and cultures is different. *Words of Type* is an ambitious effort to create a compendium of typographic concepts in different languages. Started with French and Chinese, it now also has articles in English, German, Japanese, Korean, Spanish, and the list is growing.

Tex Texin from *Translation Commons* talked about the urgency of the task of digitizing languages. By some estimates 44% percent of 7164 languages are endangered and can disappear. If we want to preserve 3000 languages in seven years, we need to digitize 428 languages per year. Tex proposed a decentralized organization of activists working on this mammoth endeavor.

John Hudson talked about the structure of the *Sanskrit* language, which left its mark on many Indic scripts. I was fascinated by its logical structure and the fact that certain letters that never occur in actual inscriptions are included in the alphabet because they correspond to the logic of the language—like missing chemical elements predicted by Mendeleev based on the logic of the periodic table.

Tawfik Jelassi talked about *inequity* in the world of digital representation. It is not just about indigenous languages spoken by a small number of people: even Arabic, with about 380 million speakers, has about 0.3% of all online content, while 52.1% of online content is in English. Jelassi described the work of UNESCO in this area, including

doi.org/10.47397/tb/46-1/tb142veytsman-face25

the initiative in declaring 2022–2032 the international decade of indigenous languages, and UNESCO World Atlas of Languages. It's a rare discussion today that avoids the issue of (so-called) artificial intelligence. Jelassi's talk was no exception: the speaker presented UNESCO Recommendations on the Ethics of Artificial Intelligence, a very useful resource in my opinion (`www.unesco.org/en/artificial-intelligence/recommendation-ethics`).

Mark Davis, Helena Aytenfisu, and Emiyare Cyril Ikwut-Ukwa talked about *CLDR*: one of the important components of the ICU library, which provides internationalization to the interfaces of computers, phones and other devices. Right now CLDR supports 168 languages on the basic level — a large achievement, but we need much more.

Sarmad Hussain, a senior director at ICANN (the world's domain name authority), talked about the effort to support *domain names* in non-Latin scripts. Since 2003 ASCII has not been the only choice for domain names: you can use Cyrillic, Arabic, and twenty more scripts in twenty-seven languages. This makes the Internet much friendlier for many people whose preferred language is not English, and whose native script is not Latin. The implementation is not trivial: one must protect people from the scammers that may use similar letters in different scripts to spoof domain names (e.g., changing Latin "a" to Cyrillic "a" and creating a fake version of a bank login page). This is a deliberate and iterative process, based on an ongoing dialogue with the multilingual user communities.

The talk by Toral Cowieson, Maroua Bezzaoui, and Samuel Minev-Benzecry discussed the work of the *Unicode Consortium*. Toral is the CEO of the consortium, while Maroua and Samuel are interns funded by the program by SILICON, UNESCO, and the Unicode consortium itself. It was very interesting to learn about the internal dynamics of Unicode authority, and the work with digitally disadvantaged languages. This talk again raised the questions touched on by Bridget Chase: who "owns" the language? How do we build trust among the communities of users?

The keynote speaker was Adam Yeo from Côte d'Ivoire, who described the work done with *Atelier National de Recherche Typographique* on Bété Script, a syllabary with 440 signs, created in 1956 by Frédéric Bruly Bouaubré. It might be the sign of the times that up to the last minute we were not sure whether Adam would make it, due to visa problems, and it took a lot of diplomatic efforts to make the in-person keynote possible.

The second day of the conference featured more technical talks.

Anshuman Pandey from the *Script Encoding Initiative* discussed digitization of Kulitan, one of the writing systems in the Philippines. Derived from the South Asian Brāhmī script, Kulitan is quite unusual: it is a vertical writing system (top to bottom, columns go from right to left). Besides Brāhmī, Kulitan has been influenced by the Korean Hangul script.

Kourosh Beigpour talked about *Laki*, the script used by Laks, 2–3 million Kurdish people in Iran. As with the other speakers involved in field work, Kourosh touched on the issue of interaction with the user communities, who often mistrust outsiders. "I am a Lak by father, which often helped a lot", noted the speaker.

Mark Jamra and Neil Patel, the authors of the well-known Kigelia type superfamily for Africa (`www.kigelia-font.com`), described their work with *Sunuwar*, a scripting system for Eastern Nepal and India state of Sakkim. Invented in 1941 by Krishna Bahadur Jenticha, the script has rather disparate letters — even with different pen angles within a single letter! It took two and a half years to digitize it. Again, a large part of the work was interaction with the community — which sometimes overrode the decisions of the designers.

Simon Charwey described the digitization of *Dangme*, the language spoken by Krobo, one of the peoples of Ghana. This is one of the projects supported by SILICON.

The talk by Frida Medrano touched an interesting application of the variable font technology: color fonts. *Kalnia Glaze*, based on her *Kalnia* typeface, offers beautiful gradients of color with subtle Victorian atmosphere (`www.fridamedrano.com/kalniaglaze`).

Gerui Wang from the *Stanford Center for East Asian Studies* discussed an unusual intersection of classical Chinese calligraphy and artificial intelligence. She compared the samples of Chinese artists and the calligraphy produced by AI and explained the deep difference between them. AI works by averaging, making everything alike, while the classic Chinese calligraphy prizes the exact opposite feature: uniqueness. This is the difference between social obedience and conformity, which reminded the speaker of passages from Horkheimer and Adorno.

The talk by Min-Young Kim discussed inscriptions in *virtual reality*. They are displayed in much more demanding situations than a text on a page or even a phone or a computer screen. Often there is a shifting background with changing illumination. It turns out that eye movement and reading physiology

is slightly different for Latin scripts and CJK, which creates additional challenges for non-English virtual reality design.

Marta Bernstein's talk discussed her research of *Italian type history* during the century of small type foundries (roughly between Bodoni's death in 1813 and the buying of Urania by the Nebiolo Typefoundry). She shared the setup of her database tools (mostly centered around Airtable) used to handle the collection of files and data.

John D. Berry told about the adventures of a designer and typesetter of *multi-script books*, including a commented edition of the Quran. A large part of his talk was devoted to tips and tricks to cajole InDesign to perform the task. As someone who has typeset several multiscript books, I should say that I had much better experience working with Babel, XeTeX and LuaTeX. It seems we in the TeX world have the best tools for complex multilingual typesetting. Berry touched on the perennial question of multilingual typesetting: should we use harmonized or contrasting typefaces for different scripts? My humble opinion is that the answer depends on the text and the reason it uses multiple languages. Different artistic and rhetorical tasks may suggest different solutions.

A remote presentation by Kateryna Korolevtseva discussed the history of *Ukrainian typefaces* and her own work on Ukrainian letterforms. The reform by Peter I in 1708 changed the shapes of Cyrillic letters. Many modern Ukrainian artists think it broke the organic feel of the text and seek Ukrainian typographic identity in the old letterforms and the manuscripts of the 16–18th centuries. Korolevtseva's own experimental typeface (to be released) is based on Skoropis, the cursive handwriting of Ukrainian scribes. Her projects can be found at `korolevtseva.com/projects`.

The subtle interplay of politics, the user community and the outsiders was demonstrated in the talk by Eason Lu about *Nüshu*, the famous "women script" of the Yao people in China. Nüshu has been used by women in their embroidery, and is intimately related to female societies, oral traditions and witchcraft. At present about 400 letters of the script are in Unicode, and are implemented in Google Noto Sans Nüshu. In modern China Nüshu was sometimes used as a tourist attraction, which caused resentment by the practitioners. For some women the script in Unicode is not the same as the "Nüshu we use" since the letters are separated from performance. How can we encode a script rooted in oral performative tradition?

We used to think about imperialism as something done exclusively by white people. This point of view is wrong: unfortunately, any humans can become imperialist if the circumstances are right. Anushah Hossain reminded about this in his talk under the provocative title *ISCII Imperialism*. The term "ASCII imperialism" is sometimes used to describe the domination of English language conventions on the Internet, especially in domain names (as discussed in the talk by Sarmad Hussain of ICANN). The multitude of Brāhmī-derived Indian scripts is encoded according to the Indian Script Code for Information Interchange (ISCII). Anushah Hossain demonstrated that the design decisions behind ISCII, later transferred to Unicode, were too heavily influenced by Devanagari, and did not completely take into account the needs of other scripts. As the different Indian ethnicities recognize their identity, these decisions spark resentment — and sometimes even political protests.

Loïc Marleix discussed his work on digitization of artificial writing system, *The Lovers Communication System* (`lovers-communication-system.com`), designed by Yukio Ota (famous for his "running man" exit sign that became a part of the ISO 7010 standard).

The talk by Sina Ahmadi discussed the *DOLMA project*: an ambitious task of collecting a multilingual parallel corpus of 50 thousand sentences in many Middle Eastern languages to train large language models. The major part of this work was community building: Ahmadi created a network with many volunteers and active contributors to collect and translate the corpus.

It was a very interesting conference showing the intersection of linguistics, computation, typesetting, social activism, art, and history. Writing and printing is a very human endeavor: animals do not write. It has been fascinating to see how this activity touches all aspects of culture.

The task of preserving our common heritage and enabling underserved communities is an important and noble task. The fact that even in these complex times organizations like SILICON are working on it gives some hope for our future.

References

[1] SILICON. YouTube channel. `youtube.com/@StanfordSILICON`

[2] B. Veytsman. Face/Interface 2023 conference: Global type design and human-computer interaction. *TUGboat* 45(1):7–10, 2024. `doi.org/10.47397/tb/45-1/tb139veytsman-face`

◇ Boris Veytsman
TeX Users Group
`https://borisv.lk.net`

Typographers' Inn

Peter Flynn

Style and fashion

Both type design and typographic design have fashions that come and go. A skilled typographer can recreate the look and feel of an historic period, or create a new look and feel for the present, or even for the future. The trick is in using typefaces and layouts that trigger readers' memories or imaginations, or somehow meet or set their expectations.

A number of web sites post periodical commentaries and forecasts about current and future trends in typography. Most of them are concerned with the advertising industry and the process of branding their clients' products. They do sometimes use typefaces we associate with body copy (plain text), but they mostly concentrate on display or ornamental types and their arrangement on the page or screen, because they are looking for a specific trigger-effect.

It is of course perfectly possible to use LaTeX in this way for advertisements and packaging, within certain limits, but LaTeX's principal strengths lie in other applications, and these too have their trends in design and the use of type. I have tried here to distil from the commentaries a few pointers that seem to me to be worth considering. Most of the sources are, of course, just doing business by selling their services, but the categories they have selected for their clients imply that they believe in the trends they are identifying. It's up to you to decide if the same holds true for your own circumstances: my personal comments are in italics.

Serif fonts. The use of serif display fonts is coming 'back into fashion' but using non-traditional or 'heritage' designs [1, 4, 6] although not everyone thinks it important enough to mention [8].

Antykwa Toruńska

Unless you are constrained by your institution, company, or publisher, try using some of the alternatives to Computer Modern in the 'Serif Fonts' category of the LaTeX Font Catalogue [5].

Irregular types. Imperfect, hand-drawn, raw, distressed, grunge, glitchy, or ragged fonts are noted for their informal, unfiltered effect [4, 6] but probably not candidates for books or theses or journals unless you are discussing type design.

Punk Nova

Knuth's Punk *font was probably among the first 'for-fun' digital fonts, but there are quite a few in the 'Other' category of the LaTeX Font Catalogue [5].*

Variable fonts. Traditional fonts come in a fixed (step-size) range of weights and measures, but newer technology allows them to be infinitely variable (in addition to using optical resizing) [1, 6, 7, 8]. See examples in [9].

LuaLaTeX and ConTeXt support OpenType variable font features [9] although some configuration may be needed.

Ragged right. Left-alignment now being preferred to justified [4].

I must admit I hadn't noticed designers preferring [display] text justified; but ragged-right setting works fine in wider settings.

AI-generated typography. There seem to be a lot of advocates of 'letting AI do the job' of selecting typefaces or generating new fonts [8]. I'm not even attempting an example of this.

I think the jury is still out on the use of AI in circumstances where humans do a better job.

Retro. Re-use of fonts from the 1970s, 80s, and 90s [1, 7, 8], including outline and bubble fonts [4, 7, 10]

QT Helvetica Black

As with the notes on serif fonts above, many of the designs that became popular during the early days of phototypesetting and electronic composition are available from the LaTeX Font Catalogue [5].

Accessibility. More emphasis is being placed on making type more readable by those with visual impairments [8], including the use of monospaced fonts [1].

```
What's accessibility?
```

This is long overdue, but I think more training is needed. See also item 3 in the list on p. 11 below.

How much of this is copied from one blog to another is unknown, as is how much of it was AI-generated (not a lot, in my opinion). But not everything is going to apply in every circumstance: if you have to produce an article for a specific journal with a fixed style (and probably a class package defining it) then your scope for change is probably limited, perhaps to zero. But if you're writing for yourself or a client, or you have the ability to change layout and fonts, it's well worth experimenting. Two resources are the LaTeX Font Catalogue [5] and the list of those typefaces installed automatically by TeX Live and usable via a package in [3, § 6.2.2.1].

Peter Flynn

doi.org/10.47397/tb/46-1/tb142inn

LaTeX processors

When I was putting together the early versions of 'Formatting Information' [2], I was still using pdfLaTeX (along with most users, I suspect). It did most of what I needed — at the time — in particular the `[utf8x]inputenc` package handled Latin–1 Western European (8–bit) characters without the need for TeX's accent macros, the `[T1]fontenc` package provided access to my PostScript Type 1 fonts, and the hyperref package provided links in the PDF output.

I continued for some years with pdfLaTeX, but eventually the inability to handle full Unicode (multibyte characters), and my own need for TrueType and OpenType fonts put it at a disadvantage compared to XeLaTeX and LuaLaTeX. I had no use case for running Lua scripts in my documents, so I started using XeLaTeX in production, and eventually switched 'Formatting Information' to use it for PDF production, and finally (after much testing and editing) started recommending it to readers and new users.

(I tried LuaLaTeX as well, of course, but I had had some problems with luaotfload, which indexes and locates fonts, which would have made it hard to recommend to beginners. There were also rumors about LuaTeX being slower than both pdfTeX and XeTeX.)

However, time marches on: pdfTeX has too many limitations and XeTeX is no longer being developed. LuaTeX is now the LaTeX team's preferred engine, and this is clearly the path to go. An extensive discussion on the `tex-live` mailing list in March 2025 made several topics clear.

1. The luaotfload package provides better font location than XeLaTeX's use of fontconfig does, which from a typographic point of view is the major benefit.
2. A number of kind people posted timing data from which it is clear that speed is not a problem on modern systems, at least not with shorter documents. If you have a 500pp document then it may need some seconds or even minutes longer than pdfLaTeX or XeLaTeX, depending on complexity.
3. LuaLaTeX provides better facilities for accessible PDFs, which is becoming a requirement for many research funding agencies (pdfLaTeX also provides them, but XeLaTeX does not).
4. LuaTeX generates a PDF directly, like pdfTeX does; unlike XeTeX which creates an intermediate (XDV) file which is then converted to PDF (reminiscent of the original TeX creating a DVI file which needed converting to PostScript).
5. I have still not found a personal use case for the Lua scripting facilities, but maybe one day.

So having committed to the switch, I am drafting this using LuaLaTeX, and the next months will be taken up with regression testing 'Formatting Information' before publishing a new edition which recommends it, and giving details of what you need to switch (which is probably almost nothing unless you have legacy PostScript requirements).

Afterthought

Many users do their LaTeXing online using *Overleaf*, *Papeeria*, or similar, especially beginners who may not be able to download and install TeX Live, or who are simply unaware that you can do so; but also students and others in the natural sciences who are writing a document collaboratively with colleagues (less common in the humanities).

A more recent arrival in this field is *Crixet* (`crixet.com`) offering similar facilities but with a different interface and editor. Welcome aboard!

References

[1] Craft Supply Co. Typography Trends 2025 — 50+ Fonts You Need to Know. `craftsupply.co/articles/font-trends/`

[2] P. Flynn. Formatting Information — A beginner's guide to formatting documents with LaTeX. *TUGboat* 23(2):115–237, 2002. `tug.org/TUGboat/tb23-2/`

[3] P. Flynn. *Formatting Information — A beginner's guide to formatting documents with LaTeX*. Silmaril, 2024. `latex.silmaril.ie/`

[4] A. Jackson. 10 Must-Follow Creative Typography Trends Of 2025. `www.designhill.com/design-blog/must-follow-creative-typography-trends/`

[5] P. Jørgensen. The LaTeX Font Catalogue. `tug.org/FontCatalogue/`

[6] P. Lee. 40 Typography Trends in 2025. `designworklife.com/typography-trends/`

[7] P. Motev. Top 10 Typography Trends for 2025 — Exploring the Bold, Dynamic, and Nostalgic in Type Design and Digital Fonts. `fontfabric.com/blog/top-typography-trends-2025/`

[8] najam. Typography Trends in 2025: What's Changing in Digital Text? `dovethemes.com/typography-trends-in-2025-whats-changing-in-digital-text/`

[9] D. Preston-Kendal, D. Carlisle, et al. TeX (LuaTeX, XeTeX, fontspec) support for OpenType variable fonts. `tex.stackexchange.com/questions/355104/`

[10] A. Wilson. The hottest typography trends to watch in 2025. `www.creativebloq.com/design/fonts-typography/the-hottest-typography-trends-to-watch-in-2025`

◇ Peter Flynn
Textual Therapy Division,
Silmaril Consultants
Cork, Ireland
Phone: +353 86 824 5333
`peter (at) silmaril dot ie`
`blogs.silmaril.ie/peter`

OpBible — a tool for creating study Bibles

Šimon Konečný and Petr Olšák

Introduction

A study Bible can be created by taking the complete text of one of the many variations of the Bible translation from the internet and adding notes to this source text. If you want to do so using TEX, and to apply your general notes to more Bible translations at the same time, then you can use the OpBible package [1]. This package was created via intensive collaboration of two authors. The first author, Šimon Konečný, is mainly a theologian and in addition an average TEX user. He brought the main ideas of what to do and he was the first and main tester of the result. He started to create his extensive note apparatus for new study Bibles in the Czech language [2]. The second author, Petr Olšák, is a TEX expert. He implemented the ideas in the TEX package.

Šimon Konečný wrote the first part of this article. He describes the package from the user's point of view. Petr Olšák added notes on technical background in the second section of this article where some interesting technical aspects are mentioned.

In the beginning was the Word

Study Bible producers would usually say *Amen!* to this claim from the very first verse of the Gospel of John. However, the vast majority of them would also add an extra-biblical *eisegesis* of their own tradition: *. . . and in the end was the InDesign.* The traditional typesetting of study Bibles from Microsoft Word to Adobe InDesign results in one particular study Bible only, in which, after the typesetting is complete, no further changes are allowed.

The OpBible package might have the potential to make this expensive process obsolete. Consider these unparalleled advantages of the OpBible macro and judge for yourself:

Price. The software is free, both libre and gratis, being public domain under an open source license. More significant for many, however, is the price which you will save on a typesetter you would otherwise have to pay for several years to manually typeset your study Bible using InDesign from Word source documents.

Time. Typesetting an entire study Bible takes minutes, not years. On a Ryzen 3 (quad-core) processor, one complete Bible took about ten minutes; on a Ryzen 7 (16 core) it is about one minute.

Flexibility. The result is not just one single version of the Bible in which you cannot make any changes. If you later decide to edit some notes, delete some or add some more, change fonts, etc., you have this new revised version immediately typeset and ready for print.

Interactive output format. The result of the processing is a PDF file that is richly augmented with active links. They make an invaluable aid for proofreading before submission to the print (see below).

Precision. A typical study Bible contains around 20,000 notes and within them around 60 to 80,000 references to Bible verses and other notes. The notes are written by humans; it is inevitable that they will contain errors from oversights or typos. It is practically beyond the human power to find and correct all of them. It is, however, within the power of a machine. OpBible prints warning messages if, for example, a note refers to a non-existent note or verse, or if the phrase you want to comment on cannot be found in the verse. In the Spirit of the Reformation Study Bible, in the note on Genesis 8:15–19 on page 24 we read: "See note on 6:19–9:29". In vain you look for such a note, because it does not exist, but after some extensive search you discover the note on 6:9–29, and — voilà — it's the right one. The reference contained a typo; it should have been 9 instead of 19. OpBible notices such occurrences and warns you of them.

You can also check for the accurateness of references to verses and notes that do exist (thus they do not trigger an error message) but for some reason they are not the right ones. OpBible helps you avoid such errors in that all references are made active links that display the immediate context of the linked location when the cursor hovers over them (if you are viewing the PDF with a good version of Evince, for example). And when you click on them, they jump straight there (but not back where you jump from, therefore you may find hovering-over preferable).

The phrase that the note comments on is highlighted with a different font (e.g., bold). Moreover, OpBible will also search for it in the Bible text inside the verse in question, and the pages will be broken in such a way that the note ends up on the same page as the phrase it is commenting on. In the case that two notes are commenting on two different words in the same verse, and the page break happens between those words, the notes will follow their phrases on their respective pages. Compare that with, for example, the NIV Study Bible, Isaiah 36:2, where the note comments on the phrase *large army* on page 1065 but you have to turn the page to 1066 before you

doi.org/10.47397/tb/46-1/tb142konecny-opbible}user{/Subtype /Link /A << /S /URI /URI (https://doi.org/10.47397

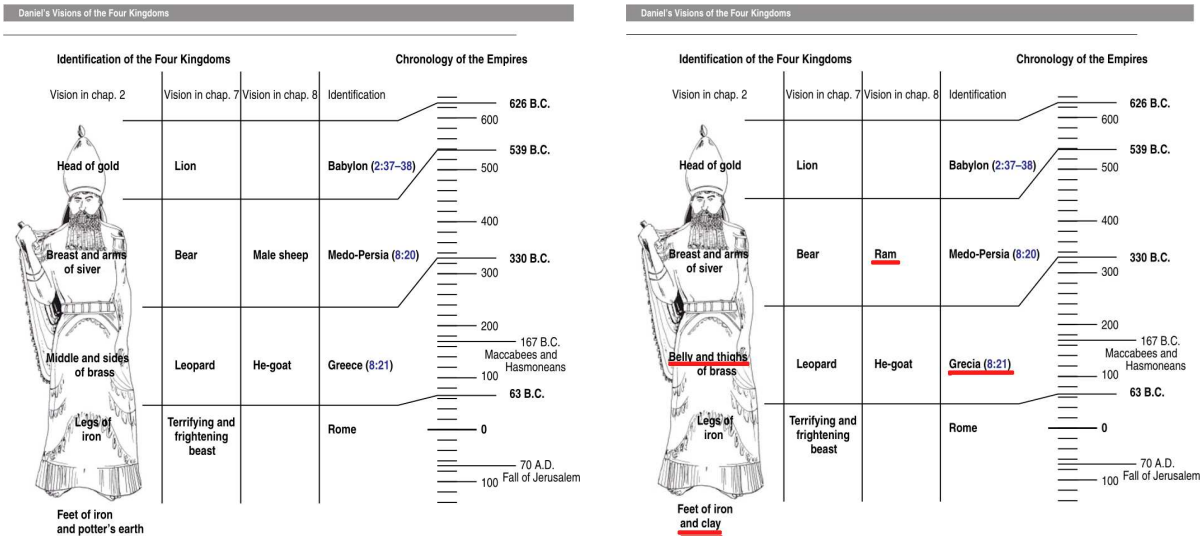


Figure 1: Two illustrations, one from BBE (left) and one from Webster (right). Differences are underlined in red (grayscaled in print) in the right-hand illustration.

see the actual large army in the biblical text. Some readers may find this annoying. With OpBible, you don't have to worry about such cases; they never occur.

Variability. The result may not be just one Bible. The notes can be written in a way that allows as many Bibles as there are different translations in a given language to be typeset with the same note apparatus.

For example, consider these six English translations of the Bible:

(1) Bible in Basic English (BBE);
(2) Jubilee2000;
(3) NETfree;
(4) Updated King James Version (UKJV);
(5) Restored Names King James Version (RNKJV);
(6) Webster.

Now let's take a brief look at Daniel, chapter 2, where the animals, countries, and other elements of Daniel's vision are rendered as follows:

- *Bear* is the same everywhere but
- *Leopard* of most translations is *Tiger* in Jubilee2000;
- the usual *Ram* becomes *Male sheep,* as well as *clay* changing into *potter's earth* in BBE;
- the usual *brass* is *bronze* in NETfree;
- the *Greece* of BBE, Jubilee2000, and NETfree is *Grecia* in UKJV, RNKJV, and Webster.

When you change the Bible version, these varying words will change accordingly not only in the biblical text but also in the notes.

Moreover, the variability is not limited to the notes only; the expressions may also change in graphs, timelines, images, maps, etc. Compare, for example these two images from BBE and Webster, depicting Nebuchadnezzar's dream in Daniel 2, and notice the differences in both the description of the statue and Greece vs. Grecia (Fig. 1).

To accomplish this, you need type them all just once in a file `vars.tex` like this:

```
\vdef {He-goat}   % BBE
      {Goat}      % Jubilee2000
      {Male goat} % NETfree
      {He goat}   % UKJV
      {Goat}      % RNKJV
      {He-goat}   % Webster
```

After that, wherever you type `\x/He-goat/` (the first entry of the set), TEX will know which version of the Bible you are currently using and pick the correct wording. This syntax allows for altering the shape (case) of the word. If you type, for example, `\x/Greece/'s king` you will see *Greece's king* in BBE, Jubilee2000, and NETfree, but in UKJV, RNKJV, and Webster you will get *Grecia's king.* This is very useful, especially in languages that decline nouns by changing their endings.

Similarly, the varying expressions can be placed over any other kind of image. Take a look at the two maps in Fig. 2 and notice the difference in spelling of the name Shushan/Susa.

The same variability applies also to the curved local names (from the tiny ones like the names of the rivers to the slightly larger ones like Persian Gulf to the really large like The Seleucids). If they differed in translations, they would change accordingly. As you might expect, the blue references on the map are active links.



Figure 2: Two maps, one from BBE (left) and one from NETfree (right). Differences are marked in red in the right-hand map. The blue references (Dan 1:2 etc.) are hyperlinks.

Another interesting aspect is the text inside the two semi-transparent rectangles. You don't need to prepare those lighter spaces in a graphical editor on a blank map in advance and then try to fit the text in. When you type your text over a map, its background will be automatically lighter (to avoid clashes with darker parts of the map) as far as the text goes. All the dimensions of the semi-transparent area, e.g., the space between the text and the border of the rectangle, or even the degree of the opacity, can be adjusted, if you are not happy with the default settings.

Formatting of the biblical text. The notes, articles and introductions are the files that the user would be modifying when creating a study Bible. What if you want to change something in the Bible itself? There are passages full of poetry where the lines need to break differently from prosaic text. The OpBible provides a way to do this non-destructively, i.e., you can format the biblical text without actually modifying it. You can tell TEX to find the particular word inside the verse, break the line (or insert a command or anything) before or after that word, and indent the next line in the desired fashion (Fig. 3).

all the land: "Peace and prosperity! [26] I have issued an edict that throughout all the dominion of my kingdom people are to revere and fear the God of Daniel.

"For he is the living God;
he endures forever.
His kingdom will not be destroyed;
his authority is forever.
[27] He rescues and delivers
and performs signs and wonders
in the heavens and on the earth.
He has rescued Daniel
from the power of the lions!"

[28] So this Daniel prospered during the reign of Darius and the reign of Cyrus the Persian.

Figure 3: Red words are those which are found in the Bible text and connected to a note printed at the same page. The color can be switched off for the final printing.

This way you can, for example, change the font for one particular word to indicate the translator's addition. You can insert control sequences not only around a verse, but also do the same thing with a chapter or an entire book. You can use the feature that allows you to place anything right after a chapter for including, for example, a reading plan, which might say something like: *Continue reading at Matthew 10,* similar to the Three Track Reading Plan of the NIV Student Bible.

How you specify all this you can find described in the documentation.

Free bonuses. After you download the Bible translation(s) that you are planning to use for your study Bible(s), you will need to make separate files with individual biblical books to be used with OpBible. The package provides a python script that does that for you.

OpBible also provides additional scripts to run multiple Bible translations at the same time so that you don't need to switch between them manually.

Some Bibles contain a topical concordance somewhere toward the end of the volume (Fig. 4). OpBible has a handy macro to help you with that. Alphabetical sorting is done at the TEX macro level; no extra program is needed.

Yet another Python script can prepare an entire Bible to be used as a source for a concordance. In this case the concordance would not be topical but exhaustive; it would show each word in the Bible and the verses where it occurs. There is a list of words you do not want to be included (*a, the, of, in, or, for, by, etc.*; you can modify the list according to your language conventions), in addition to the punctuation which the script will also delete from the source file.


Disaster 2129 Loss

The Counselor's Concordance

Disaster
• National or Community Gen 5:20, Ps 27, Ps 42, Ps 46, Ps 62, Ps 90, Ps 91, Ps 118, Ps 121, Eccl 3:1–8, Isa 41:10, Isa 43:1, Isa 61, Lam 3, Luke 13:1–5, John 11:17–44, Rom 8:28–39
Funerals Job 19:23–27, Ps 23, Ps 46, Ps 62:5–8, Ps 90, Ps 91, Ps 116:15, Ps 126:5, Ps 139, Dan 12:1–3, Isa 55:6–13, Isa 61:1–3, John 5:21–29, John 11:17–27, John 14:1–7, 27, John 20:1–11, Rom 8:37–39, Rom 14:7–9, 1 Cor 15, 2 Cor 4, 5, 2 Cor 5:8, Phlp 1:21–23, Phlp 3:20–21, 1Thess 4, 2Tim 2:8–13, 1John 3:1–3, Rev 19:1–8, Rev 20, Rev 21:1–7, Rev 22:1–5,

Loss
• of a Child 2Sam 12:18–23, Ps 8:2, Ps 139:13–16, Ps 145:1–13, Isa 49:15–16, Jer 1:5–8, Matt 18:1–5, 10, Mark 10:13–16, Luke 1:41–50, 1Cor 7:14, 1Thess 4:13–18, James 4:14,

Figure 4: The top part of a page with the concordance.

Ordinarily, including each word of the Bible in a concordance is not the most brilliant idea. Not only does it make your document some 500+ pages larger (and the TEX runtime about thirty times longer) but the result (with each word in all its forms) will be confusing for a reader. There are better ways to search for a particular phrase; OpBible by no means aspires to supplement dedicated biblical software.

However, as an OpBible user, you may find a complete concordance useful for accuracy checking. Whether you downloaded the Bible from the internet or created your own brand new Bible translation, there may be errors in the text. The complete concordance can show you anomalies like merged words or a word that begins with a comma, and at the same time it shows you the verse where it happened so you can fix your source easily.

Non-free bonus. By now you have learned about OpBible and what you can expect of it from the user's viewpoint. The extra bonus that follows is not inherently part of the TEX macros; it is an addition that sits on the top of OpBible; OpBible can run happily without it. It is currently (January 2025) under development so there is not much yet to boast about. Also, as it is not a part of the OpBible package, the open source license will not apply to it. In other words, when it is available, it might be obtainable commercially.

It is HTML output. Alongside the PDF, which is the main purpose of OpBible, you can tell TEX to produce also HTML output. This could serve not only as a web page but also as a source for mobile apps, etc. The only limitation in this direction is your imagination.

Here is how it works so far: What in PDF was a phrase in the verse that the note was commenting on is a blue-colored part of a biblical verse in HTML. What was the text of the note is now a tooltip that becomes visible when you hover your mouse over such phrase, or which you tap on in your smartphone. If the note contains other references, they become links in the tooltip with the same effect in the secondary level: you hover-over/tap and the content will show up in yet another tooltip (Fig. 5). The tooltips work up to the second level both on PCs and smartphones.

OpBible provides a text source for toys like these. If you are skilled enough programmer you can do with the data whatever you wish yourself.

Notes on the technical background

To process the Book of Books (the Bible) with TEX is quite a challenge, especially when it's a study Bible where you have to solve many new specific problems.

The OpBible package is based on OpTEX macros. This is the reason of the name part "Op". The author of the OpBible macros needed absolute control over TEX's behavior and that wouldn't be possible if LATEX macros (for example) were used. OpBible is a huge project with plenty of very specific properties never before solved by TEX macros. Only some of the main and most important OpBible features are mentioned in this article.

Core text, translation variations. The Bible text should be downloaded from the internet and transformed to text files. Each Bible book is in a single text file; each verse is on a single line prefixed by the chapter number and verse number. These files can be prepared by additional software which is a part of the OpBible package. We will call the text in these files the "core text". If there are translation variations (called "tr-vars" in this article), then each tr-var has its own file and the tr-var mark is used in the names of the files. For example the six tr-vars mentioned above have the core text of Genesis in the files `BBE-Gen.txs`, `Jubilee2000-Gen.txs`, `NETfree-Gen.txs`, `RNKJV-Gen.txs`, `UKJV-Gen.txs`, `Webster-Gen.txs`. Their contents look like this:

```
#1:1 In the beginning God created the heaven ...
#1:2 And the earth was without form, and void...
#1:3 And God said, Let there be light: and th...
```

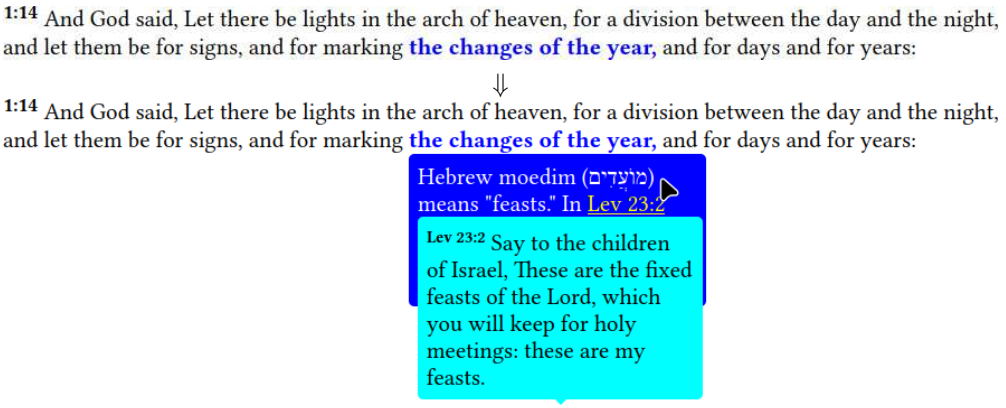


Figure 5: Notes to a selected word (and even other verses linked from the notes) are shown as tooltips in the HTML version of the study Bible.

When we say `\def\tmark{NETfree}` (for example) at the beginning of the document, then the specified tr-var is processed.

Notes. Each Bible book has not only the core text, but also the notes appended to it. It is mainly these that make a study Bible from a regular Bible. These notes are contained in separate files, for example `notes-Gen.tex` for Genesis. These files are common for all tr-vars. Authors of these notes can use `\x/phrase/` for phrases that vary in different tr-vars. The `\x` macro expands to the necessary phrase used in selected tr-var. This is not a complicated task for TEX, just the TEX primitive commands `\csname` and `\endcsname` used appropriately. The main advance is that the author of notes creates *common notes for all tr-vars*.

A more complicated task is complying with the following request: the note must be on the same page as the phrase which is being commented on. That very phrase is in core text while the note is in the note file (prefixed by `\Note` control sequence followed by the chapter:verse number and by the commented-on phrase). The notes file on the book of Daniel looks like this:

```
\Note 1:2 {And the Lord delivered}
   Israel's defeat by the Babyloniansis not ...
   breaking their covenant obligations
   (<2Kgs 17:15>, <18-20>; <21:12-15> <24:3-4>).
\Note 1:2 {These he carried off}
   Refers to the plunder of vessels
   from the temple, not to the deportation
   of captives.
```

The main rule is this: The core text downloaded from the internet must remain unchanged. So, each Bible book is processed in two steps. The whole notes file is read first and all the notes are saved in TEX's memory, arranged by verse numbers. Then TEX reads the core text of the given Bible book verse by verse. For each verse, it finds all phrases of all notes relevant to this verse and inserts control sequences in front of such phrases in the TEX memory. Then the verse is printed and the inserted control sequences expand to the `\insert` TEX primitive, i.e., they behave like footnotes. These "footnotes" are arranged in more columns, output on the same page as the core text. After that, the next verse is processed.

A typical page is divided into a top part with the core text and a bottom part with the comments. Thus the given requirement about notes on the same page as commented-on words in the core text is met. When one biblical book is complete, the TEX memory is freed and next book of the Bible is processed. There are 66 Bible books in the Protestant canon; you cannot expect TEX to swallow the entire Bible in one bite.

The phrases referred by the `\Note` commands can vary in different tr-vars. The phrases are automatically transformed with respect to the used tr-var analogously to phrases declared by the `\x` macro.

`\Note`s are connected to given chapter:verse numbers. These numbers are the same in almost all tr-vars, but a few tr-vars can include slight differences in the numbering. A configuration macro for such tr-vars is designed where the re-numbering rules of chapter:verse can be specified for selected tr-vars. Once TEX reads this data, it does internal re-numbering and the unchanged `\Note` is connected to the right verse in these specific tr-vars.

Other objects printed on the pages. There are not only notes in study Bibles, but also images, graphs, maps, timelines, etc. These objects can be dependent on tr-vars. Of course, we could create isolated graphic files for each tr-var. But it's more interesting to use the power of the `\x` macro and create a single image file common to all tr-vars and add the labels into it in TEX, i.e. using the `\x` macro.

Sometimes we need to print this variable text around curves (for example in maps, see Fig. 2).

These objects have to be put onto a page where the given chapter and verse begin. This means that the author of the study Bible specifies the placement of these objects by the chapter:verse number in the actual Bible book. TEX has to re-design the layout of the appropriate page in order to insert the image.

The timelines are typically very wide but not too tall objects, and there is another special request for them here. They have to be printed over two pages in the open book. It means that their left part is on a left page and their right part on the right one. TEX has to calculate the optimal placement of these objects with respect to the author's wish that they should be somewhere near the given chapter:verse.

Other specific objects are so-called "articles". They are in fact more extensive notes with their own title, typically occupying several pages. They begin on the page of a given chapter:verse (similar to graphs or maps). The text of an article disrupts the typical page layout of one or more subsequent pages; it is inserted in their bottom parts in, typically, a two-column arrangement and is graphically separated from the other core text and notes. The sources of articles are in separate files and TEX reads them before the second pass over the Bible book (similar to notes). Articles that span over multiple pages are balanced so that their tops cut both pages of an open book at the same level.

Each Bible book can have its own introduction. The TEX source of the introductions is prepared in separate files (`intro-Gen.tex`, for example). If the relevant file exists then the introduction is inserted at the beginning of the Bible book before the first chapter.

References, hyperlinks. A very interesting problem is creating hypertext links in the study Bible. There are thousands of them. We don't need to use hidden labels visible in the TEX source only (like in TEX sources of scientific articles) because Bible references have had a fixed format for centuries, with fixed verse numbers, chapter numbers and Bible book names. Theologians use references in the format "see Gen 3:15" and the destination is known exactly. So, the author of notes can simply write `see <Gen 3:15>` and the text between `<...>` creates the active hyperlink automatically.

The references written by authors of notes are context-dependent in most cases. The author of a note can write `see also the note on <17>n` and TEX has to know that this refers to Matt 5:17, because we are processing the chapter 5 of Matthew now. The `n` after `<17>` means that the destination is not in the core text but in the notes area where the notes on Matt 5:17 are printed. The rules of transforming context-dependent references to exact references are somewhat more complicated (five pages in the user manual), so that they respect the typical theologian's habits when using references. The example above with `\Note` shows typical usage of hyperlinked references.

Formatting the core text. Bible books are generally divided into chapters (exceptions do exist). It is easily seen from the core text sources where chapters begin: the chapter number is incremented and the next verse has number one. We can easily configure the design of chapter titles using TEX macros which are executed at beginning of each chapter.

In addition, sometimes we want to insert a subtitle between given verses in a middle of the chapter. This can be configured by external format files, because we honor the rule that the sources with core text are never changed during creating the study Bible. One reason for this rule is that one might download a new version of a selected tr-var or a new tr-var from the internet. The new files with the core text can be simply used without the loss of formatting instructions.

The format files allow insertion of arbitrary TEX commands at arbitrary places. A place is referred to by chapter:verse number in a given Bible book and also by a phrase used inside the verse. For example, some verses need to be broken into more lines with controlled indentation of each line. See Psalms or the example in Fig. 3. This line breaking and indentation can be easily controlled by special macros used in the format files.

Error handling. When the author makes a mistake or a typo that leads to a TEX error, it would normally be rather difficult to find where the problem lies, because all `\Notes` of a single Bible book have been read into TEX memory, and then the macros with `\Note` texts are executed from memory. At this point of processing, we don't know which line in which file caused the error. To help with this, OpBible prints the header of each `\Note` to the log file just before it is processed. It also prints the calculation done for creating the hyperlinks.

When authors are trying to process newly created TEX sources then they can run only selected Bible books given in a list of abbreviated names of these books. They don't need to process the entire Bible. This can make it easier to find errors.

Moreover, OpBible provides the `\checksyntax` mode in which the `\Note`s are being read and executed immediately. In this mode, they are not connected to the core text so that typos from `\Note`s can be detected more easily.

Transforming to a text file. As mentioned, each verse of the Bible is completely read into TEX's memory first and then it is processed. The same is true for notes and other material. Therefore, we can print these texts not only to the PDF output file, but we can also write them to an output text file with additional marks for defining the structure of the text. Such an output file may look like this:

```
\BibleBook{Dan}{Daniel}
\SubTitle{Jehoiakim's captivity}
\Verse{1:1}{%
    \MidNote{}{}{Vindication of Daniel and Hi...
    \MidNote{In the third year of the reign of
       Jehoiakim}{In 605 B.C., the same year ...
       king of Judah Nebuchadnezzar king of B...
\Verse{1:2}{%
    \MidNote{And the Lord gave}{Israel's defe...
       Jehoiakim king of Judah into his hand ...
```

Perhaps this example isn't very clear because this output text file isn't intended for humans. You can only see that the core text is interleaved with notes, and special marks are used here. These marks can be read by additional software which generates the HTML output. The core text is interleaved by notes in this HTML file, too. Then the HTML browser can show the core text by default and the notes can be shown as mouse-on tooltips (see Fig. 5).

This output depends on the selected tr-var, because all `\x` macros were expanded before writing to the output file.

The main idea of this feature is this: If TEX knows how to interleave the core text with the notes and how to transform the tr-vars dependent phrases then we can make TEX generate text files which can be transformed to HTML output or whatever else. The sources for such variants of study Bibles remain the same as TEX sources used for generating PDF output.

Concordances. A concordance is something like an index. It includes alphabetically sorted words or phrases with lists of references to Bible verses next to them. See Fig. 4. The list of words used in the concordance should be prepared by authors of the study Bible.

We tried an interesting experiment. We gathered all words used in the Bible, including the references to verses where the word is present. Only pronouns, prepositions and conjunctions (and punctuation) were excluded from this large list. The list included more than 370,000 words and it was alphabetically unsorted. Many words were repeated. For example the word Lord (in Czech, Hospodin) was included 6851 times. Then we ran the sorting implemented in OpTEX on this big data. We were surprised to see the results in about three minutes, which is very good for interpreters like TEX (with only sequential access to token lists). The result was many hundreds of pages, three columns on each page and hundreds of thousands of hyperlinks.

This was a somewhat impractical result; nothing like this is present in real study Bibles. But this proves that the merge sort implemented in OpTEX at the TEX macro level is an effective algorithm and that TEX is able to process large amounts of data with large numbers of hyperlinks.

Examples of outputs, conclusion. Examples of output from the OpBible package can be found at [1]. The project is distributed through CTAN too, and it is a part of the TEX Live distribution. The OpBible repository is at [3]. Not only user documentation is included in the OpBible package but comprehensive technical documentation of all OpBible macros, created using OpTEX's doc system, is also included.

As mentioned above, the OpBible package was created through collaboration of two authors. The first author, a theologian, uses TEX at a user level and he knows perfectly how study Bibles look and what the program should do. The second author is a TEX expert. Prior to starting the work, he had not much knowledge of study Bibles. This project wasn't a one-night stand because plenty of features were necessary to solve. It was a very interesting experience for the second author to get to know the needs of theologians and to reach the very limits of TEX's possibilities.

References

1. opbible.org/story-en.html
2. opbible.org/Bible.html
3. github.com/olsak/OpBible

◇ Šimon Konečný
ETS, Prague, Czech Republic
https://etspraha.cz/cs/profil/99-simon-konecny

◇ Petr Olšák
Czech Technical University in Prague
https://petr.olsak.net/

The DuckBoat — Beginners' Pond: A pilates . . . oops, PyLaTeX course

Herr Professor Paulinho van Duck

Abstract

In this installment, Prof. van Duck will show you how to programmatically create LaTeX documents or snippets using the Python library PyLaTeX.

1 Pretty Prague

Hi, (La)TeX friends!

The TUG 2024 Meeting took place in Prague, Czech Republic, last July. My assistant Carla and I attended the conference.

It was a joy spending time again with friends from all over the world! We enjoyed captivating talks, a delightful banquet, and an amazing organ concert.

Prague is one of the most beautiful cities in Europe, rich in history and culture, it is definitely worth a visit.

Carla took a selfie on Charles Bridge, she also climbed the 287 steps (of a spiral staircase without handrails) to enjoy the gorgeous view from the Great South Tower of St. Vitus Cathedral. Fortunately, ducks can fly, quack!

I am sure you know that a group of Ti*k*Zlings fans release *The Great Ti*k*Zlings Christmas Extravaganza* video every year.

This year there has been a novelty: *The Great Ti*k*Zlings Calendar 2025*! It has all the important dates from the TeX world and the Ti*k*Zling universe. You can find both the video[1] and the calendar[2] on the internet, as well as the code to create them.[3]

Good news on the `tabularray` front, too! The author, Jianrui Lyu, has hugely improved the performance of the package, removing the dependency on `l3regex`.[4] The new version (2025A) will amaze you with special effects: compilations with `tabularray` will be much faster, quack!

One of the advantages of LaTeX with respect to WYSIWYG (*What You See Is What You Get*) word processors is that a `.tex` file is just text; it can be easily generated by a program, written in any language you like.

Python is very trendy at present, since it is the de facto standard in data science and machine learning. In this installment, I will talk about PyLaTeX [1], a Python library for creating and compiling LaTeX files.

I have to warn you that the PyLaTeX project is pretty inactive. However, I contacted the author and they told me it is still maintained. If something is broken with, e.g., a new Python version, they will fix it. There are no new features planned, but the package is fairly complete, so we can safely use it.

I will assume you are able to run Python code. In case you are not, do not worry! Python is easy to learn, and there is plenty of documentation and tutorials on the internet; you can start from the official site, `www.python.org`.

I am a beginner in Python, too. I apologize in advance for any inaccuracies or inelegance. Any corrections or suggestions for improvements are welcome. Please email me, quack!

2 Quack Guide n. 10: Programming with PyLaTeX

In previous issues of *TUGboat*, there are some articles related to Python.

For instance, in [4] you find some examples where LaTeX documents are created by pure Python scripts and an introduction to PythonTeX [3], a package that allows you to enter Python code within a LaTeX document, execute it, and access its output in the original document. A more detailed overview of PythonTeX is provided by [2].

Here I would like to show you how you can generate LaTeX documents or parts of them with the Python library PyLaTeX.

This article is far from exhaustive, but I would like to give you some tips and tricks and show you some examples different from what you can find in the documentation.

PyLaTeX works on Python 3.3 or more recent. The current version also works in Python 2.7, but it is not guaranteed for the future. It is developed for GNU/Linux, but it functions on Windows, too. I run the snippets of this article using Python version 3.13.1 and PyLaTeX version 1.4.2, on Windows, without any problem.

The first thing to do is to install PyLaTeX with

```
pip install pylatex
```

[1] `vimeo.com/1038963018`

[2] `raw.githubusercontent.com/TikZlings/Extravaganza2024/main/calendar/extravaganza_calendar.pdf`

[3] `github.com/TikZlings/Extravaganza2024`

[4] `github.com/lvjr/tabularray/issues/553`

doi.org/10.47397/tb/46-1/tb142duck-pylatex

or an analogous command if you use, for example, a Python distribution like Anaconda.

Now you can use PyLaTeX for generating a full document, mostly useful when all the text is generated by Python, or for generating a snippet to include in another LaTeX document.

2.1 Generating full documents

2.1.1 Minimal working example

Let us start with the simple Python script shown in Listing 1.

Listing 1: A minimal example

```
from pylatex import Document

# Create a document
doc = Document("duckmwe")

# Put some text in it
doc.append("Hallo ducks!")

# Generate the .tex and the .pdf files
doc.generate_pdf(clean_tex=False)
```

The first line of the code imports the `Document` class from PyLaTeX library. Here, the term "class" is meant as a Python class, a sort of template for creating objects,[5] not a LaTeX document class. Even though, in this peculiar case, the Python class is used to generate the LaTeX class. I think I confused you enough, quack!

The instruction `doc = Document("duckmwe")` creates the object `doc`, the LaTeX document named `duckmwe`.

An object can have some attributes, that is, its characteristics.

In this case, since I did not specify any values, PyLaTeX will use the default: `article` as document class, and some packages loaded automatically in the preamble, such as `fontenc` and `inputenc`.

An object can also have methods, that is, functions applied to it.

`Document` works like a list, a data structure used to store collections of homogeneous items, its elements. You can use the method `append` to add elements; in the example I added a short sample text.

When we run the script, `generate_pdf` will produce the LaTeX code `duckmwe.tex` (see Box 1), compile it, and generate the PDF document `duckmwe.pdf`.

You may specify the LaTeX compiler to use with the homonymous parameter `compiler`. If it is not indicated or set to `None`, PyLaTeX will choose a fitting one on its own, starting with `latexmk` and then `pdflatex`.

By default, if the compilation returns a zero exit status (success), the method `generate_pdf` also cleans all the temporary files, `.tex` included. It only keeps the PDF document. Setting the parameter `clean_tex` to `False` avoids the deletion of the `.tex` file. With the parameter `clean` set to `False`, none of the auxiliary files are removed.

Box 1 – `duckmwe.tex`

```
\documentclass{article}%
\usepackage[T1]{fontenc}%
\usepackage[utf8]{inputenc}%
\usepackage{lmodern}%
\usepackage{textcomp}%
\usepackage{lastpage}%
%
%
%
\begin{document}%
\normalsize%
Hallo ducks!%
\end{document}
```

The output of this first minimal example is only a page with "Hallo ducks!", but if you managed to run the Python script and get the PDF, you did most of the work, quack!

Now you can enjoy customizing it as you like.

2.1.2 An example using `book`, focus on the escaping problem

Now I would like to generate a book. When I created the object `doc`, I specified the `documentclass`. I also set `fontenc` and `inputenc` to `None`, and `lmodern` and `textcomp` to `False`, to avoid loading the corresponding packages. See Listing 2.

My book has a chapter and two sections, I used the proper classes to create them. In the `import` lines, I listed all the classes used in the script.

In the first section of my book, I showed three different ways to add text that contains some LaTeX control sequences.

With the class `Command`, you have to indicate the name of the command as a parameter and, if needed: `arguments`, with the list of the mandatory arguments of the command; `options` for the optional arguments; and `packages` for a list of the packages that the command requires.

As you can observe in Box 1, PyLaTeX puts a `%` at the end of every line, including after a command name. If you need a space there, you should add `Command("space")`, translated in LaTeX as `\space`,

[5] See `docs.python.org/3/tutorial/classes.html`.

Listing 2: A `book` example with a focus on the escaping problem

```
1  from pylatex import Document, Chapter, Section, Command, NewLine, Math, Eqref, Label, Ref
2  from pylatex.package import Package
3  from pylatex.utils import NoEscape
4  from pylatex.base_classes import Environment
5
6  # Class for equation environment
7  class Eq(Environment):
8      packages = [Package("amsmath")]
9      _latex_name = "equation"
10
11 # Create a book
12 doc = Document(documentclass="book", fontenc=None, inputenc=None,
13     lmodern=False, textcomp=False)
14
15 with doc.create(Chapter("Some examples")):
16 # References to sections
17   doc.append(NoEscape("In Section "+Ref("sec:text").dumps_as_content()+" there is text."))
18   doc.append(Command("par"))
19   doc.append(NoEscape("In Section "+Ref("sec:Math").dumps_as_content()+" there are formulas."))
20   with doc.create(Section("Emphasized text", label="sec:text")):
21 # How to solve escaping problem
22     doc.append("Some ")
23     doc.append(Command("emph", arguments = ["emphasized"]))
24     doc.append(Command("space"))
25     doc.append("text.")
26     doc.append(NewLine())
27     doc.append(NoEscape("A string with \\emph{escaped backslash and NoEscape}."))
28     doc.append(NewLine())
29     doc.append(NoEscape(r"A \emph{row} string with \emph{NoEscape}."))
30   with doc.create(Section("Math")):
31 # Inline formula
32     doc.append("This is an inline formula: ")
33     doc.append(Math(data=("y=\\sum_{i=1}^{n}\\frac{1}{x_i}"), escape=False, inline=True))
34 # Displayed formula
35     doc.append("; this is a displayed formula: ")
36     doc.append(Math(data=(r"y=\sum_{i=1}^{n}\frac{1}{x_i}"), escape=False))
37     doc.append(Command(" "))
38 # Displayed and labeled formula
39     doc.append(NoEscape("and "+Eqref("eq:mylab").dumps_as_content()+" is a labeled formula:"))
40     with doc.create(Eq()):
41         doc.append(NoEscape(r"y=\sum_{i=1}^{n}\frac{1}{x_i}"))
42         doc.append(Label("eq:mylab"))
43
44 # Generate the .pdf file only
45 doc.generate_pdf("duckbook")
```

or `Command(" ")`, translated as `\␣`, a backslash followed by a space.

You may also need to add a LaTeX command directly in a string. In this case, you have to pay attention because the escaping problem arises: if you simply write

`doc.append("Some \emph{emphasized} text")`

you will get the error

`invalid escape sequence '\e'`.

In Python, escape sequences are used to specify actions such as newline (`\n`) or horizontal tab (`\t`). The backslash is used as escape character.

To tell Python that your backslash is an actual backslash and not part of an escape code, you can in turn escape it with ... a backslash, quack!

This solution could be verbose or not convenient for LaTeX code. See, for example, the inline math formula on line 33 of Listing 2.

Another way is prefacing the string with `r`, to indicate it is a *raw* string. It is clear and simple (cf. l. 36), but pay attention!

This way, any strings that you add to your object will not be escaped. Hence, use it only if you are certain that the string does *not* contain possibly unsafe code.

Of course, if you are a programmer, you already know what I explained till now, but it is not enough.

If you add

```
doc.append(r"Some \emph{emphasized} text")
```

it will give no errors but the resulting LaTeX code will be

```
Some \textbackslash{}emph\{emphasized\} text
```

that is, your PDF will show:

> Some \emph{emphasized} text

If the string does not have to be escaped when translated into LaTeX code, you have to use `NoEscape`.

Some classes, such as `Math`, have the boolean parameter `escape`. If you set it to `False`, LaTeX macros will be recognized; see Listing 2.

The `Math` class has other two parameters: `data`, for the content, and `inline`, a boolean to say if the formula is inline or displayed.

If you would like to label an equation, you could use `Environment` to create a class that generates the `equation` environment, as `Eq` in Listing 2. The parameter `_latex_name` sets the name of the LaTeX environment, and `packages` allows listing the required LaTeX packages.

Trivially, `Label` generates `\label` and `Eqref` `\eqref`. In the example, `Eqref` is within a string; the method `dumps_as_content` is used to create a string representation of it as content.

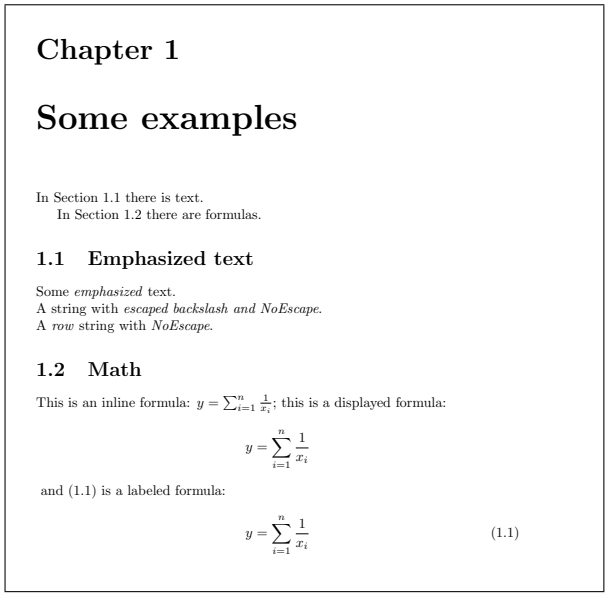
Some objects, like chapters and sections, are automatically labeled. The marker is composed of a prefix (`chap`, `sec`), a separator (`:`), and the title of the chapter, section, etc., without spaces. You can change it by setting the parameter `label`.

In the example, the automatic label of Section 2 was `sec:Emphasizedtext`; I changed it into `sec:text`.

You can refer to the label with the class `Ref`.

The last thing I would highlight in Listing 2 is the position of the name of the document. Indeed, it could alternatively be indicated as a parameter in the method `generate_pdf`. This can be useful if you would like to produce more than one PDF, with different names, with only one run of the script.

The LaTeX code generated by Listing 2 is quite verbose. I will not show it here, for the sake of brevity; the output is in Figure 1.



Chapter 1

Some examples

In Section 1.1 there is text.
In Section 1.2 there are formulas.

1.1 Emphasized text

Some *emphasized* text.
A string with *escaped backslash and NoEscape.*
A *row* string with *NoEscape.*

1.2 Math

This is an inline formula: $y = \sum_{i=1}^{n} \frac{1}{x_i}$; this is a displayed formula:

$$y = \sum_{i=1}^{n} \frac{1}{x_i}$$

and (1.1) is a labeled formula:

$$y = \sum_{i=1}^{n} \frac{1}{x_i} \tag{1.1}$$

Figure 1: Output of Listing 2

2.1.3 A `beamer` presentation, an example of automation

As you might guess from the previous example, PyLaTeX was not conceived in order to generate an ordinary document; you can do that more quickly writing LaTeX code directly. But PyLaTeX can be helpful if you would like to do something automatically.

For example, producing letters starting from a database of addresses; or exam tests, different for every student, starting from a collection of LaTeX snippets where every snippet is a question.

In Listing 3 you can find an example of automation, a presentation with a showcase of Ti*k*Zlings. The Ti*k*Zlings to include in the document are present in a list, `tlinglist` (ll. 5–6), whose elements are tuples of a kind of animal and a name.

This time, I did not list all the classes in the `import`, but took advantage of the alternative Python gives us: importing the complete package and then recalling the classes with the package name or alias (ll. 1–2) as suffix.

I would like to create a `beamer` document, but PyLaTeX does not have a class for generating the `frame` environment. Do not worry! It is easy to create it, `Frame` in the example (l. 9), using the base class `Environment`.

I also needed to add some instructions to my LaTeX preamble, but method `append` only adds elements to the document *body*. Of course, there is a solution: using `preamble.append`.

Listing 3: A `beamer` presentation, with some automation

```
import pylatex as pl
import pylatex.base_classes as plb

# List of TikZlings to include in the document
tlinglist=[("bear", r"B\"ar"), ("penguin", "Tux"), ("bug", "Marie"), ("coati", "007"),
              ("rhino", r"D\"urer"), ("squirrel", "Ratatosk"), ("meerkat", "Ambrogio")]

# Class for frame environment
class Frame(plb.Environment):
    _latex_name = "frame"

# Creation of the beamer document
doc = pl.Document(documentclass="beamer", fontenc=None, inputenc=None,
    lmodern=False, textcomp=None)

# Add code to preamble
doc.preamble.append(pl.Command("usetikzlibrary","tikzlings"))

doc.preamble.append(pl.Command("usetheme", "Madrid"))
doc.preamble.append(pl.Command("title", pl.NoEscape(r"Showcase of Ti\emph{k}Zlings")))
doc.preamble.append(pl.Command("author", "Paulinho van Duck"))

doc.preamble.append(pl.Command("setbeamertemplate", "caption"))
doc.preamble.append(pl.NoEscape(r"[numbered]"))
# or, alternatively:
# doc.preamble.append(pl.NoEscape(r"\setbeamertemplate{caption}[numbered]"))

# Title frame
with doc.create(Frame()):
    doc.append(pl.Command("maketitle"))

# Frame with the table
with doc.create(Frame(arguments="Summary table")):
    with doc.create(pl.Table()) as Table:
        Table.add_caption(pl.NoEscape(r"List of Ti\emph{k}Zlings"))
        with Table.create(pl.Tabular("llc", booktabs=True)) as Tabul:
# or, if you would like having the separators at the beginning and at the end:
#       with Table.create(pl.Tabular(r"@{\hspace{\tabcolsep}}llc@{\hspace{\tabcolsep}}",
#       booktabs=True)) as Tabul:
            Tabul.add_row(["Name", "Type", "Figure"])
            Tabul.add_hline()
            for tling in tlinglist:
                Tabul.add_row([pl.NoEscape(tling[1]), tling[0].capitalize(),
                               pl.NoEscape(pl.Ref("fig:"+tling[0]).dumps_as_content())])

# Frames with the TikZlings
for tling in tlinglist:
    with doc.create(Frame(arguments=tling[0].capitalize())):
        with doc.create(pl.Figure()) as Figure:
            with doc.create(pl.TikZ()) as TikZ:
                TikZ.append(pl.Command(tling[0],
                    options=(pl.NoEscape("speech={Hallo!}"),
                        pl.NoEscape("bubblecolour=cyan!30!white"),)))
            Figure.add_caption(pl.NoEscape(tling[1] + ", the " + tling[0]))
            Figure.append(pl.Label("fig:"+tling[0]))

doc.generate_pdf("duckbeamer", clean_tex=False)
```

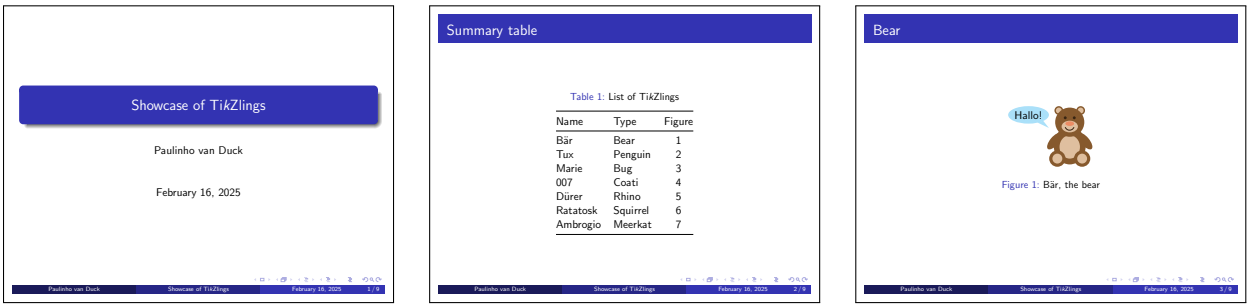


Figure 2: First three pages of the output of Listing 3

Similarly, if you need to add a package to your preamble, the method is
`packages.append(Package("⟨package-name⟩"))`

The instructions for `\setbeamertemplate` are worth mentioning. Since the class `Command` does not provide for extra options, just `extra_arguments`, I simply add them as a string, taking advantage of the PyLaTeX mania for putting a `%` at the end of every line (ll. 23–24). Alternatively, it is always possible to add the entire command as a string (l. 26).

The code for the title slide is simple; you only have to add the `Command` for `\maketitle` (l. 30).

In the second slide there is a table, automatically produced by looping on the `tlinglist`.

There are many ways to produce a LaTeX tables with Python. For example the method `to_latex` of a DataFrame object of `pandas`. Maybe it will be the subject of another article, but here let us focus on PyLaTeX.

The class `Table` generates a `table` environment, and has the method `add_caption`; guess what it is used for!

Similarly, `Tabular` creates a `tabular` environment. Among its parameters, you find the table specification, in the example `llc` (l. 36), and the boolean `booktabs`; if it is set to `True`, the package of that name is automatically loaded.

When `booktabs` is on, PyLaTeX also adds `@{}` before the first and after the last column specification. Strangely, there is no parameter to remove them. To restore the separators, you need to add `@{\hspace{\tabcolsep}}` on both sides of the table specifications (l. 38).[6]

The class `Tabular` has the method `add_row` for adding a row of cells to the table, the cell contents are separated by commas, and `add_hline` for adding a horizontal line (ll. 40–41).

The table of the example has three columns: the name of the TikZlings, within `NoEscape` because it contains LaTeX commands; the kind of animal, capitalized with the string method `capitalize`; and a reference to the figure where the TikZling is drawn. The marker of the label is composed of the string `fig:` followed by the name of the animal. (Lines 43–44.)

The table rows are automatically generated in a for loop on the list `tlinglist` (l. 42).

The remaining slides are also generated by looping on that list.

For the `figure` environment there is the specific class `Figure` (l. 49), with the method `add_caption` (l. 54).

There is also a class for the `tikzpicture` environment, `TikZ`, that automatically loads the package `tikz`. That is why I did not load the package `tikzlings` but the TikZ library of the same name.

Please observe how the commands for the TikZlings, the captions and the labels are all automatically generated from the item of the `tlinglist`.

Figure 2 shows the first three slides generated by Listing 3.

2.2 Generating LaTeX snippets

Rather than automatically generating entire documents, I think it is more usual to generate LaTeX snippets, which you can subsequently `\input` in your main document.

2.2.1 Series of images from a given path

Listing 4 creates a figure with all the PNG files present in a given directory, taking advantage of the method `listdir` from the Python module `os`.

In the example, I used the path of the LaTeX example images (l. 5); it may be different on your operating system.

I created a function, `subimage`, with a parameter, `subimagename`, that generates a `subfigure` with its `\caption` (ll. 7ff.).

A function is useful if you have to recall it more than one time. For instance, I used it to treat the first image outside of the loop and the rest within it.

[6] Thanks to samcarter for this solution: `stackoverflow.com/a/79019005/6689867`.

Listing 4: A snippet with subimages

```
import pylatex as pl
from os import listdir

# Path where the figures are (adjust for your installation)
path = "C:/texlive/2025/texmf-dist/tex/latex/mwe"

# Function for creating the subfigure
def subimage(subimagename):
  with doc.create(pl.SubFigure(position="b")) as subim:
    subim.add_image(subimagename, width=pl.NoEscape(r".35\linewidth"),
                    placement=pl.NoEscape('\\centering'))
    subim.add_caption(pl.NoEscape(subimagename))

# Create the object "Figure"
doc = pl.Figure(position="ht")
doc.append(pl.Command("centering"))

# Create a list of all the *.png documents present in the path
filelist=[file for file in listdir(path) if file.endswith('.png')]

# First image outside the loop
subimage(filelist[0])

# Loop for the rest of the images
for imagename in filelist[1:]:
  doc.append(pl.Command('space'))
  subimage(imagename)

# Caption of the figure environment
doc.add_caption("Example images")

# Generate only the .tex snippet
doc.generate_tex('duckimages')
```

I used the method `generate_tex` to only generate the LaTeX code, without compiling it (l. 33). In this case, the parameter that sets the name of the `.tex` file is required, since we do not have the `Document` object.

In Box 2 you can see part of the snippet generated by this script.

Box 2 – `duckimages.tex`

```
\begin{figure}[ht]%
\centering%
\begin{subfigure}[b]{0.45\linewidth}%
\centering%
\includegraphics[width=.35\linewidth]
        {example-grid-100x100bp.png}%
\caption{example-grid-100x100bp.png}%
\end{subfigure}%
\space%
\begin{subfigure}[b]{0.45\linewidth}%
\centering%
\includegraphics[width=.35\linewidth]
        {example-grid-100x100pt.png}%
\caption{example-grid-100x100pt.png}%
\end{subfigure}%
...
\caption{Example images}%
\end{figure}
```

If you `\input` it in your document you will get Figure 3. Remember to load the necessary packages in your preamble, in this case `graphicx` and `subcaption`, since the generated LaTeX code is only the part to put in your document body.

2.2.2 Two snippets with `tabularray`

The last example I would like to show generates two snippets using the `tblr` environment from the `tabularray` package (Listing 5).

Listing 5: Two snippets with `tabularray`

```
import pylatex as pl
from pylatex.package import Package

import csv

# Class for tblr environment
class Tblr(pl.Tabular):
    pass
    packages = [Package("tabularray")]
    _latex_name = "tblr"

# If this was not a snippet, remember to add
# doc.preamble.append(pl.Command("UseTblrLibrary","booktabs"))

# Create the first object
tblr=Tblr("colspec={llc}, hline{1,Z}={0.08em,solid}, hline{2}={0.05em,solid}")
with open("example.csv", "r") as f:
    read_data = csv.reader(f, delimiter=" ")
# Loop on the .csv lines
    for line in read_data:
        tblr.add_row([pl.NoEscape(line[0]), line[1], line[2]], strict=False)

# Generate the first .tex snippet
tblr.generate_tex("withoutbooktabs")

# Create the second object
tblr=Tblr("llc")
with open("example.csv", "r") as f:
    read_data = csv.reader(f, delimiter=" ")
# First line (headers)
    line1 = next(read_data)
    tblr.append(pl.Command("toprule"))
    tblr.add_row([pl.NoEscape(line1[0]), line1[1], line1[2]])
    tblr.append(pl.Command("midrule"))
# Loop on the rest of the .csv lines
    for line in read_data:
        tblr.add_row([pl.NoEscape(line[0]), line[1], line[2]])
    tblr.append(pl.Command("bottomrule"))

# Generate the second .tex snippet
tblr.generate_tex("withbooktabs")
```

PyLaTeX does not have a class for this environment. I created it by inheritance from the `Tabular` class (ll. 7ff.).

The tables of the example are created reading a CSV file, using a single space to separate fields rather than a comma, using the `reader` method of the Python module `csv` (l. 18). The file contains the data shown in Listing 3:

example.csv

```
Name Type Figure
B\"ar Bear 1
Tux Penguin 2
Marie Bug 3
007 Coati 4
D\"urer Rhino 5
Ratatosk Squirrel 6
Ambrogio Meerkat 7
```

The first table is created without `booktabs` (ll. 15–24), and the second one with it (ll. 26–end). In the latter case, when outputting a complete document, we would need to add a `Command` to emit `\UseTblrLibrary{booktabs}` (l. 13).

The class `Tabular`, whose attributes and methods `Tblr` inherits, checks that the number of columns is equal to the number of table specifications. For

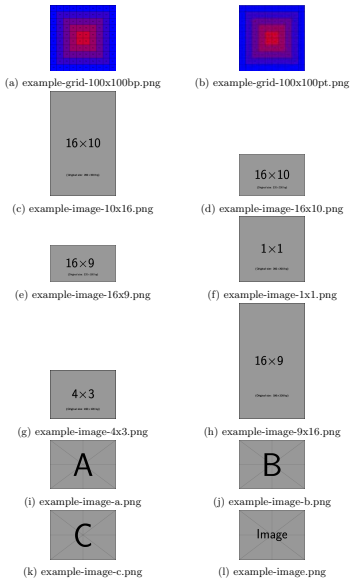


Figure 3: Output of Listing 4, after being input in a document

instance, if the table specifications are `llc`, there must be three columns.

In the first table, the table specifications also contain some `hline` parameters, so PyLaTeX is not able to calculate the correct number of columns, and would give the error:

```
Number of cells added to table (...)
 did not match table width (...).
```

You can avoid this by putting `strict=False` as a parameter of `add_row`, or by putting `width=`⟨*number-of-columns*⟩ as a parameter of the `Tblr` object.

Finally, I took advantage of the possibility of setting the name of the document as a parameter of the method `generate_tex`, and generated two different LaTeX snippets with the same Python code. One of them is listed in Box 3.

Box 3 – `withbooktabs.tex`

```
\begin{tblr}{llc}%
\toprule%
Name&Type&Figure\\%
\midrule%
B\"ar&Bear&1\\%
Tux&Penguin&2\\%
```

Name	Type	Figure
Bär	Bear	1
Tux	Penguin	2
Marie	Bug	3
007	Coati	4
Dürer	Rhino	5
Ratatosk	Squirrel	6
Ambrogio	Meerkat	7

Figure 4: Output of the snippet in Box 3, after being input in a document

```
Marie&Bug&3\\%
007&Coati&4\\%
D\"urer&Rhino&5\\%
Ratatosk&Squirrel&6\\%
Ambrogio&Meerkat&7\\%
\bottomrule%
\end{tblr}%
```

When you `\input` them in your document, with the needed packages in your preamble, they give the same result, shown in Figure 4.

3 Conclusions

I hope you find PyLaTeX useful, and remember:

LaTeX Ducks do not fear Pythons!

References

[1] J. Fennema. PyLaTeX 1.4.2 documentation. Accessed: 2025-02-15. `jeltef.github.io/PyLaTeX/current/`

[2] A. Mertz, W. Slough. A gentle introduction to PythonTeX. *TUGboat* 34(3):302–312, 2013. `tug.org/TUGboat/tb34-3/tb108mertz.pdf`

[3] G.M. Poore. The `pythontex` package, version 0.18. `ctan.org/pkg/pythontex`

[4] U. Ziegenhagen. Combining LaTeX with Python. *TUGboat* 40(2):126–128, 2019. `tug.org/TUGboat/tb40-2/tb125ziegenhagen-python.pdf`

◇ Herr Professor Paulinho van Duck
Quack University Campus
Sempione Park Pond
Milano, Italy
`paulinho dot vanduck (at) gmail dot com`

The glossaries package: Getting started

Nicola Talbot

Abstract

An introductory article on how to create glossaries and lists of abbreviations.

1 Introduction

In previous *TUGboat* articles [2, 3, 4, 5], I described some complex aspects of creating and sorting glossaries, aimed at readers who were already familiar with using the `glossaries` package [12] and its extension package `glossaries-extra` [11]. This article is aimed at readers who would like some help getting started.

2 Package loading

The `glossaries` package can be loaded in the usual way:

`\usepackage[`⟨*options*⟩`]{glossaries}`

However, if you want the enhanced support provided by `glossaries-extra` then it's better not to explicitly load `glossaries` but instead to just load `glossaries-extra`:

`\usepackage[`⟨*options*⟩`]{glossaries-extra}`

This will internally load the base `glossaries` package and make the necessary integration between the extension and base package.

If you also want to use `hyperref` [1], then this is one of the few exceptions to the general rule that `hyperref` should be loaded last. You will need to ensure that `glossaries`/`glossaries-extra` is loaded after `hyperref`.

3 Defining entries

Terms, abbreviations and symbols are all collectively referred to as "entries". They all need to be defined before they can be referenced in the document. This is best done in the document preamble (or in a separate file that can be input in the preamble).

The main command for defining entries is:

`\newglossaryentry{`⟨*label*⟩`}{`⟨*key=val list*⟩`}`

The label is used to reference the term in the document. The required keys are `name` and `description`. For example:

```
\newglossaryentry{duck}{name={duck},
 description={a waterbird}}
\newglossaryentry{zebra}{name={zebra},
 description={an African wild horse with
 black and white stripes}}
```

In the above examples, the label is the same as the name. This is usually the case when the name doesn't contain any commands or special characters.

If you need UTF-8 support, make sure that you have up-to-date versions of the packages `glossaries`, `mfirstuc` [6], `datatool` [10], and (if applicable) `glossaries-extra`. For example:

```
\newglossaryentry{ætherial}{name={ætherial},
 description={without material substance}}
```

The entry's plural form will default to the singular form with "s" appended. If this isn't the case, then the `plural` key is needed. For example:

```
\newglossaryentry{fairy}{name={fairy},
 plural={fairies},
 description={mythical being}}
```

The ⟨*key*⟩=⟨*value*⟩ interface doesn't allow paragraph breaks, so if you have a long description you can instead use:

`\longnewglossaryentry{`⟨*label*⟩`}{`⟨*key=val list*⟩`}`
`{`⟨*description*⟩`}`

For example:

```
\longnewglossaryentry{aardvark}
{name={aardvark}}
{An African mammal that feeds on ants and
termites.

Another paragraph with more information.}
```

In this next example, the label has to be different from the name, as the label must be expandable:

```
\newglossaryentry{pi}{name={\ensuremath{\pi}},
 sort={pi},
 description={the ratio of a circle's
  circumference to its diameter}}
```

This has also used the `sort` key. This is advisable if you need to use `\printnoidxglossary` or `\printglossary` (see Section 5) and the name contains commands.

An entry may have both a name and an associated symbol:

```
\newglossaryentry{pi-not}{
 name={product notation},
 symbol={\ensuremath{\prod}},
 description={indicates repeated multiplication}}
```

4 Referencing entries

Once the entries have been defined, they may be referenced in the document with:

`\gls[`⟨*options*⟩`]{`⟨*label*⟩`}[`⟨*insert*⟩`]`

This will display the text associated with the entry identified by the label. If `hyperref` has been loaded, this text will be in a hyperlink. The ⟨*insert*⟩ optional argument allows extra content (such as a possessive "'s") to be brought into the hyperlink. For example:

```
The \gls{aardvark} measured the circumference
of the \gls{duck}['s] pond and calculated that
the diameter must be $C/\gls{pi}$.
```


doi.org/10.47397/tb/46-1/tb142talbot-glossaries-intro

The aardvark measured the circumference of the duck's pond and calculated that the diameter must be C/π.

There's a similar command if the plural form is needed:

`\glspl[`⟨*options*⟩`]{`⟨*label*⟩`}[`⟨*insert*⟩`]`

For example:

```
The \glspl{fairy} lived on the \gls{ætherial}
plane.
```

The fairies lived on the ætherial plane.

If an entry has been defined with a symbol, there's a similar command to show the symbol instead of the usual text:

`\glssymbol[`⟨*options*⟩`]{`⟨*label*⟩`}[`⟨*insert*⟩`]`

For example:

```
\Gls{pi-not} (denoted \glssymbol{pi-not})
is used to represent the product of a sequence
of numbers.
```

Product notation (denoted $\prod$) is used to represent the product of a sequence of numbers.

The above example has also used one of the case-changing commands:

`\Gls[`⟨*options*⟩`]{`⟨*label*⟩`}[`⟨*insert*⟩`]`

This converts the text to sentence case.

The ⟨*insert*⟩ optional argument is particularly useful for symbols that may require super- or subscripts as they typically need to be brought inside the inner workings of the formatting.

```
Compare:
\[ \glssymbol{pi-not}_{i=1}^n x_i \]
with
\[ \glssymbol{pi-not}[_{i=1}^n] x_i \]
```

Compare:

$$\prod_{i=1}^{n} x_i$$

with

$$\prod_{i=1}^{n} x_i$$

5 Displaying a list of entries

The hyperlinks created by commands like `\gls` need a target. This is typically the corresponding line in the glossary or list of entries. There are a variety of ways to create that list.

5.1 `\printunsrtglossary`

The easiest way to create the list of entries is with:

`\printunsrtglossary[`⟨*options*⟩`]`

Glossary

duck a waterbird

zebra an African wild horse with black and white stripes

ætherial without material substance

fairy mythical being

aardvark An African mammal that feeds on ants and termites.
Another paragraph with more information.

π the ratio of a circle's circumference to its diameter

product notation ($\prod$) indicates repeated multiplication

Figure 1: `\printunsrtglossary`

This requires `glossaries-extra`. There are a number of different styles [7]. In the following, I've set the default style to `tree`:

`\usepackage[style=tree]{glossaries-extra}`

The style can also be set in the optional argument when displaying the glossary:

`\printunsrtglossary[style=tree]`

The result is shown in Figure 1. Note that this lists *all* the defined entries, including "zebra" which hasn't been referenced in the document, and the list is in the order of definition.

This is the simplest way of creating a list as it doesn't require anything extra to the usual document build. However, it does require you to take care about the order in which you define all your entries. It also doesn't provide any back-references (location lists) to indicate on what page or pages each entry was referenced.

5.2 `\printnoidxglossary`

The next simplest method for making the list of entries is to use:

`\printnoidxglossary[`⟨*options*⟩`]`

This is provided by the base `glossaries` package, so you don't need to load `glossaries-extra` if you don't require any of its supplementary features. One thing to note, however, is that the two packages have different defaults. The default behaviour with just the base `glossaries` package is to append a full stop (".") after the description. This would look odd for the "aardvark" description, which already ends with a full stop. The default behaviour with `glossaries-extra` is to omit it. So with just the base package, the `nopostdot` option is required.

Glossary

aardvark An African mammal that feeds on ants and termites.
Another paragraph with more information. 29
ætherial without material substance 29

duck a waterbird 29

fairy mythical being 29

π the ratio of a circle's circumference to its diameter 29
product notation ($\prod$) indicates repeated multiplication 29

Figure 2: `\printnoidxglossary`

This method uses a low-level sorting function provided by `datatool-base`,[1] so make sure that you also have an up-to-date version of `datatool` installed and also any applicable localisation support. The `datatool-base` package is automatically loaded by `glossaries` but may be loaded earlier. For example:

```
\usepackage[locales=en]{datatool-base}
\usepackage[style=tree,nopostdot]{glossaries}
```

The `locales` option is also supported by `glossaries` if `datatool-base` hasn't already been loaded:

```
\usepackage[locales=en,style=tree,nopostdot]
 {glossaries}
```

or load a language package first:

```
\usepackage[english]{babel}
\usepackage[style=tree,nopostdot]{glossaries}
```

This example will also need `datatool-english` [9] to be installed.

With the "noidx" method, it's necessary to activate it with:

```
\makenoidxglossaries
```

This ensures all the required information is added to the `aux` file and enables the `\printnoidxglossary` command. Unlike the other `\print...glossary` commands, this has a `sort` option:

```
\printnoidxglossary[sort=word]
```

This produces the glossary shown in Figure 2.

In both Figures 1 and 2, you may notice that there's extra vertical spacing between some of the rows, e.g., above the "duck" and "fairy" entries in Figure 2. This is the gap inserted between letter groups, which are determined by the first letter of the `sort` value. (If it hasn't been explicitly set, it's usually the same as the `name` value.) So "aardvark" and (in Figure 2) "ætherial" are in the "A" letter group and both "pi" and "product notation" are in the "P" letter group, hence there is no extra space between them.

Glossary

A

aardvark An African mammal that feeds on ants and termites.
Another paragraph with more information. 29
ætherial without material substance 29

D

duck a waterbird 29

F

fairy mythical being 29

P

π the ratio of a circle's circumference to its diameter 29
product notation ($\prod$) indicates repeated multiplication 29

Figure 3: `\printnoidxglossary` (`treegroup` style)

This vertical gap is part of the glossary style (so some may not support it) but for the predefined styles that do support it, the gap can be omitted with the `nogroupskip` option, or changed by redefining the `\indexspace` command, or, with `glossaries-extra` and `stylemods=tree`, the `\glstreegroupskip` command.

Conversely, if you want each letter group to have a heading, then select a style that supports this. For example, use `treegroup` instead of `tree` (Figure 3).

A few other points about the "noidx" method and its output in Figure 2:

- A second LaTeX run is required (as with cross-references). On the first LaTeX run there is no glossary.
- The "zebra" entry no longer shows up. This is because it hasn't been referenced in the document.
- The entries have been sorted using the sorting function in `datatool-base`, using the English localisation support.
- Each entry now has a number following the description. This is the location (page number) on which the entry was referenced. If you don't want this to appear, just use the `nonumberlist` option. The "noidx" method is unable to compact location lists. If that's required, you will need to use a different method.

[1] See the article "`datatool` v3", present issue, pp. 74–78.

5.3 `\printglossary` and indexing commands

The other methods are more complicated as they require an external indexing application to be run as part of the document build. There are two supported by the base package: MakeIndex and Xindy. Both require external files to be created that contain all the information needed to sort the entries and create the typeset glossary. This is different from both the "unsrt" and "noidx" methods which iterate over a list of labels.

The default is to use MakeIndex, but if you prefer Xindy, you can specify this as a package option:

```
\usepackage[xindy,style=tree,nopostdot]
 {glossaries}
```

In either case, you need an initialization command to open the external files and set up the commands used to write to these files:

```
\makeglossaries
```

After this, you define the entries as usual. The glossary is then displayed with:

`\printglossary[`⟨*options*⟩`]`

This won't show anything on the first LaTeX run (except with `glossaries-extra` it will show some informative text to let you know the document is missing the section, and how to fix it). It will only show the correct content once MakeIndex or Xindy has created the relevant files, so the next step is to do this.

The document build steps are now:

- Run LaTeX (`pdflatex`, `xelatex`, etc.);
- Run `makeglossaries`;
- Run LaTeX (`pdflatex`, `xelatex`, etc.).

The second step is a Perl script that works from the `aux` file, no matter whether MakeIndex or Xindy is required, or particular command line switches are used. The third step may need to be repeated. For example, if the document is in the file `myDoc.tex` then the system calls are:

```
pdflatex myDoc
makeglossaries myDoc
pdflatex myDoc
```

This is usually done by configuring your text editor or automation tool (see [8] for examples).

Aside from a few specific options, such as `sort`, most of the `\print...glossary` options are shared. For example, to set the style:

```
\printglossary[style=tree]
```

This produces the glossary shown in Figure 4.

Note that MakeIndex doesn't support UTF-8, which is why "ætherial" is placed at the end. (This example would fail with a style that shows the group headings, such as `treegroup`.) The `sort` key is needed for this case:

```
\usepackage[makeindex,style=tree,nopostdot]
 {glossaries}
\makeglossaries
\newglossaryentry{ætherial}{name={ætherial},
 sort={aetherial},
 description={without material substance}}
```

Xindy supports UTF-8 and a number of scripts. However, Xindy can have problems with names that contain commands; for example, the "pi" entry will need the `sort` key used in the earlier definition.

Glossary

aardvark An African mammal that feeds on ants and termites.
Another paragraph with more information. 902

duck a waterbird 902

fairy mythical being 902

π the ratio of a circle's circumference to its diameter 902

product notation ($\prod$) indicates repeated multiplication 902

ætherial without material substance 902

Figure 4: `\printglossary` (MakeIndex)

6 Abbreviations

Abbreviations are special types of entries that have a long and a short form. The name is set according to the style. The `glossaries` and `glossaries-extra` packages have different mechanisms for dealing with abbreviations. With just the base `glossaries` package, the style is set with:

`\setacronymstyle{`⟨*style-name*⟩`}`

and abbreviations are defined with:

`\newacronym[`⟨*options*⟩`]{`⟨*label*⟩`}{`⟨*short*⟩`}{`⟨*long*⟩`}`

For example:

```
\setacronymstyle{long-short}
\newacronym{nasa}{NASA}
 {National Aeronautics and Space Administration}
\newacronym{html}{HTML}
 {Hypertext Markup Language}
```

With the `glossaries-extra` package, the style is set with:

`\setabbreviationstyle[`⟨*category*⟩`]{`⟨*style-name*⟩`}`

and abbreviations are defined with:

`\newabbreviation[`⟨*options*⟩`]{`⟨*label*⟩`}{`⟨*short*⟩`}{`⟨*long*⟩`}`

To avoid conflict with the base package's mechanism, `glossaries-extra` redefines `\newacronym` to

use \newabbreviation with the category set to acronym. Categories (indicated by a label set with the category key) are one of the extended features provided by glossaries-extra, which allows entries in a particular category to be formatted in a specific way.

This means entries defined with \newacronym can have a different style from those that are defined with \newabbreviation. Here we define nasa as an acronym (style short), and html as an abbreviation (style long-short):

```
\setabbreviationstyle{long-short}
\setabbreviationstyle[acronym]{short}
\newacronym{nasa}{NASA}
 {National Aeronautics and Space Administration}
\newabbreviation{html}{HTML}
 {Hypertext Markup Language}
```

The optional argument for both \newacronym and \newabbreviation may have any keys recognised by \newglossaryentry, because the abbreviations are simply entries with special markup that reflects the chosen style. They are referenced in the same way as regular entries, using commands such as \gls, but their behaviour can vary depending on whether or not the abbreviation has already been referenced.

In our example, the long-short abbreviation style shows the long form on first use, followed by the short form in parentheses and then shows just the short form on subsequent use. The short style only shows the short form regardless of whether or not the entry has already been referenced. Thus:

```
First use: \gls{nasa}, \gls{html}.
Next use: \gls{nasa}, \gls{html}.
```

produces:

First use: NASA, Hypertext Markup Language (HTML). Next use: NASA, HTML.

By default, these entries will be placed in the same list as all the others. If you want the acronyms or abbreviations in a separate list, then when using just the base glossaries package use the acronyms package option:

```
\usepackage[acronyms]{glossaries}
```

With the glossaries-extra package, you can use either acronyms or abbreviations or (to further separate \newacronym from \newabbreviation) both.

7 Multiple lists

Each glossary has an identifying label and each entry is assigned to a specific glossary. The default glossary is labelled main. There are package options to create other common types of glossaries. The acronyms and (glossaries-extra only) abbreviations package options have already been mentioned. The other provided options are: symbols, numbers and index. Alternatively, you can define your own glossary with:

\newglossary*{⟨*label*⟩}{⟨*title*⟩}

When defining an entry, you can use the type key to assign the entry to a particular glossary. The \newabbreviation and \newacronym commands do this automatically to reflect the abbreviations or acronyms package options.

The type option can then be used in the optional argument of the applicable \print...glossary command. For example, with the earlier abbreviations and:

```
\usepackage[abbreviations,style=tree]
 {glossaries-extra}
\makenoidxglossaries
```

then the list of abbreviations can be displayed with:

```
\printnoidxglossary[type=abbreviations]
```

Figure 5 shows the result.

Alternatively, you can use iterative commands that display each defined glossary in turn, as in:

```
\printglossaries
\printnoidxglossaries
\printunsrtglossaries
```

Each \print...glossaries command does the corresponding \print...glossary for each glossary, in the order in which the glossaries were defined.

Abbreviations

HTML Hypertext Markup Language 32

NASA National Aeronautics and Space Administration 32

Figure 5: \printnoidxglossary (abbreviations)

8 Bib2Gls

An alternative approach is to define all your glossary entries in a bib file and use the Bib2Gls program with glossaries-extra (instead of using MakeIndex or Xindy). For example, the earlier examples can instead be defined in a file called entries.bib:

```
@entry{duck,name={duck},
 description={a waterbird}}

@entry{zebra,name={zebra},
 description={an African wild horse with
              black and white stripes}}

@entry{ætherial,name={ætherial},
 description={without material substance}}

@entry{fairy,name={fairy},plural={fairies},
```

```
 description={mythical being}}

@entry{aardvark,name={aardvark},
 description={An African mammal that feeds on
              ants and termites.

    Another paragraph with more information.}}

@symbol{pi,name={\ensuremath{\pi}},
 description={the ratio of a circle's
              circumference to its diameter}}

@entry{pi-not,name={product notation},
 symbol={\ensuremath{\prod}},
 description={indicates repeated multiplication}}

@acronym{nasa,short={NASA},
 long={National Aeronautics and Space
       Administration}}

@abbreviation{html,short={HTML},
 long={Hypertext Markup Language}}
```

We use a variety of entry types here:

- The `@entry` entry type is for general entries and corresponds to using `\longnewglossaryentry`.
- The `@symbol` entry type, used for the `pi` entry doesn't have the `sort` field, uses the label (`pi`) as the default sort value.
- The `@acronym` entry type corresponds to `\newacronym`, and the
- `@abbreviation` entry type corresponds to `\newabbreviation`.

This method needs to be used with the `record` package option:

```
\usepackage[record]{glossaries-extra}
```

If you are using `hyperref` you may prefer:

```
\usepackage{hyperref}
\usepackage[record=nameref]{glossaries-extra}
```

Any desired abbreviation styles need to be set next:

```
\setabbreviationstyle{long-short}
\setabbreviationstyle[acronym]{short}
```

The `bib` file needs to be identified:

```
\GlsXtrLoadResources[src={entries.bib}]
```

Alternatively, you can use the shorter command:

```
\glsbibdata[⟨options⟩]{⟨bib-list⟩}
```

which is a shortcut for:

```
\GlsXtrLoadResources[src={⟨bib-list⟩},⟨options⟩]
```

The entry definitions can be split across multiple files. For example, general entries may be in `general.bib`, symbols may be in `symbols.bib`, and abbreviations may be in `abbrvs.bib`:

```
\glsbibdata{general,symbols,abbrvs}
```

As shown here, the `.bib` extension may be omitted.

The way that this method works is that the `\GlsXtrLoadResources` command writes information to the `aux` file, and also inputs the applicable `glstex` file created by Bib2Gls (if it exists). The `record` option ensures that every time you reference an entry with commands such as `\gls`, a "record" is written to the `aux` file.

Bib2Gls can then read the applicable information from the `aux` file, which tells it which `bib` files to pull the data from and which entries have been referenced in the document. It will then select the required entry data, sort it, and create a `glstex` file with all the selected entries defined via `\longnewglossaryentry` (to allow for paragraph breaks), `\newacronym` or `\newabbreviation` in the desired order.

This means that the glossary can be displayed with `\printunsrtglossary` because only those entries referenced in the document have been defined, and they have already been defined in the desired order.

If the document source is in the file `myDoc.tex` then the system calls are:

```
pdflatex myDoc
bib2gls myDoc
pdflatex myDoc
```

Bib2Gls does not create letter groups by default. You will need to use the `--group` or `-g` switch to enable this:

```
pdflatex myDoc
bib2gls --group myDoc
pdflatex myDoc
```

Alternatively, most global options can be set with

```
\BibGlsOptions{⟨key=val list⟩}
```

For example, if you are using an automated build system that makes it difficult to set the command line switches, you can instead add the following to the document preamble to get letter groups:

```
\BibGlsOptions{group=true}
```

The complete example document is:

```
\documentclass{article}
\usepackage[T1]{fontenc}
\usepackage[record,abbreviations,style=tree]
 {glossaries-extra}
\BibGlsOptions{group=true}
\setabbreviationstyle{long-short}
\setabbreviationstyle[acronym]{short}
\glsbibdata{entries}
\begin{document}
The \gls{aardvark} measured the circumference
of the \gls{duck}['s] pond and calculated that
the diameter must be $C/\gls{pi}$.
The \glspl{fairy} lived on the \gls{ætherial}
```

```
plane.
\Gls{pi-not} (denoted \glssymbol{pi-not})
is used to represent the product of a sequence
of numbers.

First use: \gls{nasa}, \gls{html}.
Next use: \gls{nasa}, \gls{html}.
\printunsrtglossaries
\end{document}
```

The result is the same as in Figures 2 and 5 (but with the page number 1 in each location list).

9 Tips

- It's best to define entries in the preamble (this is enforced with Bib2Gls and the "noidx" method).
- The `glossaries-extra` package has special commands for awkward cases, such as `\glsfmttext` or `\glsfmtshort` for headings, and `\glspt` or `\glsps` for nested use inside the key value for another entry.
- If an entry is only referenced within the description of another entry, that dependent entry can't be indexed until the description in which it is referenced is typeset in the document. Dependency chains, such as:

  ```
  \newglossaryentry{duck}{name={duck},
    description={a \gls{waterfowl}}}
  \newglossaryentry{waterfowl}{
    name={waterfowl},
    description={aquatic \glspl{bird}}}
  \newglossaryentry{bird}{name={bird},
    description={creature with wings}}
  ```

  can require repeated document builds to ensure that all dependent entries appear in the glossary. Only Bib2Gls parses descriptions to search for dependencies to ensure that they are included; one extra build may still be required to ensure that the page numbers in the back references are correct.
- To prevent loading unnecessary extra packages, use the `no...` package options (such as `nosuper` or `nolong`) to omit unwanted glossary styles. With `glossaries-extra`, you may find it easier to use `nostyles` and `stylemods` to only load the style packages that you specifically need, as in:

  ```
  \usepackage[nostyles,stylemods={tree,topic},
    style=tree]{glossaries-extra}
  ```

References

[1] S. Rahtz, H. Oberdiek, The LaTeX3 Project. The `hyperref` package, 2024. `ctan.org/pkg/hyperref`.

[2] N. Talbot. Glossaries with `bib2gls`. *TUGboat* 40(1):47–60, 2019. `tug.org/TUGboat/tb40-1/tb124talbot-bib2gls.pdf`

[3] N. Talbot. `bib2gls`: selection, cross-references and locations. *TUGboat* 41(3):308–317, 2020. `doi.org/10.47397/tb/41-3/tb129talbot-bib2gls-more`

[4] N. Talbot. `bib2gls`: sorting. *TUGboat* 42(2):194–203, 2021. `doi.org/10.47397/tb/42-2/tb131talbot-sorting`

[5] N. Talbot. `bib2gls`: Standalone entries and repeated lists (a little book of poisons). *TUGboat* 43(1):44–58, 2022. `doi.org/10.47397/tb/43-1/tb133talbot-bib2gls-reorder`

[6] N. Talbot. The `mfirstuc` package, 2022. `ctan.org/pkg/mfirstuc`.

[7] N. Talbot. Dickimaw Books gallery: All predefined glossary styles, 2023. `dickimaw-books.com/gallery/glossaries-styles`.

[8] N. Talbot. Incorporating makeglossaries or makeglossaries-lite or bib2gls into the document build, 2023. `dickimaw-books.com/latex/buildglossaries`.

[9] N. Talbot. The `datatool-english` bundle, 2025. `ctan.org/pkg/datatool-english`.

[10] N. Talbot. The `datatool` package, 2025. `ctan.org/pkg/datatool`.

[11] N. Talbot. The `glossaries-extra` package, 2025. `ctan.org/pkg/glossaries-extra`.

[12] N. Talbot. The `glossaries` package, 2025. `ctan.org/pkg/glossaries`.

◇ Nicola Talbot
`dickimaw-books.com`

LaTeX News

Issue 40, November 2024 (LaTeX release 2024-11-01)

Contents

Thirty years of LaTeX 2ε 1

News from the "LaTeX Tagged PDF" project 2
- Engine support: An important update 2
- Tagging support for external packages 2
- Improved table tagging 2
- Automatic MathML tagging 2
- Change behavior of tagging sockets with two arguments 2

Changes to the LaTeX kernel 3
- Handling paragraph continuation 3
- Avoid bogus "no item" error 3
- Switch to T1 as default encoding in documents using `\DocumentMetadata` . . 3

Code improvements 3
- Avoiding key–value option clashes between classes and packages 3
- Improvement to XeTeX `\showhyphens` 3
- Improved error raised for empty hook name . . 4
- Provide counter representations for link targets 4
- Extending `\refstepcounter` 4

Bug fixes 4
- Fix wrong file type in a rollback warning 4
- Handling of global keys with spaces 4
- File list entries for rolled back packages/classes 4
- doc: `\PrintDescribeMacro` in preamble 4
- Avoid low-level error if `\ShowHooks` is used late 4
- Avoid code duplication in rollback 4
- Passing template keys using `\KeyValue` 5

Changes to packages in the amsmath category 5
- Extend support for `\dots` 5

Changes to packages in the tools category 5
- Modification to generation of the `.tex` from fileerr 5
- array: Improve `>{...}` specifier 5
- array: Tagging support for `\cline` 5
- longtable: Extend caption type 5
- longtable: Prevent `\pagegoal` exceeding maximum value 5

Changes to l3build 5

Thirty years of LaTeX 2ε

In summer 1994, i.e., thirty years ago, LaTeX 2ε saw its first public release. Back then it was meant to be an intermediate version (hence the ε) on the way to a major new version (the mythical LaTeX3) that we expected to take a couple of more years to reach maturity. It took much more than that in the end—nominally, LaTeX 2ε is still with us today.

However, under the hood, LaTeX 2ε changed a lot throughout these thirty years, as one can see, for example, when looking through the forty newsletters [2] that accompanied the LaTeX releases that happened in the meantime.

During the first two decades, the LaTeX kernel was kept largely stable with only minimal bug fix activities. During that period additional functionality was mostly provided through new or extended packages that could be loaded in the document preamble. This included many of the ideas targeted for LaTeX3, e.g., expl3 (LaTeX3 programming language), xparse (new document command interface), xtemplate (a configuration mechanism), and many others.

Initially, this approach worked well and provided good backward compatibility; however, over time it became apparent that keeping all developments confined to packages was more and more problematical. Features or bug fixes that should have been generally available, i.e., part of the kernel, were only available in packages, so a lot of dependencies between packages were introduced and resulted in convoluted code that was difficult to manage. For example, hyperref had to rewrite a lot of kernel (and package) macros, so the code and behavior of other packages had to change depending on whether or not hyperref was loaded or not.

Thus, in 2015 the LaTeX team decided to change the policy and (re)start active kernel development, see [3]. To ensure continuous backward compatibility we introduced at the same time the latexrelease package that enables users to roll back changes to the LaTeX kernel to an earlier release, in case this is necessary to successfully rerun a document produced at that time.

As a consequence of this policy change the last decade saw a larger number of enhancements and corrections that were made part of the LaTeX kernel. Overall, we can confidently say that the new approach has worked well and enabled us to modernize LaTeX and ensure that it remains relevant without compromising one of the cornerstones of LaTeX: its outstanding ability to reprocess old documents written many years ago.

doi.org/10.47397/tb/46-1/tb142ltnews40

Being able to update and modernize the kernel sources allowed us to embark in 2019 on the multi-year "LaTeX Tagged PDF" project with the goal of automatically providing accessible PDF documents with LaTeX. While there are several more project phases to complete, the milestones already reached allow users to generate PDF/UA compliant documents if the input is restricted to a (growing) subset of packages and document classes; see next section and previous newsletters.

A big change happened with the 2020-02-02 release as part of the project activity, albeit somewhat obfuscated by us as "Improved load-times for expl3". While technically correct, what it really meant is that we had finally integrated the programming layer of LaTeX3, i.e., the ideas originally sketched out around 1992. Or saying it differently: with that date the original ideas for LaTeX3 became a reality as part of the standard LaTeX kernel.

With the programming layer available under the hood we were then able to provide new concepts and extensions as part of LaTeX, e.g., the hook management system, a new mark mechanism, core functionality for tagging and PDF resource management, a consistent key/value interface, and more recently the socket and plug mechanism.

More will follow while we continue to work on modernizing LaTeX and bringing the Tagged PDF project to a truly successful completion—so stay tuned and watch this space for future announcements in the next newsletters.

News from the "LaTeX Tagged PDF" project

Engine support: An important update

As detailed below, work is progressing on the Tagged PDF project. There are many drivers for this work, including legal changes in many places which will increasingly require well-tagged PDFs including full support for mathematics. As part of the work on this, we are looking at the technical abilities of the TeX engines.

With XeTeX, it is impossible to reliably produce tagged PDFs due to engine limitations. The increasing importance of tagged PDFs means that this requires a move away from XeTeX. We will continue to address issues with XeTeX support in team-maintained LaTeX code on a best-effort basis. No *new* functionality will be added for XeTeX by the LaTeX team. It is likely that over time functionality may become more restricted, and users are urged to migrate XeTeX documents to LuaTeX.

For pdfTeX, tagging is available and we are able to support mathematics by including relevant TeX source or by using externally-generated MathML. Only LuaTeX is capable of *automatic* generation of MathML as part of a LaTeX run. Thus pdfTeX continues to be supported for existing material, but for new documents, moving to LuaTeX is recommended.

We cannot make statements about the support for other engines such as (u)pTeX, as we don't use these programs nor have in depth knowledge of their functionalities. To the best of our knowledge, core LaTeX works well with these engines, but if and to what extent tagging can be supported will remain to be seen. If relevant information becomes available to us we will provide an update in future editions of the LaTeX newsletter.

Tagging support for external packages

At `https://latex3.github.io/tagging-project/tagging-status/` we show the status of many LaTeX packages and classes with respect to PDF tagging. We also started to improve tagging support in external packages. If the `firstaid` key is used in addition to the `phase-III` key, basic commands of several packages, including amsthm and fancyvrb, can now be used.

Improved table tagging

The tagging of tabulars has been extended: it is now possible to tag row headers and to create cells that span more than one row.

The interface to this functionality is not finalized but can be accessed in the current release by specifying the row and columns to be treated as headers. For example

```
\tagpdfsetup{
 table/header-rows={1,2},
 table/header-columns={1} }
```

would specify that in the following tables the first two rows and first column of each row should be tagged as heading entries.

Similarly you may add a RowSpan attribute to tag a cell that spans two rows using:

```
\tagpdfsetup{table/multirow={2}}
```

Automatic MathML tagging

When LuaLaTeX is being used, and the luamml package is available, and if the document uses the unicode-math package, then the math module will automatically convert each math formula to MathML and use it to attach MathML associated files (or MathML Structure elements) to the tagged PDF. This new feature can be disabled with `\tagpdfsetup{math/mathml/luamml/load=false}`. More options to configure MathML tagging can be found in the documentation of latex-lab-math.

Change behavior of tagging sockets with two arguments

When calling tagging sockets with two arguments using `\UseTaggingSocket` when tagging is suspended, previous versions of LaTeX 2_ε dropped both arguments. This behavior has been changed to drop the first argument and preserve the second one instead, thereby allowing tagging sockets to be used to wrap existing content which should still appear in a non-tagging context.

Since no tagging sockets currently provided by LaTeX use two arguments we do not expect this change to affect any existing documents, but if a custom tagging socket has been defined outside of the kernel it might need to be adapted to be compatible with the new behavior.
(github issue 1500)

Changes to the LaTeX kernel

Handling paragraph continuation

Already LaTeX 2.09 offered some automation to detect whether or not text after a list or some other display environment is meant to be a continuation of the current paragraph or should start a new one. The document-level syntax for this is that a blank line after such an environment signals to LaTeX that it should start a new paragraph; whilst no blank line signals that there should be no new paragraph and the text should be considered a continuation.

Unfortunately, there were a number of cases where the original 2.09 approach failed, e.g., with

```
{⟨local customizations⟩
\begin{equation} a<b \end{equation}}
⟨some text⟩
```

the ⟨*some text*⟩ incorrectly started a new paragraph. Bug reports about this behavior can be traced back to the time LaTeX 2ε was developed, e.g., one test file from 1992 has a note that the above case was unfortunately not resolvable despite some improvements made back then. The main cause of the issue (as you probably guessed) is that the mechanism failed whenever the environment was executed within a group (`{...}`, `\begingroup`/`\endgroup`, or `\bgroup`/`\egroup` pair) that was closed before the next blank line was reached.

While most of the time this could be visually corrected by adding some explicit `\noindent`, the situation got worse when we tried to implement tagged PDFs resulting in incorrect structures or worse.

We therefore made a new attempt to resolve this problem in every situation and this new solution is rolled out in the current release.

Avoid bogus "no item" error

The commands `\addvspace` and `\addpenalty` generated the famous error message "Something's wrong—perhaps a missing `\item`" when they were encountered outside vertical mode. Most of the time this error was bogus and if not, then it was generated several times rather than once.

Once upon a time (in LaTeX 2.09) it was necessary that these commands were used only in vertical mode, but with LaTeX 2ε in 1994, we changed the internals but simply overlooked that this error message then had become useless. In this release, i.e., 30 years too late, we have finally lifted the ban and from now on this error should only show up if there is indeed a missing `\item`.
(github issue 1460)

Switch to T1 as default encoding in documents using `\DocumentMetadata`

As it is well known, the font encoding `OT1` supports only 128 characters and has various problems and quirks notably for languages different to English. Nevertheless `OT1` is the default encoding in LaTeX and this cannot be easily changed without affecting many documents as the `T1` version of the fonts have slightly different metrics.

The introduction of the `\DocumentMetadata` command, which announces *new* code and changes that can also affect the layout gives us now the opportunity to make this step. So with this version a use of `\DocumentMetadata` with (pdf)LaTeX will setup `T1` as default font encoding.[1] To ensure that scalable fonts are used, the package `cm-super` has to be installed. Users who want to revert to the `OT1` encoding in their document can do so with `\usepackage[OT1]{fontenc}`.

Code improvements

Avoiding key–value option clashes between classes and packages

In LaTeX News 35 [5] we introduced key–value option processing to the kernel. Following the standard for LaTeX 2ε options, keyval options given to the `\documentclass` line were treated as global and so parsed by every package. However, with keyvals, the likelihood of a name clash between a class-specific option and one used by a package is much higher than it is with simple strings. We have therefore refined the mechanism in the current release.

When a class uses the kernel keyval processor, any options it recognizes are recorded and any packages using the keyval processor will then *skip* these "global" options. To allow for the case where a class directly uses an option which should be global (for example `draft`), a new key property `.pass-to-packages` has been added. This can then be set to indicate that this key is not to be skipped. For example

```
\DeclareKeys{
  draft .if = {ifl@cls@draft},
  draft .pass-to-packages = true,
  mode  .store = \cls@mode
}
```

in a class would create two options, `draft` and `mode`. The `draft` option will be treated in the normal way by packages using keyvals, but they will ignore the `mode` option: it is effectively marked as "private" to the class.
(github issue 1279)

Improvement to XeTeX `\showhyphens`

When using `\showhyphens` with XeTeX, missing character warnings would be generated for any character not in Latin Modern. This has been corrected and the warnings are suppressed. *(github issue 1380)*

[1] The Unicode engines will continue to use `TU` as the encoding.

Improved error raised for empty hook name
When using the hook management, both hook and label names (if specified) should be non-empty. Before, empty hook and empty label names both raised the same label-specific error:

```
! LaTeX hooks Error: Empty code label on line ..
  Using 'top-level' instead.
```

This has now been improved. Now an empty hook name generates

```
! LaTeX hooks Error: Empty hook name on line ..
```

(github issue 1423)

Provide counter representations for link targets
To create unique target names for links the package hyperref uses a special counter representation `\theH`⟨*counter*⟩. To ensure that this counter representation exists, hyperref redefined the commands `\@definecounter`, `\@addtoreset` and `\refstepcounter`. This counter representation is also needed for the Tagged PDF project and so these augmented command definitions have now been incorporated into the kernel. Thus from now on every `\newcounter{`⟨*counter*⟩`}` will define not only `\the`⟨*counter*⟩ but also `\theH`⟨*counter*⟩.

Extending `\refstepcounter`
For many years, the package hyperref had been redefining `\refstepcounter` and adding code that creates link targets. The kernel definition has now been extended with socket interfaces that will allow hyperref to avoid the redefinitions. The new interfaces are also used by the Tagged PDF code that needs target names to resolve references between structures.

Bug fixes

Fix wrong file type in a rollback warning
When LaTeX is rolled back to date ⟨*date1*⟩ and a class or package with minimum date requirement ⟨*date2*⟩ is to be loaded, a rollback warning is raised if ⟨*date2*⟩ is later than ⟨*date1*⟩:

```
LaTeX Warning: Suspicious rollback/min-date
               date given.
  A minimal date of YYYY-MM-DD has been
  specified for package '<pkgname>'.
  But this is in conflict with a rollback
  request to YYYY-MM-DD.
```

In some cases this message showed a wrong file type, i.e., `document class '<pkgname>'` or `package '<clsname>'`. This has now been corrected.
(github issue 870)

Fix existence check of document environments
`\NewDocumentEnvironment` and friends define (or redefine) a document environment using the space-trimmed ⟨*envname*⟩, but the existence check for ⟨*envname*⟩ was done without space trimming. Thus when the user-specified ⟨*envname*⟩ consists of leading and/or trailing space(s), it may lead to erroneously silent environment declaration. For example, in

```
\NewDocumentEnvironment{myenv}{}{begin}{end}
\NewDocumentEnvironment{␣myenv␣}{}{begin}{end}
```

the first line defines a new environment `myenv` but the second line would check existence for `␣myenv␣` (which is not yet defined), then redefine `myenv` environment without raising any errors. This has now been corrected.
(github issue 1399)

Handling of global keys with spaces
If the global (class) options contained spaces around key names, `\ProcessKeyOptions` would fail to remove known keys from the list of unused global options and `\OptionNotUsed` would mistakenly add space-surrounded key names to that list. The first issue was corrected as a hotfix in patch level 1 of the November 2023 release (but unfortunately not mentioned in [6]) and the second in the current release. *(github issue 1238)*

File list entries for rolled back packages/classes
When the rollback mechanism for packages and classes was introduced in 2018 [4], loading of the selected historic release was not recorded in the file list used by `\listfiles`. This has now been corrected so that the extended usage [7]

```
\listfiles[hashes,sizes]
```

now gives more complete and less confusing info.
(github issue 1413)

doc: `\PrintDescribeMacro` in preamble
In doc version 2 it was possible to alter the definition of `\PrintDescribeMacro` and similar commands in preamble. In version 3 this stopped working because they were reset at the end of the preamble. This has now been implemented differently and changes in the preamble are possible again. *(github issue 1000)*

Avoid low-level error if `\ShowHooks` is used late
If `\ShowHooks` was used to examine a package hook after the package was loaded, a low level error resulted. This has now been corrected. *(github issue 1513)*

Avoid code duplication in rollback
When the kernel uses `\AddToHook` in a region that might be rolled back (which happens in a few places) and a document requests a rollback, then we have the situation that the hook already contains code to which we added the same (or slightly different) code during the rollback; this results in code duplication or, worse, in errors. This has now been corrected by dropping any such code chunk (if there is one) prior to adding the rollback code.
(github issue 1407)

Passing template keys using `\KeyValue`

With the move of the template code to the kernel, internal functions were reviewed to improve efficiency. However, there was an oversight in how passing key values from one setting to another was implemented, such that using `\KeyValue` could result in an infinite loop. This has now been fixed. *(github issue 1486)*

Changes to packages in the amsmath category

Extend support for `\dots`

The implementation of `\dots` in amsmath has the feature that it selects different dots depending on the symbol that follows: e.g., dots between commas would normally be on the baseline, while dots between binary or relational symbols would be raised. However, when symbols such as `\cong` were protected from expansion in moving arguments (so that they worked in places such as headings) it had the unfortunate side-effect that the `\dots` magic stopped working for them. This has now been corrected. *(github issue 1265)*

Changes to packages in the tools category

Modification to generation of the `.tex` *from fileerr*

The fileerr extraction has been modified to write `rename-to-empty-base.tex` rather than `.tex` to comply with an expected security change in TEX Live 2025. The `build.lua` file for the tools has been modified to rename `rename-to-empty-base.tex` to `.tex` after unpacking. However if using docstrip directly rather than using l3build or the unpacked zip file from CTAN, the user must now rename the file and install as `.tex`. *(github issue 1412)*

array: Improve `>{...}` *specifier*

If the argument of `>{...}` ended with a command accepting a trailing optional argument, e.g., defined for example with `\NewDocumentCommand\foo{o}{...}`, one could get low-level parsing errors. This has now been corrected. *(github issue 1468)*

array: Tagging support for `\cline`

In the last release we added tagging support for array, longtable and other tabular packages, but we overlooked that the kernel definition for `\cline` also needs modification because the rule generated by the command needs to be tagged as an artifact. Furthermore, the processing of a `\cline` looks to the algorithm as if another row is added (which is technically what happens), thus it was also necessary to decrement the internal row counter to get a correct row count. This has now been corrected in array, which is automatically loaded for tagging, so that all these packages are now fully compatible with the tagging code if it is turned on. *(github tagging issue 134)*

longtable: Extend caption type

The longtable package has been extended and now provides the command `\LTcaptype` (stemming from the ltcaption package) to change the counter and caption type used by the `\caption` command from longtable. So with `\renewcommand\LTcaptype{figure}`, a longtable will step the figure counter instead of the table counter and produce an entry in the list of figures. An empty definition, `\renewcommand\LTcaptype{}`, will suppress increasing of the counter. This makes it easy to define an unnumbered variant of longtable:

```
\newenvironment{longtable*}
  {\renewcommand\LTcaptype{}\longtable}
  {\endlongtable}
```

longtable: Prevent `\pagegoal` *exceeding maximum value*

An internal guard has been added to avoid TEX errors if `\pagegoal` is increased beyond the maximum value for a TEX dimension. *(github issue 1495)*

Changes to l3build

To support third-party developers testing their code against pre-release LATEX, a new switch `--dev` has been added to l3build. This allows the developer to run

```
l3build check
```

to run their test suite against the current release of LATEX and

```
l3build check --dev
```

to run exactly the same tests using the development release of LATEX.

References

[1] Leslie Lamport. *LATEX: A Document Preparation System: User's Guide and Reference Manual*. Addison-Wesley, Reading, MA, USA, 2nd edition, 1994. ISBN 0-201-52983-1. Reprinted with corrections in 1996.

[2] LATEX Project Team. *LATEX 2ε news 1–39*. June 2024. `https://latex-project.org/news/latex2e-news/ltnews.pdf`

[3] LATEX Project Team. *LATEX 2ε news 22*. January 2015. `https://latex-project.org/news/latex2e-news/ltnews22.pdf`

[4] LATEX Project Team. *LATEX 2ε news 28*. April 2018. `https://latex-project.org/news/latex2e-news/ltnews28.pdf`

[5] LATEX Project Team. *LATEX 2ε news 35*. June 2022. `https://latex-project.org/news/latex2e-news/ltnews35.pdf`

[6] LATEX Project Team. *LATEX 2ε news 38*. November 2023. `https://latex-project.org/news/latex2e-news/ltnews38.pdf`

[7] LATEX Project Team. *LATEX 2ε news 39*. June 2024. `https://latex-project.org/news/latex2e-news/ltnews39.pdf`

Experiences in typesetting a complex book: Consistency, graphics, languages, notes

Alan J. Cain

Abstract

This article discusses various aspects of typesetting the author's book *Form & Number: A History of Mathematical Beauty* and some of the techniques that were adopted and which have, in certain cases, formed the bases of new packages. Topics discussed include: the creation of two-page spreads showing tilings for part divisions; notes and other content in the margins; ensuring consistency in graphics; font modifications that were necessary because of unforeseen combinations involving accented letters; indexing; running heads; the build process.

1 Introduction

The book *Form & Number: A History of Mathematical Beauty* [2] was written and typeset using LuaLaTeX by the present author. This article offers a survey of some features of its typography and solutions to some of the challenges that arose. The sections that follow contain an eclectic mix of high-level discussions of strategy and low-level discussions of code. The focus is on features that appear or are relevant to a noteworthy part of the book, and thus seem more likely to be useful to others, rather than more specialized or localized tricks.

Form & Number (= *F&N*) is available on the Internet Archive[1] under a Creative Commons licence, so readers can see the effects of the typographical decisions and techniques discussed here.

Historical note. The style for *F&N* evolved from a set of lecture notes on semigroups the author produced in 2012 [1]. The author used a refined version of the style for an annotated edition, with commentary, of G.H. Hardy's *A Mathematician's Apology* [3], released in 2019. The annotated *Apology* grew out of the writing of *F&N*, and served as a test-bed for some of its typography.

2 Part divisions

The chapters of *F&N*, save for the preliminary chapter 0, are grouped into four parts. Each part division is marked by a two-page spread in which the verso page and part of the recto page are occupied by a Cairo tiling, which is made up of congruent pentagonal tiles in four orientations (see Figure 1). Each tile contains a different image that alludes to some topic discussed in the part that follows.

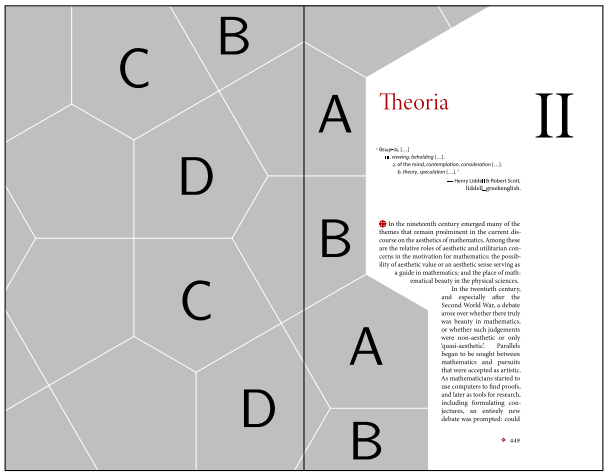


Figure 1: An illustration of the two-page spread used for part divisions in *Form & Number* and the Cairo tiling that occupies the verso page and part of the recto page. In the book, each tile shows a different graphic that relates to something in the part.

If the PDF is viewed in two-page mode with the title page centred, each two-page spread is shown correctly. This holds true in both the print version, which has a two-sided layout, and the two ebook formats, which have a one-sided layout. To ensure each spread begins on the correct page, the following macro is executed at the start of the part:

```
\cs_new:Npn\partcleartoverso
  {
    \clearpage
    \int_if_odd:nT{ \c@page }
      { \partrectofiller\newpage }
  }
```

This macro is a modification of the LaTeX command `\cleardoublepage`. It lacks an `\if@twoside` check, since the spread should begin on a 'verso' page even in `oneside` mode, and, instead of inserting a blank page, it executes `\partrectofiller`, which draws an empty skeleton of the Cairo tiling. A similar check ensures that the main matter begins on a right-hand page even in `oneside` mode, so that odd-numbered pages are 'recto' and even-numbered pages are 'verso'.

For each part, the tiling is drawn within two Ti*k*Z pictures, one on the verso page, one on the recto. At the start of the part, outside either picture, the content for each tile is specified and saved using the following macro:

[1] Ebook, large format: `archive.org/details/cain_formandnumber_ebook_large`
Ebook, small format: `archive.org/details/cain_formandnumber_ebook_small`
Version for printing: `archive.org/details/cain_formandnumber_print`

doi.org/10.47397/tb/46-1/tb142cain-setting-form

```
\cs_new:Npn \partdefinetilecontent #1#2#3
  {
    \cs_set:cpn { parttilecontent-#1-#2 }{ #3 }
  }
```

Parameters #1 and #2 are the 'column' and 'row' numbers in the tiling. (In Figure 1, BABAB and CDCD are 'columns'.) Parameter #3 is the content of each tile and is typically a fragment of Ti*k*Z code that loads an external image into a node (usually placed at (0,0)), sometimes after drawing a background. Only this content definition varies between parts. The rest of the drawing procedure is encapsulated in two macros (one for the verso page, one for the recto page) called for each of the four parts.

Inside both the verso and recto pictures, Ti*k*Z coordinates parttile-⟨*column*⟩-⟨*row*⟩-center are defined for the centres of the tiles. The positions of these coordinates are easily calculated from the current page node and the 'column' and 'row' numbers of the tile. The only difference is that a Ti*k*Z transformation xshift=\paperwidth is applied before defining the coordinates for the verso page. This xshift ensures alignment of the tiles on the two pages. Then, within each picture, for each tile that appears at least partially on that page, the following macro is called with #1 and #2 being the 'column' and 'row' number of that tile. The macro applies a shift transformation so that (0,0) is at the centre of the tile, clips to the correct shape, and executes the previously saved Ti*k*Z code that draws the content:

```
\cs_new:Npn\partdrawtilecontent #1#2
  {
    \scope[
      shift={(parttile-#1-#2-center)}
    ]
      \pgfmathtruncatemacro{\r}{
        Mod(-90*Mod(#1,2)+90*Mod(#2,2)
        +(180*Mod(#1,2)*Mod(#2,2)),360)
      }
      \use:c{ partcliptile-\r }
      \use:c{ parttilecontent-#1-#2 }
    \endscope
  }
```

The slightly intimidating \pgfmathtruncatemacro expression calculates the appropriate rotation of the tile from its 'column' number #1 and 'row' number #2 and sets \r to this value. The possible rotations 0°, 90°, 180°, 270° correspond respectively to tiles A, B, C, D in Figure 1.

There are four different clipping paths, executed by \use:c{ partcliptile-\r }, corresponding to the four rotations. Each of these macros expands to a \clip statement in Ti*k*Z syntax. (It is not possible to use Ti*k*Z's save path/use path here, since such saved paths are unaffected by transformations.) An earlier version of the tile-drawing code applied the rotation \r to a single tile outline path. But while this approach defines a mathematically correct tiling, in the rendered PDF a line of disconnected pixels where the renderer had not drawn either tile was sometimes visible. Thus the four clipping paths are designed to produce overlaps (see Figure 2) and are drawn according to the following rules: columns of tiles are drawn right-to-left (so the column BABAB in Figure 1 is drawn before CDCD), and, within columns, tiles A are drawn before tiles B, and tiles D are drawn before tiles C.

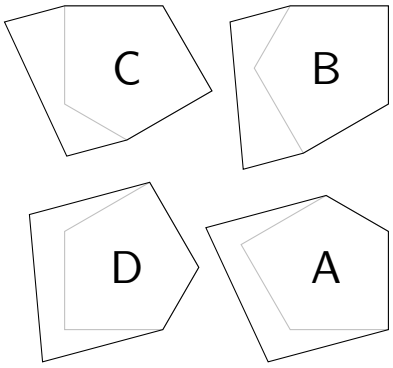


Figure 2: The four clipping paths used for tiles in part divisions.

A similar process of defining and later clipping and drawing tiles was used to create the cover, which is made up of images in the tiles of a Penrose aperiodic tiling. For the cover, the tiles have overlaps extending 'inward' towards the tile containing the title, and are drawn from outside to inside, with the tile containing the title being drawn last.

The tiles in both the part division spreads and the cover are sized so that they fit neatly into the page geometry. In the part spreads, the top and bottom edges of the page align with horizontal edges of tiles in the column CDCD. On the cover, the top and bottom edges of the page pass through vertices of the Penrose tiling. Since the small format ebook has a different page geometry from the large format, the tiles are different sizes in the two variants. Thus all the external images are scaled when they are loaded: the scaling is 1 in the large format, and is the ratio of the different page heights in the small format.

3 Sidenotes and other marginal content

F&N has a populous outer margin. The author wished to use the margin for: 1) citations and short explanatory notes; 2) small figures; 3) captions for floats; 4) full-width floats that extend into the margin; 5) running heads. Naturally, marginal content

items should not overlap, which necessitates adjustment of their vertical positions, and the amount of marginal content meant that *manual* adjustment was impractical. But automatic adjustment had to take into account the type of marginal content. For instance, a float caption should *not* move from its natural position beside the float. The running head should similarly not move from its specified position, but could occasionally be suppressed to make way for a large float caption or a full-width float.

None of the existing methods or packages for placing marginal content was entirely satisfactory, and thus a new solution was developed and has been incorporated into the (LuaLaTeX-only) package `marginalia`, which is available on CTAN.[2] An introduction to `marginalia` is the subject of a separate article.[3]

Besides the technical solution required to place marginal content, it is worth noting a detail of the font used for citations and notes in the margin. Although the main text font, Minion 3, has a 'caption' optical size (designed for text smaller than 9.9 pt), the usual 'text' optical size (designed for 9.9 pt to 13.9 pt text) is used for marginal citations and notes. This gives them a lighter appearance and cleanly separates them from the main text.

One other note-related issue concerns the command `\textsuperscript`, which is used to set footnote and sidenote marks. This command passes its parameter, prefixed with `\selectfont`, to the `\@textsuperscript` macro, which uses normal math mode superscripting to set the text inside an `\mbox`. But after a superscript or subscript, TeX inserts an additional space (by default 0.5 pt) [5, pp. 445–6]. This extra space is certainly important in math mode: compare x^2y (which has the default extra space) and x^2y (which has none). But a superscripted note mark usually appears at the end of a clause or sentence, and virtually always precedes a space or appears at the end of a line. Thus, at least for note marks, the extra space seems superfluous. And when a superscripted note mark appears at the end of a line of justified text, there is a visible indentation in the text (see the first paragraph in Figure 3).

The extra space is specified in TeX by the dimension parameter `\scriptspace` and in LuaTeX on a per-style basis via `\Umathspaceafterscript`⟨*style*⟩. In *F&N*, `\@textsuperscript` is redefined so that `\Umathspaceafterscript\textstyle` is locally set to 0 pt:

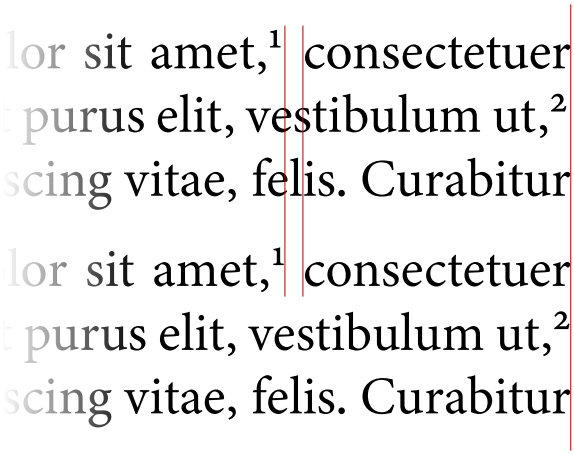


Figure 3: Text superscripts set using the standard `\@textsuperscript` (first paragraph) compared with those set using the modified version in *Form & Number* (second paragraph). [10 pt Minion 3, `microtype` protrusion and expansion disabled, magnified 2×.]

```
\cs_set_nopar:Npn\@textsuperscript #1
  {
    \group_begin:
    \Umathspaceafterscript\textstyle=0pt
    \m@th\ensuremath{
      ^{\mbox{\fontsize\sf@size\sf@size #1}}
    }
    \group_end:
  }
```

The effect is shown in Figure 3. With the modified definition, used in the second paragraph, the right side of the text seems more even. The space after the superscript in the middle of the line is also narrower, which does not seem to be a problem, at least for this font. One could append a suitable `\hskip` to the new definition of `\@textsuperscript` to restore the original mid-line spacing, since the extra space thus inserted would vanish at a line break. But, in this case, certain other definitions have to be adjusted: for instance, with the usual definition of `\footnote` (which the `\sidenote` command used in *F&N* follows), the extra space would occur between the note mark and the setting of the note, which, if a line break occurred there, could push the note to a new line and thus even to a different page. This problem can be prevented by redefining `\footnote` (and *F&N*'s `\sidenote`) to increment the counter, set the note, and set the mark last.

4 Graphics

F&N contains many diagrams, which are mostly drawn using Ti*k*Z (including `tkz-euclide` for geometrical diagrams and `pgfplots` for plots of functions) and Asymptote. Generally, Ti*k*Z is used for

[2] `ctan.org/pkg/marginalia`

[3] "`marginalia` at work", present issue, pp. 49–53

two-dimensional diagrams and Asymptote for three-dimensional diagrams. But it is important that diagrams should have a consistent appearance regardless of how they are produced.

For example, using `tkz-euclide` one draws a dot with `\tkzDrawPoint`, which creates a Ti*k*Z node that can have a different stroke and fill colour, either specified locally or set using `\tkzSetUpPoint`. Asymptote supplies the `dot` command, which simply draws a path of length 0 at the specified location. To ensure consistency, the following Asymptote command is used to mimic the effect of `\tkzDrawPoint`:

```
void draw_dot(
  triple c,
  real size=3pt,
  pen drawcolor=black,
  pen fillcolor=gray
) {
  dot(c,linewidth(size+.4pt)+drawcolor);
  dot(c,linewidth(size-.4pt)+fillcolor);
}
```

(The default `size=3pt` corresponds to the default `tkz-euclide` dot size in *F&N*.) For illustration, one of these two dots is drawn using `\tkzDrawPoint`, the other using `draw_dot`: .

A more complicated consistency problem relates to plots. Two-dimensional plots are drawn using `pgfplots`, with a style where the axes are separated from the grid on which the values are plotted. Asymptote supplies the `graph3` module, but the style of the axes is very different. (It may be possible to produce axes similar to those created using `pgfplots` using `graph3`, but the author has insufficient understanding of this module.) Thus completely new code, too long to include in full here, is used to draw the background grid and the axes. For aesthetic reasons, the axes are drawn as if they lie flat on one of the *xy*-, *xz*-, or *yz*-planes (see the example in Figure 4). The effect of laying the axis labels flat is a straightforward application of transforming an Asymptote `Label` object. For example, the axis label '*z*' is drawn by code equivalent to the following, where `axislabelsep` is a dimension:

```
label(
  rotate(90,O,Z)
    *Label("\footnotesize\(z\)",p=gray),
  (5+axislabelsep,0,0)
);
```

(Note that the *z*-axis shown in Figure 4 is actually Asymptote's *y*-axis.) A different transformation is required for each axis.

In *F&N*, all diagrams — excepting those contained in images of historical documents — and all

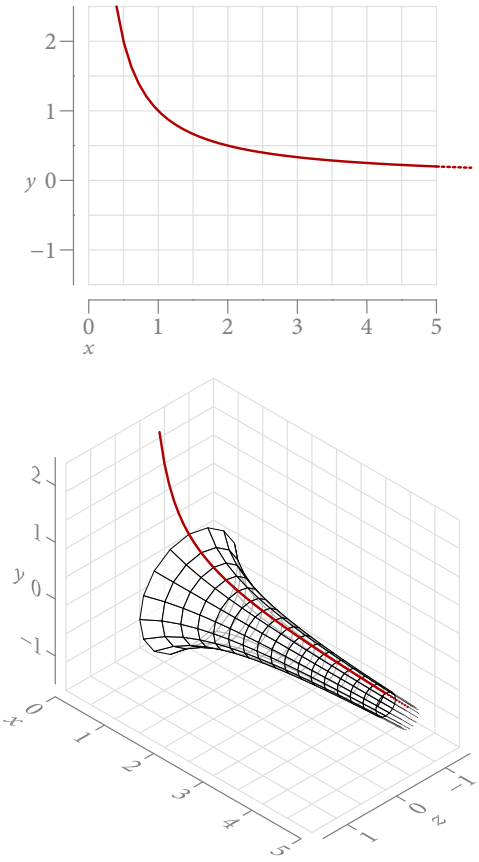


Figure 4: *Above:* The hyperbola $y = 1/x$, plotted using `pgfplots`. *Below:* Torricelli's solid, obtained by rotating the hyperbola about the line $y = z = 0$ and truncating at $x = 1$, drawn in Asymptote with axes similar to those produced by `pgfplots`.

plots other than renderings of fractals such as Mandelbrot or Julia sets are vector graphics.

Plots of surfaces are drawn as polyhedral approximations, as illustrated in Figure 4. The Asymptote code for these surfaces, and for polyhedra in general, uses the same strategy as the `polyhedron_js` module:[4] the polygonal faces are sorted according to distance from camera and then drawn furthest-to-nearest. (This is adequate for polyhedra and the surfaces required for *F&N*, but not for arbitrary collections of polygons in space: even three triangles may be situated so that each is partially in front of another from the camera's perspective.) But the code is entirely new, and encapsulates each face in an Asymptote `struct` that also carries colour and opacity information. This feature is important because in certain diagrams individual faces are coloured differently.

[4] There seems to be no official site for this module, originally written by Jens Schwaiger, but it is available from various sites, e.g. `gmaths.free.fr/asymptote/asy/polyhedron_js.asy`

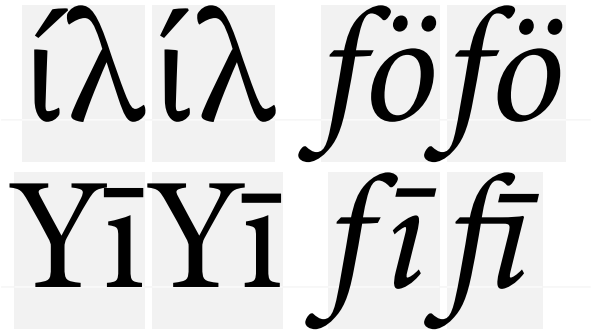


Figure 5: Comparison of original glyphs from Minion 3 (left) with the modified versions used in *Form & Number* (right). [10 pt glyphs magnified 6×.]

5 Accentuation

The text of *F&N* includes many words and names that include accented letters, including transliterations of words and names originally written in non-Latin scripts, notably Greek and Arabic. The main font is Minion 3, which has extensive coverage of accented letters in Latin, Greek, and other alphabets. But some combinations of letters involving accents are problematic, and it was necessary to create and use some modified glyphs.

For instance, when set in Minion 3, the combination ΐλ (that is, an iota with an *oxeîa* (acute) accent followed by a lambda) has the problem that the accent on the iota almost touches the lambda. A modified iota glyph with a shorter and steeper accent is used in this situation. (See Figure 5, top left.) There are numerous other such tweaks in both Latin and Greek, such as for Yī (Y followed by an i with a macron; Figure 5, bottom left), where the macron has been slightly lowered and shortened on the left. Some combinations require changes to both glyphs: for example, in Minion 3 Italic, in the combination fö (f followed by an o with a diaeresis) the diaeresis seems slightly too close to the arch of the f. Here, two modified glyphs are used: an f with a slightly shorter arch at the top, and an o with a slightly lowered diaeresis (Figure 5, top right).

Perhaps the most visible change was in the combination fī (f followed by i with a macron), which occurs frequently in transliteration of Arabic. Here the problem is the clear gap between the unmodified glyphs. Kerning them closer would bring the macron uncomfortably near the arch of the f. The solution was a new ligature, based on the usual fi ligature, with a shorter arch on the f and an i with a slightly lowered macron (Figure 5, bottom right). The new ligature is not a perfect solution, because it looks a little too 'dense' on the page, but is vastly better than the alternatives.

[It seems that few fonts contain any ligatures for an f followed by an accented i; one example is Matthieu Cortat's Louize, which has several such ligatures.[5]]

6 Date ranges

Like any historical work, *F&N* contains numerous dates and date ranges. For convenience and consistency, dates and ranges are all typeset via macros. Each date is specified (in as much detail as required) in ISO format `YYYY-MM-DD`, prefixed by `-` for a date BCE (which is a departure from the ISO standard, in which `-`⟨*year*⟩ signifies (⟨*year*⟩ + 1) BCE) and by `c` to indicate 'circa' when the date is uncertain. For instance, `\daterange{c-600}{-322}` typesets 'c. 600–322 BCE'. Note that only one era indicator BCE is necessary for this range. By contrast, `\daterange{-63}{14}` produces '63 BCE–14 CE'. For dates after 1000 CE, the era indicator is not produced.

`\daterange` also abbreviates intelligently. For example, `\daterange{1905}{1906}` yields 1905–6, but `\daterange{1914}{1918}` produces '1914–18' (*not* *1914–8), since (at least in English) a range within 10–19 should not be abbreviated. Similarly, centuries remain intact: `\daterange{1777}{1855}` produces 1777–1855 (*not* *1777–855). `\daterange` will not abbreviate ranges of dates BCE. (Using such abbreviations would create ambiguity: does *384–22 BCE end in 322 BCE or 22 BCE?) nor will it abbreviate if either the start or finish date is 'circa'.

The results produced by `\daterange` accord with style guides such as Oxford University Press's [original and *New*] *Hart's Rules* [4, 6]. The implementation is complex, because `\daterange` also handles (with suitable abbreviation) ranges that span only months or days.

7 Timeline

At the end of *F&N*, there is a multi-page timeline running from 500 BCE to 2000 CE, which shows the lifetimes of many of the most important mathematicians, philosophers, and others whose work is discussed in the book, plus some historical context. A much smaller timeline in a similar style is shown in Figure 6.

A set of macros was developed to make the creation and editing of the timeline in *F&N* as simple as possible. These macros form the basis of the package `timechart`, now available on CTAN.[6] The package is built on Ti*k*Z, but very little knowledge of Ti*k*Z is required to use it. The following is part of the code used to produce Figure 6:

[5] See the specimen at `www.205.tf/louize`

[6] `ctan.org/pkg/timechart`

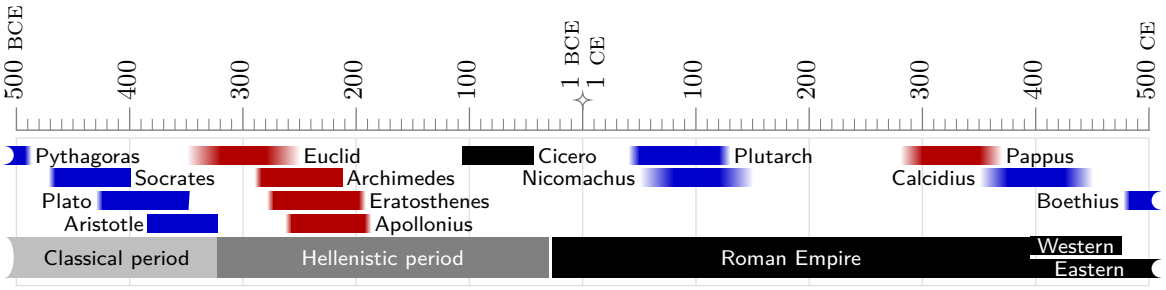


Figure 6: An example of a timeline similar to the one included in *Form & Number* (grayscaled for print).

```
\begin{timechart}[
    major tick interval=100,
    philosophy/.style={
      interval bar color=blue!80!black,
    },
      ...
  ]{-500}{500}
  \timechartinterval[philosophy,right]%
    {c-570}{c-490}{Pythagoras}
  \timechartinterval[philosophy,right]%
    {c-469}{-399}{Socrates}
      ...
\end{timechart}
```

The mandatory arguments of the enclosing environment `timechart` specify that the range of dates to be displayed is from 500 BCE to 500 CE. The optional argument is used to specify that dates at every 100 years should be marked, and to set up some Ti*k*Z styles for the intervals shown in the chart.

Within the environment, each interval (in this case, mostly lifetimes) can be specified by a single line of code (which was a design requirement). The `\timechartinterval` command takes three mandatory arguments: a start date, a finish date, and a label. Each of the dates may be prefixed with '`c`' for 'circa' to indicate uncertainty, or may be replaced by a range when definite bounds on the date are known. The `timechart` package takes care of indicating uncertainties by fading from or to transparency over a customizable length, and adds an indicator when an interval extends beyond the specified bounds of the chart.

In the lines shown, the optional argument to `\timechartinterval` has been used to apply the Ti*k*Z style `philosophy`, which is defined to specify the colour of the interval. It could also have been used, for example, to specify a different label font. The key `right` specifies the side on which to place the label for the interval.

Because the `timechart` package is built on Ti*k*Z, the user may add additional Ti*k*Z code freely, for example by adding arrows to indicate historical connections. (And, although `timechart` is written in `expl3`, the key–value configuration uses `pgfkeys` rather than `l3keys`.)

8 Image credits

F&N contains a number of images from other sources, which are either public domain or under a Creative Commons licence. Each image has an abbreviated credit alongside it, with a list of full credits and source URLs towards the end of the book. Rather than have the abbreviated and full credits separated in the source, all the credits are specified at the point the image is used (within the `figure` environment), and the full credit is written to a file `\jobname-credits.tex`, which is read back at the appropriate point.

Writing information to be read back later is a standard technique, but there is one main novelty: the automatic creation of a visual key for full credits when there are multiple images to credit within the same `figure`. This key shows where on the page each credited image appears. Figure 7 is an example of such a key and shows the positions of the two images that make up Figure 4. The rest of this section explains how this example was produced, using a slightly simplified version of the code in *F&N*.

Here is an abbreviated version of the code used for Figure 4:

```
\begin{tikzpicture}[...]
  \node (above) at (0,0)
    {\includegraphics{...}};
  \node[anchor=north] (below) at
    ($ (above.south) + (0,-5pt) $)
    {\includegraphics{...}};
  \figurekeymakenodearea{above}
  \figurekeymakenodearea{below}
\end{tikzpicture}
```

In *F&N*, the macro that places an abbreviated credit next to the image calls `\figurekeymakenodearea`; here it is called directly. `\figurekeymakenodearea`

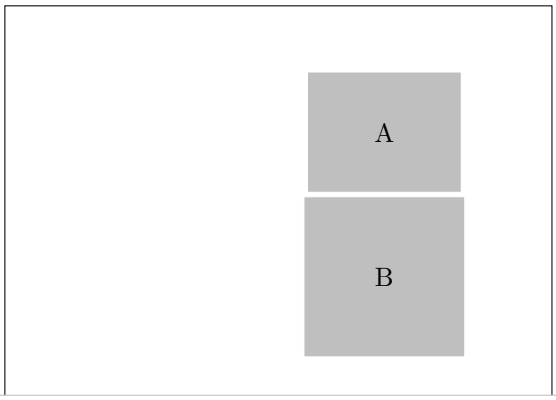


Figure 7: Example of the kind of visual key used for image credits in *Form & Number*, showing the locations on page 43 of the two images that make up Figure 4.

tells PGF/Ti*k*Z to remember the picture, extracts the coordinates of the south-west and north-east corners of the node passed as parameter `#1`, and then adds to a token list a macro `\figurekeyarearect` that takes as parameters the unique `\pgfpictureid` of the picture and the extracted coordinates:

```
\cs_new:Npn\figurekeymakenodearea #1
  {
    \pgfrememberpicturepositiononpagetrue
    \pgfextractx{\l_tmpa_dim}
      {\pgfpointanchor{#1}{south~west}}
    \pgfextracty{\l_tmpb_dim}
      {\pgfpointanchor{#1}{south~west}}
    \pgfextractx{\l_tmpc_dim}
      {\pgfpointanchor{#1}{north~east}}
    \pgfextracty{\l_tmpd_dim}
      {\pgfpointanchor{#1}{north~east}}
    \int_gincr:N\g__figurekey_areano_int
    \tl_gput_right:Ne
      \g__figurekey_content_tl
      {
        \exp_not:N\figurekeyarearect
        {\pgfpictureid}
        {
          \int_value:w
          \g__figurekey_areano_int
        }
        {\dim_use:N\l_tmpa_dim}
        {\dim_use:N\l_tmpb_dim}
        {\dim_use:N\l_tmpc_dim}
        {\dim_use:N\l_tmpd_dim}
      }
  }
```

In *F&N*, the token list is written to the external file along with the full credit data for all images within the `figure`, after which the global variables are cleared. Later, when the file is loaded, the full credit is typeset. The example here omits writing to the file and simply uses the token list directly. But the actual creation of the visual key is the same: the saved data, which is a sequence of commands of the form `\figurekeyarearect {`⟨*id*⟩`}{`*n*`}{`*x*`}{`*y*`}{`*x′*`}{`*y′*`}`, is executed within a *scaled* `tikzpicture`, which represents the page. The macro `\figurekeyarearect` is defined as

```
\cs_new:Npn\figurekeyarearect #1#2#3#4#5#6
  {
    \begin{pgfscope}
      \pgfsys@getposition{#1}
        {\figurekeyoriginpgfpoint}
      \pgftransformshift
        {\figurekeyoriginpgfpoint}
      \fill[keyarea] (#3,#4) rectangle (#5,#6);
      \path (#3,#4) --
        node {\int_to_Alph:n{#2}} (#5,#6);
    \end{pgfscope}
  }
```

`#1` is the `\pgfpictureid` of the initial picture, so the `\pgftransformshift` moves `(0,0)` to the (scaled) position representing the origin of the initial picture. Thus the subsequent commands draw a rectangle corresponding precisely to the location on the page of the node in the initial picture.

9 Indexing

F&N contains an index of subjects and an index of names. The index of names follows Knuth's practice of listing full names when known, and the policy he adopted for *The Art of Computer Programming* of supplying the 'mother tongue' version of names. It would be tiresome to include a full name in every `\index` directive for the name of a frequently-discussed person, so a preprocessing system was created that uses a 'list of shorthands' and 'list of global presentations'. For example, there is a definition in the list of shorthands saying that `Archimedes` is a shorthand for `Archimedes of Syracuse`, and there is a definition in the list of global presentations saying that `Archimedes of Syracuse` is to be presented as `Archimedes of Syracuse \origlang{Αρχιμήδης}`. (The `\origlang` macro sets its contents in square brackets, but has another purpose discussed in Section 10.) A Python script reads the `.idx` file and applies these shorthands and presentations to create a new `.idx` file, which is then run through the index processor as usual to generate the `.ind` file. The same script reads a list of valid keys and warns if any index directive contains a key not on this list, which helps prevent typos and ensure consistency in the index.

The same system of shorthands and presentations is used for the subject index, to help abbreviate various frequently-used index directives. A shorthand like `>mbeau` (where the prefatory `>` is simply to make these shorthands easy to search for) stands for `beauty in mathematics`. Shorthands can include subentries, so `>mbeau!utility` becomes `beauty in mathematics!relation to utility`.

The chosen index processor was `xindy`. Although development of `xindy` seems to have been halted for many years, it offers sufficient fine-grained control of sorting for *F&N*. Because the indexes include words and names from many languages, it seemed simpler to use a custom module for sorting. For sorting, this module maps each accented letter to its unaccented base and strips characters that are used in transliteration, such as the 'half-circle' modifiers ʿ and ʾ. (A small Python script is used to verify that the module handles all non-ASCII characters that appear in the `.idx` file.)

In the indexes of *F&N*, if a page/column break occurs within an entry, the heading of the entry and any sub-entry or sub-sub-entry is also repeated with a continuation text, as shown in Figure 8. In the default index in a LaTeX document, no such continuation texts are inserted. The package `repeatindex` inserts a continuation text for the top-level entry if a page/column break occurs within (or is triggered by) a new sub-entry or sub-sub-entry. But there are two limitations: 1) If the page/column break occurs within the locations relating to the top-level entry, nothing is inserted. 2) If the page/column break occurs *within* a sub-entry or sub-sub-entry, only the heading of the top-level entry is repeated. In *The TeXbook*, Knuth described a plain TeX method of creating such continuation texts using the mark mechanism [5, pp. 261–3], which does not have these limitations. But there are still situations in which this method can cause problems: for instance, if the keyword in the index entry spans two or more lines, it may break across a page or column and the continuation text would appear in the middle of the keyword.

The present author also prefers that the first line of a multi-line index entry should not appear last in a column, since with the continuation text there is no saving in space compared with simply ending the column *before* the index entry. Further, the author prefers that column breaks do not occur within cross-references ('see' or 'see also') in an index entry.

The indexes of *F&N* are thus produced by macros which track the amount of each column that has been filled and which typesets each index entry into a 'buffer' box and its keyword and cross-references into another box. If the second box occupies more space than remains in the column, or if the entire entry occupies more than one line and there is only space left for one line, a new column is started immediately. The buffer is output by stages, splitting off enough as much as will fit in the remaining space in the current column and triggering column breaks manually.

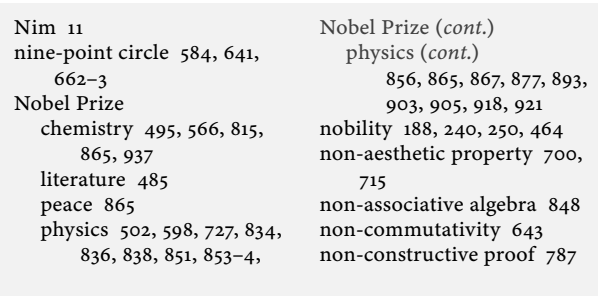

Nim 11
nine-point circle 584, 641, 662–3
Nobel Prize
chemistry 495, 566, 815, 865, 937
literature 485
peace 865
physics 502, 598, 727, 834, 836, 838, 851, 853–4,
Nobel Prize (*cont.*)
physics (*cont.*)
856, 865, 867, 877, 893, 903, 905, 918, 921
nobility 188, 240, 250, 464
non-aesthetic property 700, 715
non-associative algebra 848
non-commutativity 643
non-constructive proof 787

Figure 8: An example of index continuation text similar to those that appear in *Form & Number*.

This strategy is very dependent on an index being a highly-structured text. 'Exotic' index entries could cause problems. Nevertheless, the functionality has been released as the package `indextra`, available on CTAN.[7] At present, this package should be regarded as semi-experimental and requiring further development.

10 Bibliography and index running heads

In *F&N*, the running heads in the bibliography and index indicate the range of entries appearing on the page. Specifically, for the bibliography, the range comprises the family names of the first authors of the first and last bibliography entries on the page. For the index of names, the range comprises the family names from the first and last names on the page (even if the first name is an entry continued from the previous page). For the index of subjects, the range comprises the keywords in the first and last top-level entries on the page (again, even if the first top-level entry is continued from the previous page).

For the index, the necessary functionality is provided by the `indextra` package, which sets marks using keywords for top-level entries (including continuations) and makes available the first and last marks on each page via the macros `\indextrafirstmark` and `\indextralastmark`. Any mark thus retrieved is processed by a macro `\indextramakemark`, which by default has no effect but which can be redefined by the user. For the index of names, `\indextramakemark` is redefined to a macro which locally sets the `\origlang` macro mentioned in Section 9 to `\@gobble` (so that

[7] `ctan.org/pkg/indextra`

the 'mother tongue' version of a name is never included in the running head), and also only returns the index entry up to the first comma (so that from ⟨*familyname*⟩**,** ⟨*forename*⟩, only ⟨*familyname*⟩ appears in the running head).

For the bibliography, marks are inserted via a trick using the BibLaTeX indexing mechanism. The following code sets up a new name indexing format that inserts marks rather than index entries:

```
\NewMarkClass{bibliography}
\DeclareIndexNameFormat{marks}{%
  \InsertMark{bibliography}{\namepartfamily}%
}
```

Now the `begentry` bibmacro can be used to insert the mark using the most appropriate of BibLaTeX's name list fields:

```
\renewbibmacro*{begentry}{%
  \ifnameundef{author}{%
    \ifnameundef{editor}{%
      \indexnames[marks][1-1]{labelname}%
    }{%
      \indexnames[marks][1-1]{editor}%
    }%
  }{%
    \indexnames[marks][1-1]{author}%
  }%
}
```

The first of the two optional parameters to each `\indexnames` command applies the `marks` format; the second selects only the first name in each list. With these definitions, the family names from the relevant name list fields for the first and last bibliography entries on each page can be retrieved in the normal way using `\FirstMark{bibliography}` and `\LastMark{bibliography}` for inclusion in the running heads.

11 Building

Except for the external graphics, *F&N* is compiled as a single LuaLaTeX document, currently over 1200 pages. (For development and testing, individual draft chapters are compiled separately.) External graphics are built using Asymptote or as separate LuaLaTeX documents. Some of these graphics, such as a plot of a Lorenz attractor, depend on data computed by Python programs (often using SciPy). Some fractal plots are produced using Fractalnow or by custom Python programs.

The build process is automated, using standard Makefiles to specify dependencies and manage the generation and regeneration of graphics when necessary. On a laptop with a Core i7 2.8 GHz processor, generating all the graphics (without using parallel builds) takes about 20 min. (To save time, the external graphics are shared among the three variant formats and the individual draft chapters.) With all the graphics available, a single LuaLaTeX run of *F&N* takes over 4 min. Running `biber` takes over 1 min. Complete rebuilds, with the necessary multiple LuaLaTeX runs, are best run overnight.

12 The future

Form & Number still requires some polishing. The author is aware of flaws such as widows and orphans and intends to deal with them. The index of subjects requires a revision for greater consistency.

The full table of contents is unsatisfactory because many chapters have such long lists of sections and lower-level divisions that they split across pages. While it is usually possible and simple to *prevent* chapters splitting across pages, such a course could leave large spaces, and in any case there is one chapter that could not fit in a single page. It would be better for the table of contents to have continuation texts, analogous to those in the index.

Finally, there are other parts of the code that could be the bases of packages or Asymptote modules, but which first need *major* refactoring.

References

[1] A.J. Cain. *Nine Chapters on the Semigroup Art: Lecture notes for a tour through semigroups.* Porto & Lisbon, 2012. `archive.org/details/cain_semigroups_a4_screen`

[2] A.J. Cain. *Form & Number: A History of Mathematical Beauty.* Lisbon, 2024. `archive.org/details/cain_formandnumber_ebook_large`

[3] G.H. Hardy. *An Annotated Mathematician's Apology.* Lisbon, 2019. `archive.org/details/hardy_annotated`

[4] H. Hart. *Hart's Rules for Compositors and Readers at the University Press Oxford.* Oxford University Press, 39th ed., 1989. `archive.org/details/hartsrulesforcom0039oxfo`

[5] D.E. Knuth. *The TeXbook.* Addison–Wesley, 2021. *Computers & Typesetting*, vol. A.

[6] A. Waddingham. *New Hart's Rules: The Oxford Style Guide.* Oxford University Press, 2nd ed., 2014.

◇ Alan J. Cain
Almada
Portugal
a.j.cain (at) gmail.com
https://ajcain.codeberg.page/
ORCID 0000-0002-0706-1354

`marginalia` at work: Running heads, float captions, citations, and small figures in the margins

Alan J. Cain

Abstract

This paper is an introduction to the LuaLaTeX package `marginalia`, whose development was driven by the aim of typesetting a book with running heads, citations, and float captions in the margins. It allows the placement of non-floating marginal content anywhere, automatically adjusts positions to prevent content either overlapping or being pushed off the page, and has flexible key–value configuration.

1 Introduction

In typesetting his book *Form & Number: A History of Mathematical Beauty* [2], the present author wanted to use the margin for the following purposes, listed in decreasing order of priority:

Citations and notes. There would be *many* citations, but they would usually be short, since the chosen citation system was 'author and short title'. There would be no long discursive notes, only occasional short non-citation notes, such as to explain or clarify a '[*sic*]'.

Small figures. These might have heights around 8–10 lines of the main text, sometimes even more, but the caption would usually be short.

Captions for floats. These captions could be moderately long and might fill the entire margin beside the float.

Full-width floats. These are floats extending to the outside of the margin; in this case, the caption should be set in the margin below the float (or above for floats placed at the bottom of the page).

Running heads. The running heads should be set near the top of the margin. (Bringhurst called this style 'running shoulderheads' [1, p. 65], but the term may be non-standard.)

Of course, marginal content items should not overlap, which implies that their vertical positions must be adjusted. But float captions had to be treated specially: they should *not* move from their natural positions beside the corresponding float. The running head on a page should similarly not move from its specified position, but could occasionally be suppressed to make way for a large float caption or a full-width float. And all of this should be automatic...

This article discusses the solution that was developed, and which has been released as the LuaLaTeX package `marginalia`, available on CTAN.[1] The results can be seen in *Form & Number* (= *F&N*), which is freely available on the Internet Archive under a Creative Commons licence.[2]

2 Limitations of existing solutions

The normal `\marginpar` command would suffice for basic placement of citations and small figures (with `mparhack` to ensure that the correct margin is used in two-sided typesetting). But `marginpar`s can only be shifted *downward* if necessary to avoid overlaps. The cumulative effect can be to push content off the bottom of the page. This would be a danger when, for instance, many citations appear close together. Further, the effect of such shifting can spill from one page to the next (because of the asynchronous page-breaking of TeX). Most importantly, because `\marginpar` uses the float mechanism, it cannot be used to place captions next to floats, nor to place running heads.

The packages `marginfix` and `marginfit` both re-implement how `marginpar`s are placed, so as to ensure that nothing is shifted off the bottom of the page. `marginfit` uses a two-run solution, using the pdfTeX commands `\pdfsavepos` and `\pdflastypos` to write the vertical position of the call and the dimensions of the `marginpar` to the `.aux` file at page shipout, which allows exact placement during the next run. Again, neither package can be used to place `marginpar`s next to floats, nor to place running heads.

The `\marginnote` command, from the package of the same name, places content in the margin without using the float or insert mechanisms. Rather, the content is positioned using a calculated per-item horizontal shift inside an (invisible) `\rlap` or `\llap` from the position where the `\marginnote` command was issued. This approach allows marginal content to be placed anywhere, even from inside a float or from the page head. The `\marginnote` command also uses `\pdfsavepos` and `\pdflastxpos` to write the horizontal position where the command was called to the `.aux` file at page shipout, so that on the following run the marginal content can be automatically shifted by the appropriate distance to place it in the margin. But there is no automatic adjustment of vertical positioning to prevent overlaps.

[1] `ctan.org/pkg/marginalia`

[2] Ebook, large format: `archive.org/details/cain_formandnumber_ebook_large`
Ebook, small format: `archive.org/details/cain_formandnumber_ebook_small`
Version for printing: `archive.org/details/cain_formandnumber_print`

doi.org/10.47397/tb/46-1/tb142cain-marginalia

3 A new solution: `marginalia`

Clearly, none of the existing solutions would be entirely satisfactory for typesetting *F&N*. But the basic `marginnote` placement mechanism could work, if it were combined with saving the vertical position like `marginfit`, plus an algorithm for adjusting vertical positions that would take into account the different types of marginal content and whether they were moveable.

F&N therefore used a new solution which has been subsequently refined into the LuaLaTeX package `marginalia`. The package is written in `expl3`, but uses a Lua program to calculate the adjustments to vertical positions, which is a complicated process because of the high degree of configuration the package offers.

The `marginalia` package provides the command `\marginalia[`⟨*options*⟩`]{`⟨*content*⟩`}` to create a marginal content item. The parameter ⟨*options*⟩ sets configuration for that item via key–value syntax (using `l3keys`), and global configuration can be set similarly using the command `\marginaliasetup{`⟨*options*⟩`}`.

Figure 1 shows some of the parameters that can be controlled via ⟨*options*⟩, and which are discussed in detail in the following subsections.

3.1 Horizontal options

In one-column text, marginal content is placed by default in the outer margin (the right margin on a recto page, the left margin on a verso page). Item ③ was created using `pos=reverse`, which causes it to be set in margin opposite to the default. Because the outer and inner margins often have different widths, there are independent options `width outer` ⑨ and `width inner` ④, which specify widths for items placed in each margin, and `xsep outer` ⑥ and `xsep inner` ②, which specify the spacing between the main text and the marginal content. Thus one can use `\marginaliasetup` to define these parameters globally and the correct width and spacing will be applied to each item.

This feature is especially useful in two-column text: as with `\marginpar`, an item created for the left column is by default placed in the left margin, which is the inner margin on a recto page and the outer margin on a verso page, and an item for the right column is by default placed to the right margin. Thus the width and spacing of the item can be dependent both on the page and on the column to which it pertains.

`marginalia` can also handle layouts where the margins are not symmetrical between recto and verso pages: for instance, one can specify independently `width recto outer` and `width verso outer`.

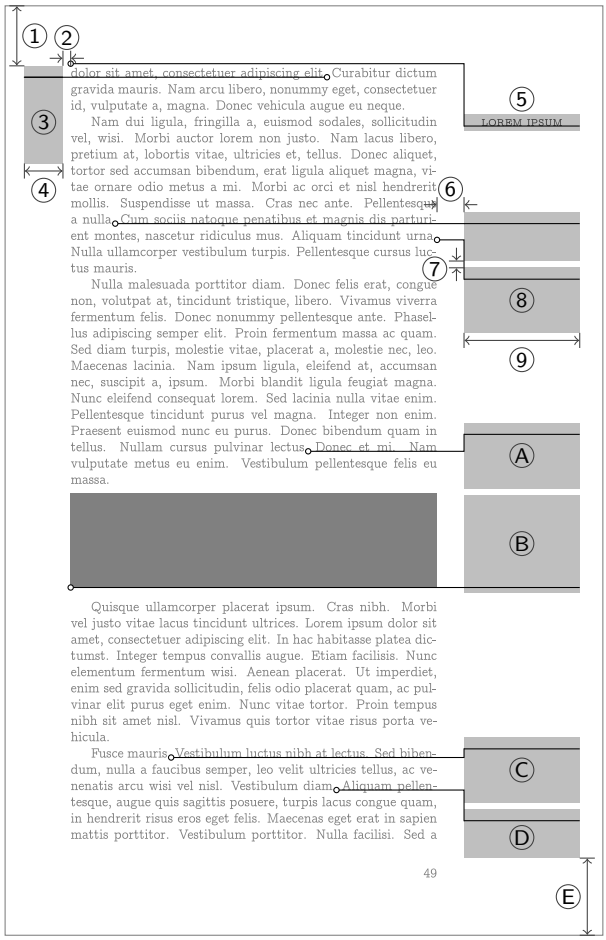


Figure 1: Illustration of some of the options provided by `marginalia` for a recto page. For clarity, a line runs from the baseline of each marginal content item to the point where the `\marginalia` command was issued. The options are discussed fully in the text, but, in abbreviated form, they are: ① ≥ `ysep page top`; ② = `xsep inner`; ③ `pos=reverse`; ④ = `width inner`; ⑤ `type=optfixed, yshift=-46pt`; ⑥ = `xsep outer`; ⑦ ≥ max{`ysep above`, `ysep below`}; ⑧ automatically moved; ⑨ = `width outer`; Ⓐ automatically moved; Ⓑ `type=fixed`; Ⓒ automatically moved; Ⓓ automatically moved; Ⓔ ≥ `ysep page bottom`.

3.2 Vertical options

Vertical spacing is similarly customizable. The minimum separation between two vertically-adjacent marginal content items ⑦ is the maximum of the value of `ysep below` for the upper item and `ysep above` for the lower item.

The motivation for this approach is that certain kinds of marginal content may require different spacing: for example, it may be desirable to have more space around a float caption in the margin than around a citation.

Similarly, the minimum separation between an item and the top and bottom of the page are specified by `ysep page top` ① and `ysep page bottom` Ⓔ.

`marginalia` tries to adjust the position of marginal content so that the specified minimum separation is maintained. By default, the adjustments are made *downwards*: item ⑧ has been moved down from its natural position to maintain spacing with the item above, which has not been moved. But leaving item Ⓒ in its natural position would force item Ⓓ to be moved too close to the bottom of the page, so Ⓒ has been moved *upwards*.

When it is impossible to fit all the marginal content with the specified spacing, a package warning is emitted.

3.3 Style options

Not illustrated in Figure 1 are options affecting the style applied to the content of a marginal content item. It is possible to specify independently styles for items placed in the outer and inner margins on a recto page via `style recto outer` and `style recto inner`, and similarly on a verso page via `style verso outer` and `style verso inner`. For instance, in the outer margins, one may want to have marginal notes set ragged, with the rag towards the outside of the page (that is, `\raggedright` in recto outer margins, `\raggedleft` in verso outer margins). In the inner margin, one may want to have the opposite, and also to use smaller text if the margin is narrower.

Within the ⟨*content*⟩ of each item, the page number on which it appears (based on the data written to the `.aux` file on the previous run) is available in a counter `\marginaliapage`. When relevant, the column number is also available via a counter `\marginaliacolumn`. This information can be used for purposes not foreseen by the `style` options.

3.4 Different types of marginal content

Ⓑ was created using the option `type=fixed`, because it contains a caption for a float. This option specifies that the item will not be moved from its default position. Thus, in order to maintain the specified vertical separation, Ⓐ has been automatically moved upwards.

Finally, there is ⑤, which was set from the page head (which has height zero) to place a running header in the margin. It has been moved downward manually from its default position using the option `yshift` and has been created with `type=optfixed`. This option specifies that the item will never be *moved* automatically from its specified position, but it may be *disabled* automatically on a particular page if it would clash with another item set with `type=fixed`, such as a float caption.

3.5 Legacy parameters and commands

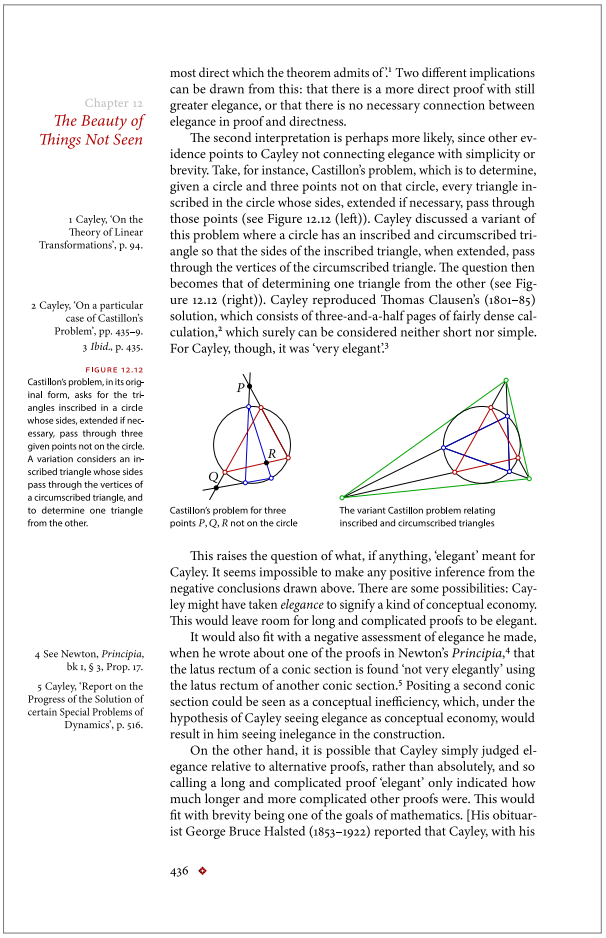
The usual LaTeX dimensions that influence the placement of `marginpar`s, namely `\marginparwidth` (the width of a `marginpar`), `\marginparsep` (horizontal distance between a `marginpar` and the main text block), and `\marginparpush` (minimum vertical space between `marginpar`s), serve only to set default values for the various keys `width` ⋯, `xsep` ⋯, and `ysep` ⋯ at package load time, and are otherwise not used by `marginalia`. The command `\reversemarginpar` (which switches the margin in which subsequent `marginpar`s are placed) has no effect on `marginalia`, since the option `pos=reverse` produces the same effect.

4 Implementation choices

The most complicated part of `marginalia` is computing the automatic adjustment of vertical positions and the parallel process of deciding when items with `type=optfixed` should be disabled. The required computation depends on multiple parameters which may vary between items. Thus the computation is carried out by a Lua program. In principle, it would be possible to implement a pure LaTeX version, but the author has not thought seriously about writing such an implementation and suspects this task to be beyond his `expl3` skills.

On the first run, the LaTeX part of the package writes information about each item into the `.aux` file, including the page number, the position on the page of the call to `\marginalia` (as supplied by `\lastxpos` and `\lastypos`, which are the LuaTeX equivalents of pdfTeX's `\pdflastxpos` and `\pdflastypos`), the height and depth of the typeset item, and the specified values of `yshift` and the various `ysep` ⋯. In the next LuaLaTeX run, when the `.aux` file is read back, the information is stored in a Lua table. Then the Lua program performs the necessary computations for each page and, within each page, for each margin. The basic algorithm is similar to `marginfix` and `marginfit`: shift items downward to avoid clashes, then upward to avoid items being pushed off the page while still avoiding clashes. But the details are more complicated because some items have fixed positions and because the minimum vertical spacing can be set on a per-item basis.

Finally, when the second LuaLaTeX run encounters the original call to `\marginalia`, the computed adjustment is loaded from Lua and used to position the item.


Chapter 12
The Beauty of Things Not Seen

most direct which the theorem admits of'.[1] Two different implications can be drawn from this: that there is a more direct proof with still greater elegance, or that there is no necessary connection between elegance in proof and directness.

The second interpretation is perhaps more likely, since other evidence points to Cayley not connecting elegance with simplicity or brevity. Take, for instance, Castillon's problem, which is to determine, given a circle and three points not on that circle, every triangle inscribed in the circle whose sides, extended if necessary, pass through those points (see Figure 12.12 (left)). Cayley discussed a variant of this problem where a circle has an inscribed and circumscribed triangle so that the sides of the inscribed triangle, when extended, pass through the vertices of the circumscribed triangle. The question then becomes that of determining one triangle from the other (see Figure 12.12 (right)). Cayley reproduced Thomas Clausen's (1801–85) solution, which consists of three-and-a-half pages of fairly dense calculation,[2] which surely can be considered neither short nor simple. For Cayley, though, it was 'very elegant'.[3]

1 Cayley, 'On the Theory of Linear Transformations', p. 94.

2 Cayley, 'On a particular case of Castillon's Problem', pp. 435–9.

3 *Ibid.*, p. 435.

FIGURE 12.12
Castillon's problem, in its original form, asks for the triangles inscribed in a circle whose sides, extended if necessary, pass through three given points not on the circle. A variation considers an inscribed triangle whose sides pass through the vertices of a circumscribed triangle, and to determine one triangle from the other.

Castillon's problem for three points *P*, *Q*, *R* not on the circle

The variant Castillon problem relating inscribed and circumscribed triangles

This raises the question of what, if anything, 'elegant' meant for Cayley. It seems impossible to make any positive inference from the negative conclusions drawn above. There are some possibilities: Cayley might have taken *elegance* to signify a kind of conceptual economy. This would leave room for long and complicated proofs to be elegant.

It would also fit with a negative assessment of elegance he made, when he wrote about one of the proofs in Newton's *Principia*,[4] that the latus rectum of a conic section is found 'not very elegantly' using the latus rectum of another conic section.[5] Positing a second conic section could be seen as a conceptual inefficiency, which, under the hypothesis of Cayley seeing elegance as conceptual economy, would result in him seeing inelegance in the construction.

On the other hand, it is possible that Cayley simply judged elegance relative to alternative proofs, rather than absolutely, and so calling a long and complicated proof 'elegant' only indicated how much longer and more complicated other proofs were. This would fit with brevity being one of the goals of mathematics. [His obituarist George Bruce Halsted (1853–1922) reported that Cayley, with his

4 See Newton, *Principia*, bk 1, § 3, Prop. 17.

5 Cayley, 'Report on the Progress of the Solution of certain Special Problems of Dynamics', p. 516.

436 ❖

Figure 2: A verso page from (the print variant of) *Form & Number*, showing the running head, a float caption, and several citations. The first citation has been moved down from its natural position to below the running head. The second and third citations have been moved up so as not to clash with the figure caption, which has a fixed position.

5 Using the solution

This section shows how `marginalia` was used in *F&N* to typeset citations and short notes, float captions, and running heads. The code snippets below are simplified but functional versions of the real code. A sample page, containing all three kinds of marginal content, is shown in Figure 2.

5.1 Citations and short notes

To set up numbered sidenotes, the following code suffices:

```
\newcounter{sidenote}
\newcommand*\sidenote[1]{%
  \stepcounter{sidenote}%
  \textsuperscript{\thesidenote}%
  \marginalia[
    style recto outer=
      {\raggedright\footnotesize},
    style verso outer=
      {\raggedleft\footnotesize},
  ]{%
    \leavevmode\llap{\thesidenote~}#1%
  }%
}
```

This code elides various details. (The real code used in *F&N* essentially follows the implementation of `\footnote` in LaTeX 2ε, and uses the package `perpage` so that note numbering starts at 1 on each page, but this simplified version is similar to that used in the `marginalia` package documentation.) Notes set with this `\sidenote` command will have the default minimal vertical separation from adjacent marginal content.

For citations, the usual `biblatex` methods were used to define citation commands `\sidecite` and `\sidecites`, which are analogous to `\footcite` and `\footcites`.

5.2 Float captions

This is a simplified version of the re-definition of the `\caption` command used for floats in *F&N*:

```
\renewcommand\caption[1]{%
  \refstepcounter{\@captype}%
  \marginalia[
    valign=b,
    type=fixed,
    ysep above=10pt,
    ysep below=10pt,
    style={\sffamily\footnotesize},
  ]{%
    \noindent
    {\scshape\csname fnum@\@captype\endcsname}%
    \par
    \noindent #1\par
  }%
}
```

Thus `\caption` sets a marginal content item with the option `valign=b`, which aligns the baseline of the bottom line of the item with the call to `\marginalia`. It is set with `type=fixed`, so it will not be automatically moved from its position beside the float. To make it distinct, it is set in justified sans-serif, with a larger minimal vertical separation from adjacent marginal content than usual.

5.3 Running heads

Because running heads were placed in the margins, the usual head itself was not needed, so `\headheight` and `\headsep` were set to 0 pt (via the `nohead` option

of the geometry package). Then, using the titleps package, the headings page style was defined to call \marginalia from the header. The following is a simplified version:

```
\renewpagestyle{headings}{%
  \sethead{%
    \marginalia[
      type=optfixed,
      yshift=-46pt,
      ysep below=48pt,
      style recto outer=
        {\raggedright\large\itshape},
      style verso outer=
        {\raggedleft\large\itshape},
    ]{%
      \chaptertitle
    }%
   }{}{}
  \setfoot{}{}{\thepage}
}
```

The option yshift=-46pt positions the running head so that its first line is aligned with the fourth line of the main text, since 46 pt = \topskip + 3 × \baselineskip in the book class with option 10pt. The option ysep below=48pt ensures that no other marginal content on the page will be placed closer to the running head than this distance, which is equivalent to four lines of the main text. (In the running head in Figure 2, the italic red text 'The Beauty of Things Not Seen' is the current chapter title, and its first line is aligned with the fourth line of the text block. The grey text above, giving the current chapter number, is raised above this line by code not shown here.) The option type=optfixed specifies that the running head can be disabled if it is necessary to place an item of type fixed, such as a float caption.

6 Another application: memoir-style sidebars and sidefootnotes

An approximation to the sidebars and sidefootnotes supplied by the memoir class can also be created using marginalia. Sidebars are set in the margin, starting near the top of the page and can continue to another page. Continuing to another page is beyond the present capabilities of marginalia, but to place the marginal content at the top of the margin, one could use:

```
\newcommand*\sidebar[1]{%
  \marginalia[
    style recto outer=
      {\raggedright\footnotesize},
    style verso outer=
      {\raggedleft\footnotesize},
    yshift=\paperheight,
  ]{#1}%
}
```

On the first run, anything set with this \sidebar command will be shifted off the top of the page. The adjustment on subsequent runs will move the topmost such item to ysep page top from the top of the page, and other such items will stack below it. (However, marginalia currently does not reorder marginal content items, so an item set with this \sidebar command will not migrate above any other items set with \marginalia.)

Similarly, sidefootnotes are footnotes set at the bottom of the margin. One could define an analogous \sidefootnote command similar to \sidebar by setting yshift=-\paperheight. On the second run, items set with such a \sidefootnote will be stacked at the bottom of the margin, with the lowest a distance ysep page bottom from the bottom of the page.

Acknowledgements

The author thanks Tânia Paulista for commenting on a draft of this article.

References

[1] R. Bringhurst. *The Elements of Typographic Style*. Hartley & Marks, version 3.0 ed., 2004.

[2] A.J. Cain. *Form & Number: A History of Mathematical Beauty*. Lisbon, 2024. archive.org/details/cain_formandnumber_ebook_large

◇ Alan J. Cain
Almada
Portugal
a.j.cain (at) gmail.com
https://ajcain.codeberg.page/
ORCID 0000-0002-0706-1354

Typesetting code with syntax highlighting: Introducing minted version 3

Geoffrey M. Poore

Abstract

The minted [5] package for LaTeX typesets code with syntax highlighting from the Pygments [1] library for Python. With version 3, minted enhances security and usability while adding new features and simplifying future development. The package includes a new Python executable called `latexminted` that is compatible with restricted shell escape. Usability is improved by new debugging and error reporting capabilities.

The redesign makes possible several additional features, including support for custom lexers and the ability to select snippets of code with regular expressions. The new architecture also allows future features to be developed using Python.

1 Introduction

The minted package provides commands and environments for typesetting code with syntax highlighting. It provides a `\mintinline` command for inline code within paragraphs, a `minted` environment for blocks of multi-line code, and an `\inputminted` command for bringing in external files (Figure 1). minted passes code to Python for syntax highlighting with the Pygments library, and then the highlighted output is typeset using the fancyvrb [12] and fvextra [6] packages for LaTeX.

In July 2023, I received a TeX Development Fund grant from the TeX Users Group to develop minted version 3. My grant proposal focused on two objectives: (1) eliminating existing shell escape security vulnerabilities, and (2) allowing minted to be extended using Python in addition to LaTeX macros.

I released minted version 3 in September 2024. This paper provides an overview of the new version focused on new capabilities. An accompanying paper (see "New tools for LaTeX package authors", this issue, 60–68) details new LaTeX and Python packages that I created for LaTeX package authors as part of the development process.

2 Shell escape

minted provides syntax highlighting via the Pygments library for Python. Code must be passed from LaTeX to Python, highlighted, and then sent back to LaTeX. The Python executable that performs syntax highlighting is invoked via `\ShellEscape` [11]. This brings the potential for security vulnerabilities.

2.1 Vulnerabilities

Prior to minted version 3, the code to be highlighted was saved to a temp(orary) file, then processed with the `pygmentize` application that is bundled with Pygments. Finally, the saved output was brought back into LaTeX via `\input`. Running `pygmentize` via `\ShellEscape` required special permission. Unrestricted shell escape, which allows any executable to be run, could be enabled by invoking LaTeX with the command-line option `-shell-escape` (TeX Live) or `-enable-write18` (MiKTeX). It was also possible to enable `pygmentize` by modifying the TeX configuration. This approach brought security vulnerabilities:

- Unrestricted shell escape allows arbitrary commands to be executed via `\ShellEscape`. It cannot be limited to `pygmentize`. Compiling documents from untrusted sources is equivalent to running untrusted code.
- Highlighting settings were passed to `pygmentize` as command-line options. This involved multiple layers of escaping. First, options must be correctly escaped as LaTeX to give the desired literal text. Second, options must be escaped in an operating-system dependent manner to be compatible with the system shell. Any bugs in escaping could result in invalid options so that `pygmentize` fails. With unrestricted shell escape, bugs could also be exploited so that a `\ShellEscape` that was intended to run `pygmentize` would also invoke arbitrary additional commands.

2.2 The `latexminted` executable

minted version 3 includes a new Python executable called `latexminted` that replaces `pygmentize`. It is designed to avoid security vulnerabilities while also allowing new minted features to be implemented in Python.

By default, (LA)TeX allows "restricted shell escape", shell escape but only for trusted executables such as `bibtex` and `kpsewhich`. This does not require `-shell-escape` or `-enable-write18`, or equivalent modifications to TeX configuration. Trusted executables access the file system in a limited manner compatible with TeX's security model and cannot run arbitrary additional programs. `pygmentize` could never be a trusted executable because its `-x` option allows custom lexers and formatters to be loaded from `.py` files, which is equivalent to arbitrary code execution. In contrast, `latexminted` was designed from the start to be compatible with restricted shell escape and has been accepted as a trusted executable by TeX Live.

doi.org/10.47397/tb/46-1/tb142poore-minted3

LaTeX source

```
\begin{minted}{python}
class KeyDefaultDict(collections.defaultdict):
    '''
    Default dict that passes missing keys to the factory function, rather than
    calling the factory function with no arguments.
    '''
    __init__: Callable[[Callable[[Any], Any]], None]
    default_factory: Callable[[Any], Any]
    def __missing__(self, key):
        if self.default_factory is None:
            raise KeyError(key)
        self[key] = self.default_factory(key)
        return self[key]
\end{minted}
```

Output

```
class KeyDefaultDict(collections.defaultdict):
    '''
    Default dict that passes missing keys to the factory function, rather than
    calling the factory function with no arguments.
    '''
    __init__: Callable[[Callable[[Any], Any]], None]
    default_factory: Callable[[Any], Any]
    def __missing__(self, key):
        if self.default_factory is None:
            raise KeyError(key)
        self[key] = self.default_factory(key)
        return self[key]
```

Figure 1: An example of the `minted` environment for syntax highlighting, with LaTeX source (top) and output (bottom). The example code is from the latex2pydata Python package. The environment takes a single mandatory argument, which is the name of the programming language or lexer that is used for highlighting. Optional arguments provide customization of typesetting, appearance, and highlighting. The `minted` package also provides a `\mintinline` command for inline code within paragraphs and an `\inputminted` command for external files.

The `latexminted` executable accesses the file system via the new `latexrestricted` [3] Python library. This uses TeX configuration settings to determine which files are permitted for reading and writing. With TeX Live, `latexrestricted` restricts file system access based on the `texmf.cnf` configuration settings for the following variables:

- `openin_any`
- `openout_any`

With MiKTeX, it restricts file system access based on the `miktex.ini` configuration settings for the following variables:

- `[Core]AllowUnsafeInputFiles`
- `[Core]AllowUnsafeOutputFiles`

These TeX Live and MiKTeX variables determine whether LaTeX can access files anywhere in the file system, or only in locations such as the current document directory or a designated output directory. These variables also determine whether "dot files" (filename begins with ".") can be accessed. Dot files are typically used for configuration, so if LaTeX has permission to write dot files then it can affect any executables that depend on them.

Even with a trusted executable that is compatible with restricted shell escape, properly handling command-line options presents several challenges.

- In restricted shell escape mode (the default), the LaTeX executable handles the quoting and escaping of command-line options. Under non-Windows operating systems, it quotes options using single quotation marks, which means that options cannot contain literal single quotation

marks. This is a significant limitation for a package like minted that works with code.

- If unrestricted shell escape is enabled for some reason, then the LaTeX executable no longer quotes and escapes command-line options. This must now be performed via LaTeX macros in an operating-system dependent manner. Any bugs in this process will result in errors at best, and may potentially be exploited to run arbitrary commands.

The `latexminted` executable avoids these issues by only accepting command-line options that do not require quoting or escaping. All options consist of ASCII alphanumeric characters, so regardless of shell escape mode, no quoting or escaping is ever needed. Since this significantly limits the data that can be passed as command-line options, most data is instead transferred to `latexminted` via a temp file.

Communicating data via a temp file requires a new approach compared to prior versions of minted. Before version 3, temp files were only used for code; all highlighting settings and other data were communicated via command-line options. Now, temp files must contain code, highlighting settings, and other data, so a data serialization format is needed. minted saves this data to a file in the Python literal format [10], via the new latex2pydata LaTeX package [7]. Then the `latexminted` executable loads this data using the new latex2pydata Python package [4], which builds on Python's ast [9] module to deserialize data.

Python literal format resembles JSON [2] for numerical, string, list, and dictionary data types. It is ideal for working with LaTeX for two reasons. First, Python literal format supports multi-line string literals, so it is possible to write the contents of a LaTeX environment to a file one line at a time, without having to accumulate the entire environment contents within a single macro or otherwise access it all at once. Second, Python literal format only requires that quotation marks and backslashes be escaped within typical strings, so implementing serialization in LaTeX is straightforward and efficient.

The latex2pydata LaTeX package also makes it possible to reduce temp file usage significantly. In minted version 2, code was written to a temp file, and then an MD5-based hash of the temp file plus highlighting settings was used to check for a cached copy of highlighted code. latex2pydata allows data to be buffered in memory as a sequence of LaTeX macros, and only written to file later if desired. It can calculate the MD5 hash of the buffered data without writing it to a file, and then this hash is used to check for a cached copy of highlighted code. The data is only ever written to a temp file if a cached copy of highlighted code does not exist, so that a temp file is necessary for communicating the data to `latexminted` for highlighting.

3 Packaging and installation

Because minted combines a LaTeX package with a Python executable, there is the potential for packaging and installation to be more complicated than it would be for a normal package.

Prior to version 3, installing and configuring minted was typically a multi-step process. The LaTeX package might be installed with a LaTeX package manager, and then Pygments and its `pygmentize` application might be installed with a Python package manager. Depending on system configuration, `pygmentize` might need to be added to `PATH` before use. LaTeX editing software might need to be configured to locate `pygmentize`, and would need to run LaTeX with unrestricted shell escape unless the configuration was modified to enable this.

With version 3, minted can now be installed just like a regular LaTeX package, perhaps via the TeX Live package manager, and then used without any additional steps or special configuration. The `latexminted` executable and all required Python libraries, including Pygments, are bundled with the LaTeX package. Libraries are bundled as Python wheels [8], which are zip archives with a `.whl` file extension that contain a built package in a standardized format. Because the `latexminted` executable is bundled with the LaTeX package, no `PATH` modifications are needed beyond those to use LaTeX itself. Because `latexminted` is compatible with restricted shell escape, modifying compilation to enable unrestricted shell escape is no longer necessary.

While this approach to packaging simplifies package installation and configuration, it does bring one disadvantage. Because the bundled version of Pygments is accessed as a Python wheel, it cannot be modified or customized with Python plugin packages. When plugin packages are needed, `latexminted` can be installed separately as a regular Python package within a Python installation. Then this separate version of `latexminted` can be given priority on `PATH` above the version that is bundled with the minted LaTeX package. `latexminted` is available as a Python package from the Python Package Index (PyPI) at `pypi.org/project/latexminted`.

4 Errors and debugging

minted errors can originate within either LaTeX or Python. This can make debugging more challenging than normal.

debug_example.tex

```
\documentclass{article}
\usepackage{minted}
\begin{document}
\begin{minted}{undefined}
x = 1
\end{minted}
\end{document}
```

Compile log error

```
! Package minted Error:
 => ./debug_example:4: Pygments lexer
"undefined" is unknown.
```

Figure 2: An example of minted's new error reporting capabilities. The document `debug_example.tex` attempts to use an undefined lexer in a `minted` environment. This error cannot be detected until the code, lexer, and other data have been passed to the `latexminted` Python executable. `latexminted` converts the error from a Python exception into a format suitable for LaTeX and then passes the error message back, where it is reported as a standard LaTeX error. The error message contains the source file and line number for each error.

Prior to minted version 3, any errors that occurred while `pygmentize` was running resulted in raw Python error messages in the LaTeX compilation log. These Python errors might then trigger additional LaTeX errors due to missing highlighted code.

With version 3, when minted passes code and settings to the `latexminted` executable, it also includes source file and line number information. If an error occurs while `latexminted` is running, the error message is converted into a form suitable for LaTeX, annotated with source information, passed back to LaTeX via temp file, and finally converted into a LaTeX error message (Figure 2). Debugging no longer involves skimming the log file for raw Python error messages.

There is also a new `debug` package option to help in cases when debugging remains difficult. This preserves the temp files that are used to communicate between LaTeX and Python, instead of cleaning them up automatically. It also writes additional information to the compilation log, such as the source file and line number at each location where code is highlighted.

5 Extending with Python

The move from Pygment's `pygmentize` executable to the new `latexminted` executable allows new features to be implemented in Python. Since minted now has its own dedicated Python executable, Python features are no longer limited to what is provided by `pygmentize`.

LaTeX source

```
\begin{minted}[
  rangeregex=\\\\usepackage.+,
  rangeregexmatchnumber=2
]{latex}
\usepackage{fvextra}
\usepackage{minted}
\end{minted}
```

Output

```
\usepackage{minted}
```

Figure 3: An example using the new `rangeregex` option, which allows a snippet of code to be selected using a Python regular expression. The regular expression is used to select a line containing "`\usepackage`" from within a `minted` environment. There are two lines that match, and `rangeregexmatchnumber` is used to select the second. minted has other new options such as `rangestartstring` and `rangestopstring` that allow snippets to be selected using literal strings rather than regular expressions.

5.1 Selecting code snippets

There are several new options that allow a snippet of code to be selected for display. Options such as `rangestartstring` and `rangestopstring` allow a snippet to be selected based on delimiting strings of text. Meanwhile `rangeregex` allows a snippet to be selected using a Python regular expression (Figure 3). If there are multiple non-overlapping matches for `rangeregex`, then `rangeregexmatchnumber` can be used to select the desired match.

These options pass delimiting strings or regular expressions from LaTeX to Python, and then Python performs the search. A delimiting string or regular expression can be defined in two ways. It can be provided as a single macro that gives the desired text when fully expanded. Alternatively, it can be provided as escaped text, in which each character is interpreted literally except that backslash escapes of ASCII punctuation characters are allowed to give the literal characters ("\\" for backslash, "`\#`" for "#", and so on).

5.2 Custom keywords

Pygments currently includes lexers for approximately 600 languages and other text formats. If those plus the existing Pygments plugins are insufficient, it is

LaTeX source

```
\begin{minted}{python}
default_factory: Callable[[Any], Any]
\end{minted}

\begin{minted}[
  extrakeywordstype={Callable,Any}
]{python}
default_factory: Callable[[Any], Any]
\end{minted}
```

Output

```
default_factory: Callable[[Any], Any]
default_factory: Callable[[Any], Any]
```

Figure 4: An example of one of minted's new `extrakeywords` options. In this Python code, by default Pygments does not provide any highlighting for the type hints. The `extrakeywordstype` option allows new keywords for types to be defined, and then these will be highlighted. This allows existing lexers to be customized without requiring any explicit Python programming. There is support for customizing all keyword tokens supported by Pygments.

also possible to create your own custom lexer using Python.

One of the most common cases for creating a custom lexer is adding additional keywords to a built-in Pygments lexer. minted provides several new options for adding keywords, including `extrakeywords` for general keywords, `extrakeywordsconstant` for constants, `extrakeywordstype` for types (Figure 4), and similar options covering all keyword tokens supported by Pygments. Behind the scenes, these options create a copy of the current lexer that has been extended with additional keywords. No explicit Python programming is required.

5.3 Custom lexers

In more complex cases, it may be necessary to create your own custom lexer in Python. A custom lexer may be packaged as a Pygments plugin and installed within a Python installation. minted has always supported this approach, since it simply modifies the existing Pygments installation. It is also possible to load a custom lexer from a `.py` file. The `pygmentize` application supports this with its `-x` option, but this was never officially supported by minted version 2. Loading a custom lexer from a `.py` file has significant security implications since the file may contain arbitrary code unrelated to Pygments.

minted version 3 adds support for loading custom lexers from `.py` files. It is convenient to be able to bundle a custom lexer file with a LaTeX document and avoid modifications to the Pygments installation. However, custom lexers in `.py` files are disabled by default. Enabling a given lexer file involves modifying the minted config file `.latexminted_config` by adding an entry of this form:

```
"custom_lexers": {
  "<filename>": "<SHA-256 hash of file>"
}
```

Once a file has been enabled, then the filename can be used as a lexer name in minted options. If there are multiple versions of a given lexer file, a list of SHA-256 hashes can be provided instead of a single hash.

The `.latexminted_config` file can be in the user's home directory or `TEXMFHOME`. It is also possible to enable config files in document root directories, but that is disabled by default since it could allow documents from untrusted sources to enable code execution via custom lexers.

6 An example of implementing a new option with Python

In many cases, it is now easy to implement new options in Python. The `literatecomment` option serves as an example. `literatecomment` specifies a string of text that will be removed from the beginning of each line of code, but only if it appears at the beginning of all lines of code. This option is useful for working with literate programming formats, such as the `.dtx` format commonly used for writing LaTeX packages. In a `.dtx` file, all lines of documentation text begin with a percent symbol "`%`", to differentiate them from the LaTeX package code. Using "`literatecomment=\%`" with `minted` will strip these percent symbols before typesetting code that is part of the documentation.

Implementing `literatecomment` involved only a single line of code on the LaTeX side of the minted package. There is a list of options that are passed from LaTeX to Python, and "literatecomment" needed to be added to this list with a default value of the empty string.

On the Python side, within the `latexminted` executable, "literatecomment" needed to be added to a list of options that should have string values, added to a list of options that are applied to code before highlighting, and added as a function argument to the function that processes code before highlighting. Finally, the option was actually implemented within that function in five lines (Figure 5). In the future,

```
if literatecomment:
    code_lines = code.splitlines(True)
    if all(line.startswith(literatecomment) for line in code_lines):
        len_literatecomment = len(literatecomment)
        code = ''.join(line[len_literatecomment:] for line in code_lines)
```

Figure 5: The Python implementation of the new `literatecomment` option. Since minted now has its own Python executable, `latexminted`, it is now possible to pass options to Python and implement them entirely within `latexminted`. If a value for `literatecomment` is defined, this splits `code` into individual lines. If all lines begin with `literatecomment`, then `code` is redefined to remove `literatecomment` from the beginning of lines. This is useful for working with literate programming formats. For example, in the `.dtx` format commonly used for writing LaTeX packages, all lines of documentation text begin with a percent symbol "%", and this must be stripped before any code that is part of the documentation is typeset.

additional options may be implemented in Python with similar efficiency.

7 Conclusion

minted version 3 brings a new `latexminted` Python executable for highlighting and then typesetting code. This executable is compatible with restricted shell escape, so it avoids the security vulnerabilities commonly associated with external software. The executable is bundled with all required Python libraries, so it also avoids the complexity that often comes with external programs. Because minted now has its own dedicated Python executable, along with a standardized method of communicating data from LaTeX to Python provided by latex2pydata, it is now possible to implement new features using Python. Defining a new Python-based option can be as simple as adding the name of the option within the minted LaTeX package, and then extending the `latexminted` executable with code to process that option.

The new version of minted provides an example for package authors of how a LaTeX package can securely communicate with an external executable using a standardized data format, and how an external executable can be designed to respect LaTeX security settings. For packages using external Python executables, the tools required to do this are available in the new latexrestricted and latex2pydata packages for Python.

References

[1] G. Brandl, Pygments contributors. Pygments: Python syntax highlighter, version 2.19.1, 2025. `pygments.org`

[2] Ecma International. ECMA-404: The JSON Data Interchange Syntax, 2017. `ecma-international.org/publications-and-standards/standards/ecma-404`

[3] G.M. Poore. latexrestricted — Python library for creating executables compatible with LaTeX restricted shell escape, version 0.6.2, 2024. `pypi.org/project/latexrestricted`

[4] G.M. Poore. latex2pydata — load data from LaTeX in Python literal format, version 0.5.0, 2025. `pypi.org/project/latex2pydata`

[5] G.M. Poore. The minted package, version 3.6.0, 2025. `ctan.org/pkg/minted`

[6] G.M. Poore. The fvextra package, version 1.12.0, 2025. `ctan.org/pkg/fvextra`

[7] G.M. Poore. The latex2pydata package, version 0.5.0, 2025. `ctan.org/pkg/latex2pydata`

[8] PyPA. Binary distribution format, 2024. `packaging.python.org/en/latest/specifications/binary-distribution-format`

[9] Python Software Foundation. ast — Abstract Syntax Trees, 2024. `docs.python.org/3/library/ast.html#ast.literal_eval`

[10] Python Software Foundation. Lexical analysis, 2024. `docs.python.org/3/reference/lexical_analysis.html#literals`

[11] The LaTeX Project Team. The shellesc package, version 1.0d, 2023. `ctan.org/pkg/shellesc`

[12] T. Van Zandt, H. Voss. The fancyvrb package, version 4.5c, 2024. `ctan.org/pkg/fancyvrb`

◇ Geoffrey M. Poore
1050 Union University Dr
Jackson, TN 38305
USA
gpoore (at) gmail dot com
https://gpoore.github.io/

New tools for LaTeX package authors: Buffering, hashing, caching, serializing, and executing

Geoffrey M. Poore

Abstract

This paper provides an overview of new tools for LaTeX package authors that I created during the development of minted [7] version 3.

The fvextra [8] package can now save the contents of an environment verbatim in memory, in a form that allows line-by-line iteration. Then the saved contents or a modified version can later be typeset or written to an external file.

The new latex2pydata LaTeX package [9] allows key-value data to be written to a file in Python literal format [11]. Alternatively, data can be accumulated in an in-memory buffer which allows hashing or modification before optionally being written to a file. The new latex2pydata Python package [6] provides data loading and processing capabilities for data saved by the latex2pydata LaTeX package. Together, these packages make it possible to pass structured data from LaTeX into Python.

The new latexrestricted [5] Python package provides access to the file system and subprocesses in a manner consistent with LaTeX security settings. This simplifies the process of creating Python executables that meet the security requirements for LaTeX restricted shell escape.

1 Introduction

The minted package for LaTeX typesets code with syntax highlighting from the Pygments [1] library for Python. In July 2023, I received a TeX Development Fund grant from the TeX Users Group to develop version 3 of the minted package. As part of this process, I created several new LaTeX and Python packages geared toward package authors, and also extended the existing fvextra package for LaTeX. This paper provides an overview of the new capabilities provided by these packages.

The minted package functions by collecting code within a LaTeX document and passing the code to Python for syntax highlighting with Pygments. Then Python passes the highlighted output back in LaTeX format. In minted version 3, each part of this process relies on new or improved packages intended for package authors.

2 Saving and processing verbatim environments with fvextra

The minted package needs to save verbatim text and then process it multiple times. Code that is to be highlighted cannot be processed in a single pass. First, code is combined with settings and hashed (`\pdf@mdfivesum` from pdftexcmds [4]) to check for the existence of a cached copy of highlighted code. If this does not exist, then code with settings is passed to Python for highlighting. Finally, if that fails, there is an error and also an attempt to typeset the code verbatim without syntax highlighting. The necessary capabilities for handling verbatim text are provided by enhancements to the fvextra package.

2.1 New fvextra features

The fvextra package extends the fancyvrb [18] package, along with related features for verbatim text. fvextra's new `VerbatimBuffer` environment stores its contents verbatim in a "buffer", a sequence of numbered macros each of which contains one line of the environment. The "buffered" lines can then be typeset or iterated over for further processing.

The `\VerbatimInsertBuffer` command typesets a buffer as a fancyvrb environment such as `Verbatim` (like `verbatim` with extra features) or `BVerbatim` (boxed `Verbatim`). The `\InsertBuffer` command interprets a buffer as LaTeX input, using `\scantokens`. The result is essentially the same as if the buffered text had been included literally at the insertion point. `\InsertBuffer` can also be used to typeset a buffer verbatim by setting the `wrapperenvname` option to a verbatim environment that will be used to wrap the buffer during interpretation. When these insertion commands are combined with `VerbatimBuffer`, it is possible to display LaTeX source and its output without duplicating the LaTeX source or using a temp(orary) file (Figure 1).

The `\IterateBuffer` command iterates over buffered lines from beginning to end. It takes a single mandatory argument: `\IterateBuffer{`⟨*macro*⟩`}`. During iteration, ⟨*macro*⟩ is applied to each line in the buffer. Iteration can be stopped before the end of the buffer with `\IterateBufferBreak`. When `\IterateBuffer` is combined with `\InsertBuffer` or `\VerbatimInsertBuffer`, it is possible to typeset a buffer in a manner that depends on the buffer content (Figure 2).

It is also possible to calculate the MD5 hash of a buffer with `\BufferMdfivesum`, write a buffer to a file with `\WriteBuffer`, and delete a globally defined buffer with `\ClearBuffer`.

2.2 Comparison with other approaches to saving text verbatim

fvextra's `VerbatimBuffer` environment and associated commands provide an efficient way to save and

doi.org/10.47397/tb/46-1/tb142poore-extools

LaTeX source

```
\newcommand{\after}{
 \begin{tcolorbox}[colback=black!10]
 \VerbatimInsertBuffer
 \end{tcolorbox}
 \InsertBuffer}

\begin{VerbatimBuffer}[afterbuffer=\after]
An example using the \textsf{minted}
package:
\begin{minted}{latex}
\usepackage{minted}
\end{minted}
\end{VerbatimBuffer}
```

Output

An example using the \textsf{minted} package:

```
\begin{minted}{latex}
\usepackage{minted}
\end{minted}
```

An example using the minted package:

```
\usepackage{minted}
```

Figure 1: An example using fvextra's new `VerbatimBuffer` environment to typeset LaTeX source along with its output, without using a temp file. The `VerbatimBuffer` environment saves its contents verbatim in a buffer, or sequence of numbered macros, with one macro per line. `afterbuffer` provides commands that are invoked after buffering is complete, before the end of the environment. In this case, `\VerbatimInsertBuffer` is used to typeset the buffer verbatim, with a frame provided by the tcolorbox [14] package. Then `\InsertBuffer` interprets the buffer as LaTeX input, using `\scantokens`. By default, buffers are only defined locally within `VerbatimBuffer` environments, so they are only accessible via `afterbuffer`. It is also possible to define buffers globally with the `globalbuffer` option, and have multiple global buffers by setting `buffername`. When a buffer is global, multiple `VerbatimBuffer` environments can be used to accumulate text within a single shared buffer.

reuse verbatim text. There are several other approaches to saving text that bring different tradeoffs for usability and performance.

A traditional way to save text verbatim and then process it multiple times is to write the text to a temp file, and then read this file multiple times as necessary. fancyvrb provides a `VerbatimOut` environment that can be paired with `\input` or fancyvrb's own `\VerbatimInput`. fvextra provides a `VerbatimWrite` environment that is similar to `VerbatimOut`, except that it allows multiple environments to share a temp file and it uses `\detokenize` to guarantee verbatim saving. The temp file approach is straightforward and can have excellent performance with a fast hard drive. The disadvantage is that it generates a temp file that must be cleaned up. Under some circumstances, using temp files can also require management of open files to avoid "No more room for a new `\write`" errors.

The alternative is storing the contents of an environment in one or more macros or token registers. The fancyvrb package provides a `SaveVerbatim` environment that saves its contents in a single macro for later use. Similarly, the scontents [3] package provides an `scontents` environment that saves its contents in a `seq` (sequence) datatype [15], which is defined in the LaTeX3 kernel. Both `SaveVerbatim` and `scontents` accumulate environment contents one line at a time, redefining the storage macro after each line to add the current line. Because of that constant redefinition of a single storage macro as lines are added, this approach is less efficient than `VerbatimBuffer` (Table 1). However, the `VerbatimBuffer` approach of one macro per line can make it more complex to change the scope of saved text from local to global or to duplicate the saved text. In cases when a single storage macro is desired, the best performance can likely be obtained by saving text with the `VerbatimBuffer` approach, and then converting the per-line storage macros into a single storage macro as a final step.

3 Assembling data and communicating it to Python with latex2pydata

After the minted package saves code to an in-memory buffer using `VerbatimBuffer`, this code along with settings must be communicated to Python for syntax highlighting. Code and settings are communicated via temp file. The latex2pydata package for LaTeX allows this data to be saved to a file using Python literal format. Then the latex2pydata package for Python can load this data for processing.

LaTeX source

```
\newcommand{\formatter}{}
\ExplSyntaxOn
\seq_new:N \l_color_match_seq
\NewDocumentCommand\lineprocessor{ m }
{
  \regex_extract_once:nnNTF {color:\s*(\w+)\s*;} {#1} \l_color_match_seq
    {\tl_set:Nx \l_color_name {\seq_item:Nn \l_color_match_seq {2}}
     \edef\formatter{\noexpand\color{\l_color_name}}
     \IterateBufferBreak}
    {}
}
\ExplSyntaxOff

\newcommand{\after}{
  \IterateBuffer{\lineprocessor}
  \VerbatimInsertBuffer[formatcom=\formatter]}

\begin{VerbatimBuffer}[afterbuffer=\after]
/* When this is typeset, it will be red */
div .class {
  color: red;
}
\end{VerbatimBuffer}
```

Output

```
/* When this is typeset, it will be red */
div .class {
  color: red;
}
```

Figure 2: An example using fvextra's new `VerbatimBuffer` environment to typeset text in a manner that depends on text content. A snippet of Cascading Style Sheets (CSS) is saved using a `VerbatimBuffer` environment. Then (`afterbuffer`), `\IterateBuffer` iterates over the saved lines while applying the `\lineprocessor` command to each one. `\lineprocessor` uses the l3regex [17] package to search for a color name in each line. If a color name is found, it is saved in `\formatter`, and iteration is halted via `\IterateBufferBreak`. Finally, the buffer is typeset verbatim with `\VerbatimInsertBuffer`. `\formatter` is applied to the typeset text as a format command (`formatcom`), so that the typeset text will have the color that was extracted from the buffer.

3.1 latex2pydata package for LaTeX

The latex2pydata package for LaTeX serializes data using Python literal format. Python literal format resembles JSON [2] for numerical, string, list, and dictionary data types. For example, a dictionary mapping a string "key" to a string "value" can be represented like this: `{"key": "value"}`.

Python literal format is ideal for working with LaTeX for two reasons. First, it supports multi-line string literals, so it is possible to write the contents of a LaTeX environment to a file one line at a time, without having to accumulate the entire environment contents within a single macro or otherwise access it all at once. Second, Python literal format only requires that quotation marks and backslashes be escaped within typical strings, so implementing serialization in LaTeX is straightforward and efficient. In contrast, JSON lacks multi-line string literals and requires that several additional characters be escaped.

While Python literal format supports arbitrary nesting of data structures, the latex2pydata package currently limits the raw data that it transmits to two forms:

- `dict[str, str]`
- `list[dict[str, str]]`

Table 1: Performance comparison for several methods of saving text verbatim and then typesetting the saved text. Environments were used to save lines of text consisting of 80 "X" characters, and then the saved text was typeset verbatim. Compile times are for documents using a pre-compiled preamble containing fvextra, fancyvrb, and scontents, so that results do not depend on individual package load times. Compile times are averages of 10 compilations on a Mac mini M4 using pdflatex from TEX Live 2024. For larger numbers of lines, fvextra's `VerbatimBuffer` has better performance than fancyvrb's `SaveVerbatim` or scontents's `scontents` because it saves each line of text in its own macro, rather than accumulating all text in a single macro that is redefined after each line to append the current line. The difference between using fvextra's `VerbatimBuffer` and a temp file is typically negligible.

Package: saving environment + typesetting command	100 lines	1000 lines	5000 lines
fvextra: `VerbatimBuffer` + `\VerbatimInsertBuffer`	53 ms	80 ms	209 ms
fvextra: `VerbatimWrite` + `\VerbatimInput` (temp file)	51 ms	81 ms	200 ms
fancyvrb: `SaveVerbatim` + `\UseVerbatim`	55 ms	328 ms	6593 ms
fancyvrb: `VerbatimOut` + `\VerbatimInput` (temp file)	53 ms	80 ms	192 ms
scontents: `scontents` + `\typestored`	70 ms	857 ms	18442 ms

That is, a dictionary mapping strings to strings, or a list of such dictionaries. `dict[str, str]` is a natural way to represent a LaTeX command or environment with its arguments and associated settings, and `list[dict[str, str]]` is a natural way to represent a sequence of commands and environments. These restrictions on raw data types simplify implementation and allow the package to perform basic data checking within LaTeX. As a byproduct, the subset of the Python literal format that is used for serializing data is sufficiently simple that it would be straightforward to process the data with programming languages other than Python.

It is possible to transmit other basic Python data types, including arbitrarily nested data structures, by combining the latex2pydata package for LaTeX with the latex2pydata package for Python (Section 3.2).

3.1.1 Writing data directly to a file

The latex2pydata package allows data to be written directly to a file. First, a file must be specified. It is possible to use an existing file handle, or provide a filename and let latex2pydata automatically generate and manage a file handle:

- `\pydatasetfilehandle{`⟨*filehandle*⟩`}`
- `\pydatasetfilename{`⟨*filename*⟩`}`

Once the file is ready, data can be written. There are several commands for starting and ending dictionary and list data types:

- `\pydatawritedictopen`
- `\pydatawritedictclose`
- `\pydatawritelistopen`
- `\pydatawritelistclose`

After a dictionary has been opened, it is possible to write key-value data. This can be done all at once, or the key and value can be written individually:

- `\pydatawritekey{`⟨*key*⟩`}`
- `\pydatawritevalue{`⟨*value*⟩`}`
- `\pydatawritekeyvalue{`⟨*key*⟩`}{`⟨*value*⟩`}`

All of these commands read their arguments verbatim, and can take either brace-delimited arguments (as shown above) or arguments delimited by a repeated character (like `\verb`). This simplifies the process of writing literal strings to a file. There is also `\pydatawritekeyedefvalue{`⟨*key*⟩`}{`⟨*value*⟩`}`, which treats the value as macros that are fully expanded (`\edef`) before writing.

Keys are limited to single-line strings, but multi-line values are supported, since it must be possible to transmit the contents of an environment. There are two ways to write a multi-line value. First, the `\pydatawritekey` commmand can be used to write a key, and then the `pydatawritemlvalue` environment will write its contents as the corresponding value. Second, after a key is written with `\pydatawritekey`, the value can be generated with three commands that manually manage the value delimiters and content:

- `\pydatawritemlvalueopen`
- `\pydatawritemlvalueline{`⟨*line*⟩`}`
- `\pydatawritemlvalueclose`

`\pydatawritemlvalueline` is used for each line of the value. Like other `value` commands, it reads its argument verbatim.

As data is written to a file, latex2pydata tracks the data state. Any attempt to write dictionary or list delimiters when they are not allowed, or failure

to write keys and values in correctly ordered pairs, immediately results in an error that halts compilation. This can prevent clearly invalid data from being passed to external software.

3.1.2 Accumulating data in memory

latex2pydata is designed to communicate data to external software, which will then typically generate some sort of output for inclusion in the document. If the external software implements caching, running the external software may only be necessary once for any given data input. latex2pydata provides features to simplify cache implementation.

Instead of writing keys and values directly to an external file, latex2pydata can accumulate them in a "buffer", a sequence of numbered macros each of which contains one line of serialized data. The latex2pydata buffers are implemented in exactly the same way as the fvextra buffers, so they are fully compatible. First, a buffer must be specified with `\pydatasetbuffername{`⟨*buffername*⟩`}`. Then keys and values can be added with commands similar to those for writing to a file:

- `\pydatabufferkey{`⟨*key*⟩`}`
- `\pydatabuffervalue{`⟨*value*⟩`}`
- `\pydatabufferkeyvalue{`⟨*key*⟩`}{`⟨*value*⟩`}`
- `\pydatabufferkeyedefvalue{`⟨*key*⟩`}{`⟨*value*⟩`}`

The `pydatabuffermlvalue` environment is available for multi-line values, or they may be managed manually:

- `\pydatabuffermlvalueopen`
- `\pydatabuffermlvalueline{`⟨*line*⟩`}`
- `\pydatabuffermlvalueclose`

Once key-value data is buffered, its MD5 hash can be calculated with `\pydatabuffermdfivesum`, which is fully expandable and uses `\pdf@mdfivesum` from pdftexcmds. When a given data set is first communicated to an external program, this MD5 hash can be used to generate the filename for output that is based on this data. Then, during subsequent LaTeX runs, the MD5 hash of buffered data can be used to check for cached output.

If key-value data is buffered but cached output does not exist, then the data can be written to a previously configured file with `\pydatawritebuffer`, and this file can be processed with external software. Because buffered data contains only keys and values, any necessary list and dictionary opening delimiters must be written before the buffer, and then any companion list and dictionary closing delimiters must be written afterward.

It is possible to write additional key-value pairs before or after the buffered data is written. For example, the minted package buffers all key-value data that is necessary to perform syntax highlighting, including the code that is to be highlighted. The MD5 hash of this buffer is used to check for a cached version of the highlighted code. If there is no cached output, then minted writes this buffered data to a file. However, it also writes additional information, including the current LaTeX source filename and line number. This information is not needed to highlight the code, so it is not buffered or included in the hash. Instead, this information is useful during the actual highlighting process, since it makes possible error messages that include full tracebacks to the LaTeX source where data originated.

Figure 3 provides an example that demonstrates data buffering, hashing, and writing.

3.2 latex2pydata package for Python

Data saved with the latex2pydata LaTeX package can be loaded with Python's ast [10] module, which provides a `literal_eval()` function. Data can be read from a file, then deserialized with `literal_eval()` into Python data types. Because `literal_eval()` is limited to evaluating Python literal data types, this is a safe operation; in contrast with Python's regular `eval()` function, arbitrary code execution is not permitted with `literal_eval()`. The latex2pydata LaTeX package serializes all values as strings, so only string values are possible if ast is used for deserialization.

Non-string value types are possible when the latex2pydata LaTeX package's schema capabilities are combined with the latex2pydata Python package. A schema is defined by setting a value type for each key: `\pydatasetschemakeytype{`⟨*key*⟩`}{`⟨*type*⟩`}`. Once a complete schema has been defined, it can be written to the data file with `\pydatawritemeta`, which writes all defined metadata including the schema. When the latex2pydata Python package loads a data file, it also extracts the schema if it exists. For each key-type pair in the schema, the corresponding key-value pair in the data is retrieved, and the value is evaluated as a Python literal. If this does not yield a Python object with the specified type, then an error is raised.

If data values are primarily strings, with only a few values that need to be evaluated as Python literals, defining a full schema could be tedious. `\pydatasetschemamissing{`⟨*fallback*⟩`}` defines the fallback behavior for values that are not defined in the schema. By default, if a schema is defined, then a missing value results in an error. It is possible instead to leave missing values unmodified as strings,

LaTeX source

```
\pydatasetbuffername{exbuff}
\pydatabufferkeyvalue{key1}{value1}
\pydatabufferkey{key2}
\begin{pydatabuffermlvalue}
multi-line
value 2
\end{pydatabuffermlvalue}

\pydatasetfilename{data.txt}
\pydatawritedictopen
\pydatawritebuffer
\pydatawritekeyvalue{key3}{value3}
\pydatawritedictclose
\pydataclosefilename{data.txt}

MD5 hash of the buffered data:\\
\texttt{\pydatabuffermdfivesum}
\par\vspace{2em}
Data written to \texttt{data.txt}:
\VerbatimInput{data.txt}
```

Output

MD5 hash of the buffered data:
77C72E0435DB94721D3034AF0E668434

Data written to `data.txt`:

```
{
"key1": "value1",
"key2":
"""\
multi-line
value 2
""",
"key3": "value3",
}
```

Figure 3: An example of data buffering, hashing, and writing with the latex2pydata LaTeX package. After a buffer name is specified, two pairs of key-value data are added to the data buffer. Then this is written to a file `data.txt`, along with an additional key-value pair that is not buffered. Finally, the MD5 hash of the buffered data is displayed, along with the content of `data.txt`. The data saved in `data.txt` is serialized in the Python literal format. This can be loaded using Python's ast module or with the latex2pydata Python package, which provides enhanced features including key paths, schema support, and non-string value types.

or to evaluate them as Python literals and accept any data type for the resulting value.

The latex2pydata Python package also provides a convenient shorthand for nested dictionaries. Dictionary keys received from LaTeX are restricted to ASCII alphanumeric characters plus the underscore, and then periods are interpreted as separators between elements in a key path. Thus, setting a key `main.sub` to `value` within LaTeX will result in the nested dictionary `{"main": {"sub": "value"}}` when the data is loaded with the latex2pydata Python package.

Figure 4 provides an example with a schema and key paths.

4 Creating LaTeX-compatible Python executables with latexrestricted

After the minted package saves code along with settings to a temp file, it launches a Python executable called `latexminted` via `\ShellEscape` [16] to perform syntax highlighting. The Python executable needs to determine the location of the temp file based on the LaTeX configuration. Then it must read the temp file, perform syntax highlighting, and save the highlighted output in a manner that is consistent with LaTeX security settings.

The new latexrestricted Python package provides access to LaTeX configuration, as well as access to the file system and subprocesses that is restricted in a manner consistent with this configuration. The result is a Python executable that meets the requirements for LaTeX restricted shell escape.

(These settings are technically implemented by the TeX binaries, not the LaTeX format, but we'll ignore that since the packages described here are for LaTeX.)

4.1 Access to LaTeX configuration

The latexrestricted Python package provides access to the LaTeX configuration through the `latex_config` object, which is an instance of its `LatexConfig` class. Some of the key features of `latex_config` are summarized below.

The `latex_config` object has several properties related to directories.

- `latex_config.tex_cwd` is the current working directory at the time latexrestricted was imported and initialized. This makes it easy to change

LaTeX source that saves data to a file

```
\pydatasetfilename{schema_keypath_data.txt}

\pydatasetschemakeytype{string}{str}
\pydatasetschemakeytype{integer}{int}
\pydatasetschemakeytype{list_of_bools}{list[bool]}
\pydatasetschemamissing{evalany}
\pydatawritemeta

\pydatawritedictopen
\pydatawritekeyvalue{string}{"12345"}
\pydatawritekeyvalue{integer}{12345}
\pydatawritekeyvalue{list_of_bools}{[True, False, True]}
\pydatawritekeyvalue{keypath.subkey_set_of_ints}{{1, 2, 3}}
\pydatawritekeyvalue{keypath.subkey_none}{None}
\pydatawritedictclose

\pydataclosefilename{schema_keypath_data.txt}
```

Python console that loads and displays saved data

```
>>> import latex2pydata
>>> import pathlib
>>> import pprint
>>> data = latex2pydata.load(pathlib.Path('schema_keypath_data.txt'))
>>> pprint.pprint(data, sort_dicts=False)
{'string': '12345',
 'integer': 12345,
 'list_of_bools': [True, False, True],
 'keypath': {'subkey_set_of_ints': {1, 2, 3}, 'subkey_none': None}}
```

Figure 4: An example of a schema and key paths in communicating data from LaTeX to Python (grayscaled for print). By default, all values are communicated as strings. If a schema is defined, then each value is instead evaluated as a Python literal, and the resulting data type is checked against the schema. It is possible to avoid creating a complete schema by using `\pydatasetschemamissing{`⟨*fallback*⟩`}` to specify fallback behavior for data values that are missing from the schema. The default `error` setting raises an error. The example uses `evalany`, which evaluates all missing values as Python literals and accepts any data type as the result. There is also `verbatim`, which keeps any missing values as the verbatim text that was received from LaTeX, without any evaluation.

the current working directory within part of a Python executable, and then later return to the LaTeX working directory.

- `latex_config.texlive_bin` is the path to the TeX binary directory, if the Python executable is launched via `\ShellEscape` under TeX Live. If MiKTeX is used instead, then the binary directory will be in `latex_config.miktex_bin`.

There are several properties that summarize LaTeX security settings for file system access:

- `latex_config.can_read_dotfiles`
- `latex_config.can_read_anywhere`
- `latex_config.can_write_dotfiles`
- `latex_config.can_write_anywhere`

For TeX Live, these summarize the following variables in `texmf.cnf`:

- `openin_any`
- `openout_any`

For MiKTeX, these `latex_config` properties summarize the following variables in `miktex.ini`:

- `[Core]AllowUnsafeInputFiles`
- `[Core]AllowUnsafeOutputFiles`

The `*dotfile` properties describe whether dot files (filename begins with ".") can be read/written. The `*anywhere` properties describe whether files anywhere in the file system can be read/written, or only those under the current working directory, `TEXMFOUTPUT`, and `TEXMF_OUTPUT_DIRECTORY`. (The `TEXMFOUTPUT`

environment variable defines a fallback location if the current directory is not writable. With TeX Live, it can also be defined in `texmf.cnf`. The `TEXMF_OUTPUT_DIRECTORY` environment variable is defined by TeX Live when the `-output-directory` command-line option is used.)

There are also properties that provide access to settings that may come from either the LaTeX configuration or environment variables.

- `latex_config.TEXMFHOME` contains the value of the corresponding variable obtained from `kpsewhich` or `initexmf`.
- `latex_config.TEXMFOUTPUT` contains the value of the corresponding environment variable if it is defined, and otherwise the value obtained from `kpsewhich` or `initexmf`.
- `latex_config.TEXMF_OUTPUT_DIRECTORY` contains the value of the corresponding environment variable if it is defined.

4.2 Restricted file system access

Python provides multiple ways to read and write files, including the `open()` function and the pathlib [12] module. pathlib provides a `Path` class for representing paths. For example, `p = Path('doc.tex')` would create an object `p` that represents the path, and then `p.read_text()` would read the file, decode the bytes with the current locale encoding, and return the resulting string of text.

The latexrestricted library implements subclasses of pathlib's `Path` class that obey LaTeX security settings, which are accessed through the `latex_config` object (Section 4.1). If a Python executable only interacts with the file system through latexrestricted's `RestrictedPath` classes, then errors are raised for any attempts to access the file system in ways that violate LaTeX security settings.

latexrestricted provides subclasses of `Path` that implement LaTeX security settings in different ways. The two major subclasses are summarized below. There are several additional subclasses that provide more stringent restrictions, for cases when more limited access is sufficient.

4.2.1 `StringRestrictedPath`

The `StringRestrictedPath` class follows the approach taken in LaTeX's own file system security. File paths are analyzed as strings to determine whether they can be read or written; the file system is never consulted. When read/write locations are restricted by LaTeX configuration, paths are restricted using the following criteria:

- Relative paths are relative to the LaTeX working directory.
- Absolute paths must be under `TEXMFOUTPUT` or `TEXMF_OUTPUT_DIRECTORY`.
- Paths cannot contain "`..`" to access a parent directory, even if the parent directory is a valid location.

When read/write locations are restricted, it is still possible to access locations elsewhere in the file system, if there are symlinks in the permitted locations that go elsewhere.

Under Windows (including Cygwin), writing files with file extensions in `PATHEXT` (for example, "`.exe`") is also disabled. All `RestrictedPath` classes have this restriction, but you can create custom subclasses that relax or eliminate this.

All `RestrictedPath` classes provide methods for representing common LaTeX directories. For example, `StringRestrictedPath.tex_cwd()` returns a path object representing the LaTeX working directory. There are similar methods for `TEXMFOUTPUT` and `TEXMF_OUTPUT_DIRECTORY`.

4.2.2 `ResolvedRestrictedPath`

The `ResolvedRestrictedPath` class interacts with the file system instead of analyzing paths as strings. It resolves any symlinks and "`..`" references in paths using the file system before determining whether paths are valid locations for reading/writing. When read/write locations are restricted, paths are restricted using the following criteria:

- Resolved paths must be under the LaTeX working directory, resolved `TEXMF_OUTPUT_DIRECTORY`, or resolved `TEXMFOUTPUT`.
- Relative paths are resolved relative to the LaTeX working directory.

Unlike `StringRestrictedPath`, paths are allowed to contain "`..`", and `TEXMF_OUTPUT_DIRECTORY` and `TEXMFOUTPUT` can be accessed via relative paths.

Because paths are resolved before being compared with permitted locations, it is not possible to access locations outside the LaTeX working directory, `TEXMF_OUTPUT_DIRECTORY`, and `TEXMFOUTPUT` via symlinks in those locations. Since this is more secure, minted uses `ResolvedRestrictedPath` by default. There is a minted configuration setting for switching to `StringRestrictedPath` when additional symlink flexibility is desired.

4.3 Restricted subprocesses

Python's subprocess [13] module provides a `run()` function for launching subprocesses. This is not compatible with LaTeX security settings for restricted shell escape because it permits any executable, not just executables designated as trusted in LaTeX configuration. (Trusted executables are designated in

TEX Live's `shell_escape_commands` in `texmf.cnf` or MiKTEX's `[Core]AllowedShellCommands[]` in `miktex.ini`.)

latexrestricted defines `restricted_run()`. This serves as a wrapper around `run()` that enforces LATEX security settings, as obtained from the `latex_config` object. It is always possible for `restricted_run()` to invoke `kpsewhich` (as well as `miktex-kpsewhich`) and `initexmf`. These are necessary to access LATEX configuration values, so it is not practical to prohibit them. If shell escape is enabled (restricted or full), then it is also possible to run permitted restricted shell escape executables. Permitted executables must be located in the same directory as `kpsewhich` (TEX Live) or `initexmf` (MiKTEX), or be on `PATH`. Executables must not be in locations writable by LATEX. For added security, locations writable by LATEX cannot be under an executable's parent directory, to guard against executable resources or configuration being written by LATEX. Additional restrictions are detailed in the latexrestricted documentation.

`restricted_run()` forces subprocesses to run with Python's subprocess setting `shell=False`. This launches child processes without calling a system shell. As a result, there is no need to quote or escape the arguments passed to the child process, and an entire category of potential vulnerabilities is avoided.

5 Conclusion

As a byproduct of the development of minted version 3, I created new tools for package authors that fall into three categories. The fvextra LATEX package has new capabilities for saving and processing the contents of environments in memory, without the use of temp files. This is useful in typesetting and also in communicating verbatim text to Python or other external software. The new latex2pydata LATEX and Python packages allow key-value data to be assembled within LATEX and then communicated to Python. The latexrestricted Python package simplifies the process of creating Python executables that respect LATEX security settings for file system access and subprocesses.

Together, these packages significantly simplify the process of creating LATEX packages that interface with external Python software. I hope that they can also serve as a template and inspiration for future LATEX packages involving other programming languages.

References

[1] G. Brandl, Pygments contributors. Pygments: Python syntax highlighter, version 2.19.1, 2025. `pygments.org`

[2] Ecma International. ECMA-404: The JSON Data Interchange Syntax, 2017. `ecma-international.org/publications-and-standards/standards/ecma-404`

[3] P. González. The scontents package, version 2.1, 2024. `ctan.org/pkg/scontents`

[4] H. Oberdiek and Oberdiek Package Support Group. The pdftexcmds package, version 0.33, 2020. `ctan.org/pkg/pdftexcmds`

[5] G.M. Poore. latexrestricted — Python library for creating executables compatible with LATEX restricted shell escape, version 0.6.2, 2024. `pypi.org/project/latexrestricted`

[6] G.M. Poore. latex2pydata — load data from LATEX in Python literal format, version 0.5.0, 2025. `pypi.org/project/latex2pydata`

[7] G.M. Poore. The minted package, version 3.6.0, 2025. `ctan.org/pkg/minted`

[8] G.M. Poore. The fvextra package, version 1.12.0, 2025. `ctan.org/pkg/fvextra`

[9] G.M. Poore. The latex2pydata package, version 0.5.0, 2025. `ctan.org/pkg/latex2pydata`

[10] Python Software Foundation. ast — Abstract Syntax Trees, 2024. `docs.python.org/3/library/ast.html#ast.literal_eval`

[11] Python Software Foundation. Lexical analysis, 2024. `docs.python.org/3/reference/lexical_analysis.html#literals`

[12] Python Software Foundation. pathlib — Object-oriented filesystem paths, 2025. `docs.python.org/3/library/pathlib.html`

[13] Python Software Foundation. subprocess — Subprocess management, 2025. `docs.python.org/3/library/subprocess.html`

[14] T. Sturm. The tcolorbox package, version 6.4.1, 2024. `ctan.org/pkg/tcolorbox`

[15] The LATEX Project. The l3seq module, version 2025-01-18, 2025. `ctan.org/pkg/l3kernel`

[16] The LATEX Project Team. The shellesc package, version 1.0d, 2023. `ctan.org/pkg/shellesc`

[17] The LATEX3 Project. The l3regex package, version 2024-12-09, 2024. `ctan.org/pkg/l3regex`

[18] T. Van Zandt, H. Voss. The fancyvrb package, version 4.5c, 2024. `ctan.org/pkg/fancyvrb`

◇ Geoffrey M. Poore
1050 Union University Dr
Jackson, TN 38305 USA
gpoore (at) gmail dot com
https://gpoore.github.io/

Piping YAML files into LaTeX3 key–values

Vít Starý Novotný, Marei Peischl

Abstract

In this article, we show how YAML files can be used with LaTeX3 key–values to set document metadata and configure LaTeX packages.

Introduction

While TeX documents may look pretty on paper, their sources can be challenging to understand for both authors and external tools. By incorporating structured markup languages — such as Markdown for text and YAML for metadata — we can improve human- and machine-readability, enhance text editor support, and enable reliable conversions to other formats.

In our previous articles, we demonstrated how the Markdown package for TeX can process YAML files. For simple files with regular structures, the `jekyllDataRenderers` option [2, Section 2.1] offers a basic parser that prioritizes ease of setup over fine control. Conversely, more complex or irregular YAML files can be managed with a low-level, event-based parser [4]. However, there was a notable gap in structured approaches for parsing medium-to-high-complexity YAML files that followed regular patterns.

Outline

In version 3.10.0 of the Markdown package, we introduced the `jekyllDataKeyValue` option to pipe YAML metadata into LaTeX3 key–values [5]. Like the `jekyllDataRenderers` option, it offers a declarative method for defining how different file segments are processed without the need to manage low-level parsing details. However, unlike `jekyllDataRenderers`, which allowed only a single definition for list items and which treated all scalars as strings, the new option `jekyllDataKeyValue` allows users to target specific items within lists and differentiate between scalar data types.

Since the detailed behavior of this new option is already documented elsewhere [3], this article focuses on two practical examples instead. First, in Section 1, we illustrate the primary use case: piping YAML files to dedicated key–value modules. Then, in Section 2, we explore an unintended but compelling scenario in which a YAML file is piped to the root module. In this scenario, users can configure *any* existing LaTeX3 package that has a key–value interface from a single YAML config file.

1 Piping to a dedicated module

Following a long-standing tradition [1, 2], we start by a simple LaTeX document with YAML metadata.

```
\documentclass[a5paper]{article}
\usepackage{markdown}
```

First, we define a LaTeX3 key–value module `example/module` that processes YAML metadata:

```
\ExplSyntaxOn
\str_new:N \g_example_title_str
\seq_new:N \g_example_authors_seq
\keys_define:nn
  { example / module }
  {
```

The key–value stores the raw document title in a variable `\g_example_title_str` and passes the formatted title to the command `\title`:

```
    title / programmaticString .str_gset:N
      = \g_example_title_str,
    title .code:n = { \title { #1 } },
```

Furthermore, the key–value also stores a variable-length list of authors in a sequence variable titled `\g_example_authors_seq`:

```
    authors / unknown .code:n = {
      \seq_put_right:Nn
        \g_example_authors_seq
        { #1 }
    },
  }
```

Next, we define a Markdown snippet `metadata` that pipes the YAML metadata to the previously defined LaTeX3 key–value and typesets the title page:

```
\markdownSetupSnippet
  { metadata }
  {
    jekyllDataKeyValue = example / module,
    renderers = {
      jekyllDataEnd = {
```

Before typesetting the title page, the snippet also passes the comma-delimited list of authors to the command `\author`:

```
        \exp_args:Nx
          \author {
            \seq_use:Nn
              \g_example_authors_seq
              { ,~ }
          }
        \AtBeginDocument { \maketitle }
      },
    }
  }
\ExplSyntaxOff
```


doi.org/10.47397/tb/46-1/tb142starynovotny-piping-yaml

Finally, we pass example YAML metadata to the previously-defined Markdown snippet and typeset the variable `\g_example_title_str` with the raw title:

```
\begin{yaml}[snippet=metadata]
title: Some title that includes _markup_
authors: [Jane Doe, John Moe]
\end{yaml}
\begin{document}
The raw document title is: \itshape
\ExplSyntaxOn
\str_use:N \g_example_title_str .
\end{document}
```

Typesetting this document with LuaLaTeX produces the following result:

Some title that includes *markup*

Jane Doe, John Moe

March 21, 2025

The raw document title is: *Some title that includes _markup_.*

2 Flexible package configuration

In the preceding, we wrote `jekyllDataKeyValue = example/module` to restrict YAML files to a dedicated key–value module. By omitting the module name, we can provide access to *any* module, which allows us to configure many LaTeX3 packages.

First, we create a LaTeX document that loads several LaTeX3 packages with key–value interfaces:

```
\documentclass[a5paper]{article}
\usepackage{chemformula}
\usepackage{lua-widow-control}
\usepackage{phonenumbers}
```

Next, we start a Markdown document with a YAML metadata block that configures the packages:

```
\usepackage[jekyllData, jekyllDataKeyValue,
  rawAttribute, texMathDollars]{markdown}
\begin{document}
\begin{markdown}
---
chemformula:
  decimal-marker: $\cdot$
lua-widow-control:
  draft: true
phonenumbers:
  area-code: place
  area-code-sep: space
  country: UK
---
```



Finally, we show that the configuration has been applied by using commands from the packages:

```
Phone number:
`\phonenumber{02079460345}`{=latex}

Chemical reaction:
`\ch{A + B -> 3.14159}`{=latex}
\end{markdown}
\end{document}
```

Typesetting this document with LuaLaTeX produces the following result:

Phone number: London 7946 0345 infinite
Chemical reaction: A + B ⟶ 3·14159 infinite

(The light gray "infinite" at right is from the draft mode of `lua-widow-control`.)

While not all LaTeX packages support key–values, many do — check for `\keys_define` in the source!

Conclusion

TeX offers few guardrails. In this article, we demonstrated how the Markdown package for TeX can separate document metadata and package configuration into structured YAML files. This separation enhances general usability, enabling users to focus on content rather than on juggling TeX macros.

References

[1] V. Novotný, D. Rehák, et al. Markdown 2.15.0: What's new? *TUGboat* 43(1):10–15, 2022. doi.org/10.47397/tb/43-1/tb133novotny-markdown

[2] V. Novotný. Markdown 2.17.1: What's new, what's next? *TUGboat* 43(3):276–278, 2022. doi.org/10.47397/tb/43-3/tb135novotny-markdown

[3] V. Starý Novotný. Piping YAML metadata into LaTeX3 key–values. github.com/witiko/markdown/discussions/517

[4] V. Starý Novotný. Markdown themes in practice. *TUGboat* 45(2):211–220, 2024. doi.org/10.47397/tb/45-2/tb140starynovotny-markdown-themes

[5] J. Wright. Programming key–value in expl3. *TUGboat* 31(1):83–87, 2010. tug.org/TUGboat/tb31-1/tb97wright-l3keys.pdf

◇ Vít Starý Novotný
Studená 453/15
Brno 63800, Czech Republic
witiko (at) mail dot muni dot cz
github.com/witiko

◇ Marei Peischl
Gneisenaustr. 18
Hamburg 20253, Germany
marei (at) peitex dot de
peitex.de

dtxmacrofont: Resizable macrocode in doc implementations

John E. Beach

Abstract

macrocode environments in doc implementation text are typeset in the \small font size. The dtxmacrofont package allows the package author to select the font size through four declarations and reimplements \MacroFont and \AltMacroFont to typeset macrocode in the selected font size.

Description

macrocode environments are typeset using the font attributes and size declarations defined in \MacroFont. The default is small typewriter. A different font size may be selected using a \MacroFont⟨*Fontsize*⟩ declaration. The font attributes may be changed by redefining \MacroFont.

\MacroFont (*decl.*)

macrocode environments containing module code *might*, depending on the value in StandardModuleDepth, be typeset using the font attributes and size declarations defined in \AltMacroFont. The default for module code is *small slanted typewriter*. The font size is identical to that used by \MacroFont, i.e., at any given instant in time \MacroFont and \AltMacroFont will typeset macrocode using the same font size (but different font attributes).

\AltMacroFont (*decl.*)

As mentioned, the \MacroFont and \AltMacroFont font size is controlled using the declarations:

- \MacroFontSmall, the default.
- \MacroFontFootnotesize.
- \MacroFontScriptsize.
- \MacroFontTiny.

\MacroFontSmall (*decl.*)
\MacroFontFootnotesize (*decl.*)
\MacroFontScriptsize (*decl.*)
\MacroFontTiny (*decl.*)

The following macrocode block shows the number of columns available for each font size declaration. The line numbers are added during typesetting using the CodelineNo counter and are \scriptsize for \MacroFontSmall (two steps less) and \tiny otherwise.

```
%23456789 123456789 123456789 123456789 123456789 123456789 123456789 12
%======= \MacroFontSmall (72)
%23456789 123456789 123456789 123456789 123456789 123456789 123456789 1234567890
%======= \MacroFontFootnotesize (80)
%23456789 123456789 123456789 123456789 123456789 123456789 123456789 123456789 1
%======= \MacroFontScriptsize (91)
%23456789 123456789 123456789 123456789 123456789 123456789 123456789 123456789 123456789 123456789 12345678
%====== \MacroFontTiny (128)
```

Individual blocks of macrocode may also have their font size changed — the TEXnique used when typesetting the above macrocode. A \MacroFont⟨*Fontsize*⟩ declaration immediately before a block of macrocode resizes that block — just don't forget to restore the font size afterwards (or make the change local by enclosing it in braces):

```
%␣␣␣{\MacroFontFootnotesize
%␣␣␣␣\begin{macrocode}
%23456789␣123456789␣123456789␣123456789␣12345...
%=======␣\MacroFontFootnotesize␣(80)
%␣␣␣␣\end{macrocode}
%␣␣␣}␣%^^A␣\MacroFontFootnotesize
```


doi.org/10.47397/tb/46-1/tb142beach-dtxmacrofont

Implementation

```
⟨*package⟩
\NewDocElement[macrolike = true,                % Declaration
               toplevel  = true,
               idxtype   = \noexpand\textit{\DeclarationColor{}decl.},
               idxgroup  = Declarations,
               printtype = \textit{\DeclarationColor{}decl.}
              ]{Declaration}{declaration}
\NewDocumentCommand{\Declaration}    {m}{\texttt{\string#1}}
\NewDocumentCommand{\DeclarationNew}  {}{\NewDocumentCommand}
\NewDocumentCommand{\DeclarationUsage}{m}{%
    \DescribeDeclaration{#1}%
    \Declaration{#1}%
}
\NewDocumentCommand{\DeclarationColor}{}{\color{Black}}
```

`\MacroFont` (*decl.*) Font for typesetting `macrocode`.

```
\def\MacroFont{%
    \fontencoding\encodingdefault%
    \fontfamily\ttdefault%
    \fontseries\mddefault%
    \fontshape\updefault%
    \ifthenelse{\equal{\doc@MacroFontSize}{small}}{\small}%
   {\ifthenelse{\equal{\doc@MacroFontSize}{footnotesize}}{\footnotesize}%
   {\ifthenelse{\equal{\doc@MacroFontSize}{scriptsize}}{\scriptsize}%
   {\ifthenelse{\equal{\doc@MacroFontSize}{tiny}}{\tiny}%
   {\small%
   }}}}%
}
```

`\AltMacroFont` (*decl.*) Font for typesetting module code.

```
\def\AltMacroFont{%
    \fontencoding\encodingdefault%
    \fontfamily\ttdefault%
    \fontseries\mddefault%
    \fontshape\sldefault%
    \ifthenelse{\equal{\doc@MacroFontSize}{small}}{\small}%
   {\ifthenelse{\equal{\doc@MacroFontSize}{footnotesize}}{\footnotesize}%
   {\ifthenelse{\equal{\doc@MacroFontSize}{scriptsize}}{\scriptsize}%
   {\ifthenelse{\equal{\doc@MacroFontSize}{tiny}}{\tiny}%
   {\small%
   }}}}%
}
```

`\doc@MacroFontCurrent` (*decl.*) `\doc@MacroFontCurrent` keeps track of the font currently being used for typesetting. To allow the definitions of `\MacroFont` and `\AltMacroFont` to be changed in the document ⟨*preamble*⟩, the selection of the current font is deferred until the beginning of the document.

```
\AtBeginDocument{\let\doc@MacroFontCurrent\MacroFont}
```

`\doc@MacroFontSize` (*decl.*) `\doc@MacroFontSize` keeps track of the font size currently being used for typesetting.

```
\DeclarationNew{\doc@MacroFontSize}     {}{\small}%
```

`\MacroFontSmall` (*decl.*) Declarations to change the `\MacroFont` font size.

```
\DeclarationNew{\MacroFontSmall}        {}{%
    \def\doc@MacroFontSize{small}%
    \def\theCodelineNo{\rmfamily\scriptsize\arabic{CodelineNo}}%
}
```

\MacroFontFootnotesize (*decl.*)
```
\DeclarationNew{\MacroFontFootnotesize} {}{%
    \def\doc@MacroFontSize{footnotesize}%
    \def\theCodelineNo{\normalfont\tiny\arabic{CodelineNo}}%
}
```

\MacroFontScriptsize (*decl.*)
```
\DeclarationNew{\MacroFontScriptsize}   {}{%
    \def\doc@MacroFontSize{scriptsize}%
    \def\theCodelineNo{\normalfont\tiny\arabic{CodelineNo}}%
}
```

\MacroFontTiny (*decl.*)
```
\DeclarationNew{\MacroFontTiny}         {}{%
    \def\doc@MacroFontSize{tiny}%
    \def\theCodelineNo{\normalfont\tiny\arabic{CodelineNo}}%
}
```

Make `\small` `macrocode` be the default.

```
\MacroFontSmall
```

⟨/package⟩

References

[1] Frank Mittelbach. *The doc and shortvrb Packages*, pp. 25ff., macrocode line 105ff. `https://mirrors.ctan.org/macros/latex/base/doc.pdf`.

◇ John E. Beach
Columbus, Ohio, USA
`johnebeach (at) hot mail (dot) com`

datatool v3: Performance, localisation, databases and more

Nicola Talbot

Abstract

A major new release of datatool (version 3.0) has improved performance with added localisation support. This article gives an overview of key new features.

1 Introduction

In December 2022, following on from a major update to glossaries-extra (and intertwined mfirstuc, glossaries and bib2gls changes), I finally started a long-overdue rewrite of datatool [7]. A large part of the new code uses LaTeX3, although there are still a few old commands that haven't been updated. Two years later, in March 2025, I finally released datatool version 3.0. Unfortunately, I slipped up on the packaging and accidentally missed out the new files. Version 3.0.1 followed on soon after as a bit of a damp squib. I'm sorry for any disruptions caused by this.

The datatool bundle consists of the following:

- the main datatool package;
- the databar package for plotting bar charts;
- the datapie package for plotting pie charts;
- the dataplot package for plotting 2D line charts or scatter plots;
- the person package for mail merging or document templates (added 2009);
- the databib package (added in August 2007, but it's better to use biblatex [2] now);
- the datagidx package (added in January 2013, but it's better to use the \printnoidxglossary command provided with glossaries [9] now).

The underlying package that's used by all the above is datatool-base. This deals with parsing (to determine data type and numeric value), sorting, and providing an interface to lower-level arithmetical functions.

Originally, the commands provided by the fp package [3] were used to perform floating point arithmetic. In version 2.10, the math package option was added to switch from fp to pgfmath [4], if preferred. As of version 3.0, there are new options: math=l3fp (use l3fp functions) or math=lua (use \directlua). The new default is lua (if LuaLaTeX) or l3fp (otherwise).

2 Datum variables

In the context of datatool, a "plain number" is a value that consists of a sequence of digits 0–9, optionally prefixed by a single + or -, and (for decimals) using a decimal point to separate the integer part from the fractional part. For example, 82145 (integer) or 48271.3 (decimal).

A "formatted number" is a number that is formatted using the designated number group separator and decimal character, where the number may optionally have a currency symbol before or after it. The default is a group separator of comma and a decimal point. For example, 82,145 (integer) or 48,271.3 (decimal) or \$48,271.30 (currency).

Arithmetic commands that expect formatted numbers in the arguments, such as \DTLadd, need to parse the values, perform the appropriate calculation, and then format the result. If the result is used in another calculation, the formatted value would then have to be parsed.

With v3.0, the result is now stored as a "datum variable" which contains not only the formatted text but also the plain number value (if applicable). This reduces the amount of parsing required.

The new command \DTLparse may be used to parse some text, which may or may not be numerical. The result is a datum variable, and the component parts (formatted text, numeric value, currency symbol and data type) can all be extracted. This underlying function is used by all parsing commands.

For example, with the default settings:

```
\DTLparse\result{\$48,271.30}
Formatted: \result.
Currency symbol: \DTLdatumcurrency{\result}.
Value: \DTLdatumvalue{\result}.
Type:
\DTLgetDataTypeName{\DTLdatumtype{\result}}.
```

This produces:

> Formatted: $48,271.30. Currency symbol: $.
> Value: 48271.30. Type: currency.

The currency symbol needs to be declared. Common symbols are added by default. The following hides the currency symbol:

```
\newcommand{\mypound}{\textbf{\large\pounds}}
\DTLparse\result{\mypound 48,271.30}
Formatted: \result.
Currency symbol: \DTLdatumcurrency{\result}.
Value: \DTLdatumvalue{\result}.
Type:
\DTLgetDataTypeName{\DTLdatumtype{\result}}.
```

This produces:

> Formatted: **£**48,271.30. Currency symbol: .
> Value: . Type: string.

The value and currency symbol are both empty since the data is considered non-numeric, because the

doi.org/10.47397/tb/46-1/tb142talbot-datatool3

parser doesn't recognise `\mypound` as a currency symbol. `\result` will only expand as far as the formatted value. So if `\edef\tmp{\result}` was added to the end of the last example, `\mypound` would not be expanded. For a full expansion, you need `\DTLusedatum{`⟨*datum-var*⟩`}`.

If the original value needs reformatting (for example, the source the data was obtained from was a little untidy) then `\DTLparse` can be instructed to do this:

```
\DTLsetup{numeric={auto-reformat}}
\DTLparse\result{-\$12,345.328}\result.
\DTLparse\result{\$-12345.3289}\result.
```

This produces:

−$12,345.33. −$12,345.33.

This has adjusted the sign and rounded to two decimal places.

Scientific notation is now supported. If the auto-reformat setting is on and `siunitx` [11] has been loaded, then scientific notation will be reformatted with `\si`. The original value can again be extracted:

```
\DTLsetup{numeric={auto-reformat}}
\DTLparse\result{1.3e4}
Formatted: \result.
Value: \DTLdatumvalue{\result}.
```

This produces (assuming `siunitx` is also loaded):

Formatted: 1.3×10^4. Value: 1.3e4.

Although `datatool-base` is mostly concerned with parsing, there are also commands for converting from plain numbers to decimal or currency:

```
\DTLdecimaltocurrency{12345.6}{\price}
Price: \price.
\DTLdecimaltolocale{12345.6}{\decimalval}
Decimal: \decimalval.
```

This produces:

Price: $12,345.60. Decimal: 12,345.6.

These commands are used by the auto-reformat feature (but not for scientific notation).

3 Localisation

There is now also localisation support. This is separated into support for regions (currency and number formatting, see Section 3.1) and languages (see Section 3.2). Searching for the appropriate localisation files is dealt with via `tracklang` (see, for example, [5]). Therefore, make sure you also have an up-to-date version of `tracklang` [10].

The localisation support is a little more complex than for `datetime2` (for which `tracklang` was originally written). Suppose an English document that doesn't use either `babel` or `polyglossia` has:

```
\usepackage[locales={en-BE,en-GB,en-US}]
 {datatool}
```

This requires support for English in three regions: BE (Belgium), GB (United Kingdom) and US (United States). The packages `datatool-regions` [8] and `datatool-english` [6] are needed, and this document will load the region files `datatool-BE.ldf`, `datatool-GB.ldf`, and `datatool-US.ldf`, and the language file `datatool-english.ldf`.

In this case, there are no `\captions...` hooks, but each LDF file provides its own hook to switch in this situation:

- `\DTLenLocaleHook` (switch to English orthography);
- `\DTLBELocaleHook` (switch to Belgian number formatting and currency);
- `\DTLGBLocaleHook` (switch to UK number formatting and currency);
- `\DTLUSLocaleHook` (switch to US number formatting and currency).

If a language package has been loaded and `tracklang` has a mapping from its own internal labelling system to the dialect label used by the language support, then these hooks will be added to the captions hook.

Some regions with multiple official languages may have different number formatting according to the language. For example, Canada may have a decimal point in English but a decimal comma in French. This is dealt with by the region file checking for the existence of a function that combines both the language tag and region tag in its name. The applicable language bundle needs to supply support for that specific language and region combination.

For example, `datatool-english` provides the file `datatool-en-CA.ldf`. If French localisation support were to be developed, it would not only need to provide `datatool-french.ldf` but also `datatool-fr-CA.ldf` (for Canadian French), `datatool-fr-BE.ldf` (for Belgian French), etc.

However, for the most part, the language and region files are independent of each other, which is why `en-BE` may be specified as a locale, even though there's no "en-BE" LDF file.

Suppose now that you have a document where `tracklang` only detects the root languages:

```
\documentclass[french,dutch,german,english]{book}
\usepackage{babel}
\usepackage[locales={BE}]{datatool-base}
```

In this case, `datatool-base` will add the region `BE` to each of `tracklang`'s dialects that don't have an associated region. Now `\DTLenLocaleHook` can be added to `\captionsenglish` but the region hook will also be added.

If all tracked dialects already had an associated region, then `und-BE` would be added to the list of tracked dialects instead. In that case, the `\DTLBELocaleHook` would need to be used explicitly in order to switch to the settings for that region, since there's no applicable language hook.

In the following, the `babel` dialects already have an associated region so the `locales` option isn't necessary:

```
\usepackage{babel}
\babelprovide{english-ca,french-ca}
\usepackage{datatool-base}
```

3.1 Regions

Region support is provided by the package `datatool-regions`, which needs to be installed separately. There are currently only a limited number of available regions. More may be added via pull request on GitHub.[1] There is an interactive Perl script provided to create a file based on your responses.

Each region file can provide options specific to that region. These can be set with:

`\DTLsetLocaleOptions{`⟨*module*⟩`}{`⟨*key=value list*⟩`}`

If you want to make use of these options, then switch on auto-reformatting so that the style commands governed by those options can be included in the formatting.

For example, the GB region has a numeric style called "education" that will allow a normal space or a thin space as the number separator when parsing, but will use `\,` (thin space) when formatting. (The default is the "official" style, which uses a comma number group separator.)

```
\DTLsetup{numeric={auto-reformat}}
\DTLsetLocaleOptions{GB}{number-style=education}
```

The following switches to each region in turn, to parse and reformat according to that region:

```
\DTLGBLocaleHook
\DTLparse\priceGB{£12 345.239}
\DTLUSLocaleHook
\DTLparse\priceUS{\$54,321.989}
\DTLBELocaleHook
\DTLparse\priceBE{24.678,42€}
```

All regional-sensitive commands can be reset:

```
\DTLresetRegion
\DTLdecimaltocurrency{12345.6}{\price}
```

Now each of the earlier datum variables are displayed in the text along with their embedded value:

```
No region price: \price.
(Value: \DTLdatumvalue{\price}.)
BE price: \priceBE.
(Value: \DTLdatumvalue{\priceBE}.)
US price: \priceUS.
(Value: \DTLdatumvalue{\priceUS}.)
GB price: \priceGB.
(Value: \DTLdatumvalue{\priceGB}.)
```

This produces:

No region price: ¤12,345.60. (Value: 12345.6.)
BE price: 24.678,42€. (Value: 24678.42.)
US price: $54,321.99. (Value: 54321.989.)
GB price: £12 345.24. (Value: 12345.239.)

Changing regional settings won't have an effect unless or until the specified region is active:

```
\DTLUSLocaleHook
\DTLsetLocaleOptions{US}{
 currency-symbol-prefix,
 currency-symbol-sep=thin-space
}
Prefixed US price: \priceUS.

\DTLGBLocaleHook
\DTLsetLocaleOptions{GB}{number-style=official}
No change to GB price separator: \priceGB.
VAT: \DTLmul\vat{\priceGB}{0.2}\vat.
(Value: \DTLdatumvalue{\vat}.)
```

This produces:

Prefixed US price: US$ 54,321.99.
No change to GB price separator: £12 345.24.
VAT: £2,469.05. (Value: 2469.0478.)

While the options that affect the way the currency is displayed may alter content that has already been formatted, options that affect the number group and decimal characters won't change already formatted content. In general, it's best to set up all the localisation options in the preamble before any data parsing takes place.

3.2 Language

The language files not only provide fixed text translations (as is usual for language support) but also provide support for the language's alphabet when sorting. Old versions of `datatool-base` provided `\dtlsortlist` for sorting a comma-separated list according to a comparison handler function. Modern LaTeX kernels provide `\clist_sort:Nn` which is far more efficient than the old method used by `datatool-base`. With version 3.0, `\dtlsortlist` has been reimplemented to use `\seq_sort:Nn`. However, there is a new command that works better with localisation:

`\DTLsortwordlist{`⟨*clist-var*⟩`}{`⟨*handler-cs*⟩`}`

There is also a lower-level command for sequence variables (now used by `\printnoidxglossary`):

`\datatool_sortwordseq:NN` ⟨*seq-var*⟩ ⟨*handler-cs*⟩

[1] `github.com/nlct/datatool-regions`

In this case, the handler function isn't a comparison macro but is used to preprocess the sort value. It works along a similar principle as Java's collation keys (which is used by `bib2gls`). In the case of Java, the sort value is converted into a series of bits. Since TEX doesn't provide an array datatype, a token list is used instead, where each token's character code represents a subset of bits (eight for single-byte tokens).

Bear in mind that we are only concerned with the character code of each token. The token list is not intended to be typeset and may include control codes (with category code "other"). A sequence variable is then set so that each item consists of both the pseudo-byte array token list and the original value. The sequence is then sorted using a comparison function that extracts the token list and performs a simple character code comparison. Afterwards, the original values can be extracted from the sorted sequence.

There are four predefined handler functions:

`\DTLsortwordhandler{`⟨*original*⟩`}{`⟨*tl-var*⟩`}`
`\DTLsortwordcasehandler{`⟨*original*⟩`}{`⟨*tl-var*⟩`}`
`\DTLsortletterhandler{`⟨*original*⟩`}{`⟨*tl-var*⟩`}`
`\DTLsortlettercasehandler{`⟨*original*⟩`}{`⟨*tl-var*⟩`}`

Each function converts ⟨*original*⟩ into a pseudo-byte array that's stored in the token list variable ⟨*tl-var*⟩. First a protected expansion is applied to the original content. The letter handlers then strip all hyphens and spaces. They then all use the default handler that applies the localisation support. Finally, the result is purified to remove any formatting commands and the case-insensitive handlers convert to lowercase.

Each language file needs to redefine (in its hook):

`\DTLCurrentLocaleWordHandler{`⟨*tl-var*⟩`}`

to convert the content of the supplied token list variable. The trick here is to ensure that any letters or punctuation that may be used in the language are converted to one or more characters that reflect the relative order in that language's alphabet. Support for foreign letters may be useful, but since this is done by regular expression, the more complex the expression, the longer the processing time.

English is quite straight-forward. The basic Latin characters can remain as they are. Common letters from the Latin-1 Supplemental block can be converted to their unaccented counterpart. For example, è, é, ê and ë can be replaced with `e`, and æ can be replaced with `ae`. Punctuation characters are prefixed with 0x22. This ensures that they are all placed before any letters. Currency signs are all prefixed with 0x24, which keeps them all together. If the conversions result in identical sort values, a character code comparison of the original values will be used to differentiate them. For example:

```
\DTLenLocaleHook
\newcommand{\mylist}{résumé,zebra,ætherial,
resupply,resume}
\DTLsortwordlist{\mylist}{\DTLsortwordhandler}
\DTLformatlist{\mylist}.
```

ætherial, resume, résumé, resupply and zebra.

The language support can be reset. This shows the result if there's no localisation:

```
\DTLresetLanguage
\newcommand{\mylist}{résumé,zebra,ætherial,
resupply,resume}
\DTLsortwordlist{\mylist}{\DTLsortwordhandler}
\DTLformatlist{\mylist}.
```

This produces:

resume, resupply, résumé, zebra & ætherial.

Control codes can be useful if a diacritic or ligature is a distinct letter of the alphabet. For example, if "Æ" should come after "A" and before "B" then replace "Æ" with 0x41 0x7F.

The `datatool-english` bundle includes support for Old English (Anglo-Saxon) for both ang-Latn (Latin script) and ang-Runr (Runic script). This is mainly included to provide an example of how to support extended Latin and non-Latin alphabets. Here, "script" refers to what is used in the source code. For example, if a package is used that provides a command called, say, `\runic` where `\runic{fuþorc}` is rendered as runes in the PDF, then the script should be specified as Latin, not Runic, so the locale would need to be `ang-Latn`.

4 Databases

The `datatool` package deals with data arranged in rows and columns. This can either be defined within the document via commands or obtained from a CSV (or TSV) file. There are new commands for reading and writing:

`\DTLread[`⟨*options*⟩`]{`⟨*filename*⟩`}`
`\DTLwrite[`⟨*options*⟩`]{`⟨*filename*⟩`}`

The older commands have been redefined in terms of these new commands. The `format` option specifies the file type. For example, `csv` for CSV files or `tsv` (which will locally change the category code of the Tab character). The `csv-content` option may be: `tex` (data is valid LATEX syntax) or `literal` (literal content that will require converting special characters like `$` to commands) or (v3.2+) `no-parse` (data should not be parsed, the fastest setting for text-only content).

If the data contains numeric values that may need plotting or aggregating, switch on the "store-datum" option to store each item in the datum format for easy access to the numeric value without having to constantly re-parse. For example, to handle a CSV file that has one column with `\$` prices and another with `£` prices:

```
\usepackage[locales={en-GB,en-US}]{datatool}
\DTLsetup{store-datum}
\DTLread[format=csv,csv-content=tex,name=data]
 {profits.csv}
```

If the numeric values are plain numbers (with no currency symbols) then use the `csv-content=no-parse` option. You will also need `data-types`. If omitted, the data will be considered non-numeric. For example, if all columns should be considered decimals:

```
\DTLsetup{store-datum}
\DTLread[format=csv,name=data,
 csv-content=no-parse,data-types={decimal}]
{results.csv}
```

The old database sorting command `\dtlsort` has been rewritten, but there is a newer command analogous to `\DTLsortwordlist`:

`\DTLsortdata[`⟨*options*⟩`]{`⟨*db-name*⟩`}{`⟨*criteria*⟩`}`

Some commands have quite lengthy names and need to have the database name supplied in an argument. This can be quite tiresome, so a new general purpose action command is provided:

`\DTLaction[`⟨*options*⟩`]{`⟨*action-name*⟩`}`

The default database name can be set:

```
\DTLsetup{default-name=products,
 store-datum,numeric=auto-reformat}
```

The "products" database can now be created in the document:

```
\DTLaction{new}% same as \DTLgnewdb{products}
\DTLaction[assign={Product={Élite Book},
  Price={\$2,345}, Quantity=4}]{new row}
\DTLaction[assign={Product={Elephant Puzzles},
  Price={\$2.99}, Quantity=20}]{new row}
\DTLaction[assign={Product={Zebra Print},
  Price={\$2.99}, Quantity={1,500}}]{new row}
```

The "sort" action internally uses `\DTLsortdata` and the "display" action internally uses `\DTLdisplaydb`:

```
\begin{table}
\caption{Price then Quantity Descending}
\label{tab:productprice}
\centering
\DTLaction
 [assign={Price=desc,Quantity=desc}]{sort}
\DTLaction{display}
\end{table}
```

The result is shown in Table 1. For long tables, use "display long" instead, which will load and use the `longtable` package instead of `tabular` [1].

Table 1: Price then Quantity Descending

Product	Price	Quantity
Élite Book	$2,345.00	4
Zebra Print	$2.99	1,500
Elephant Puzzles	$2.99	20

Table 2: Product Name Ascending

Product	Price	Quantity
Elephant Puzzles	$2.99	20
Élite Book	$2,345.00	4
Zebra Print	$2.99	1,500

In Table 1, the order is numerical as the "Price" column has been identified as currency by the parser. String columns will be sorted using the current localisation setting. For example,

```
\begin{table}
\caption{Product Name Ascending}
\label{tab:productname}
\centering
\DTLaction[assign={Product}]{sort}
\DTLaction{display}
\end{table}
```

This produces Table 2 (assuming English localisation has been set up).

References

[1] D. Carlisle. The `longtable` package, 2024. `ctan.org/pkg/longtable`.

[2] P. Kime, M. Wemheuer, P. Lehman. The `biblatex` package, 2024. `ctan.org/pkg/biblatex`.

[3] M. Mehlich. The `fp` package, 1999. `ctan.org/pkg/fp`.

[4] PGF/TikZ Team. The `pgf` package, 2023. `ctan.org/pkg/pgf`.

[5] N. Talbot. Localisation of TEX documents: `tracklang`. *TUGboat* 37(3):337–351, 2016. `tug.org/TUGboat/tb37-3/tb117talbot.pdf`

[6] N. Talbot. The `datatool-english` bundle, 2025. `ctan.org/pkg/datatool-english`.

[7] N. Talbot. The `datatool` package, 2025. `ctan.org/pkg/datatool`.

[8] N. Talbot. The `datatool-regions` bundle, 2025. `ctan.org/pkg/datatool-regions`.

[9] N. Talbot. The `glossaries` package, 2025. `ctan.org/pkg/glossaries`.

[10] N. Talbot. The `tracklang` package, 2025. `ctan.org/pkg/tracklang`.

[11] J. Wright. The `siunitx` package, 2025. `ctan.org/pkg/siunitx`.

◇ Nicola Talbot
`dickimaw-books.com`

SpeLaTeX — Speech-enabled LaTeX

Walter Daems and Paul Levrie

Abstract

Text-to-speech software has existed for quite some time. However, it still struggles to properly read mathematical expressions and equations. The SpeLaTeX package, in conjunction with a Perl script, allows you to transform your text into a document with a hyperlink for every chunk of text, equation, table or figure. These hyperlinks, when clicked, cause your media player to read the content out loud, coping properly with mathematics and graphics.

The package targets an audience with reading disorders, as well as people looking for help in focusing on the text by having it read out loud while also reading it silently.

At present, the package does not rely on PDF tags for practical reasons, but that is probably where it is heading in the long term.

1 Introduction

The prevalence of significant reading disorders is estimated between 2% and 20% across studies. The value depends heavily on the definition used for the disorder, and the threshold that is used to judge whether they are significant [4].

At our institute, the University of Antwerp in Belgium, the number of students with reading and/or writing disorders (or at least are aware of these disorders) is increasing. Approximately 5% of the students are registered with such a disorder and the number has been rising steadily over the past 10 years. This number is probably an underestimation as some students choose not to register their disorder.

A large portion of the study materials we offer to students is still written material. We believe that this will continue to be the case, even given the multimedia and AI options that have become mainstream. Let's not go into this debate for now. However, written course texts are suboptimal for students with reading disorders.

For small texts, reading them out loud and recording them using a voice recorder to create an aid for our target group, is still feasible. We have done that in the past for exam assignments. However, for bigger texts (like course syllabi) this is beyond the time a professor can afford spending on students. Yes, economic laws also govern teaching! SpeLaTeX solves this issue by transforming a LaTeX manuscript into a PDF with hyperlinks to automatically generated speech (audio) files [9].

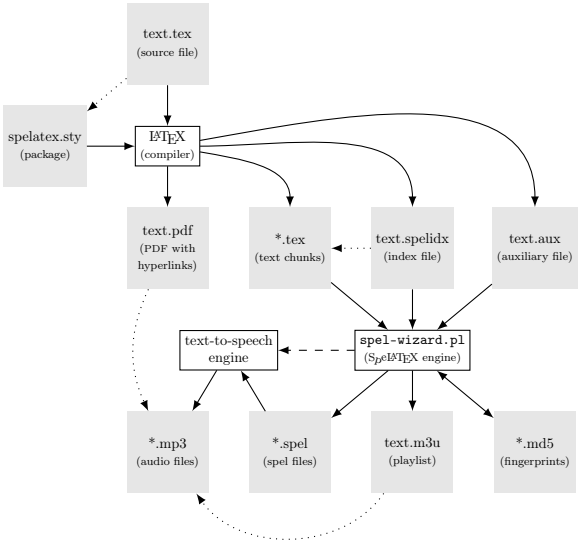


Figure 1: Basic tool setup of the SpeLaTeX package; filled boxes indicate files, outlined boxes indicate tools; solid lines indicate the use or creation of files, dotted lines indicate references, dashed lines indicate a control relationship.

2 The operational flow

How does SpeLaTeX work? Let's go through the flow as outlined in Fig. 1. The process consists of two steps: compilation and incantation.

Compilation

We start at the top with your manuscript (`text.tex`) by loading the `spelatex.sty` style file and embedding every chunk of your document that needs to be converted to speech in a `spelchunk` environment. When you compile your manuscript, a PDF file will be generated that contains clickable hyperlinks to audio files that need to be generated later in the process.

As shown, besides the PDF, as a side effect of the compilation, (1) every text chunk will be written to a separate `.tex` file in a dedicated subdirectory, and (2) a `.spelidx` index file will be generated that contains information on the chunks that need to be converted to speech, including extra information to help the text-to-speech conversion. As usual, (3) an auxiliary file (`.aux`) is generated that is also needed in the text-to-speech conversion.

Incantation

These three source files are used by the Perl script `spel-wizard.pl` to generate (a) plain English text (`.spel` files) for all chunks to be converted to speech, (b) mediaplayer playlists (`.m3u`) for sections that gather more than one chunk and (3) fingerprint files (`.md5`). These fingerprint files are preserved between LaTeX compilations to keep track of when a text

doi.org/10.47397/tb/46-1/tb142daems-spelatex

chunk changed. If the fingerprint of a chunk does not correspond to its old value, then the `spel-wizard.pl` script will launch the text-to-speech converter that reads the `.spel` file and transforms it into an audio file (in mp3 format[1] in Fig. 1). These are the audio files (and the `.m3u` files) that the PDF file references. Keeping track of the fingerprints avoids the costly action of regenerating audio files for chunks that did not change.

For a `text.tex` source file, this flow just takes two commands (with `$` being your shell prompt):

```
$ lualatex text
$ spel-wizard.pl -v text
```

The `-v` flag allows you to follow the text-to-speech conversion in a little bit more detail.

Unfortunately, the fact that multiple tools are required for this flow to work (your LaTeX compiler, a Perl interpreter and the `SpeL::Wizzard` module, and a text-to-speech tool) makes the setup no *walk in the park*. We'll try to explain this as best we can in section 4.

3 Getting a taste of it

You can go to the web page of SpeLaTeX on CTAN (`https://ctan.org/pkg/spelatex`) and right-click on the "Package documentation" to save it to disk. Then open the file with a PDF reader — but avoid your web browser opening the document; use evince, okular or Adobe Reader instead.

Many of the structure elements of this text, like the title, the author block and the (sub)section headers, can be clicked on. This will cause your browser to pop up and (if the file association of the `.mp3` file extension on our system correctly connects your browser to your media player) play the spoken version of the element you just clicked. If not, then skip this section and try the instructions below in section 4.

In addition, you will see small triangles after the (sub)section headers near the right margin of the text. If you click these, the entire (sub)section will be spoken out loud for you. Similarly, there are triangles at the left margin before each paragraph, which will speak that paragraph.

Finally, continue to section 6 of the document and click on some of the equations you see there. The entire equation area is clickable and will cause the formula to be spoken out loud for you.

That's all you need to know to be able to use a PDF that has been generated using SpeLaTeX. Where do the audio files come from? They are available as part of the package on the CTAN servers (and their mirrors). Note that linking to files on servers is not the usual behavior. It's generally better to make a PDF that links to local files. This will save you quite a bit of internet traffic and also avoid the inevitable delay when you are not on a fast network.

Did you notice that this article — if you are reading a PDF version — is also SpeLaTeX-enabled? We have hidden the markers (by specifying the package option `markervisibility=none`) to avoid confusion when reading this article on paper. Try clicking a section heading, or where the markers normally are. It works!

4 Setting things up

4.1 Preparing your PDF reader

Most PDF readers work out of the box. In some cases you need to allow your reader to open links to local files. It should give you an error message if it refuses to open such links. Also, make sure the file extensions `.mp3` (or whatever format you choose to use in the future) and `.m3u` link to your music player. Most media players are capable of playing these formats. We can recommend SoX, totem or VLC on GNU/Linux, QuickTime player on macOS, and VLC on Windows. Why not the standard Windows Media Player? The latter requires you to declare every folder containing audio files you want to play as a Media Player library. If you don't mind this extra hurdle: no problem, go ahead.

4.2 Installing the package

Three parts are needed to get the SpeLaTeX system operational:

1. The LaTeX part
2. The Perl part
3. The Text-to-speech part

The LaTeX part

If you are using a major TeX distribution like TeX Live or MiKTeX, the latest version of the package will be installed on your system. If it is not, use the package manager to install it manually. On TeX Live this is as simple as running the following command in a shell:

```
$ tlmgr install spelatex
```

For MiKTeX, start the MiKTeX console and search for the SpeLaTeX package in the Packages section. Right click on it and press 'install'.

If you don't like package managers, you also can use the style file `spelatex.sty` directly. Just make sure it is somewhere on your LaTeX search path.

[1] The mp3 format has been chosen for this article because it is best supported over all platforms and media-players. Note, however, that other formats exist that provide a better quality vs. size ratio, e.g., opus (in an ogg container).

The Perl part

On GNU/Linux and macOS, a Perl interpreter is normally part of the system installation. If not, install a Perl interpreter and its package manager (`cpanminus`) using your system's software package manager. Then, use the perl package manager to install the module `SpeL::Wizard` as follows:

```
$ cpanm SpeL::Wizard
```

On Windows, we recommend that you install Strawberry Perl [10]. Then open a *Command Prompt* and type

```
$ cpan SpeL::Wizard
```

This will install the `spel-wizard.pl` script on your system. Make sure it is in your search path.

The text-to-speech part

This is probably the hardest part. The starting point is a configuration file for `spel-wizard.pl`. You can check the SpeLaTeX package documentation to see what the various options are for its location. To keep things simple in this article, we'll assume you put it in your home directory on GNU/Linux or macOS or in your `%userprofile%` folder on Windows, and that you name it `tts.conf`.

This configuration file determines the text-to-speech tool that will be used during incantation, including the voice you prefer.

We can recommend two free options for a text-to-speech tool:

- For GNU/Linux and macOS: Festival [5].[2]
- For Windows: Balabolka [3].

Read the documentation of these packages to learn how to install them. Make sure your `PATH` environment variable contains the path to the correct executable.

Here are working configuration files for each of these systems:

(for Festival)

```
[engine]
tts=festival.pl

[languagetags]
english=en-gb

[voices]
english=en1_mbrola
```

(for Balabolka)

```
[engine]
tts=balabolka.pl

[languagetags]
english=en-us

[voices]
english=Zira
```

The `tts=festival` line specifies which text-to-speech script (on the left, `festival.pl`) to call for the conversion. The language tag `english=en-gb` specifies which internationalization (i18n) module (`en-gb`) the spel-wizard script should use for the selected babel language (`english`). Finally, the voices section specifies which voice (`en1_mbrola`) to use for the selected babel language (`english`). For multilingual documents you will need multiple lines in both the languagetags and voices sections.

A conversion script (e.g. `festival.pl`) can be written in any available scripting language. It takes three arguments. In order:

1. the name of the `.tex` file to convert to an audio file,
2. the name of the audio file; the file extension will determine which format will be generated, and
3. the voice to be used.

If you want to integrate your own text-to-speech tool, take a peek into the existing scripts. If you don't manage yourself to create a working setup, you're welcome to ask help from us. If it is a text-to-speech tool that might be of use to others, we will be glad to help you out (and incorporate the setup in the SpeLaTeX package).

Personally, we often use AWS Polly [2] and Azure TTS [7] for high-quality voices. Conversion scripts are provided with the package; however, you need to apply for your own Amazon AWS or Microsoft Azure account and perform some setup involving configuration and credential files. It is beyond the scope of this article to explain this in detail.

5 Using the package

Once the setup is working, using the package is fairly straightforward. The following plan should get you going. We assume that `manuscript.tex` is the name of your document.

1. Make a subdirectory `manuscript-spel` in the directory that contains `manuscript.tex`.
2. Add the following lines to your preamble:
   ```
   \usepackage[english]{babel}
   \usepackage{spelatex}
   ```
   This will select the language you need and load the SpeLaTeX package. Note that the SpeLaTeX package only supports English and Dutch at the time of writing this article. Do you want to contribute your own language? Contact us.
3. Replace every `\item` by `\spelitem` and wrap the text after it in curly braces.
4. Wrap every paragraph in a `spelchunk` environment or put it as an argument to the `\<<<` macro. You will find that the `spelchunk` environment is a bit more robust, while the triple-lesser-than-macro alternative is a bit less visually intrusive in your text editor.

[2] Many package managers can install them; look for the keywords `festival` and `festvox`.

5. Provide the non-textual elements of your document (e.g. graphics) with an appropriate annotation, by embedding them in a `spelchunk` environment followed by a `spelchunkad` environment that contains a textual description of the non-textual element (the suffix `ad` stands for 'audio description').

   Then, execute (assuming we use LuaLaTeX):[3]

```
$ lualatex manuscript
$ spel-wizard.pl -v manuscript
$ evince manuscript.pdf # okular, acroread, ...
```

and enjoy!

6 Example

Below is a minimal working example, that you can start from, to explore the package. Don't forget step one of the plan that we described in the previous section, to make a `...-spel` directory.

The example uses a macro that we did not describe above: `\spelmacad`. Check the documentation for that.

```
\documentclass[a4paper]{article}

\usepackage[margin=1in]{geometry}
\usepackage[english]{babel}
\usepackage{spelatex}
\usepackage{tikz,pgfplots}

\title{Simple \spelatex-example}
\date{11/09/2024}

\begin{document}
\maketitle
\section{Basic use}
\begin{spelchunk}
  You just have to embed every paragraph in a
  spelchunk environment. The alternative is to
  use the triple-lesser-than-macro (as is
  done in the next paragraph), which does
  exactly the same, though it is less robust.
\end{spelchunk}

\<<<{We illustrate how to treat lists with a
  description of the taste of some fruits:}
\begin{description}
\spelitem[lemon]{tastes sour}
\spelitem[banana]{tastes sweet}
\end{description}

\<<<{
  Figure~\ref{fig:example} is also audio
  enabled, both in the caption and with a
  separate comment.
}
```

[3] On Windows, omit the `.pl` extension in the second command.

```
\begin{figure}
  \centering
  \begin{spelchunk}[arealink]
    \begin{tikzpicture}
      \begin{axis}[xlabel=$x$,
        ylabel=$y$,zlabel=$z$,grid=both]
        \addplot3[patch,shader=faceted interp,
        samples=10] {x^2-y^2};
        \fill (axis cs:0,0,0)
          circle[radius=1pt] node[above] {$p$};
      \end{axis}
    \end{tikzpicture}
  \end{spelchunk}
  \begin{spelchunkad}
    This is the graph of a biquadratic surface.
    Note the point $p$ which we call a saddle
    point. This is a point where the first
    partial derivatives are zero and the second
    partial derivatives in $x$ and $y$ have
    opposite signs.
  \end{spelchunkad}
  \caption{A simple biquadratic surface}
  \label{fig:example}
\end{figure}

\section{Some equations}
\<<<{
  The \texttt{arealink} option causes the link
  to cover the entire area of the equation.
  This section shows also some advanced
  features that are explained in the
  documentation of the package.
}
\newcommand\dd[1]{\,\mathrm{d}#1}
\spelmacad{eu}{e}
\spelmacad{dd}[1]{d #1}

\begin{spelchunk}[arealink]
\begin{equation}
  \int_{0}^{+\infty} \mathrm{e}^{-cx}\dd{x}
    = \frac{1}{c}
\end{equation}
\end{spelchunk}
\<<<[arealink]{
  The arealink option also works with the
  triple-lesser-than-macro alternative.
}
\end{document}
```

7 Discussion

In this section, we will first discuss the difficulty of joining a free-form typesetting language like LaTeX with the requirement of parsing the constructs to convert them to meaningful language again. Then we'll discuss the things that went well in developing the package, and also the things that are suboptimal and even fragile.

7.1 LaTeX: Praise and critique

LaTeX is built around the principle of separating content and formatting. While writing the document, the author can focus on the content, indicating the structure using sectioning and enumeration commands. It supports non-textual material such as graphics and equations in a structured way. The LaTeX compiler does all the formatting and bookkeeping for textual aids, like references, tables and so forth. The end result looks magnificent. We all love it. So far the praise.

However, when it comes to mathematics, LaTeX is a lot more liberal. That is for good reason: mathematical notation is also quite liberal. And we must admit: we all like to create our own special notation, and more than once we have used illogical constructions, or worse, reused existing constructions by 'overloading' them. In addition, how to read mathematical equations out loud is not fixed in rules that are generally agreed on. This creates a problematic starting position for a tool that reads the equations based on parsing their LaTeX code.

Below are a few examples of our critique.

Simple vs. rigorous reading
Consider the following:

$$y = \sqrt{\sin \frac{x}{x+1} - 1} \tag{1}$$

A simple way to read this could be: "y equals the square root of the sine of x over x plus one minus one." Note that this incantation is full of ambiguity. A more rigorous way of reading this expression would be: "y equals the square root of the sine of x over x plus one, this ends the fraction and ends the sine minus one, this ends the square root."

We opted for the first version, based on the fact that we assume the text is in front of the reader when the text is being spoken. The latter version would allow for listening to the text in a podcast-like style, without the document in front of you. However, we think that only a few people possess the mental ability to keep an overview of an equation with all the nested 'end' clauses.

In short, SpeLaTeX opts for *visually supported reading*. Maybe in the future an option will be implemented to allow you to make the choice yourself at incantation time.

Function argument vs. grouping brackets
Consider the following:

$$y = c\,(a+b) \tag{2}$$

$$y = f(x+1) \tag{3}$$

If the two lines were shown to you separately, you'd probably read the former as "y equals c times a plus b" and the latter as "y equals f of x plus one". This is based on our capability to use contextual information to see which notation has been used. However, SpeLaTeX is not an AI tool and does not have this capability.

In the current implementation, a single letter followed by brackets is recognized as a function. One can force reading it as group brackets by using `\left` and `\right` before the brackets. This has been done in the equation pair above.

Switching between math and text mode
When reviewing articles of researchers in our group, we often correct a typesetting error made when specifying a variable with a subscript that is an abbreviation, such as τ_{ff}. In such a case you need to make clear to LaTeX that the subscript is not itself a mathematical expression, therefore you need to enclose the double f in a `\mathit` (as above) or `\mathrm` command, such that it is not typeset as τ_{ff}. If your subscript contains spaces (which we don't recommend), you might even need to use `\textit` or `\textrm`. SpeLaTeX also needs this clarity, in order not to read the construct as "tau f times f".

Meta-annotations
LaTeX allows you to put nice extras in relation to subexpressions (such as braces) to clarify their meaning, such as:

$$H(s) = \frac{\overbrace{\frac{\alpha}{\alpha+1}}^{\equiv B_1} s + 1}{\underbrace{(\beta-1)}_{\equiv A_2} s^2 + \underbrace{2\zeta}_{\equiv A_1} s + \underbrace{\delta+3}_{\equiv A_0}} \tag{4}$$

We have chosen not to read that extra information, as it complicates reading the original equation itself. Your feedback or suggestions on these issues are welcome.

Units
We are big fans of the `siunitx` package [8] as it forces you to use correct units and typesets them flawlessly. In addition, if you refrain from using the abbreviations the package provides, e.g. writing `\nano\meter` instead of `\nm`, SpeLaTeX works out of the box, by just removing the backslashes. As always, *Joseph is right* in the implementation of his packages.

High-level constructs
It is difficult to give meaning to high-level constructs if one typesets them on a low level. Consider as an

example:

$$(x+y)^n = \sum_{i=0}^{n} \binom{n}{i} x^{n-i} y^i \tag{5}$$

If one typesets this using the `\matrix` environment, it will be read out loud as a matrix, instead of a binomial coefficient.

However, if the `\binom` macro is used, and provided an appropriate audio description, such as

```
\spelmacad{binom}[2]{%
  the binomial coefficient of #2 out of #1,}
```

the construct reads out loud like a charm.

Conclusion

Many more examples could be shown, but the general advice should be clear by now: *be explicit and keep the description at a high level!*

7.2 SpeLaTeX: The good, the bad and the ugly

Usability of the resulting PDF

Overall, we think that the learning curve of using a PDF that is equipped with hyperlinks to the audio files is smooth and short. After a while, even working with a PDF that only contains the right-hand-side markers, requiring you to click on the hidden link-area before the first line of a paragraph, feels very natural.

Direct access to the required material is possible by directly clicking before a paragraph or on an equation, while more linearly sequential access is facilitated using the right-hand-side markers.
Disadvantages that we see:

- you need to distribute your document as a file plus a folder full of audio files; embedding the document in an archive file like zip or tar alleviates this problem. Common document formats (like `.docx`) do the same. We even might claim our own file extension.
- a document will take a lot more disk space, which is not very good for sustainability.

Advantages we see:

- bundling the audio files with a document allows for providing high-quality audio.
- the costs are kept low for the reader, as the author pays for the high-quality audio generation.

In the medium term, embedding the audio files in the PDF itself is a logical choice. It avoids the zoo of audio files. We opted not to do that now, as the current PDF viewers struggle with playing the audio files without requiring extra mouse clicks to confirm various things or to save them to disk first.

In the long term, embedding a plain-text version of the graphical material (like equations and descriptions of graphical material or tables) as tags inside the PDF, such that the reader of your PDF viewer can take care of the text-to-speech conversion is also a logical choice. We opted not to do that either, as most PDF readers do not support this feature. In addition, the quality of the built-in readers is not as high as the currently available AI voices from some major providers (like Google and Amazon).

In any case, having a PDF that is read out loud properly is a big advantage for students suffering dyslexia, both for course material as well as for (exam) assignments.

Overhead when preparing a manuscript

Only part of the document is equipped automatically with links (title, date, author, footnotes, (sub)section headers, ...). The overhead of having to encase all the paragraphs manually in `spelchunk` environments is considerable. It also puts a big visual burden on managing your manuscript in an editor. For this reason, the triple-lesser-than-macro alternative has been implemented, although it is not as robust.

The culprit for the encasing requirement is the fact that paragraphs are rather inaccessible in LaTeX. We considered auto-wrapping them using LaTeX3 constructs. It is possible, but not elegant either, as (sub)section headers and include files keep getting in the way of the wrapping process. Any fresh ideas on this front would be very welcome!

In an earlier version of the SpeLaTeX package we had planned to make an orgmode exporter available for Emacs to take care of this. However, the user base for that is too small. Creating it would deflect the focus from solving the real problem within the LaTeX engine.

Flexibility of the package

The flexibility of the package has not been highlighted much in this article. However, it is made multilingual with proper Babel support and i18n support in the Perl `SpeL::Wizard` module.

In addition, both the `\spelmacad` macro and the `spelchunkad` environment allow you to override any result of `spel-wizard.pl` that you are not fully pleased with. So, you can work around our bugs or other problems when needed.

If you are interested in learning more about this in a subsequent article, we will be happy to devote some time to that.

Fragility of the package

The SpeLaTeX package redefines most sectioning commands, as well as titlepage information, footnotes, etc. This makes the package fragile with respect to

compatibility with other packages that do similar things. Maybe some LaTeX3 hooks can be used for our implementation, without having to redefine these commands.

The Perl `SpeL::Wizard` uses a regular expression grammar parser, which is very capable, but also slow and not easy to debug. *Not a concern for me as a user of the package*, you might think. However, sometimes it even fails to finish in an acceptable time (it continues looping, without matching the correct grammatical construct). This might cause the script to stall. Very annoying. The infinite flexibility of LaTeX, allowing the user to write equations in exotic ways, does not help either. We did not yet implement a timeout mechanism to break free from these loops, but it is on the todo list.

Immaturity of the package

The first version of SpeLaTeX was implemented back in 2008. Then, due to professional constraints, we were not able to continue working on it. We started working on it again in 2023, with the single goal to finish the package as soon as possible to be able to involve the community in the further development. If not for actual help in coding, then at least in providing feedback.

This explains why most of the code still heavily relies on rather primitive TeX constructs, rather than on the more powerful LaTeX3 [6] machine gun that package writers can use these days. This makes the package rather ugly and inconsistent in the current version. We have no doubt that it will mature over time.

8 Missing in action

Many issues have not been discussed in this article, such as:

- the stubbornness of some text-to-speech converters which makes SpeLaTeX's life difficult,
- its multilingual capabilities and how they have been implemented,
- how integration with other packages has been conceived (e.g., `amsmath` [1]), and
- how other text-to-speech services may be integrated in the tool chain.

These might be subjects for a subsequent article.

9 Help!

We can help you ...

The current SpeLaTeX package is in active development. That means that there will be errors. If you find any, please signal them to us. We will be happy to follow up on them and try to help you to find a temporary workaround.

... and you can help us

By asking our assistance with chunks that are not converted well, you help us to improve the package and the `spel-wizard.pl` script as well. However, expect us to judge whether the constructs that you try to push through SpeLaTeX make sense. This especially holds for equations. You must be prepared to bring structure to them. Only in this way can we make them parsable by the `spel-wizard.pl` script. No doubt, discussions will emerge on how to properly read specific mathematical constructs.

And you can contribute even more. If you speak a language that we don't master, and would like to use it with SpeLaTeX, you can volunteer in translating the internationalization (i18n) module in the `SpeL::Wizard` module. You'll earn a spot in the thank-you section of the package.

10 Conclusion

We think SpeLaTeX deserves a spot in the current range of tools supporting people with reading disabilities. It also supports you in creating richer texts in general. Though it is still in development, it is usable in its current state. We look forward to your feedback!

References

[1] amsmath — AMS mathematical facilities for LaTeX. `ctan.org/pkg/amsmath`.

[2] Amazon Polly. `aws.amazon.com/polly`.

[3] Balabolka text-to-speech program for MS Windows. `cross-plus-a.com/balabolka.htm`.

[4] C. Di Folco, A. Guez, et al. Epidemiology of reading disability: A comparison of DSM-5 and ICD-11 criteria. *Scientific Studies of Reading*, 26(4):337–355, 2022. `doi.org/10.1080/10888438.2021.1998067`

[5] The Festival speech synthesis system. `www.cstr.ed.ac.uk/projects/festival`.

[6] The LaTeX project. `latex-project.org/latex3/`.

[7] Microsoft Azure TTS. `learn.microsoft.com/en-us/azure/ai-services/speech-service/text-to-speech`.

[8] siunitx — A comprehensive (SI) units package. `ctan.org/pkg/siunitx`.

[9] SpeLaTeX — Create PDF documents with hyperlinks to audio fragments. `ctan.org/pkg/spelatex`.

[10] Strawberry Perl for MS Windows. `strawberryperl.com/`.

◇ Walter Daems and Paul Levrie
University of Antwerp, Belgium
`walter dot daems (at) uantwerpen dot be`

BibBreeze: A LaTeX package to automate bibliography data reconstruction

Amir Mohammad Tahsiri

Abstract

The LaTeX package BibBreeze, written using LaTeX3, is a referencing package that automates bibliography reconstruction, eliminating manual effort in reference handling. It reorders disorganized fields for bibliography entries, fills in missing fields, and produces polished, reference style-compliant bibliographies optimized for researchers, academics, and writers. Currently, the package's referencing style is designed for APA, both `numeric` and `authoryear` in-text citations, with more styles to come.

Using powerful packages such as `expl3` and fundamental features such as `regex`, BibBreeze automatically extracts essential information from bibliography (BibTeX) entries and formats it according to the referencing guidelines. Furthermore, it distinguishes between entry subtypes such as proceedings papers (presented, published in journals or books) all identified by `@inproceedings`, using targeted checks. These features are especially valuable for users managing large reference lists who need precise formatting.

1 Introduction

LaTeX users typically integrate references into their documents by downloading BibTeX citations and adding them to a `.bib` file. However, these entries are often incomplete or unorganised: a single field may contain data for multiple fields, while others may omit critical information.

This will result in the referencing style package printing incorrect and incomplete information in the bibliography. This is due to current referencing packages only focusing on the style guidelines and not on analysing the content of bibliography entries (however, to the best of this developer's knowledge, current packages have trouble even adhering to the strict APA referencing guidelines).

Therefore, the user will have to manually reorder or find online the field entries for the `.bib` file. However, the referencing style package presented here, called BibBreeze, intelligently automates the process of reordering or finding field entries to a significant degree whilst assuring strict adherence to APA (7th edition) guidelines.

2 Motivation and significance

Examples of entry types with missing or unorganised data fields for BibTeX entries will be presented below to give the readers a clearer understanding of what problems can occur using existing APA referencing packages and the methods BibBreeze plans to use to handle them. To give additional perspective as to why BibBreeze is needed, its methods will be compared with non-LaTeX bibliography management packages that resemble the same methods.

2.1 Issues of bundled and incomplete fields

A reputable and widely used source for downloading BibTeX citations in computer science is DBLP (`dblp.org`), as it often has more complete information than the official publisher's site. However, this completeness usually results in multiple fields being bundled into one.

For example, consider a conference proceedings paper (`@inproceedings`): the `booktitle` field typically bundles information pertaining to the conference `location` and the `date` it was held, as here:

```
@inproceedings{DBLP:conf/icmas/Wooldridge00,
  author={Michael J. Wooldridge},
  title={Computationally Grounded Theories of Agency},
  booktitle={4th International Conference on
             Multi-Agent Systems, {ICMAS}
             2000, Boston, MA, USA,
             July 10-12, 2000},
  pages     = {13--22},
  publisher = {{IEEE} Computer Society},
  year      = {2000},
  url       = {https://doi.org/10.1109/ICMAS.2000.858426},
  doi       = {10.1109/ICMAS.2000.858426},
  timestamp = {Thu, 23 Mar 2023 13:59:57
               +0100},
  biburl    = {https://dblp.org/rec/conf/
               icmas/Wooldridge00.bib},
  bibsource = {dblp computer science
               bibliography, https://dblp.org}}
```

Listing 1: Bibliography entry with bundled fields in `booktitle`

In this entry, the `booktitle` field combines the conference `title`, `location`, and `date`. As the reference is a proceedings presentation, APA (7th edition) guidelines require it to be formatted as follows:

> Wooldridge, M. J. (2000, July 10–12). *Computationally Grounded Theories of Agency.* [Paper presentation]. 4th International Conference on Multi-Agent Systems (ICMAS 2000), Boston, MA, USA. `doi.org/10.1109/ICMAS.2000.858426`

As it can be seen, the conference date has to be printed after the year in parenthesis. The paper title must be italic, and it is required to indicate the conference type in "[ ]" (in this case, a paper presentation). On the other hand, if the conference paper is part of an edited book, according to APA (7th edition), a reference would be formatted as follows (a new example):

> Drawel, N. Qu, H. Bentahar, J., & Shakshuki, E. (2020). Specification and automatic verification

doi.org/10.47397/tb/46-1/tb142tahsiri-bibbreeze

of trust-based multi-agent systems. Future Generation Computer Systems, 107, 1047–1060. https://doi.org/10.1016/j.future.2018.01.040

It is required that the conference location and date be omitted; only the conference title is printed.

To this developer's knowledge, even the most comprehensive existing referencing or bibliography management packages, such as the standard BibLaTeX APA package [2] or Springer Nature's BibTeX package [1], as well as several packages tested from Overleaf and CTAN, lack automated identification and extraction of bundled fields to reorder them correctly. Instead, they print the entire contents of each field consecutively. This developer's tests reveal that even when fields like `location` or `date` are separate (from the `booktitle` field), they fail to be recognised. Moreover, these packages also output verbatim lengthy fields, such as `abstract` or `note` fields.

In regards to printing only the required fields, existing packages fail to recognise when required fields are missing. For example, suppose a proceedings paper published in an edited book lacks a critical field like `editor`. In that case, current packages do not flag the issue or prompt users for corrections, but simply reproduce the entry unchanged. An example:

```
@inproceedings{10.5555/3491440.3491441,
    author ={Ahmadi, Saba and Ahmed, Faez and
            Dickerson, John P. and Fuge, Mark
            and Khuller, Samir},
    title ={An algorithm for multi-attribute
            diverse matching},
    year ={2021},
    isbn ={9780999241165},
    booktitle ={Proceedings of the Twenty-Ninth
                International Joint Conference
                on Artificial Intelligence},
    articleno ={1},
    pages ={7--11},
    location ={Yokohama, Yokohama, Japan},
    series ={IJCAI'20}}
```

Listing 2: Entry with missing `editor` field

In this example, the presence of an `isbn` field is evidence that this proceedings paper is part of an edited book. However, due to the abovementioned issue of unorganised and/or missing data fields, it does not have an `editor` field.

2.2 Issues with recognising entry subtypes

Another critical need that BibBreeze addresses is automatically distinguishing subtypes of a specific entry. For instance, it can distinguish whether a proceedings paper is a presentation, published in a journal, or included in a book, despite each of these sharing the `@inproceedings` identifier in the `.bib` file. Without this package, users need to manually identify these distinctions.

Sometimes missing fields extend beyond the `editor` field to another critical field such as `isbn`, rendering subtype recognition impossible, and requiring manual retrieval of fields from online sources. In this case, the `BibBreeze` package yet again automates this process by relying on the `URL` (the paper's online source) and entering the contents in the `biburl` field to distinguish between entry subtypes. This is because the `biburl` is easily accessible, thus simplifying retrieval for users; more importantly, it almost always contains information about the subtype of an entry. For example, in this entry:

```
@inproceedings{DBLP:journals/entcs/RaimondiL04,
    author= {Franco Raimondi and Alessio Lomuscio},
    editor= {Wiebe van der Hoek and Alessio
            Lomuscio and Erik P. de Vink and
            Michael J. Wooldridge},
    title= {A Tool For Specification And
            Verification Of Epistemic
            Properties In Interpreted Systems},
    booktitle= {1st International Workshop on
                Logic and Communication in
                Multi-Agent Systems, {LCMAS}
                2003, Eindhoven, The Netherlands,
                June 29, 2003},
    series= {Electronic Notes in Theoretical
            Computer Science},
    volume= {85},
    pages= {176--191},
    publisher= {Elsevier},
    year= {2003},
    doi= {10.1016/S1571-0661(05)82609-X},
    timestamp= {Thu, 08 Dec 2022 14:13:49 +0100},
    biburl= {https://dblp.org/rec/journals/
            entcs/RaimondiL04.bib},
    bibsource= {dblp computer science
                bibliography, https://dblp.org}}
```

Listing 3: Entry with `biburl` field

The entry has an `editor` field, indicating the conference proceedings has been "edited" and published in a book and this entry is a section in that book. This is technically accurate. However, 'Electronic Notes in Theoretical Computer Science' is considered to be a journal. Therefore, this entry is required to be printed as a journal article. This is identifiable from the `biburl` field as it contains the term `journals`.

2.3 Subjective & objective evaluation of methods

In addition to comparing BibBreeze with existing LaTeX-based packages, let's compare this package to `bibtexparser` [3], a Python library designed for parsing `.bib` files. While the BibBreeze package also parses through entry fields to unbundle them

or distinguish entries, `bibtexparser` parses through entries to extract fields as strings into Python dictionaries. This opens the door to tasks like analysing citation patterns (e.g., counting entries by year or author), reformatting references (e.g., transforming `.bib` files into JSON, XML, or CSV), or feeding raw data into web platforms. Such a design can shine in database management of bibliography data, where structured, field-level access is key. By contrast, BibBreeze leverages fundamental functionality (i.e., regex) and creatively designed logic to semantically interpret and recognise the content of BibTeX entry fields automatically. Built directly into LaTeX, it streamlines the process, side-stepping the need to script parsed data back into `.tex` files as `bibtexparser` users must (if the purpose of its entry parsing is field content identification).

3 Software description

The main algorithms that make BibBreeze unique are the `\ExtractFieldInfo` macro (listings 4 and 5) and the bibliography driver for `@inproceedings` (listing 6) located in the `BibBreeze.bbx` file in the Github repository referenced below. These handle extraction of bundled fields and distinguishing between subtypes of an entry, respectively.

The code presented here illustrates the algorithms; the actual code in the package as distributed on Github and CTAN has been updated.

3.1 The `\ExtractFieldInfo` macro

The `\ExtractFieldInfo` macro (line 12) takes an input field and splits it into a sequence of items based on commas (lines 14–15). The main processing loop `\l_my_continue_bool` (line 16) then iterates through this sequence from *right to left* (lines 20–101). This direction for iteration is a delicate matter: since the conference title may contain commas as punctuation (not separators), starting from the right ensures that the title is identified correctly after other fields (e.g., `date` and `location`) are extracted. By prioritising the right-most items, the macro avoids misinterpreting the commas that are parts of the title as separate fields.

At the start of the loop (line 20), a regex checks the last item in the sequence for a specific token string (pattern): a four-digit string (i.e., a year) preceded by at least one non-digit character: `{.\d{4}\Z}`[1] (line 24). If this token string is found at the right-most item in the list (line 26), it means the year is embedded within the title (i.e., "{TITLE ACRONYM} 2025") and that all items in the list belong to the `title` of the paper. Therefore, the macro wraps this year (and its acronym) in parentheses (line 27) and sets it alongside the remaining items as the paper `title` (lines 28–30). After this, the loop stops (line 32). Such a token string is considered unique amongst other possible list items (i.e., `stand-alone-year`), and by taking advantage of this item's unique token string, the macro can avoid unnecessary searches such as `location` and `date`, efficiently isolating the title and saving computational effort.

However, if the last item does not match the mentioned pattern, the macro pops the next item (line 35) and checks if the item in the list matches the string "proceedings" (line 37). If so, it discards (pops) this item, as it is not relevant to the `title`, `date`, or `location` (lines 39–40). Next, the macro uses another regex to check for a `stand-alone-year` `{^\d{4}}`[2] (with no preceding characters) in the current item (line 42). If found, the year is discarded (line 44), and the next item is checked for a month (e.g., "January") to store in the date variable (lines 46–48). However, if a `stand-alone-year` is not found, the macro does not check for other items and returns to the start of the loop as the `title` is the only remaining item.

If a `date` is identified, the next item(s) are assumed to be the `location`. However, `location` fields can vary: they typically include two items (city and country), but if the country is `USA`, a third item (state) is present. To handle this, the macro checks if the first `location` item is `USA` (line 53). If so, it uses a nested while loop to extract the next two items (city and state); otherwise, it extracts only the next item (city) (lines 53–67). These items are stored in a temporary sequence `\l_my_location_list_seq` from *left to right* to preserve (first out, first in) their original order (lines 51, 68). The nested loop ensures all location items are accounted for before exiting (lines 72–74).

After processing the `location`, the macro returns to the main loop and rechecks for the embedded year pattern (line 24) (in case the `booktitle` field had begun with the string "proceedings"). If no special conditions remain, the macro builds the title by adding items to `\g_my_title_tl` variable from left to right, separating them with commas and spaces (lines 91–95). This ensures the `title` is formatted consistently, even if it contains internal commas. Finally, the macro checks if the input sequence is empty to exit the loop or continue processing (lines 96–101).

[1] `\d` represents a digit, `\d{4}` means a four-digit string, the "`.`" before matches one character, and `\Z` matches at the end of the string.

[2] Same as before, except that `{^}` means no preceding characters, i.e., matches at the beginning of the string.

3.2 The `inproceedings` bibliography driver

The bibliography driver for `inproceedings` operates as follows. First, `\ExtractFieldInfo` is called (line 112) as it contains functions to allow the `title`, `location`, and `date` be printed (refer to the Github repository). Next, the driver examines whether the `editor` field is undefined (line 113). If so, it searches for the `biburl` field. If that is also undefined (line 115), it looks for an `isbn` field (line 117). In the rare case that none of these fields are defined, it prints a message prompting the user to retrieve the `biburl` from the url of its online source (line 118). The address bar almost always contains a specific key word identifying the type of paper.

If the entry includes an `isbn` field, we can confirm the proceedings paper is part of an edited book, as this field is unique to published books. In this case, the driver proceeds to execute the `EditorMode` function (line 119). The `EditorMode` function formats the output for references within an edited book. A potential question arises: if the `editor` field is undefined at this stage, what does it mean to execute a function designed for printing the `editor` field? The `EditorMode` function addresses this by outputting *"Find editors from publisher's source!"* if the `editor` field is absent.

Next, the driver handles cases where the `biburl` field is defined. It calls `\ExtractURLInfo` (again, see the repository) which uses a regex command to scan for terms like *"article"* or *"journals"*. If a match occurs, we can confidently classify the paper as a journal article, so the code prompts the user *"This paper is an article, find the correct BibTeX reference!"* (lines 122–129). If no match is found — indicating terms like `conference`, `proceedings`, or `chapter` — and the `editor` field is absent (with incomplete data already accounted for), the paper is presumably a paper presentation and it will print accordingly (lines 132–141).

The driver then addresses scenarios where the editor field is defined but where both `biburl` and `isbn` are undefined (lines 147–149). If so, it prints *"Find biburl from online source!"* (line 150). However, if `isbn` is defined, we can confirm the paper belongs to an edited book, and the driver prints using `EditorMode` (line 151). When `biburl` is defined, the process mirrors the earlier logic (lines 154–158). However, this time if the paper is not a journal article, it is part of an edited book, and `EditorMode` is applied (line 159).

4 Illustrative examples

Figure 1 shows an example of the package's output. It not only adheres to the strict guidelines of APA (7th edition) referencing style but also automatically reorders all unorganised fields. Furthermore, in cases where completely filling fields is not possible, it prompts the user in red text (grayscaled for print) on how to fix the issue; see items [2] and [16].

5 Impact

Through comparison with other referencing packages and a detailed explanation of the algorithms presented above, BibBreeze reduces time-consuming manual editing of incomplete and or messy bibliography data to a significant degree.

Beyond outperforming other packages, the BibBreeze package leverages powerful and fundamental techniques like `regex` commands and the `expl3` package designed for robust list manipulation and iteration. In addition, the package's innovative methods prioritise both efficiency and accuracy. The use of fundamental commands, powerful packages, and innovative methods embody a high flexibility within this package, making it adaptable to other referencing styles. With minor to moderate modifications, this package can be used for other referencing styles such as Harvard or Chicago, etc., broadening its utility for diverse academic needs.

6 Conclusions

Typesetting systems such as TEX possess significant untapped potential. This project shows how bibliography management can become more intelligent and time-saving through the development of the BibBreeze package. However, it is noteworthy that this approach does not appear to have been widely adopted by other developers designing referencing packages.

References

[1] godkingjay. springer-nature-latex-template. `github.com/godkingjay/springer-nature-latex-template.git`, 2023. Version 3.

[2] plk. biblatex-apa. `github.com/plk/biblatex-apa.git`, 2010. Version 1.3c.

[3] sciunto-org. bibtexparser. `github.com/sciunto-org/python-bibtexparser.git`, 2013. Version 1.4.3.

◇ Amir Mohammad Tahsiri
`mohammad_tahsiri (at) hotmail dot com`
`https://github.com/mamadgit/BibBreeze.git`
ORCID 0009-0007-3230-4147

References

[1] Find biburl field from online source (address bar)!

[2] Ahmadi, S. Ahmed, F. Dickerson, J. P. Fuge, M., & Khuller, S. (2021). An algorithm for multi-attribute diverse matching. In Find editors from publisher's source! (Ed.), *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence* (pp. 7–11).

[3] Alechina, N., & Logan, B. (2010). Computationally Grounded Account of Belief and Awareness for AI Agents. In O. Boissier A. E. F. Seghrouchni S. Hassas & N. Maudet (Eds.), *Proceedings of The Multi-Agent Logics, Languages, and Organisations Federated Workshops (MALLOW 2010)* (pp. 1–14). CEUR-WS.org.

[4] Alechina, N., & Logan, B. (2016). Verifying Systems of Resource-Bounded Agents. In A. Beckmann L. Bienvenu & N. Jonoska (Eds.), *Pursuit of the Universal - 12th Conference on Computability in Europe (CiE 2016)* (pp. 3–12). Springer. https://doi.org/10.1007/978-3-319-40189-8_1

[5] Alur, R. Henzinger, T. A., & Kupferman, O. (2002). Alternating-time temporal logic. *J. ACM*, *49*(5), 672–713. https://doi.org/10.1145/585265.585270

[6] Cohen, P. R., & Levesque, H. J. (1990). Intention is choice with commitment. *Artificial Intelligence*, *42*(2), 213–261. https://doi.org/10.1016/0004-3702(90)90055-5

[7] Drawel, N. Bentahar, J. Laarej, A., & Rjoub, G. (2020). Formalizing Group and Propagated Trust in Multi-Agent Systems. In C. Bessiere (Ed.), *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI 2020)* (pp. 60–66). ijcai.org. https://doi.org/10.24963/ijcai.2020/9

[8] Drawel, N. Bentahar, J., & Shakshuki, E. (2017). Reasoning about Trust and Time in a System of Agents. *Procedia Computer Science*, *109*, 632–639. https://doi.org/10.1016/j.procs.2017.05.369

[9] Drawel, N. Qu, H. Bentahar, J., & Shakshuki, E. (2020). Specification and automatic verification of trust-based multi-agent systems. *Future Generation Computer Systems*, *107*, 1047–1060. https://doi.org/10.1016/j.future.2018.01.040

[10] Elgot-Drapkin, J. J., & Perlis, D. (1990). Reasoning situated in time I: basic concepts. *J. Exp. Theor. Artif. Intell. 2*(1), 75–98. https://doi.org/10.1080/09528139008953715

[11] Fagin, R. Halpern, J. Y. Moses, Y., & Vardi, M. Y. (1995). *Reasoning About Knowledge*. MIT Press. https://doi.org/10.7551/mitpress/5803.001.0001

[12] Hintikka, J. (1962). *Knowledge and belief : an introduction to the logic of the two notions*. Cornell University Press.

[13] Hoek, W., & Wooldridge, M. J. (2003). Towards a Logic of Rational Agency. *Log. J. IGPL*, *11*(2), 135–159. https://doi.org/10.1093/jigpal/11.2.135

[14] Lomuscio, A. Qu, H., & Raimondi, F. (2009). MCMAS: A Model Checker for the Verification of Multi-Agent Systems. In A. Bouajjani & O. Maler (Eds.), *Computer Aided Verification* (pp. 682–688). Springer Berlin Heidelberg.

[15] Raimondi, F. (2006). *Model checking multi-agent systems* [Doctoral dissertation, University College London, UK] dblp computer science bibliography. http://discovery.ucl.ac.uk/5627/

[16] This paper is an article, find correct BibTeX reference!

[17] Rao, A. S., & Georgeff, M. P. (1991). Modeling rational agents within a BDI-architecture. In A. James F. Richard & S. Erik (Eds.), *Proceedings of the Second International Conference on Principles of Knowledge Representation and Reasoning* (pp. 473–484). Morgan Kaufmann Publishers Inc.

[18] Wooldridge, M. J. (2000, July 10-12). *Computationally Grounded Theories of Agency* [Paper presentation]. 4th International Conference on Multi-Agent Systems (ICMAS 2000), Boston, MA, USA. https://doi.org/10.1109/ICMAS.2000.858426

Figure 1: Output of BibBreeze package for various conditions; warnings for author are printed in red (grayscaled for print)

```
\ExplSyntaxOn
\seq_new:N \l_my_input_seq
\tl_new:N \l_my_input_tl
\tl_new:N \l_my_item_tl
\tl_new:N \g_my_date_tl
\tl_new:N \g_my_location_tl
\tl_new:N \g_my_title_tl
\seq_new:N \l_my_location_list_seq
\int_new:N \l_my_counter_int
\bool_new:N \l_my_continue_bool
\bool_new:N \l_my_nested_loop_bool
\NewDocumentCommand{\ExtractFieldInfo}{m}
{
  \tl_set:Nx \l_my_input_tl {\expandafter\use:n{#1}}
  \seq_set_split:NnV \l_my_input_seq {,} \l_my_input_tl
  \bool_set_true:N \l_my_continue_bool
  \tl_gset:Nn \g_my_title_tl {}
  \tl_gset:Nn \g_my_location_tl {}
  \tl_gset:Nn \g_my_date_tl {}
  \bool_while_do:Nn \l_my_continue_bool
  {
    \seq_get_right:NN \l_my_input_seq \l_my_item_tl
    \tl_trim_spaces:N \l_my_item_tl
    \regex_match:nVTF {.\d{4}\Z} \l_my_item_tl
    {
      \seq_pop_right:NN \l_my_input_seq \l_my_item_tl
      \tl_set:Nx \l_my_parenthesized_item_tl {( \l_my_item_tl )}
      \tl_gset:Nx \g_my_title_tl {
        \seq_use:Nn \l_my_input_seq {,~}
        \c_space_tl \l_my_parenthesized_item_tl
      }
      \bool_set_false:N \l_my_continue_bool
    }
    {
      \seq_pop_right:NN \l_my_input_seq \l_my_item_tl
      \tl_trim_spaces:N \l_my_item_tl
      \tl_if_eq:nVT {Proceedings} \l_my_item_tl
      {
        \seq_pop_right:NN \l_my_input_seq \l_my_item_tl
        \tl_trim_spaces:N \l_my_item_tl
      }
      \regex_match:nVTF { ^\d{4} } \l_my_item_tl
      {
        \seq_pop_right:NN \l_my_input_seq \l_my_item_tl
        \tl_trim_spaces:N \l_my_item_tl
        \regex_match:nVT { ^(J|F|Mar|A|May|Jun|Jul|Aug|S|O|N|D) } \l_my_item_tl
        {
          \tl_gset:Nx \g_my_date_tl {\l_my_item_tl}
          \seq_pop_right:NN \l_my_input_seq \l_my_item_tl
          \tl_trim_spaces:N \l_my_item_tl
          \seq_put_left:Nx \l_my_location_list_seq {\l_my_item_tl}
        }
```

Listing 4: `\ExtractFieldInfo` definition in `BibBreeze.bbx`

```
      \tl_if_eq:nVTF { USA } {\l_my_item_tl}
      {
        \int_set:Nn \l_my_counter_int {2}
      }
      {
        \int_set:Nn \l_my_counter_int {1}
      }
      \bool_set_true:N \l_my_nested_loop_bool
      \bool_while_do:Nn \l_my_nested_loop_bool
      {
        \int_compare:nTF {\l_my_counter_int > 0}
        {
          \seq_if_empty:NF \l_my_input_seq
          {
            \seq_pop_right:NN \l_my_input_seq \l_my_item_tl
            \seq_put_left:Nx \l_my_location_list_seq {\l_my_item_tl}
            \tl_trim_spaces:N \l_my_item_tl
            \int_decr:N \l_my_counter_int
          }
          \int_compare:nT {\l_my_counter_int == 0}
          {
            \bool_set_false:N \l_my_nested_loop_bool
          }
        }
        {
          \bool_set_false:N \l_my_nested_loop_bool
        }
      }
      \tl_gset:Nx \g_my_location_tl { \seq_use:Nn \l_my_location_list_seq {,~} }
      \seq_if_empty:NF \l_my_input_seq
      {
        \bool_set_true:N \l_my_continue_bool
      }
      {
        \bool_set_false:N \l_my_continue_bool
      }
    }
    {
      \tl_if_empty:NF \g_my_title_tl
      {
        \tl_gput_left:Nn \g_my_title_tl {,~}
      }
      \tl_gput_left:Nx \g_my_title_tl {\l_my_item_tl}
      \seq_if_empty:NTF \l_my_input_seq
      {
        \bool_set_false:N \l_my_continue_bool
      }
      {
        \bool_set_true:N \l_my_continue_bool
      }
    }
  }
}
```

Listing 5: `\ExtractFieldInfo`, continued

```
\DeclareBibliographyDriver{inproceedings}
{
  \noindent
  \usebibmacro{bibindex}
  \ExtractFieldInfo{\thefield{booktitle}}
  \ifnameundef{editor}
  {
    \iffieldundef{biburl}
    {
      \iffieldundef{isbn}
      {\hspace{-5pt}\printtext{Find biburl field from online source (address bar)!}}
      {\EditorMode}
    }
    {
      \ExtractURLInfo{\thefield{biburl}}{article}
      {
        \hspace{-5pt}\printtext{This paper is an article, find correct BibTeX reference!}
      }
      {
        \ExtractURLInfo{\thefield{biburl}}{journals}
        {
          \hspace{-5pt}\printtext{This paper is an article, find correct BibTeX reference!}
        }
        {
          \hspace*{-8pt}\printnames{author}
          \setunit{\labelnamepunct}\newblock
          \printfield{year}
          \setunit{\labelnamepunct}\newblock
          \printfield{title}
          \setunit{\labelnamepunct}\newblock
          \printtext{[Paper presentation]\adddot}
          \PrintTitletl\addcomma
          \PrintLocationtl\adddot
          \printfield{doi}
        }
      }
    }
  }
  {
    \iffieldundef{biburl}
    {
      \iffieldundef{isbn}
      {\hspace{-5pt}\printtext{Find biburl field from online source (address bar)!}}
      {\EditorMode}
    }
    {
      \ExtractURLInfo{\thefield{biburl}}{article}
      {\hspace{-5pt}\printtext{This paper is an article, find correct BibTeX reference!}}
      {
        \ExtractURLInfo{\thefield{biburl}}{journals}
        {\hspace{-5pt}\printtext{This paper is an article, find correct BibTeX reference!}}
        {\EditorMode}
      }
    }
  }
}
```

Listing 6: Bibliography driver for `inproceedings`, from `BibBreeze.bbx`

Efficient input file processing with TEX: A case study

Martin Ruckert

Abstract

This paper illustrates the use of the TEX profiler for program analysis. Using as an example the index generation of the `cweb` system for PDF documents, it shows how finite state machines can be implemented using TEX macros, and that input file processing using this technique is not only highly efficient but also leads to clean and compact code.

1 Introduction

In my last article in *TUGboat* [1], I presented a new tool for TEX programmers: a TEX macro profiler. In that article, I analyzed `texprof.tex`, the profiler's documentation, as an example. One result of the short analysis was that creating a PDF file with pdfTEX is significantly slower than creating a DVI file with TEX, and that the difference in speed is caused by macros that are used only when producing PDF output. These macros process the index file created by the `cweb` system and create links from the section numbers mentioned in the index back to the pages where the sections occur. The macros in question are part of the `cweb` system and consist of just 18 lines (see Fig. 4). Obviously they are a good target for optimization.

The present article uses the TEX profiler for a deeper analysis, clarifies the cause of the poor performance of the code, and explains how to write efficient code to handle this and similar cases of input file processing. The new code will make pdfTEX run almost as fast as the original TEX in this case.

2 Analysis

2.1 Runtime

Measuring the runtime of a program is a complicated undertaking. Modern CPUs run multiple concurrent processes on multiple cores, some of them performance cores, others efficiency cores, and the operating system's scheduling algorithms change the clock rates up and down according to computation needs or processor temperature. While I tried to control all these effects as well as possible, the runtimes given in the following all have a margin of error between 1 % and 5 %.

Using the "real" time reported by the UNIX `time` command, TEX needs 0.251 s while pdfTEX needs 2.03 s to process `texprof.tex`. When running the respective profiler, I observe 0.270 s for TEXprof and 1.30 s for pdfTEXprof. TEXprof is slightly slower (7 %) than TEX because of the overhead of profiling (even when profiling is switched off). In contrast, pdfTEXprof is faster than pdfTEX because the profiler does not produce a PDF output file. When I disable the generation of the PDF file by pdfTEX using its `-draftmode` option, pdfTEX needs only 1.42 s. If I run pdfTEXprof with profiling switched on using the `-prof` option, pdfTEXprof needs 1.90 s to process the input and 0.4 s to write the profile data (45 MB) to the `texprof.tprof` file after profiling is completed.

Running `texprofile -T texprof` on the profile data shows the top ten lines of TEX's input that account for the largest share of the runtime. The results shown in Fig. 1 are slightly edited to fit the current column width. (Similar editing is applied to other figures as well.) The table clearly shows that lines 156–173 are responsible for a major share of the slowdown. The call graph, as described in [1], gives further details.

line	percent	absolute	count
156	19.63 %	456.71 ms	233 200
157	12.74 %	296.40 ms	146 349
158	8.91 %	207.35 ms	147 401
159	5.61 %	130.41 ms	120 279
172	3.47 %	80.74 ms	16 159
173	3.18 %	73.87 ms	38 842
24	2.89 %	67.30 ms	29 919
shipout	2.47 %	57.52 ms	813
linebrk	2.29 %	53.25 ms	28 926
168	1.90 %	44.13 ms	106 202

Figure 1: `texprofile -T texprof`

Next, I focus on the generation of the index by inserting `\profileon` before processing the index file, and `\profileoff` after it. Then I run pdfTEXprof again, this time without the `-prof` option because switching profiling on and off is now done by the inserted primitives. The time, as measured by the profiler, is 1.07 s just for typesetting the index. The table of TEX commands generated by `texprofile -C texprof` (see Fig. 2) shows that the `assign_toks` command is responsible for a significant share of the runtime — and one might rightly wonder why executing such a simple command is so expensive. It is followed by `def` and `let` in second and third place.

For a more detailed inspection, I generate the complete table of all time measurements by running `texprofile -R -m -n texprof`. I add the `-m` option (optimize for machine readability) because I plan to import the output to a spreadsheet for further analysis, and I add the `-n` option (include the

doi.org/10.47397/tb/46-1/tb142ruckert-faster

time	percent	count	command
474.95 ms	36.94 %	404 294	assign_toks
345.14 ms	26.84 %	157 248	def
257.53 ms	20.03 %	231 351	let
51.32 ms	3.99 %	93 515	extension
35.72 ms	2.78 %	123 036	assign_int
27.06 ms	2.10 %	105 859	prefix
15.92 ms	1.24 %	41 252	right_brace
12.49 ms	0.97 %	12 159	system
12.33 ms	0.96 %	27 666	make_box
12.15 ms	0.95 %	53 697	spacer

Figure 2: `texprofile -C texprof`

sequence number of each time measurement) to be able to sort the time measurements back in their natural order after sorting the data according to some other criteria of interest. The table contains 1 384 256 entries and so I filter out the `assign_toks` commands by using `grep`. This gives still 404 294 entries and so I focus on the single line 172 using `grep` a second time. As shown below, line 172 is a key line of the algorithm under investigation. I end up with a table with 14 935 entries and import the file into a spreadsheet.

Looking at the running times of the individual `assign_toks` commands, one can observe that they start with a low value, then steadily increase, until they start again with a low value. This can be seen graphically more easily than from the pure numbers; see Fig. 3, which plots the runtimes of 2000 consecutive measurements of `assign_toks` in line 172. Sorting the table entries by their runtime reveals that the overall lowest value for the command is 0.337 μs, while the maximum is 58.713 μs. This is quite a difference!

2.2 Code

To further understand what is going on, it is necessary to understand the purpose of the code (see Fig. 4) and its implementation.

The purpose is simple: The `cweb` system outputs an index file containing lines like this:

```
\I\\{above}, \[207], 1045, 1177, 1178, 1179.
```

which means that the identifier *above* is defined in section 207 and is used in sections 1045, 1177, 1178, and 1179. A section that defines an identifier is enclosed in `\[...]` and should be underlined in the index. To make the index more useful in a PDF document, the section numbers need to be converted to PDF links that can be clicked to navigate to the respective section. This is achieved using the macro `\pdflink`. It takes two arguments: the text of the link, which is the section number, and the name of the link, which for simplicity is the section number as a roman numeral. So the number `207` from our example needs to become `\pdflink{207}{ccvii}`. (Newer versions of the macro package do not require the roman numerals, but the principle is the same.)

I will not go into all the details of the implementation given in Fig. 4. You can find an analysis of the code's operation in [1]. Here I will just focus on the token list assignments. Let's start with the macro `\makenote` in line 172, which is expanded for each section number in an index entry. Its main action is calling `\addtokens`. `\toksB`, a token list register that contains all the `\pdflink`s created previously for the current index entry, is the first argument of `\addtokens`. The second argument is the code for a new `\pdflink` taking the section number from `\toksC`.

In our example, let's assume the `\pdflink` to section 207 is already in `\toksB`; then TEX's log file (using `\tracingmacros=1`) shows how `\addtokens`, defined in line 156, is appending the link to section 1045:

```
\addtokens #1#2->\edef\addtoks
   {\noexpand #1={\the #1#2}}\addtoks
#1<-\toksB
#2<-\noexpand\pdflink
   {\the\toksC}{\romannumeral\the\toksC}

\addtoks ->\toksB={\[\pdflink{207}{ccvii}],
    \pdflink{1045}{mxlv}}
```

When processing the body of `\addtokens`, TEX finds `\edef\addtoks`, creating a new definition with an expanded body containing an assignment to `\toksB`. Immediately after that definition is complete, TEX expands `\addtoks`, and there it finds `\toksB={...}`. When TEX finally expands `\toksB`, it triggers the `assign_toks` command. The profiler associates this command with line 172 because the control sequence `\toksB` comes from line 172 and was passed as a parameter to `\addtokens` and then to `\addtoks` without being expanded.

As the token lists assigned to `\toksB` get longer and longer, the assignments get slower and slower, as we will see in the next section.

The considerable extent of this slowdown is visualized in Fig. 3. Out of the 14 935 measurements, the graph shows only 2000 measurements, starting at measurement number 8000. Each of the 2000 data points corresponds to one `\pdflink` and each triangle in the graph corresponds to one index entry. The biggest triangle in the graph, starting with measurement 8623, corresponds to the index entry for *null* which references 317 different sections. At the

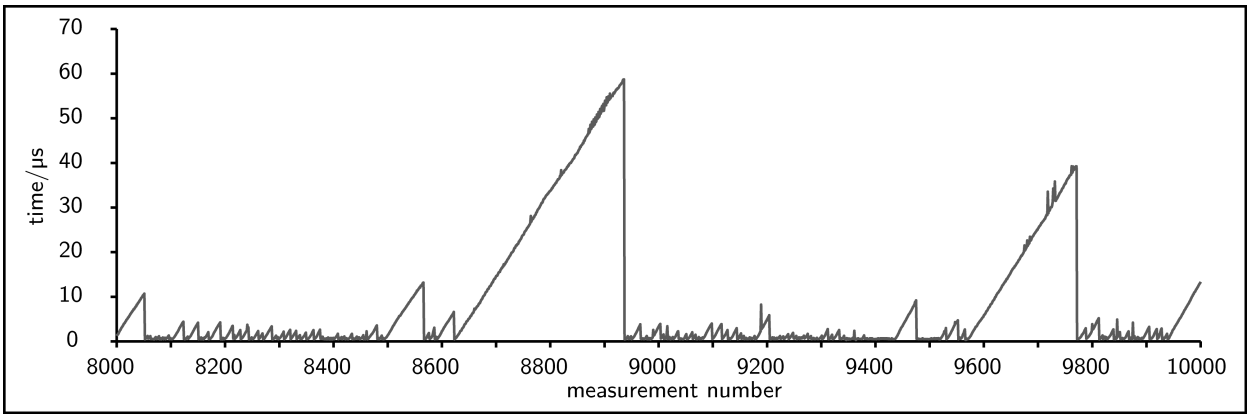


Figure 3: Runtimes of `assign_toks`

```
\def\addtokens#1#2{\edef\addtoks{\noexpand#1={\the#1#2}}\addtoks}
\def\poptoks#1#2|ENDTOKS|{\let\first=#1\toksD={#1}%
  \ifcat\noexpand\first0\countB=`#1\else\countB=0\fi\toksA={#2}}
\def\maketoks{\expandafter\poptoks\the\toksA|ENDTOKS|%
  \ifnum\countB>`9 \countB=0 \fi
  \ifnum\countB<`0
    \ifnum0=\countC\else\makenote\fi
    \ifx\first.\let\next=\maketoksdone\else
       \let\next=\maketoks
       \addtokens\toksB{\the\toksD}
       \ifx\first,\addtokens\toksB{\space}\fi
    \fi
  \else \addtokens\toksC{\the\toksD}\global\countC=1\let\next=\maketoks
  \fi
  \next
}
\def\makenote{\addtokens\toksB
  {\noexpand\pdflink{\the\toksC}{\romannumeral\the\toksC}}\toksC={}\global\countC=0}
```

Figure 4: Lines 156 to 173 of `cwebacromac.tex` (version 1.1)

beginning of the index entry, `\toksB` is reset to an empty list, and while the 317 links of the entry accumulate in `\toksB`, the runtime of the `assign_toks` command rises from $0.466\,\mu$s when adding the first link to an empty `\toksB` to $58.713\,\mu$s when adding the last link to `\toksB`, when it already contains 316 links.

The next index entry after "*null*" is the entry for "null delimiter", which references only two sections. The little triangle formed by these two runtime measurements (450 ns and 586 ns) is barely visible.

The runtime apparently depends linearly on the length of `\toksB`. As a consequence, the processing of an index entry with n sections is proportional to $n^2/2$ as visualized in Fig. 3 by the triangular areas between the x-axis and the curve. A look at TEX's implementation of the `assign_toks` command explains why this is the case.

2.3 TEX's implementation

The `assign_toks` command removes the "`=`" from the input and finds the "`{`". It puts the left brace back into the input and calls *scan_toks*, a general procedure for reading a token list from the input. The first thing *scan_toks* does is check for the mandatory left brace (the one just pushed back into the input) and removing it. Then it makes a copy of the tokens it finds in the input. The `\edef` has already expanded the body of `\addtoks` (except the token after `\noexpand`) so TEX finds the copy of `\toksB` made by `\the#1` and the tokens of `#2`. It copies these tokens from the input into a list of newly allocated tokens until the matching right brace is found.

Before the pointer to the freshly allocated list can be stored in `\toksB`, the old pointer in `\toksB` must be properly deallocated. So TEX decrements the reference count for the old token list, causing it to go down to zero, which means that the whole token list must be deallocated. Deallocation of a token list requires traversing the complete token list to find the last element of the list because TEX's token lists are implemented as singly linked lists.

In summary, the `assign_toks` command requires two traversals of the token list in `\toksB`: one traversal of the old value to deallocate it, and one traversal of the new value to copy it from the body of `\addtoks`.

One more copy of the old value is made by `\the` when creating the body of `\addtoks` and yet another traversal of the body of `\addtoks` is necessary to deallocate it before `\edef` can replace the old definition of `\addtoks` by the new one. The time of both operations is charged to TEX's `def` command.

Further copies of token lists are needed when `\maketoks` and `\poptoks` remove one token at a time from `\toksA`. The time to copy `\toksA` to the argument stack when calling `\poptoks` is charged to the first command of `\poptoks`'s body which is `\let\first`. This is, perhaps unjustly, the reason why the `let` command appears to be so expensive.

2.4 Possible improvements

Using token list registers as the code of Fig. 4 does is neither very readable nor efficient. The code uses the token list register `\toksB` as a queue or buffer, adding new tokens at the end for each new section number read from the index file. Once a complete index entry is parsed, the token list in `\toksB` is copied to TEX's input stack and `\toksB` is flushed. Appending tokens to the end of a linked list is never very efficient — especially if the list is long. If you have to do it, keep the list short. For example, it would be better to send each `\pdflink` to TEX's input stack as soon as it is finished instead of collecting all links for the entire entry.

But even removing tokens from the beginning of a token list, as done in lines 158 and 159 with the tokens in `\toksA`, is a slow operation. It is done by copying the entire list to the input stack, appending `|ENDTOKS|` as a sentinel, removing the first element from the input stack, copying the remainder of the input stack up to the sentinel into a new token list, deallocating the old token list, and storing the new token list.

It would certainly be easy enough, if demand for such a feature were strong enough, to add new token list register access primitives to TEX: one primitive to add a token at the beginning of the list and another to remove a token from the beginning of the list would be easy to implement, very fast, and have runtimes independent of the size of the list. This would enable TEX macro programmers to use token list registers as stacks. And stacks come in very handy for all kinds of parsing problems. Changing TEX's internal implementation of token list registers slightly, to use circular linked lists, would allow primitives adding tokens at either end of the list in constant time. Then token lists could also be used as queues.

3 Scanning with finite state machines

Here I propose a different approach, not using token lists at all. It is general enough to be extended to many different parsing problems, is very efficient, and I would claim its code is even more readable than the code of Fig. 4. The class of parsing problems that the method described in the following can handle is the class of "regular" expressions. An expression is "regular" if the syntax rules describing it are not recursive. As an example, one can describe the syntax of the list of section numbers by the following rules:

⟨*list*⟩*:* ⟨*section*⟩ (`,␣`⟨*section*⟩)⋆ `.` *;*

This rule explains what a ⟨*list*⟩ looks like. After the colon follows the explanation. It consists of a ⟨*section*⟩ followed by zero or more occurrences of a comma, a space, and another ⟨*section*⟩. Characters that occur literally in the input are shown in typewriter font. In regular expressions, the star means "repeat zero or more times", and parentheses are used to group things together. At the end of the list there is a (literal) period. The final (italic) semicolon just marks the end of the rule.

The next rule explains that a ⟨*section*⟩ has two forms: ⟨*definition*⟩ or ⟨*use*⟩. As is conventional with regular expressions, a vertical bar is used to denote the "or".

⟨*section*⟩*:* ⟨*definition*⟩ | ⟨*use*⟩ *;*

As we have seen, section numbers that define an identifier are enclosed in `\[...]`.

⟨*definition*⟩*:* `\[`⟨*use*⟩`]` *;*

We are almost done, but there is one slight complication: The section number might be followed by `\*` if the section was changed by a change file. There is no need to know here what a change file is, it's enough to know that there might be a `\*` in the input after the number and it should be typeset using the `\lapstar` macro. (Don't confuse `*`, a star in the input, with ⋆, the repetition operator.)

⟨*use*⟩*:* ⟨*number*⟩ | ⟨*number*⟩`\*` *;*

It remains to explain what a ⟨*number*⟩ is; in our case, nothing more than one or more digits:

⟨*number*⟩: ⟨*digit*⟩ ⟨*digit*⟩⋆ ;

⟨*digit*⟩: 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 ;

These simple rules define exactly the format of the input one should expect in the index file. The theory of parsing tells us that it is possible to process a regular expression with a finite state machine. Without bothering you with the details of the theory, a finite state machine is a machine that is, as the name suggests, always in one of a finite number of states. In any state, it will look at the next item in the input and based on the current state and what it finds in the input, it decides what to do and what its next state should be.

If we program in TEX, we write one macro for every state and pass it the next token from the input as a parameter. The macro then uses a few `\if`s to inspect the parameter and decide what to do and what the next state should be. Now, "doing" something in TEX means reading input and expanding it. So the things a macro wants to do and the macro that implements the next state must be written to TEX's input stack to be processed after the macro has made its decision. For this purpose a much underestimated primitive of TEX comes in handy: the primitive `\aftergroup`.

`\aftergroup` takes a single token as an argument and saves it, unexpanded, on TEX's stack used for grouping. You probably have used this stack already many times. If you write `{\bf...}`, you open a new level of grouping, switch to the bold face font, and when you end the group with the closing brace, you expect that TEX remembers what font you had before the group and restore it. The information about the font in use before the group is saved on this stack. Because using this stack to save things and restore them at the end of a group is so common, it has a very efficient implementation. For `\aftergroup`, saving the token and pushing it back on TEX's input stack is a matter of a few machine instructions.

So let us consider the various states of the machine we want to build one at a time. Fig. 5 gives a graphical representation of what we are about to build.

3.1 State A

We call the start state of our machine state "A" and the macro `\stateA`. The `\stateA` macro takes one parameter `#1` that will contain the first token from the input. At the start, we expect a ⟨*list*⟩ in the input, which starts with a ⟨*section*⟩, which is either a ⟨*definition*⟩ or a ⟨*use*⟩. The machine can tell the former from the latter, because the former starts with the `\[` macro. So we use `\ifx` to test if `#1` is the `\[` macro. If that is the case, we use `\aftergroup` (Oops! That should remind us that we should have started a group in the first place.) to push `#1` back in the input. We consider how to define the `\[` macro properly later. In the other case we know that the next thing in the input is a ⟨*use*⟩ and we will define state B to handle that. To get into state B, we use `\aftergroup\stateB`. And to arrange things so that the finite state machine will find the first token of ⟨*use*⟩ in state B, we also use `\aftergroup#1`. After all that, we are done with the `\ifx` and we close the group. State A is a typical example of a state that looks at the next token without truly removing it from the input. But it decides, based on this token, what to do next. Here is the code for `\stateA`:

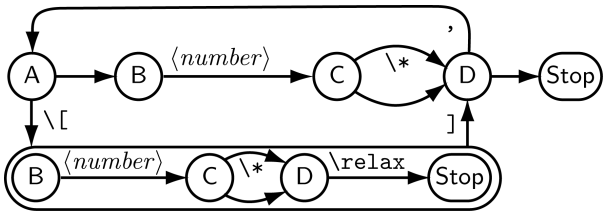


Figure 5: The finite state machine

```
\def\stateA#1{{% begin of group
  \ifx\[#1\aftergroup#1%
  \else\aftergroup\stateB\aftergroup#1%
  \fi
}}% end of group
```

3.2 State B

In state B, we know that the next thing in the input is a ⟨*use*⟩ which always starts with a ⟨*number*⟩ which in turn starts with a digit: one of the characters 0, 1, ..., 9. We could therefore start state B with an `\if` to find out what digit is in `#1`, process it, and go to a new "find out if there are more digits and process them" state.

But numbers are so common in input files that we will take a shortcut. TEX already knows how to read a number from the input. After a count register, TEX scans an optional equal sign and then scans a number and puts it into the register. Fine! But how do we tell TEX that it should continue with state C after scanning the number? We can't use `\aftergroup` here, but `\afterassignment` comes to the rescue. TEX puts its argument, a single unexpanded token, into a global variable which it will examine just after an assignment has been performed. This is the code for `\stateB`:

```
\newcount\stateBnumber
\def\stateB{\afterassignment\stateC
  \stateBnumber
}
```

If you read this short macro, you may wonder if it would not have been possible to pack all that into the `\else` case in `\stateA`, and if that would not have been slightly faster. The answer is yes. But having it in a separate macro is more readable. And we do not want to sacrifice readability for speed if not absolutely necessary.

3.3 State C

When we get into state C, the number is in the `\stateBnumber` count register and we can create the `\pdflink`. Then we check the next token which is in `#1`, and if it is `\*`, we typeset it as `\lapstar`, otherwise we put it back into the input, and we have reached the end of the ⟨*use*⟩ which is also the end of the ⟨*section*⟩. At this point the finite state machine goes into state D. State C is a typical example of a state that handles an optional token.

```
\def\stateC#1{%
  \pdflink{\the\stateBnumber}%
     {\romannumeral\the\stateBnumber}%
  {\ifx\*#1\lapstar\aftergroup\stateD
   \else\aftergroup\stateD\aftergroup#1%
   \fi
}}
```

3.4 State D

The finite state machine gets into state D when it has reached the end of a ⟨*section*⟩. So we have to check the next token to see if it is a comma; if so, typeset a comma and a space and continue with scanning the next ⟨*section*⟩. At this point, we observe that we have already a state that scans a ⟨*section*⟩, namely state A. So we do not need to define a new state again, but instead enter a state that is already defined. This is an important observation. Without it, we would not have a finite state machine, and infinite state machines are quite impractical. If the next token is not a comma, we put it back in TEX's input stack and end the scanning.

```
\def\stateD#1{{%
  \ifx,#1,\space\aftergroup\stateA
  \else#1\fi
}}
```

3.5 The `\[...]` macro

Now that we're used to defining states, we could as well have made the title of this subsection "State E", and instead of putting `\[` back into the input in state A, we could have put `\stateE` into the input. But all this would have just been a different name for the same macro. The macro we have to define now expects a ⟨*use*⟩ followed by a `]` in the input.

But there is a subtle difference between these two alternatives. Calling the next macro `\stateE` would suggest that we continue to follow the "finite state machine pattern" which would have the following consequence: State E would be in the same situation as state B with a ⟨*use*⟩ in the input, but we can not simply reuse state B because somewhere in the future there will be the matching `]` lurking in the input, and after state B, inevitably there will be state C and then D. And state D is not prepared to handle the `]`. Of course it would be possible to modify state D to ignore the extra `]`. But this would not be a clean solution. The scanner would then accept the extra `]` even without the `\[` and it would be harder to know where to end the underlining that is implied by the use of `\[`. Hence a clean solution would be to create duplicates of states B, C, and D and call them states E, F, and G. Then in state G there would be a check for the mandatory `]` and the end of the underline before state G would continue with state D.

Of course it is not desirable to duplicate a whole sequence of states just to remember that there was an opening `\[` and check for the closing `]`. For this reason, we keep the macro name `\[` and have TEX's parameter passing algorithm find the closing `]` and pass all the tokens between the brackets as argument to the macro. The advantage of this approach is that we can now feed the argument tokens to the `\stateB` macro to scan the ⟨*use*⟩. To make sure that `\stateD` eventually finds a token that terminates the scanning gracefully, we add a `\relax`. After the ⟨*definition*⟩ is completely processed, we call `\scanD` to check for a comma and continue scanning. This all may sound complicated, but the code is simple:

```
\def\[#1]{%
  $\underline{\stateB #1\relax}$\stateD}
```

What the code here does is call the scanner recursively inside the scanner. If you know the theory behind all that, you might hope that this technique will extend the capabilities of the scanner beyond the limits of regular expressions, but this is not the case because TEX's parameter passing algorithm will not look for a matching `]` but simply for the first `]`.

3.6 Runtime

After going through all this work, what is the effect on the runtime?

Comparing Fig. 6 with Fig. 2 one can see that the first three entries, `assign_toks`, `def`, and `let`

time	percent	count	command
53.28 ms	27.56 %	93 516	`extension`
24.10 ms	12.46 %	116 255	`right_brace`
22.06 ms	11.41 %	112 021	`after_group`
18.28 ms	9.45 %	83 504	`left_brace`
12.23 ms	6.33 %	12 159	`system`
11.06 ms	5.72 %	50 810	`other_char`
9.22 ms	4.77 %	27 666	`make_box`
7.14 ms	3.69 %	30 160	`assign_int`
6.16 ms	3.19 %	12 006	`par/match`
5.59 ms	2.89 %	24 586	`spacer`

Figure 6: `texprofile -C` improved

no longer appear among the top ten entries. Instead `extension`, with slightly more than 50 ms, moves from fourth to first place. `right_brace`, not unexpectedly, increases its share from 16 ms to 24 ms and is now second place, followed by `after_group` and `left_brace`. `system` follows again with 12 ms.

For the analysis in section 2, I have used the TEX profiler to measure the total runtime (1.90 s) and the runtime of just the index generation (1.07 s). With the new code, the total runtime is 1.00 s, and just for the index generation, I measure 0.16 s. So the time for the index processing goes down by about 0.9 s. The new code is more than six times faster than the old code and its runtime depends strictly linearly on the size of the input. For pdfTEX, I can only measure the total runtime: It goes down from 2.03 s (or 1.42 s in draftmode) to 1.25 s (or 0.64 s in draftmode). The speedup is about 0.8 s and the reduction in the total runtime is about 40 % respectively 50 % — a considerable improvement.

4 Conclusion

The TEX profiler is a useful tool to find sources of inefficiencies in TEX macro code. Furthermore, it is possible to quantify inefficiencies and improvements.

The use of TEX's token lists can get quite inefficient when the lists are long and the operations on the lists go beyond simple copy and paste. Using token lists to define the body of a control sequence can hide the fact that the control sequence is extremely long and as a consequence expensive to allocate and to deallocate. Hiding such complexities is critical if the definitions and redefinitions are part of long running loops.

Loops to process input files can in many cases be implemented using finite state machines. Finite state machines are a common abstraction and can lead to clean and understandable code that is also highly efficient.

References

[1] Martin Ruckert. Profiling TEX input files. *TUGboat*, 45(2):203–210, 2024. `https://tug.org/TUGboat/tb45-2/tb140ruckert-profiler.pdf`

◇ Martin Ruckert
Hochschule München
Lothstraße 64
80336 München
Germany
`martin.ruckert (at) hm dot edu`

A DVI driver for R

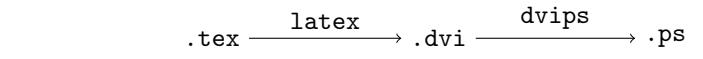
Paul Murrell

Abstract

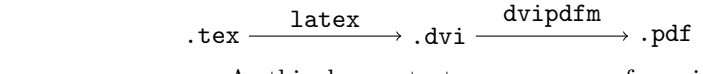
The R package `xdvir` provides a DVI driver that converts DVI files into R objects. This makes it possible to integrate LaTeX fragments as labels and annotations within R plots.

1 Introduction

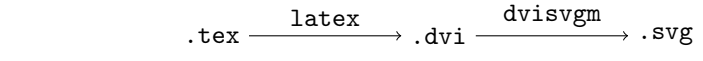
In the late 1990s I wrote my PhD thesis [2] in LaTeX. This involved editing `.tex` files within an X11 terminal using the `vi` editor, running `latex` on the `.tex` (LaTeX) files to generate `.dvi` (DVI) files, and then running `xdvi` to view the result on screen and `dvips` to generate `.ps` (PostScript) files for printing.



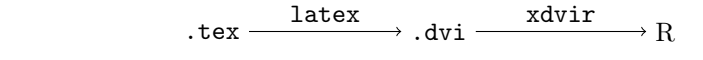
These days it is far more likely that `.tex` files are processed directly to `.pdf` (PDF) files using `pdflatex`. It is even more likely that LaTeX code is written in a WYSIWYG editor and processed to PDF at the press of a button, but the old ways still exist and they still work. For example, rather than going straight to PDF, we can generate DVI and then use a different DVI driver, `dvipdfm(x)`, to generate PDF.



As this demonstrates, one reason for going via DVI is that we can use different DVI drivers to generate different final formats. Another example is the `dvisvgm` driver which produces `.svg` (SVG) files.



This article describes a new DVI driver called `xdvir`. It is a little unusual because it is not a command line tool and it does not generate output files. Instead, this driver is an add-on package for the R language [5] and it processes `.dvi` files to generate R objects.



2 A simple example

In order to demonstrate the `xdvir` driver, we will work with a very simple LaTeX document, shown below. This LaTeX code is stored in a file called `stdnorm.tex`.

```
\documentclass{standalone}
\begin{document}
$Z \sim \mathcal{N}(0, 1)$
\end{document}
```

We can generate a DVI file from that LaTeX document with the following command. The `$` prompt is shown to indicate that this is typed in a command line interface, or a shell. The result is a file called `stdnorm.dvi`.

```
$ latex stdnorm.tex
```

We can generate a PDF file from the DVI file with the following command. The result is a file called `stdnorm.pdf`, which is shown below the command.

```
$ dvipdfm stdnorm.dvi
```

$$Z \sim \mathcal{N}(0, 1)$$

3 The `xdvir` driver

We need to run R to make use of `xdvir`. From a command line this just involves typing the letter "R", as shown below. Subsequent code examples will be within R, which will be indicated by a different prompt.

```
$ R
```

Within R, we load the R package `xdvir`.

```
R> library(xdvir)
```

The simplest thing we can do with `xdvir` is to view a DVI file. The following R code views the DVI file `stdnorm.dvi`. To keep things readable, the R output has been trimmed on its right edge, special operations have been removed, and we only show up to the first `set_char` operation (the math-italic Z).

```
R> readDVI("stdnorm.dvi")
```

```
pre           version=2, num=25400000, den=
              comment= TeX output 2025.02.2
bop           counters=1 0 0 0 0 0 0 0 0 0,
down3         a=-4736287
push
right3       b=-4736287
pop
right3       b=-4736287
y4            a=1073741823
push
down4         a=-1073741823
y0
push
down4         a=-1073086463
push
push
push
down3         a=-163840
fnt_def_1     fontnum=30, checksum=19506028
              fontname=cmmi10
fnt_num_30
set_char_90   5a
```


doi.org/10.47397/tb/46-1/tb142murrell-xdvir

This is one useful feature of the `xdvir` driver. It can print DVI files in a human-readable format, similar to tools like `dvitype` or `dviasm`.

4 Rendering LaTeX in R

A more useful feature of the `xdvir` package is the ability to render DVI files. A current limitation of `xdvir` rendering is that it requires font file paths in DVI font definitions and glyph indices in `set_char` operations. This means that we need to generate DVI using either a recent LuaTeX engine or the XeTeX engine. Furthermore, if we are generating mathematical equations, we need to make use of the LaTeX package `unicode-math`. Given these limitations, our simple example becomes the following LaTeX code.

```
\documentclass{standalone}
\usepackage{unicode-math}
\begin{document}
$Z \sim \mathcal{N}(0, 1)$
\end{document}
```

The following (shell) command generates (extended) DVI output from this LaTeX code using the XeTeX engine. The result is a file `stdnorm.xdv`.

```
$ xelatex --no-pdf stdnorm.tex
```

The following (R) code reads that DVI file into R and renders it. The image that R produces is shown below the code.

```
R> xdv <- readDVI("stdnorm.xdv")
R> render(xdv)
```

$$Z \sim \mathcal{N}(0, 1)$$

This result from `xdvir` is identical to the result from `dvipdfm` in Section 2 because the image above was saved from R as a PDF file. However, R is capable of producing a variety of graphics formats. This includes PDF and SVG, but also includes on-screen display and a range of raster formats, such as PNG and JPEG, plus a set of high-quality raster formats via the R package `ragg` [4].

5 Integrating LaTeX with R plots

All that we have done so far with `xdvir` is view a DVI file, which tools like `dviasm` can already do, and render a DVI file to a PDF file, which tools like `dvipdfm` can already do. The distinguishing feature of `xdvir` is that it allows LaTeX code to be integrated within an R plot.

As a convenience, `xdvir` allows the user to specify just a fragment of LaTeX code rather than a complete LaTeX document like we have been working with so far. For example, the following R code creates an R character value that contains a LaTeX fragment for the mathematical equation from previous examples.

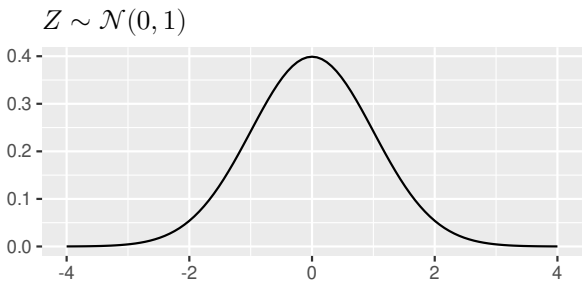


Figure 1: A `ggplot2` plot in R with a title that is based on a LaTeX fragment.

```
R> latexFrag <-
R+   r"($Z \sim \mathcal{N}(0, 1)$)"
R> latexFrag
[1] "$Z \\sim \\mathcal{N}(0, 1)$"
```

We can render that LaTeX fragment in R with the following R code. Behind the scenes, the fragment is augmented to produce a complete LaTeX document, the LaTeX document is processed with XeTeX to generate a DVI file, the DVI file is read into R and rendered. The resulting image, which should be getting familiar by now, is shown below the code.

```
R> grid.latex(latexFrag)
```

$$Z \sim \mathcal{N}(0, 1)$$

The purpose of that convenience is that R users can specify a LaTeX fragment when specifying the labels on an R plot. For example, the following code draws a plot using the popular R package `ggplot2` [7] and uses the LaTeX fragment above as the plot title. The result is shown in Figure 1.

```
R> library(ggplot2)
R> gg <- ggplot() +
R+   xlab(NULL) + ylab(NULL) +
R+   geom_function(fun=dnorm,
R+                  args=list(mean=0, sd=1),
R+                  xlim=c(-4, 4))
R> gg +
R+   labs(title=latexFrag) +
R+   theme(plot.title=element_latex())
```

It is also possible to position LaTeX fragments relative to the coordinate system within an R plot. The following R code uses the R package `gggrid` [3] to do this in a `ggplot2` plot.

```
R> library(gggrid)
```

An R function `label()` is defined to draw our LaTeX fragment at a location within the plot, centred on the x-position and with the baseline at the y-position.

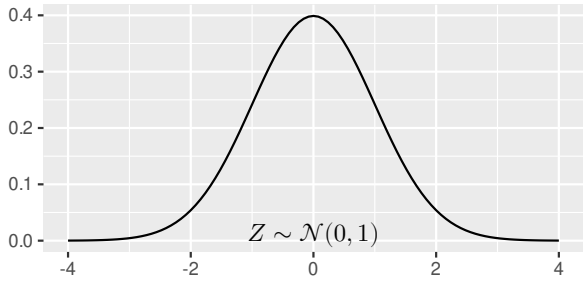


Figure 2: A `ggplot2` plot in R with an annotation that is based on a LaTeX fragment. The annotation is aligned with the coordinate system in the `ggplot2` plot.

```
R> label <- function(data, coords) {
R+   latexGrob(latexFrag,
R+              coords$x, coords$y,
R+              vjust="baseline")
R+ }
```

The following R code adds that label to the plot at the location $(0, 0)$. The result is shown in Figure 2.

```
R> gg +
R+   grid_panel(label,
R+               data=data.frame(x=0, y=0),
R+               aes(x, y))
```

6 Conclusion

This article has described a new DVI driver that generates R objects from DVI files. As already mentioned, `xdvir` is limited to DVI files that are generated by the XeTeX engine or a recent LuaTeX engine. Rendering of DVI files is also limited, by default, to standard LaTeX, plus a small set of LaTeX packages. For example, the LaTeX packages `fontspec` and `xcolor` are supported, so LaTeX fragments can contain changes in font family and text colour. The Ti*k*Z package is also supported, so Ti*k*Z pictures can be included as "labels" in R plots. It is also possible for R users to add support for further packages.

I would like to acknowledge the work of Yihui Xie in producing the TinyTeX distribution and the R package `tinytex` [8, 9]. The `xdvir` driver leans heavily on those tools for typesetting LaTeX documents to DVI files. I also have to acknowledge that the `xdvir` driver is not unique. In the Python world [6], it is possible to use LaTeX to format all text labels on a `matplotlib` plot [1].

While I hope that there will be some excitement from R users once they realise that they can use LaTeX in R plot labels, this article is more about what might get LaTeX users excited about the R package `xdvir`. To be honest, I only expect enthusiasm from R users who know a bit about LaTeX, so I can only reasonably expect enthusiasm from LaTeX users who know a bit about R. I hope that that is not an empty set.

References

[1] J.D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi.org/10.1109/MCSE.2007.55

[2] P. Murrell. *Investigations in Graphical Statistics*. Ph.D. thesis, The University of Auckland, 1998.

[3] P. Murrell. *gggrid: Draw with 'grid' in 'ggplot2'*, 2022. R package version 0.2-0. CRAN.R-project.org/package=gggrid

[4] T.L. Pedersen, M. Shemanarev. *ragg: Graphic Devices Based on AGG*, 2024. R package version 1.3.3. CRAN.R-project.org/package=ragg

[5] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2025. www.R-project.org/

[6] G. Van Rossum, F.L. Drake. *Python 3 Reference Manual*. CreateSpace, Scotts Valley, CA, 2009.

[7] H. Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2016. ggplot2.tidyverse.org

[8] Y. Xie. TinyTeX: A lightweight, cross-platform, and easy-to-maintain LaTeX distribution based on TeX Live. *TUGboat* 40(1):30–32, 2019. tug.org/TUGboat/tb40-1/tb124xie-tinytex.pdf

[9] Y. Xie. *tinytex: Helper Functions to Install and Maintain TeX Live, and Compile LaTeX Documents*, 2024. R package version 0.54. github.com/rstudio/tinytex

◇ Paul Murrell
paul (at) stat.auckland.ac.nz
www.stat.auckland.ac.nz/~paul/
ORCID 0000-0002-3224-8858

A new take on paragraphs

Hans Hagen, Mikael P. Sundqvist

1 Introduction

The excellence of the Knuth–Plass algorithm for breaking paragraphs into lines is one of the reasons for the success of TEX. It is very fast (built upon dynamic programming) and powerful (it can combine both justification and hyphenation in one go). The algorithm is built to use so-called demerits in a cost function to determine the optimal breakpoints.

The paragraph builder is however limited to at most three runs over each paragraph to get the job done. In this article we will describe some new ideas and tools regarding the process of paragraph building. What we describe is already available in LuaMetaTEX and ConTEXt. The main new feature is that it is now possible to have an arbitrary number of runs over each paragraph and to configure them independently.

If TEX is an example of "The Art of Programming", then we might approach some of its building blocks as pictures. These come in flavors; some are concrete and show a scene that leaves no doubt about what is pictured. Others are more abstract and can let us imagine or experience something and anyway leave interpretation to the viewer. When old paintings are restored quite often layers under the top layer show something different. The canvas might have been repurposed or we can see intermediate (even different) versions of what the final result is. Modern paintings can use paint that was hip at that moment but was not durable over a long term, so drastic measures are needed.

The par builder code in LuaMetaTEX has all these aspects: features were added, some on top of others, the code and algorithm is open for interpretation, some tricks relate to the toolkit used. This makes fundamental extensions hard and a rewrite has the danger of losing compatibility. To quote from Knuth's TEX source:

> This particular part of TEX was a source of several subtle bugs before the correct program logic was finally discovered; readers who seek to improve TEX should therefore think thrice before daring to make any changes here.

The original TEX par builder is (at least for us) not something that immediately reveals its workings. Since the logic is a bit fuzzy to us, we have to be able to analyze what we see. There are many parameters like `\pretolerance` and `\tolerance`, as well as badness, penalties, demerits and all of these plus slack in a line leads to a conclusion about how bad breakpoints are. Add to that comparing neighboring lines with respect to how much the applied spacing differs.

On top of that ε-TEX added some layers (like last line related) and pdfTEX added even more due to expansion and protrusion. We can see some remnants of Omega (Aleph) like local boxes, too. The LuaTEX approach separated the hyphenation, ligature building and kerning from the main task. Then LuaMetaTEX added more control, various new features, and node list normalization from the perspective of access by Lua.

So, the whole picture becomes more complex and abstract over time. And one indeed has to be careful when adding new features to it. Some comments in the source indicate that coming to the right solution has been a step-wise process. Just like painters made their own paint we have all kinds of helpers. We can trace what TEX does, and what solution was considered best. We can do that visually as well as via extensive logging. These helpers have been invaluable in the work to extend the paragraph builder.

The idea to use more runs over the paragraph is however not new. In D.E. Knuth's *Digital Typography* we can read the following:

> On the other hand, some paragraphs are inherently difficult, and there is no way to break them into feasible lines. In such cases the algorithm we have described will find that its active list dwindles until eventually there is no activity left; what should be done in such a case? It would be possible to start over with a more tolerant attitude toward infeasibility (a higher threshold value for the adjustment ratios). TEX takes the attitude that the user wants to make some manual adjustment when there is no way to meet the specified criteria, so the active list is forcibly prevented from becoming empty by simply declaring a breakpoint to be feasible if it would otherwise leave the active list empty. This results in an overset line and an error message that encourages the user to take corrective action.

Maybe it was the limitations of computers at that time that prevented more runs? So, given the faster computers and already opened-up code base, which permits extensive visual tracing, we decided to play with multiple passes, a mechanism that will be discussed below. When documenting this we occasionally went back to Knuth's descriptions, like the ones above, and admit that some started making sense only in retrospect. For instance the "so the

doi.org/10.47397/tb/46-1/tb142hagen-paragraphs

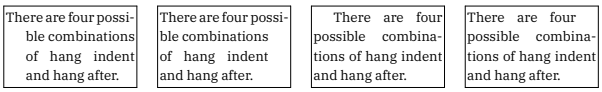


Figure 1: hangindent

active list is forcibly prevented from becoming empty by simply declaring a breakpoint to be feasible" action was something that we had to circumvent in order to let additional passes kick in at all. We still learn.

2 The traditional par builder

Before we move on and discuss the possibility of using more paragraph passes, we will discuss in a bit more detail how the traditional par builder works. This will help us to better understand the various extensions in LuaMetaTEX.

For good order a paragraph is a horizontal list, wrapped over lines. This happens either in a `\vbox` or in the main vertical list (page). When such a list is broken, TEX has to keep track of the current width of a line. That width can change depending on the so-called par shape or hanging indentation.

We show the hanging indentation in Figure 1, which makes clear that it adds a constraint. We can also have an indentation on the first line, left and right par fill skip (last line) as well as left and right init skip (first line). Then there are left and right skip but these are the same for every line. All this means that the par builder has to keep track of the current line, in order to set the current width.

If we render table cells, or captions, or a narrow quote, or text flowing around an image, the width can be a limiting factor and combined with penalizing hyphenation, multiple hyphens in a row, specific demands like inline math, the solution space can become cramped but we will notice that the engine can quite well deal with these situations, unless of course we leave no room, for instance by setting every penalty that plays a role to 10000 or demerits to the maximum number possible.

It is also good to keep in mind that a macro package can have features that interfere with what otherwise would be a pristine paragraph. Think of a forced linebreak (`\crlf`), binding words (using `~`), switching fonts and thereby spacing, ligatures and kerning, changing to a language with fewer or more short words, compound words and possibly different hyphenation rules, verbatim, which normally runs wider and doesn't hyphenate.

TEX first tries to break the paragraph list into lines without using hyphenation, within the constraints of the `\pretolerance` value. If this fails a second pass will use `\tolerance` as the constraint, with hyphenation enabled. The verdict is also influenced by various penalties, for instance those that penalize one or more hyphens at the end of lines. If the outcome is still not right, a third pass permits `\emergencystretch` to be applied.

The decision to enter a next pass is determined by a valid result. So, if a pass processes the whole list within the constraints we have a result and no further passes are done. When there is no result, the next pass will be entered. We can skip the first pass by setting `\pretolerance` to -1, and the third pass won't happen if we have no emergency stretch. It is important to have a final pass, because TEX has to make sure to provide a result. Thus, we can have the following cases.

1. pretolerance tolerance
2. pretolerance tolerance stretch
3. tolerance
4. tolerance stretch

Take the first situation: we run the pretolerance pass, if there is a valid result we quit, otherwise we run the tolerance pass which is tagged final and therefore will be forced to always have a result by dealing with troublesome breakpoints. There is no third pass because emergency stretch is zero. This is where the majority of TEX users end up.

In the second case we can succeed after the pretolerance pass and quit, or carry on with the tolerance pass, where again we can succeed or carry on. The stretch pass is the final one and it must result in something.

The third case is final right from the start so the first pass will always result in something, no matter how bad. The fourth case can succeed after the tolerance pass but can carry on with the last pass, which then must return a result.

Traditionally, TEX can break lines

- at glue (after words, not usually inside math),
- at a kern followed by glue,
- at a discretionary (hyphenation),
- due to a penalty (also inside math).

We will look at many examples below, and for them we will (re)use a paragraph written by the famous mathematician and physicist P.A.M. Dirac.[1]

As TEX runs over a paragraph, a badness value is attached to each possible breakpoint. With the default parameter settings of `\pretolerance` to `100` and `\tolerance` to `200`, many of the possible breakpoints are discarded, since the badnesses attached to them are greater than the tolerance; they would

[1] From his article "Pretty Mathematics", *Internat. J. Theoret. Phys.* vol. 21, no. 8–9, pp. 603–605, 1981/82. Presented at the Dirac Symposium, Loyola University, New Orleans, May 1981.

I can give a good example of this procedure. At one time, in 1927, I was playing around with three 2×2 matrices whose squares are equal to unity and which anticommute with one another. Calling them $\sigma_1, \sigma_2, \sigma_3$, I noticed that if one multiplied them into the three components of a momentum so as to form $\sigma_1 p_1 + \sigma_2 p_2 + \sigma_3 p_3$, one obtained a quantity whose square was just $p_1^2 + p_2^2 + p_3^2$. This was an exciting result, but what use could one make of it?

Figure 2: Feasible breakpoints are marked with vertical bars. Here we are using the default settings `\pretolerance=100` and `\tolerance=200`.

1	1	0	13	529	loose	glue
	2	0	3	490169	decent	penalty
	3	0	45	3025	tight	math
2	4	1	7	818	decent	glue
	5	3	6	25781	decent	glue
	6	3	38	27829	tight	glue
3	7	4	8	1142	decent	glue
	8	4	1	939	decent	glue
	9	4	15	12054	tight	glue
	10	6	0	27929	decent	glue
	11	5	0	31710	tight	glue
4	12	7	54	5238	loose	glue
	13	8	14	1515	loose	glue
	14	7	14	1242	decent	glue
	15	8	8	1039	decent	glue
	16	7	8	2511	tight	glue
	17	10	1	28050	decent	glue

11	5 3
12	7 4 1
13	8 4 1
14	7 4 1
15	8 4 1
16	7 4 1
17	10 6 3

pass : 1 demerits : 1139
subpass : P looseness : 0
subpasses : 0

Figure 3: A table representing the tree TEX uses for the example paragraph. The various columns are described on p. 108.

simply lead to non-optimal (within our measurement of tolerance) lines. In ConTEXt we can use the pair of commands `\startshowbreakpoints` and `\stopshowbreakpoints` to show the possible breakpoints; see Figure 2.

In fact, each breakpoint (except the first one, which sits at the start of the paragraph and has index 0) points to a previous breakpoint; a tree is

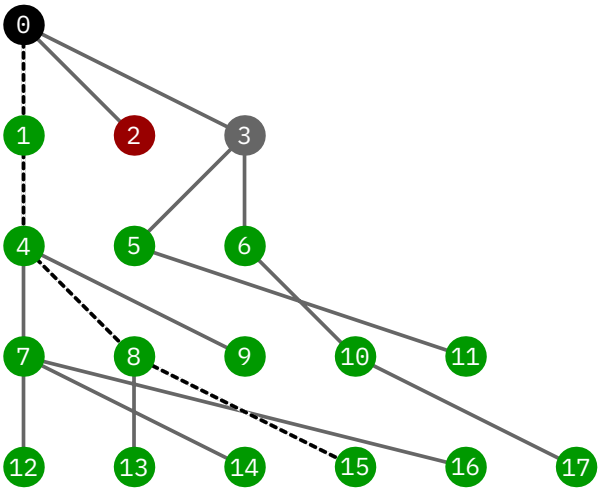


Figure 4: The tree TEX has after going over the paragraph, with the selected path dashed.

built. With `\showbreakpoints` we can get some information about the breakpoints that TEX kept until it was time to make the choice; see Figure 3. We will discuss the details a bit later when we have introduced the relevant concepts.

We can also draw a representation of the tree with `\drawbreakpoints`, where each line represent the possible breakpoints for different lines; see Figure 4. Some of the nodes are kept by TEX until the end, even though no later node points back to them.

By setting the parameters `\pretolerance` to `-1` and `\tolerance` to `10000`, we can fool TEX into viewing all possible breakpoints in a paragraph as feasible, not throwing away any of them; see Figures 5 and 6.

We have mentioned badness and tolerance, and that breakpoints are discarded if their badness is larger than the tolerance. We will next explain how badness values are calculated. To do that, we first need the *adjustment ratio*.

When TEX starts to run over a paragraph, it knows the desired length of each line. Usually these lengths are fixed, but they can vary a bit depending on (hanging) indentation, or even more advanced par shapes. While running over the paragraph, there

I can give a good example of this procedure. At one time, in 1927, I was playing around with three 2×2 matrices whose squares are equal to unity and which anticommute with one another. Calling them σ_1, σ_2, σ_3, I noticed that if one multiplied them into the three components of a momentum so as to form $\sigma_1 p_1 + \sigma_2 p_2 + \sigma_3 p_3$, one obtained a quantity whose square was just $p_1^2 + p_2^2 + p_3^2$. This was an exciting result, but what use could one make of it?

Figure 5: All possible breakpoints considered by TEX.

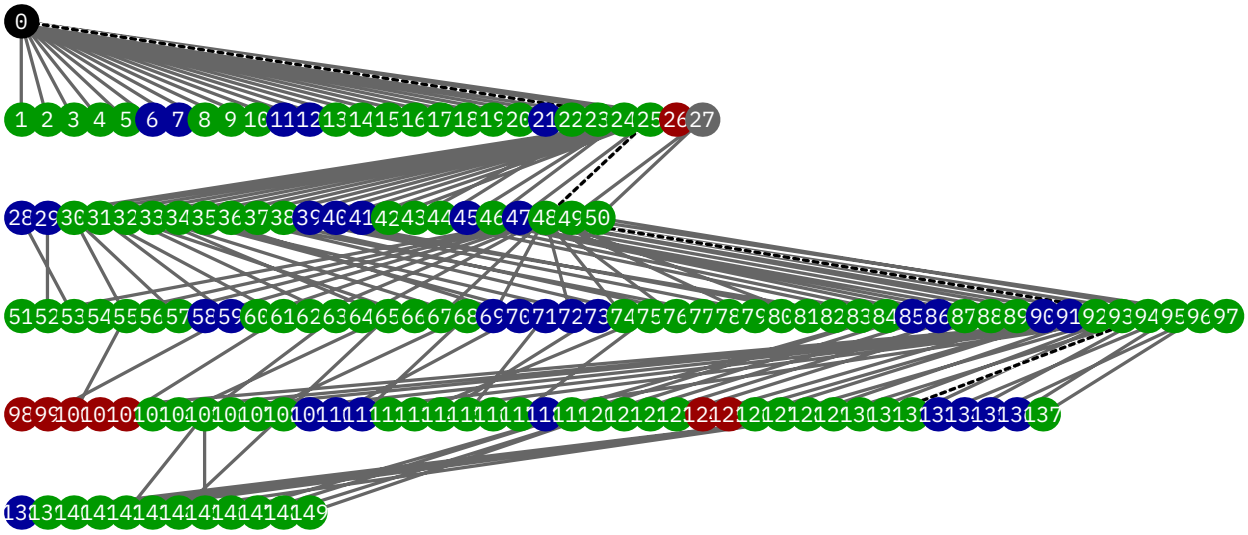


Figure 6: All possible breakpoints, drawn in a cramped manner. We observe that among all solutions, some are one line longer than the selected solution.

will be a few active nodes. At the start, it is only the one that we have marked with a 0 in the upper left of the figures. Then every possible breakpoint points back to node 0. When TEX goes on, it will check for each possible breakpoint in turn, whether it, when pointing back to 0, will give a line that is OK according to the rules set up. If so, it will add that point to the list of active nodes and move on. Once we come to a point where the first line would be more than completely filled, it will deactivate node 0. Hopefully, there will be new active nodes to test new possible breakpoints against (if not, the run fails). Other active nodes will also be deactivated in a similar way as TEX moves on.

When TEX creates an active node it also creates a so-called passive node, which carries additional information. It is the passive nodes that point to the previous breakpoints and build the tree. There can be multiple nodes pointing to a node, but only the one with the fewest demerits in each fitness class is kept. If an active node is deactivated, the passive nodes are not cleaned up. (This is also why we can generate the tree graphics.)

We now assume that we have a new possible breakpoint, and an active one that it might be able to point back to. Let ℓ be the desired length of the corresponding line (which is known). Let L be the total *natural width* of what we have so far (without stretch and shrink), calculated from the active breakpoint, considered at the moment, up to the current candidate. Also, let $Y > 0$ be the total *stretchability* and $Z > 0$ be the total *shrinkability*. (Negative values are possible but we leave them out of this discussion.) Define the adjustment ratio r as

$$r = \begin{cases} (\ell - L)/Y \,, & L < \ell \,; \\ 0 \,, & L = \ell \,; \\ (\ell - L)/Z \,, & L > \ell \,. \end{cases}$$

The closer r is to 0, the less stretch or shrink is needed. When $r = -1$, all available shrink is asked for, and when $r = 1$ all stretch is needed.

The *badness* β of the breakpoint is defined as (here $\lfloor x \rfloor$ denotes the integer part of a real number x)

$$\beta = \begin{cases} +\infty \,, & r < -1 \,; \\ \lfloor 100|r|^3 + 0.5 \rfloor \,, & \text{otherwise.} \end{cases}$$

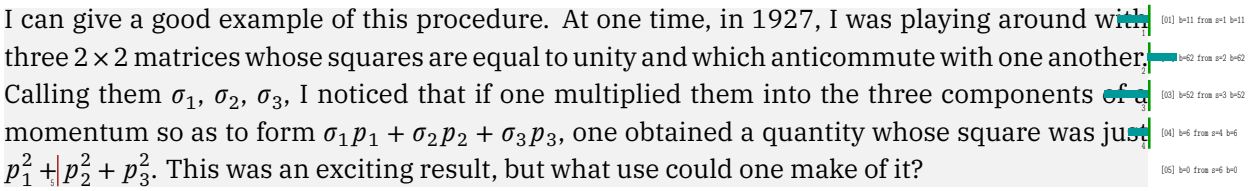


Figure 7: The turquoise bars at right indicate how much the first, third and fourth lines are stretched and therefore a bit loose. The second line is shrunk and therefore tight.

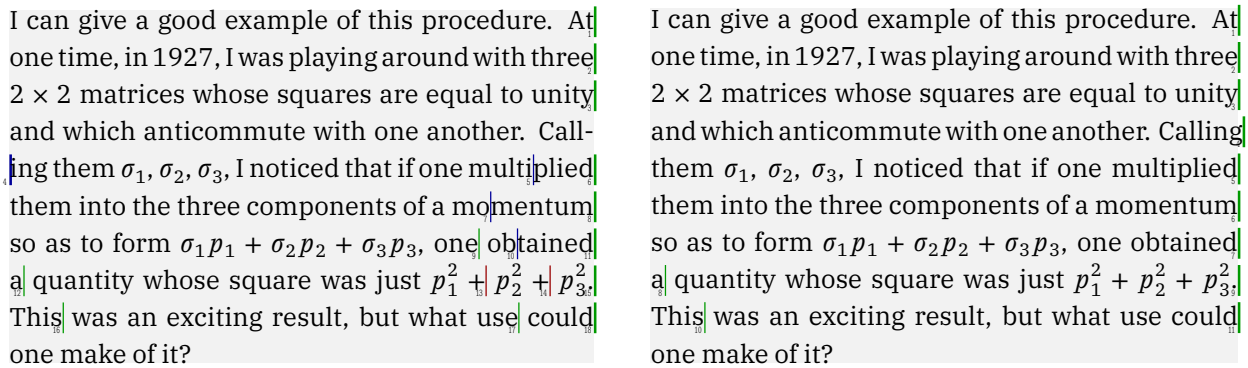


Figure 8: Left: a narrow paragraph, with hyphenation enabled. Right: The same narrow paragraph, with hyphenation disabled.

The badness does not depend on the sign of r. TEX never allows more shrink than specified, and therefore the badness is defined to be infinite if $r < -1$. We emphasize that TEX *does* allow more stretch than what is specified, so $r > 1$ is allowed in the second line of the badness calculation above.

It is possible in ConTEXt to show beside each line the badness values that were calculated for each used breakpoint; see Figure 7. The turquoise bars at right (grayscaled for print) indicate if the line is set tight or loose.

In Figure 8 we have set the paragraph on a narrower width. To the left, with hyphenations enabled, we get a solution that is OK, while to the right the word Calling is sticking out. Here the second run failed and TEX gave up on finding a good solution.

In TEX, hyphenations are controlled by penalties. We want to avoid hyphenation if possible, but we do not want to disable it completely, since then TEX will sometimes fail to find feasible solutions. Each hyphenated line costs a `\hyphenpenalty`, and there is a competition between different costs that determine how TEX will break the paragraphs.

Going even narrower, we will eventually end up in a situation where none of the first two passes succeed, with or without hyphenations enabled. With `\emergencystretch` unset (left in Figure 9), we get overfull lines sticking out. We absolutely want to avoid that, if at all possible. To the right we set `\emergencystretch` to `2em`. This means that each line gets an amount of `2em` extra stretch to distribute. For this paragraph `2em` was enough for TEX to find a solution without any line sticking out.

As discussed above, when TEX reads a paragraph, it will put possible breakpoints in a tree-like structure (in fact a single-linked list, with new entries added at the beginning). If the badness is greater than the (pre)tolerance, the breakpoints will be discarded. Once the paragraph is read, TEX will work on the tree with the breakpoints that survived, as shown in Figures 3 and 4 (unless there was a complete failure, and then TEX will typically produce a paragraph where some line sticks out, as in Figure 9, left). To choose among the possible breakpoints, TEX calculates demerit values for each possible solution, and the final choice is the path in the tree that adds up to the least total amount of demerits.

Let us explain the content in Figure 3. The first column denotes what linebreak we are looking at, so for example the three first rows all correspond to the breakpoint after the first line. Since there are three such rows there are exactly three possible line breaks. The second column is an index for the possible breakpoints. The third column shows what breakpoint each breakpoint is pointing at. We see that the first three point at 0, which means the beginning of the paragraph. The fourth breakpoint is pointing at breakpoint 1. The values in the fourth

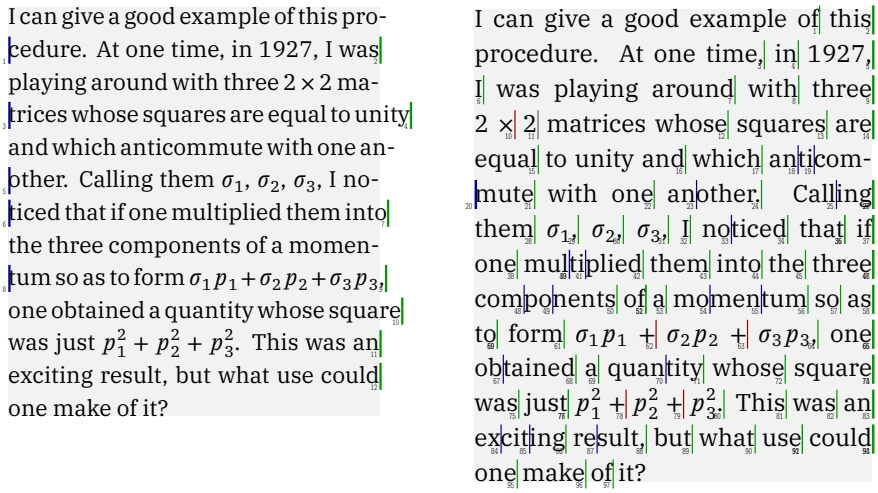


Figure 9: Left: a narrow paragraph. TEX has failed to find a solution with both its first runs, and we see several overfull lines. Right: The same paragraph with the emergency run enabled by setting `\emergencystretch` to `2em`.

column are the badness values. The fifth column keeps the (accumulated) demerits. The sixth column shows the fitness class of the break (more on that below) and the seventh column shows the type of break. The breakpoints that have colored numbers are the ones that are used.

After that we get a summary on what paths in the tree were still valid at the end; here there were seven, and $15 \to 8 \to 4 \to 1$, marked in color, was the chosen one. Looking at the paragraph we note that the breakpoints 15 and 16 sit at the same place. The difference is that they point to different previous breakpoints (8 and 7, respectively).

In the summary we also see what subpass was used (`P` means the pretolerance run). We also see that the total demerits was 1139, that we did not use (extra) paragraph passes, and not looseness (more on that later).

Next, we describe how the demerits are calculated. For each breakpoint, the following happens. Let β be the badness, ℓ the line penalty (by default set to 10), π the possible penalty, and α the additional demerits that correspond to a certain breakpoint (but that usually comes from a combination with the previous one). Then the demerits value δ for that breakpoint is defined by

$$\delta = \begin{cases} (\ell+\beta)^2 + \pi^2 + \alpha\ , & \text{if } \pi \geq 0\ ; \\ (\ell+\beta)^2 - \pi^2 + \alpha\ , & \text{if } -\infty < \pi < 0\ ; \\ (\ell+\beta)^2 + \alpha\ , & \text{if } \pi = -\infty\ . \end{cases}$$

The penalty π can come, for example, from a hyphenation (`\hyphenpenalty` is often set to 50) or a break inside a formula. The only places where traditional TEX breaks inside formulas are after binary operators such as + (default penalty 700) and binary relations such as = (500). This means that hyphenations are preferred over breaks inside formulas.

The additional demerits, α in the formula, come from the interplay of neighboring lines. There is for example `\doublehyphendemerits` (10000) that gets added when consecutive hyphenated lines are considered during line breaking, `\finalhyphendemerits` (5000) that is added if the final breakpoint of the paragraph is hyphenated and also `\adjdemerits` (10000) that is added when consecutive lines are considered to be incompatible with each other, say one with fitness class tight and one with loose. The values given above in parentheses are the ones that Knuth set up for plain TEX; they have survived also in other macro packages.

We stress that in the formula for demerits, the $\ell + \beta$ and π are squared, while α is not. This means that $\alpha = 10000$ corresponds to a penalty $\pi = 100$. This is important to have in mind when setting up the parameters, since when TEX chooses the line breaks, these are the kind of terms that compete with each other, and that influence the final choice.

Traditional TEX has four fitness classes: tight, decent, loose and very loose. They are attached to breakpoints and depend on the badness. The adjacent demerits are added when we jump over at least one fitness class. Looking at Figure 10, we see that it happens when we go from A to either C or D (from tight to either loose or very loose), or when we go from B to D (decent to very loose). The same amount is added both for jumps from A to C and from A to D, even though the latter is likely worse.

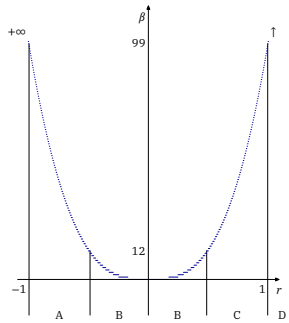


Figure 10: Traditional fitness classes. A: tight, B: decent, C: loose and D: very loose. The graph shows the badness β as a function of the adjustment ratio r. The jumps are due to the integer part in the formula (we want integers). The division into fitness classes is done so that the subintervals on the r-axis have the same length. This gives the thresholds 12 and 99 for the badness.

Problematic paragraphs can be tweaked manually. We can locally increase the tolerance to make TEX accept less good solutions, we can enable font expansion in order to stretch or shrink characters slightly, and thus enable line breaks that otherwise would be considered bad. Excessive use of expansion might lead to visually incompatible lines and ugly results. We can also, as mentioned and shown above, set the `\emergencystretch` to a positive value, and hope for a good final run.

Sometimes, the last line of a paragraph is too short. It can look bad if indentation is enabled and the size of the indentation is approximately the same, or even bigger, than the width of the last line in the paragraph before. One way to avoid such short last lines is to use orphan penalties (we will come back to them). If set to 10000, TEX may no longer break between the last two words.

When optimizing for the number of lines to stay on a page, or to add one extra, it is sometimes possible to shorten (or lengthen, but that usually does not give good results) paragraphs with help of `\looseness`. As an example, look at Figure 11. By adding `\looseness-1` we ask for a paragraph that is one line shorter than the number of lines that the optimal paragraph considered by TEX has, if possible. In this case it succeeded.

We emphasize once more that this will only work if there is a case among the possible solutions that TEX has collected that has the number of lines asked for. Also, an otherwise-successful pretolerance run might be discarded in the hunt for a paragraph with the number of lines asked for. In LuaMetaTEX the user will get a message in the log that tells if it was successful or not. The `\looseness-1` is local,

I can give a good example of this procedure. At one time, in 1927, I was playing around with three 2×2 matrices whose squares are equal to unity and which anticommute with one another. Calling them σ_1, σ_2, σ_3, I noticed that if one multiplied them into the three components of a momentum so as to form $\sigma_1 p_1 + \sigma_2 p_2 + \sigma_3 p_3$, one obtained a quantity whose square was just $p_1^2 + p_2^2 + p_3^2$. This was an exciting result, but what use could one make of it?

I can give a good example of this procedure. At one time, in 1927, I was playing around with three 2×2 matrices whose squares are equal to unity and which anticommute with one another. Calling them σ_1, σ_2, σ_3, I noticed that if one multiplied them into the three components of a momentum so as to form $\sigma_1 p_1 + \sigma_2 p_2 + \sigma_3 p_3$, one obtained a quantity whose square was just $p_1^2 + p_2^2 + p_3^2$. This was an exciting result, but what use could one make of it?

Figure 11: By using `\looseness-1`, the paragraph is shortened by one line.

bound to the current paragraph, so it is completely a manual tweak.

3 Introducing paragraph passes

We are now ready to discuss the LuaMetaTEX extension of the traditional paragraph builder, where we set up and use our own paragraph passes (par passes for short). The idea is that more runs on each paragraph, and the possibility of configuring the relevant parameters for each run independently, will give a higher quality in general. The model with badness, penalties and demerits is kept, and we also keep the same logic when running over the paragraph to decide which breakpoints to keep and which to throw away. We did in fact test altering the formulas both for badness (for instance, why the cube?) and demerits, but we did not see any improvements.

Let us introduce the new concept by studying examples, where we step-wise show what we can do and the different parameters available. We will use some low-level setups here. Later we will indicate a few possible high-level interfaces available to ConTEXt users. We start with a very simple example where we use two passes, the first with tolerance set to 50 and the second with tolerance set to 100. We disable hyphenation.

```
\parpasses 2
  hyphenation   0
  tolerance    50
next
  tolerance   100
\relax
```

Here, the `\parpasses 2` specifies that we will use two passes. The keyword `next` is the divider for successive par pass setups. Values are inherited, so hyphenation is still disabled in the second run. The specification ends with `\relax` just to be sure that we do not read on. We can enable these par passes with `\linebreakpasses 1`, and have a look at the test paragraph.

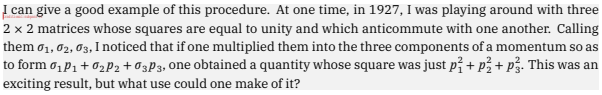

I can give a good example of this procedure. At one time, in 1927, I was playing around with three 2×2 matrices whose squares are equal to unity and which anticommute with one another. Calling them $\sigma_1, \sigma_2, \sigma_3$, I noticed that if one multiplied them into the three components of a momentum so as to form $\sigma_1 p_1 + \sigma_2 p_2 + \sigma_3 p_3$, one obtained a quantity whose square was just $p_1^2 + p_2^2 + p_3^2$. This was an exciting result, but what use could one make of it?

Figure 12: With the same text width as in Figure 2 it happens that we get the same breakpoints with the par passes enabled.

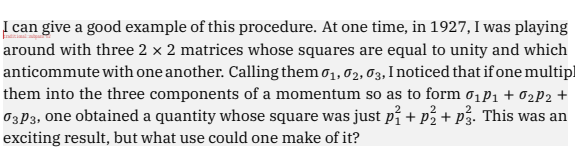

I can give a good example of this procedure. At one time, in 1927, I was playing around with three 2×2 matrices whose squares are equal to unity and which anticommute with one another. Calling them $\sigma_1, \sigma_2, \sigma_3$, I noticed that if one multiplied them into the three components of a momentum so as to form $\sigma_1 p_1 + \sigma_2 p_2 + \sigma_3 p_3$, one obtained a quantity whose square was just $p_1^2 + p_2^2 + p_3^2$. This was an exciting result, but what use could one make of it?

Figure 13: Hyphenation is disabled in the defined par pass. A narrower width leads to a problematic paragraph.

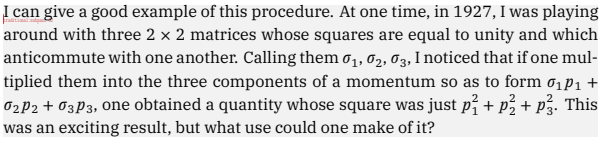

I can give a good example of this procedure. At one time, in 1927, I was playing around with three 2×2 matrices whose squares are equal to unity and which anticommute with one another. Calling them $\sigma_1, \sigma_2, \sigma_3$, I noticed that if one multiplied them into the three components of a momentum so as to form $\sigma_1 p_1 + \sigma_2 p_2 + \sigma_3 p_3$, one obtained a quantity whose square was just $p_1^2 + p_2^2 + p_3^2$. This was an exciting result, but what use could one make of it?

Figure 14: Hyphenation enabled in the second run.

This happens to work out well; we get the same result as the traditional parbuilder gave us. If we go a bit narrower, we will get an overfull line pretty soon, since the tolerance is low and we do not hyphenate: see Figure 13.

One way to prevent the overfull line could be to enable hyphenation in the second run; see Figure 14. Except for the lower tolerance, this is close to the traditional TEX setup that we started with, with a pretolerance run and a tolerance run, only a bit stricter.

```
\parpasses 2
  hyphenation   0
  tolerance    50
next
  hyphenation   1
  tolerance   100
\relax
```

In the example above, we specified the values of a few parameters. Then the logic follows the traditional paragraph builder: TEX does a run with the settings of the first pass. If it is successful, we are done. If not, it will run the second pass, and since we have no more passes, it is marked as a final pass. This means that TEX will make sure that something is returned, even if it fails to fulfill the constraints of the parameter values. That is why we got an overfull line in Figure 13, with hyphenations disabled. Another option for fixing the overfull line would be to increase the tolerance. A value of 200 would work in this case.

4 Hyphenation

We just saw how one can turn hyphenations on and off in par passes. The outer value of `\hyphenpenalty` (50) will be used; for technical reasons, it cannot be changed inside the par passes. Therefore, we have introduced the keyword `extrahyphenpenalty`, that adds to the exterior `\hyphenpenalty`.

```
\parpasses 2
  hyphenation           0
  tolerance            50
next
  hyphenation           1
  tolerance           100
  extrahyphenpenalty 150
\relax
```

Here we have set `extrahyphenpenalty` to 150. It is additive, so with `\hyphenpenalty` set to 50, the total penalty for breaking at hyphens becomes 200. The outcome would in this case be the same, whatever finite value of `extrahyphenpenalty` we give, because this is essentially the only solution that is available. We would need an extremely high value (9950) to prevent the hyphenated line, but that would just mean that we forbid hyphenations, so we could then equally well not enable it. And we saw that the paragraph did not come out well without hyphenation.

It is also possible to influence hyphenations by setting the parameters `doublehyphendemerits` and `finalhyphendemerits`. We need to remember that these are indeed demerits, and therefore of the order of penalties squared.

5 Font expansion

It has become very popular in so-called microtypography to use font expansion, i.e. to stretch and shrink glyphs just slightly. This can reduce the amount of hyphenations needed, and it can also even out the spacing a bit, leading to better paragraphs. An excessive use of expansion quickly becomes ugly, as we can see in many newspapers, with narrow columns.

One of the nice aspects of par passes is that one can apply expansion selectively. It is indeed possible to enable and disable expansion. Below we disable it in the first run (it might have been enabled outside the par passes with `\setupalign[hz]`) and then enable it in the third run by doing `adjustspacing 3` to expand glyphs and font kerns. We set the step to 1, maximum shrink to 10 (that means 1%) and maximum stretch to 15 (that means 1.5%). These values might seem small, but, by using several paragraph passes, one can increase the values in the latter passes, and thereby not use more than needed.

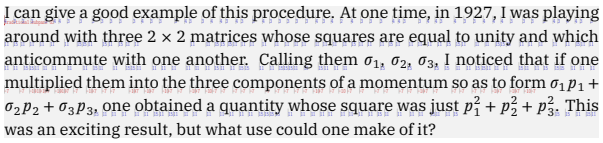


Figure 15: The tight paragraph, with expansion. The blue and red numbers indicate the amount of stretch and glue, respectively.

```
\parpasses 3
    tolerance             50
    hyphenation            0
    adjustspacing          0
next
    tolerance            100
next
    adjustspacing          3
    adjustspacingstep      1
    adjustspacingshrink   10
    adjustspacingstretch  15
\relax
```

Note that we disabled hyphenation in this setup. The narrow paragraph, that before introduced the lines sticking out, now typesets OK; see Figure 15. We get a linebreak inside a formula, and we will soon come back to that problem.

In the paragraph in Figure 15 we used the command `\showmakeup[expansion]` to show the amount of stretch and shrink for each character. In ConTEXt it makes sense to set `expansion=quality` as a font feature. This will take the difference in characters into account when spreading the stretch and shrink.

```
\definefontfeature[default][default]
  [expansion=quality]
```

For those who are familiar with expansion in the other engines we remark that in LuaMetaTEX we implemented it a bit differently. For instance, we have an expansion as well as compression factor per glyph. Instead of 'freezing' the step, stretch and shrink in a font definition, we can change it any time. Sensible values can still be bound to a specific font switch but when we set the adjustment properties in a pass those values are taken instead. Moreover, instead of initializing the glyph compression and expansion factors when a font is loaded we (can) delay this until it is needed. Experiments demonstrated that it is less often needed that one might think, so not all fonts need this to be set up. For this reason, font expansion in the par passes has only a small impact on run time.

6 Mathematics

When developing this extension we were also busy with extending math support in the engine and as a consequence we took math into account. For instance, we have a parameter that can influence the inter-atom penalties.

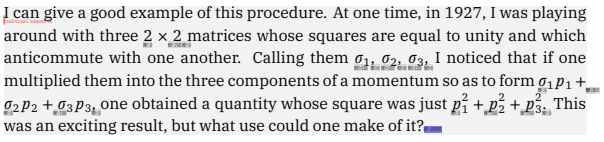


Figure 16: A narrow paragraph. We get penalties before short formulas and after binary operators and binary relations. In this case the penalty does not prevent the formula from being broken.

```
mathpenaltyfactor 500
```

This reduces several math penalties by 50%. To minimize the number of breaks inside math, we can start out with a large `mathpenaltyfactor` in the first run, and decrease it during later runs. We consider the narrow paragraph, but under more natural tolerance values, and without hyphenation (Figure 16).

```
\parpasses 2
    tolerance   100
    hyphenation   0
next
    tolerance   200
\relax
```

We get the default penalty of 700 after binary operators (the plus signs). We would also get 500 after binary relations, if we had any. We do also get the ConTEXt specific penalties of 150 before short formulas. We do indeed get a line break inside the formula. If we want a run that prohibits both the breaks after the plus signs and before the short formulas we need to multiply by a factor that ensures that both are 10000 or more. In this case 67 is sufficient; see Figure 17.

```
\parpasses 2
    tolerance            100
    hyphenation            0
    mathpenaltyfactor 67000
next
    tolerance            200
\relax
```

This was not especially successful, since we kept the high value through all paragraph passes. We reiterate that it might be better to forbid those breaks

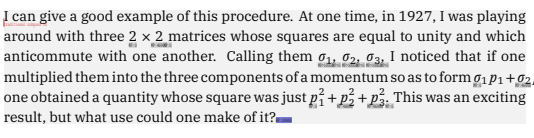


Figure 17: With a high enough `mathpenaltyfactor`, we can forbid TEX from breaking inside formulas and before short formulas. In this case it was not successful.

I can give a good example of this procedure. At one time, in 1927, I was playing around with three 2 × 2 matrices whose squares are equal to unity and which anticommute with one another. Calling them σ_1, σ_2, σ_3, I noticed that if one multiplied them into the three components of a momentum so as to form $\sigma_1 p_1 + \sigma_2 p_2 + \sigma_3 p_3$, one obtained a quantity whose square was just $p_1^2 + p_2^2 + p_3^2$. This was an exciting result, but what use could one make of it?

Figure 18: Paragraph set with `\emergencystretch=2em`, with OK results.

I can give a good example of this procedure. At one time, in 1927, I was playing around with three 2 × 2 matrices whose squares are equal to unity and which anticommute with one another. Calling them $\sigma_1, \sigma_2, \sigma_3$, I noticed that if one multiplied them into the three components of a momentum so as to form $\sigma_1 p_1 + \sigma_2 p_2 + \sigma_3 p_3$, one obtained a quantity whose square was just $p_1^2 + p_2^2 + p_3^2$. This was an exciting result, but what use could one make of it?

Figure 19: Paragraph set with `\emergencystretch=1em`, with an overfull line.

in the first pass(es) and then maybe decrease the factor for later runs.

The `mathpenaltyfactor` also works in combination with forward and backward penalties, which can be used to try to avoid line breaks in the beginning or at the end of a longer inline math formula. A possible setup for these is given below.

```
\mathforwardpenalties  2 200 100
\mathbackwardpenalties 2 200 100
```

These will add a penalty of 200 to the first and last available breakpoints in an inline math formula, and a penalty of 100 to the second and second from last.

7 We have an emergency!!

Oh, just kidding! The word emergency in the traditional TEX primitive `\emergencystretch` might have been a bit unfortunate, since it is not a bad idea to enable it, sparingly of course.

If we set the emergency stretch to `2em` in the example with low tolerance, we do indeed get the break inside the formula (Figure 18). With emergency stretch set to `1em` above, it won't help (Figure 19).

```
\parpasses 2
  hyphenation         0
  tolerance          50
next
  tolerance         100
  emergencystretch   2em
\relax
```

There are more "cheats". In Figure 20 we use an emergency stretch of 1em and also mess with the width of the paragraph, to the right. The 20 here means 2%.

```
\parpasses 2
  hyphenation           0
  tolerance            50
next
  tolerance           100
  emergencystretch      1em
  emergencyleftextra    0
  emergencyrightextra  20
\relax
```

I can give a good example of this procedure. At one time, in 1927, I was playing around with three 2 × 2 matrices whose squares are equal to unity and which anticommute with one another. Calling them σ_1, σ_2, σ_3, I noticed that if one multiplied them into the three components of a momentum so as to form $\sigma_1 p_1 + \sigma_2 p_2 + \sigma_3 p_3$, one obtained a quantity whose square was just $p_1^2 + p_2^2 + p_3^2$. This was an exciting result, but what use could one make of it?

Figure 20: Paragraph set with `emergencyrightextra`.

I can give a good example of this procedure. At one time, in 1927, I was playing around with three 2 × 2 matrices whose squares are equal to unity and which anticommute with one another. Calling them σ_1, σ_2, σ_3, I noticed that if one multiplied them into the three components of a momentum so as to form $\sigma_1 p_1 + \sigma_2 p_2 + \sigma_3 p_3$, one obtained a quantity whose square was just $p_1^2 + p_2^2 + p_3^2$. This was an exciting result, but what use could one make of it?

Figure 21: Paragraph set with `emergencywidthextra`.

I can give a good example of this procedure. At one time, in 1927, I was playing around with three 2 × 2 matrices whose squares are equal to unity and which anticommute with one another. Calling them σ_1, σ_2, σ_3, I noticed that if one multiplied them into the three components of a momentum so as to form $\sigma_1 p_1 + \sigma_2 p_2 + \sigma_3 p_3$, one obtained a quantity whose square was just $p_1^2 + p_2^2 + p_3^2$. This was an exciting result, but what use could one make of it?

Figure 22: Paragraph set with `emergencypercentage`.

Another one is `emergencywidthextra`: use a different width when the line breaks are decided, but not apply it in the end. This means it only works out well if lines have stretch and shrink. In the example in Figure 21 we use 2% extra width. This should probably only be used in true emergencies, if at all.

```
\parpasses 2
  hyphenation           0
  tolerance            50
next
  tolerance           100
  emergencystretch      1em
  emergencywidthextra  20
\relax
```

We have so far set the emergency stretch explicitly, in terms of font `em` units. If we have hanging indentation or parshapes, the widths of different lines in the paragraph will vary. One can then argue that it makes more sense to set the amount of emergency stretch as a percentage of the line width, even if it does not matter for most paragraphs. In Figure 22 we set the stretch to 4% of the line width, which was sufficient this time.

```
\parpasses 2
  hyphenation           0
  tolerance            50
next
  tolerance           100
  emergencypercentage  40
\relax
```

It might happen that `\emergencystretch` is set to a positive value outside of the par pass setups (for example via `\setupalign[stretch]`). When we go in to the par passes, we can use `emergencyfactor`

I can give a good example of this procedure. At one time, in 1927, I was playing around with three 2×2 matrices whose squares are equal to unity and which anticommute with one another. Calling them σ_1, σ_2, σ_3, I noticed that if one multiplied them into the three components of a momentum so as to form $\sigma_1 p_1 + \sigma_2 p_2 + \sigma_3 p_3$, one obtained a quantity whose square was just $p_1^2 + p_2^2 + p_3^2$. This was an exciting result, but what use could one make of it?

Figure 23: Shortening a paragraph with a higher `\linepenalty`.

to handle that. We can start by setting it to 0 in the first pass to be sure to disable the emergency stretch, and then update it to a positive value in a later run to enable it.

```
\parpasses 2
  hyphenation         0
  emergencyfactor     0
  tolerance          50
next
  tolerance         100
  emergencyfactor  1000
\relax
```

8 More penalties

An example above showed the `extrahyphenpenalty` parameter, which is specific to paragraph passes. There are a few more penalties available. The ones below can also be set by primitives. An orphan penalty can prevent a line break before the last word in a paragraph (we come back to that one), and a toddler penalty might prevent a line break before a single glyph.

```
linepenalty    100
orphanpenalty  200
toddlerpenalty 200
```

We show one example with `linepenalty`. Earlier we used `\looseness-1` to get the paragraph one line shorter. In Figure 23 we succeed in obtaining the same paragraph by increasing the `linepenalty` from 10 to 100. It is, however, difficult to predict when it will work.

```
\parpasses 2
    tolerance    50
    hyphenation   0
    linepenalty 200
next
    tolerance   100
\relax
```

It's worth mentioning that the first versions of TEX did not come with the `\linepenalty` parameter. The corresponding number was then 1 instead of the 10, which is probably used everywhere now.

The orphan penalties can be problematic if set too aggressively, in particular for short paragraphs that often occur in novels with a lot of dialogue. In Figure 24 we see such a problematic example, where

This is just a short sentence that is just a bit longer than one line.

Figure 24: A one-liner with too-strict orphan penalties.

This is just a short sentence that is just a bit longer than one line.

Figure 25: A one-liner with strict orphan penalties and multipliers.

Some write: I owe you one.

The kids write: I o u 1.

Figure 26: Penalized toddlers.

we have prohibited breaks before the last word by setting the penalty there to 10000.

```
\parpasses 1
    tolerance       100
    orphanpenalties 1 10000
\relax
```

To avoid this problem we have factors that can be used. Below we multiply by 0.1 if the paragraph has one line break, 0.5 if it has two and 1.0 if it has more than two. We see in Figure 25 that this is sufficient; we can now break before the last word.

```
\parpasses 1
    tolerance       100
    orphanpenalties   1 10000
    orphanlinefactors 3 100 500 1000
\relax
```

Let us also show an example where we set toddler penalties both to the left and the right. If you are able to zoom in Figure 26, you will see that we get penalties of 50 sitting to the right of the single character letters, and 25 to the left of the leftmost one. The `\parfillrightskip` was set to 0pt here, to get a bit extra space between the words so that the penalties show better. We do not know if there are languages where single-letter words can be stacked like this.

```
\parpasses 1
    tolerance       100
    toddlerpenalties 1 options 2 50 25
\relax
```

9 Being more granular

The next LuaMetaTEX extension: It is possible to specify the number of fitness classes to be used. We saw before that traditional TEX uses four: tight, decent, loose and very loose. By invoking

```
\setupalign[granular]
```

we enable more. You can see in Figure 27 that they are evenly spread out regarding the adjustment ratio,

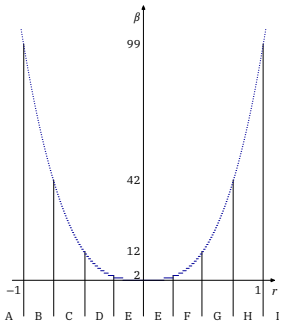


Figure 27: Granular fitness classes. A: very tight. B: tight. C: almost tight. D: barely tight. E: decent. F: barely loose. G: almost loose. H: loose. I: very loose.

I can give a good example of this procedure. At one time, in 1927, I was playing around with three 2×2 matrices whose squares are equal to unity and which anticommute with one another. Calling them $\sigma_1, \sigma_2, \sigma_3$, I noticed that if one multiplied them into the three components of a momentum so as to form $\sigma_1 p_1 + \sigma_2 p_2 + \sigma_3 p_3$, one obtained a quantity whose square was just $p_1^2 + p_2^2 + p_3^2$. This was an exciting result, but what use could one make of it?

Figure 28: The test paragraph, set with more granular fitness classes.

```
1   1   0  13    5529  almostloose  glue          10   6   0  40429  decent      glue        14  7 4 1
    2   0   3  490169  barelytight  penalty   4   11   8  14  11515  almostloose glue        15  10 6 3
    3   0  45  10525  tight        math          12   7  14   6242  decent      glue        pass     : 4  demerits : 6139
2   4   1   7    5818  barelyloose  glue          13   8   8   6039  decent      glue        subpass  : 1  looseness :  0
    5   3   6  43281  barelyloose  glue          14   7   8  15011  almosttight glue        subpasses : 2
    6   3  38  35329  almosttight  glue          15  10   1  40550  decent      glue
3   7   4   8    6142  barelyloose  glue      11  8 4 1
    8   4   1    5939  decent       glue      12  7 4 1
    9   5  15  43906  almostloose  glue      13  8 4 1
```

Figure 29: Information on the breakpoints that TEX used for the paragraph in Figure 28.

and how they are related to the badness values. This should be compared with Figure 10.

The granular fitness classes are defined by a `\specificationdef` command (more about them later). The classes are defined to spread evenly over the adjustment ratios, just as in the non-granular situation.

```
\permanent
\specificationdef \granularfitnessclasses
  \fitnessclasses 9
  99
  42 % .75
  12 % .50
   2 % .25
   0 % .00
   2 % .25
  12 % .50
  42 % .75
  99
```

It becomes more meaningful to enable the granular mode if we also configure how these fitness classes are to be used. As previously mentioned, in traditional TEX the `\adjdemerits` is added whenever we jump over at least one fitness class when going from one line to the next. We can use `adjacentdemerits` in the par passes. For example,

```
adjacentdemerits 4 0 5000 7500 10000
```

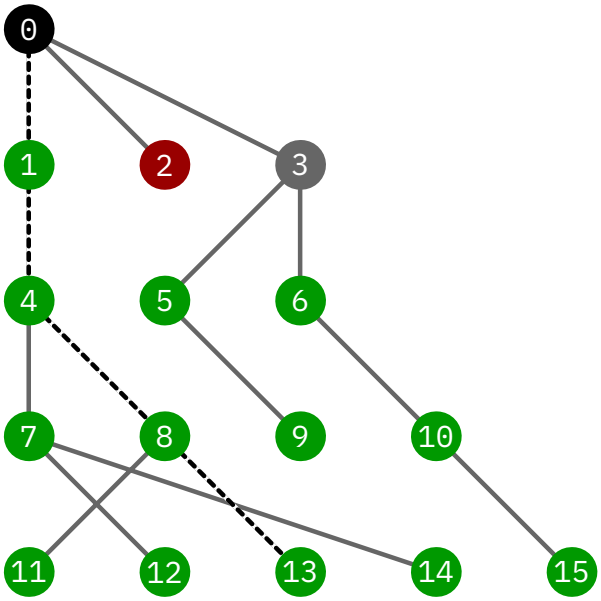


Figure 30: The tree corresponding to the paragraph in Figure 28.

defines four levels of adjacent demerits. For two consecutive linebreaks with neighboring fitness classes, no demerits is added. If we jump one step 5000 is added, jumping two steps cost 7500 and three steps (or more) cost 10000.

In Figure 29 we see fitness classes that we did not see before, such as barely tight and almost loose. We see in Figure 30 that the tree is slightly different from the one in Figure 4.

We got here the same linebreaks as for the traditional parbuilder (Figure 2). But in the traditional case we had fitness classes loose, decent, decent, which means that we paid no demerits for them. Now we got almost loose (cost 5000), barely loose (0), decent (0). Thus, the total demerits in this case landed at 6139 instead of 1139.

Going back to the default fitness classes, one can use

```
\fitnessdemerits 0
```

and to go granular in only one par pass, one can use

```
...
    fitnessclasses    \granularfitnessclasses
    adjacentdemerits \granularadjacentdemerits
next
    classes           \matchallfitnessclasses
...
```

where the `\granularadjacentdemerits` have been defined to be compatible with the more granular fitness classes. The `classes` parameter (a bitset) tells the builder to check all set classes; the constant is a generous `"FF`.

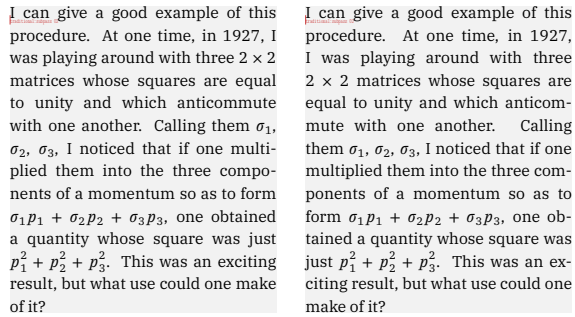


Figure 31: Left: A narrow paragraph set with `doublehyphendemerits` set to 0. Right: The same paragraph with `doublehyphendemerits` set to 300000.

On a bigger project, we have seen only a few changes when enabling the granular setup. Since they are few it is difficult to say something general about quality, but we expect that the neighboring lines are slightly more compatible.

10 Other demerits

We recall that it is the demerits of the paragraph that TEX uses as a cost function to select the best set of line breaks; the solution with minimal demerits wins. We emphasize again that the additional demerits are not added to singular breakpoints, but to combinations of breakpoints that fulfill some condition.

We have already seen how the granular fitness classes could be used, together with `adjdemerits` (defaults to 10000), or rather the plural version `adjacentdemerits`, to be able to detect smaller differences in badness values between consecutive lines. There are other demerits we can set. In Figure 31, we compare results with `doublehyphendemerits` set to zero (left), and set to a high value (right), preventing two consecutive lines from being hyphenated.

We also have `finalhyphendemerits` that can be used to discourage the last breakpoint from being hyphenated. Its default value is 5000.

The settings are equivalent to the primitives and more about them can be found in regular TEX documentation. We also have twin demerits:

```
lefttwindemerits  2000
righttwindemerits 2000
```

These discourage line breaks where words at the beginning or end of lines are the same; be aware that this doesn't prevent mid-line occurrences. And of course it puts more constraints on the solution and has to work with other constraints. More about this feature can be found in another recent TUGboat article.[2]

[2] "Twin demerits", Hans Hagen and Mikael P. Sundqvist, *TUGboat*, vol. 45, no. 2, pp. 362–369, 2024, `https://tug.org/TUGboat/tb45-3/tb141hagen-twins.pdf`.

11 Conditionally entering par passes

We have seen several examples of using par passes, where the standard logic of TEX is kept: if TEX is happy after a run, we are done with the line breaking. It is possible to also enter par passes conditionally. There are three main criteria that we can use. The valid criteria keys are `demerits`, `threshold` and `class`:

- `demerits`: the overall measure that TEX uses to select the best choice.
- `threshold`: over- or underfull lines.
- `classes`: compatibility between successive lines.

The first is not that useful because it is hard to come up with some good numbers. Longer paragraphs typically have higher demerits than shorter, and for very long paragraphs some shortcuts are taken and large values get clipped in order not to overflow numbers. We will discuss the `classes` option soon.

In the traditional paragraph builder it is difficult to go back and deduce what decisions were made during the runs, and how and why they were made. The values are not kept, except for the demerits, but those are recalculated as we go.

When we add more passes we don't know in advance what is the final pass, but we need one because in the end we must have a result. We could of course always add a final one automatically but then we might just as well take the last one anyway. The multiple pass mechanism will always do the regular pretolerance and tolerance passes but we can set the values in a par pass definition. We have two situations:

1. When we have a criterion in the first par pass, we will first do the two tolerant passes. The second tolerant pass is a final pass so we do have a result but we check for further actions in the list of par passes.
2. When we have three or more par passes, the first two will act like tolerance passes when there are no criteria. Again the second one is a final pass that can have a follow up.

Here we have a single par pass of the first category:

```
\parpasses 1
  threshold          0.025pt
  tolerance        300
  emergencyfactor 1000
\relax
```

and here is an example of the second:

```
\parpasses 3
  tolerance        100
next
  tolerance        200
next
```

```
  threshold          0.025pt
  tolerance        300
  emergencyfactor 1000
\relax
```

The second one has no criteria, so the last pass becomes the final pass, which kicks in when none of the previous ones gave an acceptable solution.

The `classes` key is the most difficult one to describe. In ConTEXt we can add

```
\parpasses 1
  classes            \indecentparpassclasses
  tolerance        300
  emergencyfactor 1000
\relax
```

and that needs an explanation. When TEX looks at lines it will use adjacent demerits to penalize neighboring lines that are space wise incompatible. The `\indecentparpassclasses` condition will let you enter the par pass if there are any lines that ar flagged as not being decent.

In ConTEXt, when using the granular mode described above, we have these constants defined:

```
\integerdef\verylooseparpassclass   "0001
\integerdef\looseparpassclass       "0002
\integerdef\almostlooseparpassclass "0004
\integerdef\barelylooseparpassclass "0008
\integerdef\decentparpassclass      "0010
\integerdef\barelytightparpassclass "0020
\integerdef\almosttightparpassclass "0040
\integerdef\tightparpassclass       "0080
\integerdef\verytightparpassclass   "0100

\integerdef\allparpassclasses       "FFFF
```

The definition of `\indecentparpassclasses` is then:

```
\integerdef\indecentparpassclasses\numexpr
    \allparpassclasses
  - \decentparpassclass
\relax
```

As you see the condition is really using a bitset, but it is easier to have names for them. There are a few others predefined:

```
\almostdecentparpassclasses
\looseparpassclasses
\tightparpassclasses
```

and you can define your own just as we showed above.

In addition to the `demerits`, `threshold` and `classes` criteria mentioned above, we can also decide if entering (using) a par pass with the following keys

- `ifadjustspacing`: enter if `expansion` is enabled.
- `ifemergencystretch`: enter if `emergencystretch` is enabled.
- `ifglue`: enter if there is anything to stretch or shrink.
- `iftext`: enter if the paragraph has text (glyphs/discretionaries).
- `ifmath`: enter if the paragraph has math.
- `unlessmath`: enter if the paragraph does not have math.

A new block of parameters is marked by `next`. With `quit` processing passes can be stopped, and `skip` will bypass a pass. These last two are mostly for testing.

To sum up, we have three situations:

- traditional mode, up to three passes,
- mixed mode, first two traditional passes and then additional ones,
- par passes that likely include traditional setups,

and we have various different ways to condition on entering the par passes.

12 A bit of infrastructure

For administrative purposes we have the directives `callback`, `identifier`, and `linebreakchecks`, as well as `linebreakoptional` to select what optional content to enable.

We can add an identifier:

```
\parpasses 3
  identifier         1
  tolerance        100
next
  tolerance        200
  hyphenation        1
next ifemergencystretch
  emergencyfactor 1000
\relax
```

This identifier will be used in reporting and in ConTEXt we can relate this to a more meaningful name, like ‘default’. We can avoid altering the current par pass by defining an alias:

```
\specificationdef \parpassdefault \parpasses 3
  identifier         1
  tolerance        100
next
  tolerance        200
  hyphenation        1
next ifemergencystretch
  emergencyfactor 1000
\relax
```

We use a generic `\specificationdef` and by just issuing the given name the par pass is activated. However, one also has to set `\linebreakpasses` to a positive value to let it do its work.

13 Changing par passes locally

We saw how to use `\looseness` to manually (try to) tweak a single paragraph in the traditional par builder. We have a similar local par pass mechanism.

With \parpassesexception we can locally use a specified par pass setup for the current paragraph. It has to be called just before the paragraph in question, as in:

```
\parpassesexception \mylocalparpasses
Paragraph comes here ...
```

will use the par passes setup \mylocalparpasses, which must have been previously defined with a \specificationdef. This opens it up for a simple but complete local control when needed.

14 After breaking the paragraph into lines

Breaking a paragraph into lines and, at some asynchonous, point breaking pages are separate processes. The first process has related penalties and demerits that are part of the decision making that are no longer relevant once the work is done. The second process also has penalties to consider, for instance widow and club penalties. These are inserted between lines by the par builder because it is that routine that, after optimal breakpoints have been determined calls out to a post line break routine that constructs the lines. The lines themselves as well as various glue and penalties, plus possible \vadjust and \insert material, are added to a current list of contributions that is eventually transferred to the page. So, it makes sense to mention them here.

It goes unnoticed, but the broken line is in practice no more than a begin and end point in the horizontal list that enters the routine. Every line is just a range and although the decisions were made using glue and optionally font expansion, the original nodes are still there. So, when that range has to become a line, the horizontal pack routine is called to wrap it into a \hbox, and, as with any horizontal box construction, it will recalculate what the final glue will be and what expansion is applied, based on what the par builder decided. Also, before packaging, the left and right skip, indentation, paragraph related shape measure etc. are injected. This somewhat redundant effort is fast enough not to be of impact.

An important activity in this packing is that we (when enabled) can normalize the line. Depending on what line we are, we have a lot of skips to consider (for practical purposes items that are actually kerns, like indentation, also use glue nodes):

```
leftskip lefthangskip leftparskip leftinitskip
indentskip [content] correctionskip
rightinitskip rightparskip righthangskip
  rightskip
```

We also make sure that direction nodes are balanced and math is well indicated across lines. Discussing this process is beyond what this article focuses on, but you can imagine that it involves some code. This pays off in nicer code at the Lua end when we want to mess with the lines afterwards.

The abovementioned widow and club penalties (plus some more) are taken from the singular and plural commands \widowpenalty, \widowpenalties, \clubpenalty, \clubpenalties. In LuaMetaTEX these values are stored in the initial par node.

For the record: there are uet more penalties that matter, for instance we have \shapingpenalty that can prevent breaks in a parshape or hanging setup and \singlelinepenalty that penalizes a two line result. We don't discuss how display math is handed in line breaks and wrapping up, but just mention that we can have a display formula that combines with the previous and upcoming paragraph. In that case the builder sees three paragraphs as one and display math as three lines, which of course influences what is seen as the current line width when shaping the whole. It doesn't affect the discussed break mechanism. In ConTEXt we handle display math differently, so we have not added features for mixed-in display math.

A par pass definition is what (in LuaMetaTEX) we internally call a specification command. Other examples of specification commands are \parshape and the mentioned plural penalties. Each of their values is a pointer to a node, and because the amount of data can differ these have a variable size. In LuaTEX they are taken from the regular pool and when the set of values change another sized one is needed. Released nodes are kept in a pool but one can think of scenarios where too many different sizes will create a bit of a mess. This is why in LuaMetaTEX we allocate the variable part dynamically as an independent 'array' of parameters.

The reason for mentioning these details is that, because of the decoupling between the handful of primitives and the way their information is stored, it started making sense to provide ways to create variables as with registers. This brings us to an example:

```
\widowpenalties 4 2000 500 250 0
```

We can also say:

```
\specificationdef \lesswidowpenalties
  \widowpenalties 4 2000 500 250 0
\relax
```

and then use \lesswidowpenalties to enable this set of penalties.

The \specificationdef command can also be used to define par passes, as we saw above. Using these definitions is not only faster but also has the advantage that we can provide interfaces in ConTEXt in the way we like. It also makes it easier to

reset the plural penalties to default values. An even more important feature is that we can get rid of the singulars which is a big benefit because of the way the engine works. When ε-TEX introduced these plurals it had to remain compatible so this is what happens there:

- When `\widowpenalties` is set, `\widowpenalty` is ignored.
- When `\widowpenalties` is reset, `\widowpenalty` kicks in again.

Resetting `\widowpenalties` is done with:

```
\widowpenalties 0
```

That said, what about the following?

```
\def\widowpenalty{\widowpenalties 1 }
```

This is very close to what we want but because the last value is used for all that follows, we would need this to be compatible.

```
\widowpenalty     500
\widowpenalties 2 500 0
```

This is why we end up with:

```
\permanent\protected\untraced\def\widowpenalty
  {\widowpenalties\minusone}
```

where the negative one sets an option to not reuse the last value. The prefixes declare that the command can't be redefined when overload protection is enabled, that it doesn't expand (in e.g. an `\edef`) and that in tracing it gets reported without its meaning, so basically the users see a primitive. The advantage of this approach is that we only have to deal with one variable. Of course users should be aware of this but few will set these plurals explicitly, leaving that to ConTEXt.

The plural widow and club penalties can result in better results but also add constraints. This means that we can get less full pages or when we have stretch in the white space (if present) possibly inconsistent spacing. We can handle this by limiting the stretch in the vertical spacing combined with overall vertical scaling that we call vz, analogous to hz (Hermann Zapf's initials for expansion); it was Hermann who suggested to us to play with this because "No reader will notice a few percent vertical scaling of the page". Limiting stretch is an engine feature (in the page builder) while vertical scaling is a ConTEXt trick. Applied to the large test document this also helps to make it look great.

Another new feature is that when a paragraph is eventually broken across a page, you might want to distinguish between a left and right page of a spread. It is therefore possible to do this:

```
\widowpenalties 3
  options \numexpr8 + 4\relax % largest + double
    5000 7500
     250  500
       0    0
\relax
```

This says: use higher penalties for the right page and when you overlap with club penalties use the larger of the widow and club penalty, i.e., we do not want to add them. The `options` is a bitset that differs per specification.

The `\adjdemerits` parameter controls what demerits get added to lines that have a distance of more than one step in the fitness sequence tight, decent, loose, very loose. In LuaMetaTEX we have more control over this; for instance we can, as we have seen, have more steps. In that case we also apply different demerits for every distance and even have accumulated demerits. This is controlled by `\adjacentdemerits` and we can redefine the traditional parameter like this:

```
\permanent\protected\untraced\def\adjdemerits
  {\adjacentdemerits\minusone}
```

So, to summarize this part: setting up and using par passes to get better results is worth the effort, but part of this often also involves making sure the vertical penalties are right. This is bound (applied) to the result of line breaking.

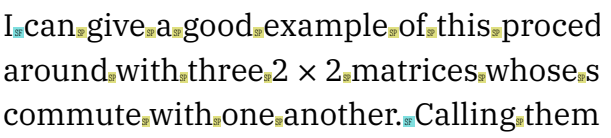
15 Tracing and debugging

It would be impossible for us to develop these new features without extensive testing, and the testing would be very difficult to do without tracing. There are two ways to trace what the engine is doing: built-in (hardcoded in the engine) reporting, and ConTEXt trackers that use Lua to add visual or report textual information. The first one is probably not that useful unless you need to know what goes on deep inside; the second can help you improve a specific document setup.

When `\tracingpenalties` is set to 1, you will get reports like this, where `l` and `r` refer to the left and right page of a spread where the values kick in when the page is broken in a double sided layout:

```
[linebreak:  interline penalty, line 1, index 1, delta 101, total 101]
[linebreak:  club l penalty, line 1, index 1, delta 100, total 201]
[linebreak:  club r penalty, line 1, index 1, delta 100, total 201]
[linebreak:  interline penalty, line 2, index 2, delta 101, total 101]
[linebreak:  interline penalty, line 3, index 3, delta 101, total 101]
[linebreak:  interline penalty, line 4, index 4, delta 101, total 101]
[linebreak:  interline penalty, line 5, index 5, delta 101, total 101]
[linebreak:  interline penalty, line 6, index 6, delta 101, total 101]
[linebreak:  interline penalty, line 7, index 7, delta 101, total 101]
[linebreak:  interline penalty, line 8, index 8, delta 101, total 101]
[linebreak:  widow l penalty, line 8, index 1, delta 101, total 202]
[linebreak:  widow r penalty, line 8, index 1, delta 101, total 202]
```

When the value is set to 2, you will also get lines that report the `\shapingpenaltiesmode` value that was applied. This is a bitset that determines what



Spaces between words and after punctuation.



Spaces after initials and punctuation.



Space factors and stretch.

Figure 32: Showing spaces

penalties will be applied when we have a hanging situation.

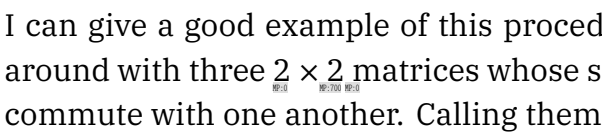
```
[linebreak: penalty, line 1, best line 10,
 prevgraf 0, mode "FF (i=1 c=4 w=2 b=8)]
```

Another tracing option is the traditional TEX `\tracingparagraphs` parameter, which reports a lot and even more when its value exceeds 1. Probably more interesting is `\tracingpasses`, which reports the parameters used, and, when set to more than 1, also reports details over the decisions made. We mention also `\tracingtoddlers` and `\tracingorphans` that might come in handy.

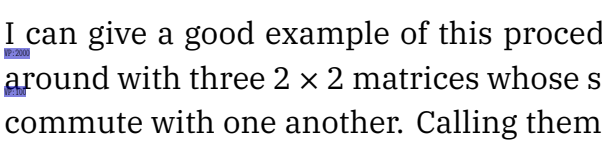
When we discussed and tested these extensions with ConTEXt users, there was some confusion about `\looseness`. These parameters can, as we have explained, be used to increase or decrease the number of lines relative to the optimum, if possible. Any change to the involved parameters might spoil the ability to get that extra line. With `\tracinglooseness` you get some information about the attempts to fit the demands. When tracing with the trackers that show all possible breakpoints it quickly becomes clear that TEX doesn't discard bad solutions as it goes forward but keeps them around till (at the end of a successful pass) it tries to loosen.

While developing features like these it helps very much to see what we're dealing with. For instance, TEX distinguishes between spaces between words (that become glue) and spaces after punctuation (influenced by the space factors). With `\showmakeup[space]` you can show both (Figure 32). This example also shows another feature: space factoring applied after uppercase characters, in this case shown but not applied. Think of situations like 'D.E. Knuth'. More control over space factors is part of the optimizations because we have ways to limit the maximum stretch, just like TEX already limits the shrink.

Similarly, we can use `\showmakeup [hpenalty]` to see where horizontal penalties are applied and



Math has plenty of penalties.



Vertical penalties are added between lines.

Figure 33: Showing penalties.

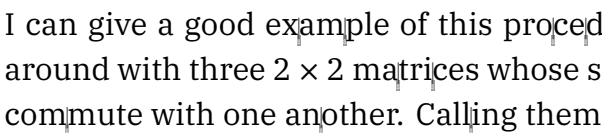


Figure 34: We look at all discretionaries, but only longer words get hyphenated.

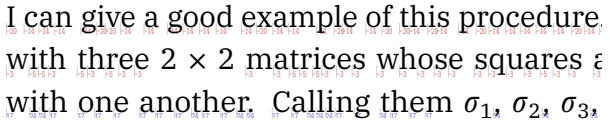


Figure 35: Expansion kicks in.

`\showmakeup [vpenalty]` for vertical penalties; see Figure 33.

Hyphenation results in injected discretionary nodes; `\showmakeup [discretionary]` lets us see them. The ones at ends of lines eventually get replaced by the content of pre and post fields but we can show the places where they were seen in the rest. We can show them because in LuaMetaTEX we keep track of such decisions in the glyph nodes so we know at what places hyphenation is possible; see Figure 34.

Expansion is another feature that we might want to track, and `\showmakeup [expansion]` reveals it, see Figure 35.

16 Larger example used in a math book

We have experimented a lot with a first year analysis book that Mikael has written with his colleague Tomas Persson, in Swedish. We emphasize that the settings we have ended up using might not fit everyone, but they did seem to work well for this book.

We use five par passes, and we don't enter any of them conditionally; we quit directly if TEX is happy after a run. Our strategy is trying to avoid both hyphenations and breaking inside of mathematics, as much as possible.

The book contains 3023 paragraphs. A vast majority, 2697 paragraphs, are done by the first run.

This is one of the reasons that the extra par passes do not add much overhead.

The first run is a typical pretolerance run. We use no expansion, no emergency stretch, and we accept no hyphenations. Also, the math penalties (inside formulas and before short formulas) are multiplied by 20. This means that they reach at least 10000 and thus such breaks are prohibited.

We follow up with a run with a slightly higher tolerance, and also a very small allowed font expansion, with a stretch of at most 1% and a shrink of at most 0.5%. This one is used 192 times.

In the third run we switch expansion off again, but allow for a tolerance of 300; this is used only five times. In the fourth run we go back to tolerance 200 but increase the possible amount of expansion, and get 76 paragraphs. Finally, in the fifth run we enable hyphenation, but add 200 to its penalty, we increase the amount of font expansion allowed, enable some additional emergency stretch, and also reset the math penalties to the outer values. This run takes care of the 53 remaining paragraphs.

```
\startsetups align:pass:analysisbook
\parpasses 5
  identifier           \parpassdefaultone
  tolerance            100
  adjustspacing          0
  emergencyfactor        0
  hyphenation            0
  mathpenaltyfactor  20000
next
  tolerance            200
  adjustspacing          3
  adjustspacingstep      1
  adjustspacingshrink    5
  adjustspacingstretch  10
next
  tolerance            300
  adjustspacing          0
next
  tolerance            200
  adjustspacing          3
  adjustspacingshrink   20
  adjustspacingstretch  40
next
  tolerance            400
  hyphenation            1
  extrahyphenpenalty   200
  adjustspacing          3
  adjustspacingshrink   30
  adjustspacingstretch  60
  emergencystretch       1\bodyfontsize
  emergencyfactor     1000
  mathpenaltyfactor   1000
\relax \stopsetups
\newinteger\parpassdefaultone
\parpassdefaultone\parpassidentifier{analysisbook}
```

Eftersom E är begränsad enligt lemma 2.41, har (a_k) en konvergent delföljd (a_{n_k}) enligt Bolzano–Weierstraß sats (sats 2.23). Låt A beteckna delföljdens gränsvärde. Vi vill visa att $A \in E$.

traditional

Eftersom E är begränsad enligt lemma 2.41, har (a_k) en konvergent delföljd (a_{n_k}) enligt Bolzano–Weierstraß sats (sats 2.23). Låt A beteckna delföljdens gränsvärde. Vi vill visa att $A \in E$.

par passes

Figure 36: Avoiding hyphenations.

With the last line, we can set up these par passes with

```
\setupalignpass[analysisbook]
```

And with

```
\enabletrackers[paragraphs.passes=summary]
```

we get a summary in the log file. Here we find how many times each run was used and also what the paragraphs contained (`t` is text, `d` is discretionary and `m` is math).

```
'subpass 01', count 2697, states 93:t-- 199:td- 1:--m
  145:t-m 44:t-- 549:td- 135:t-m 1443:tdm 18:td- 69:tdm 1:tdm
'subpass 02', count 0192, states 35:td- 1:t-m 138:tdm
  6:td- 12:tdm
'subpass 03', count 0005, states 5:tdm
'subpass 04', count 0076, states 12:td- 57:tdm 2:td- 5:tdm
'subpass 05', count 0053, states 1:t-m 10:td- 40:tdm
  1:td- 1:tdm
```

Before comparing some outputs, let us first make clear that it is only a few paragraphs that change. This is good, we do not want to alter TEX's usual very high quality output.

Since we enable hyphenations only in the last run, we get fewer hyphenations. We show one example in Figure 36.

It is possible to disallow line breaks before short math formulas by manually inserting a maximum penalty. Knuth calls them "ties", and it has become standard to use a tilde to type them, as in `Om~$A$`. With our math penalties setup we do not need that manual tweak in the source. Compare the results in Figure 37, where the stricter math penalties avoids short formulas at the beginning of lines.

Line breaks inside formulas can become very ugly, compare the results in Figure 38.

The orphan penalties sometimes help us to prevent just a word or a symbol at the end of the line. Compare the results in Figure 39.

We also show one example of a paragraph where TEX fails with the traditional runs but succeeds with the par passes, see Figure 40.

In addition to the previously mentioned settings for the math book, we also use

Funktionsbegreppet är oerhört viktigt och centralt inom matematiken. Om A och B är två mängder, så tänker vi ofta på en funktion från A till B som en regel som till varje element i A tilldelar ett entydigt bestämt element i B. Om f är en funktion från A till B, så skriver vi $f\colon A \to B$. Det element i B som funktionen f tilldelar ett element $a \in A$ betecknas $f(a)$. Mängden A kallas *definitionsmängden* för f och mängden B kallas *målmängden* eller *värdeförrådet* till f. Ibland kommer vi att skriva D_f för definitionsmängden för f.

traditional

Funktionsbegreppet är oerhört viktigt och centralt inom matematiken. Om A och B är två mängder, så tänker vi ofta på en funktion från A till B som en regel som till varje element i A tilldelar ett entydigt bestämt element i B. Om f är en funktion från A till B, så skriver vi $f\colon A \to B$. Det element i B som funktionen f tilldelar ett element $a \in A$ betecknas $f(a)$. Mängden A kallas *definitionsmängden* för f och mängden B kallas *målmängden* eller *värdeförrådet* till f. Ibland kommer vi att skriva D_f för definitionsmängden för f.

par passes

Figure 37: Avoiding manual ties before math.

Lösning. Eftersom $a^{1/k} = 1/(1/a)^{1/k}$ räcker det att visa påståendet för $a > 1$. För $a > 1$ och $k \in \mathbb{N}$ är $a^{1/k} > 1$. Bernoullis(11) olikhet (se övning 2.4) med $x = a^{1/k} - 1$ ger

traditional

Lösning. Eftersom $a^{1/k} = 1/(1/a)^{1/k}$ räcker det att visa påståendet för $a > 1$. För $a > 1$ och $k \in \mathbb{N}$ är $a^{1/k} > 1$. Bernoullis(11) olikhet (se övning 2.4) med $x = a^{1/k} - 1$ ger

par passes

Figure 38: Avoiding line breaks inside math.

Ibland vill man starta summationen på något annat tal än 1. Frågan om konvergens/divergens för serier beror inte på hur de första termerna uppför sig, utan endast hur a_k beter sig för stora värden på k. Vi kommer därför att använda beteckningen $\sum a_k$ för att beteckna serien med termer a_k, där den undre gränsen är något fixt heltal (ofta 0 eller 1) och den övre gränsen är $+\infty$.

traditional

Ibland vill man starta summationen på något annat tal än 1. Frågan om konvergens/divergens för serier beror inte på hur de första termerna uppför sig, utan endast hur a_k beter sig för stora värden på k. Vi kommer därför att använda beteckningen $\sum a_k$ för att beteckna serien med termer a_k, där den undre gränsen är något fixt heltal (ofta 0 eller 1) och den övre gränsen är $+\infty$.

par passes

Figure 39: Avoiding orphan words or symbols.

Serien $\sum(-1)^k/k$ är konvergent, och eftersom $0 < a_k < 1/k$ så är serien $\sum(-a_k)^k/k$ absolutkonvergent eftersom $\sum 1/k^{k+1}$ är konvergent (jämför till exempel med den konvergenta p-serien $\sum 1/k^2$). Det följer att serien $\sum(-1)^k a_k$ är konvergent. Alltså konvergerar $\sum a_k x^k$ precis då $-1 \le x < 1$.

traditional

Serien $\sum(-1)^k/k$ är konvergent, och eftersom $0 < a_k < 1/k$ så är serien $\sum(-a_k)^k/k$ absolutkonvergent eftersom $\sum 1/k^{k+1}$ är konvergent (jämför till exempel med den konvergenta p-serien $\sum 1/k^2$). Det följer att serien $\sum(-1)^k a_k$ är konvergent. Alltså konvergerar $\sum a_k x^k$ precis då $-1 \le x < 1$.

par passes

Figure 40: Avoiding line breaking failure (overfull line).

```
\setupalign
  [hanging,     % protrusion
   depth,       % align stuff at bottom of page
   profile,     % even line spacing if possible
   granular,    % a granular setup
                %   (classes and adjacentdemerits)
   lesswidows,  % a strict widow setup
   lessclubs,   % a strict club setup
   lessorphans, % a strict orphan setup
   lessbroken,  % try to avoid hyphenations over pages
   strictmath]  % strict math penalty setup
```

The `lessbroken` option makes sure that we penalize hyphenations that run from right to left pages more than from left to right pages. This make sense in a book, where one has to turn pages.

Let us also mention that these extra par passes do not increase compilation time much. We have kept track of some of the compilation times for the book (290 pages) while working. The par passes were enabled in the September 2023 run (but the setup has been evolving).

```
December  2022 | total runtime: 12.109 seconds
July      2023 | total runtime: 7.997  seconds
September 2023 | total runtime: 8.306  seconds
April     2024 | total runtime: 9.739  seconds
April     2024 | total runtime: 18.439 seconds
                    (with synctex and tagging)
September 2024 | total runtime: 9.290  seconds
```

So, it has not slowed down much. This is of course non-scientific, but the runs were done on the same computer. If it was only the par passes that had changed (it is not), one second extra is not a big deal. With synctex and tagging enabled, the compilation time doubles.

17 Another test paragraph

In D.E. Knuth's *Digital Typography* he uses a rather math-dense paragraph as a showcase. We display that paragraph below, in a few different text widths, with the traditional run, and with the par passes from the last section, used in the math book.

In Figure 41 we note that the traditional paragraph builder is hyphenating and breaking inside one of the formulas. The third subpass manages without both.

In Figure 42 (`\hsize`=240pt), the traditional parbuilder fails, with an overfull line. Here we need subpass 5. The first line is a bit loose, but overall it looks OK.

In Figure 43, the text block is quite narrow (180pt). The traditional builder fails again, while subpass 5 succeeds. We get another broken formula and a few hyphenated lines, but given the constraints it is not too bad.

18 Summary

We have discussed an extension to the traditional Knuth–Plass paragraph builder, implemented in LuaMetaTEX, and available today to ConTEXt users.

The main new feature is the possibility of having an arbitrary number of runs over each paragraph, with independent setups for each run. With a setup where the first two runs are similar to the traditional pretolerance and tolerance runs, most ordinary paragraphs are taken care of by them, leaving only the more difficult paragraphs to be handled by the later runs. This means that the impact on speed is negligible.

We have also introduced a few new penalty and demerit parameters, and made others more configurable, with plural versions and sometimes also with the possibility to keep track of left and right pages.

The next step is to test this on various types of documents and to provide a few standard setups that make sense for the users who don't want to mess with details. There are already a few users who are up and running on their book projects, and they seem to be very satisfied.

We also touched upon how the result of the paragraph builder can influence the page builder. The process of building pages is at a first glance simpler than the building the paragraphs, since we merely add content to a vertical list until it is deemed as being full, and then ship out the page. On the other hand one has to handle footnotes, floats, sections, columns, and so on, and that greatly complicates the matter. We intend to study this process in a future project.

◇ Hans Hagen
Pragma ADE

◇ Mikael P. Sundqvist
Department of Mathematics
Lund University

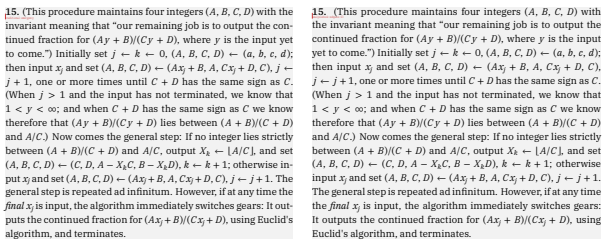


Figure 41: A test paragraph from Knuth's *Digital Typography*; `\hsize` is 300pt (scaled for *TUGboat*). Left: traditional TEX; right: par passes setup, which manages to avoid hyphenation and breaking inside formulas.

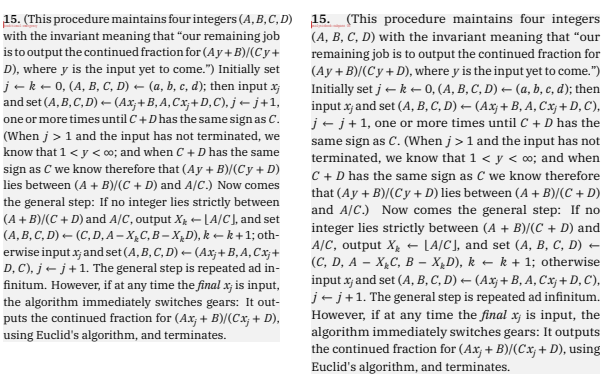


Figure 42: The test paragraph with `\hsize` = 240pt. Left: traditional TEX has an overfull line; right: par passes setup succeeds.

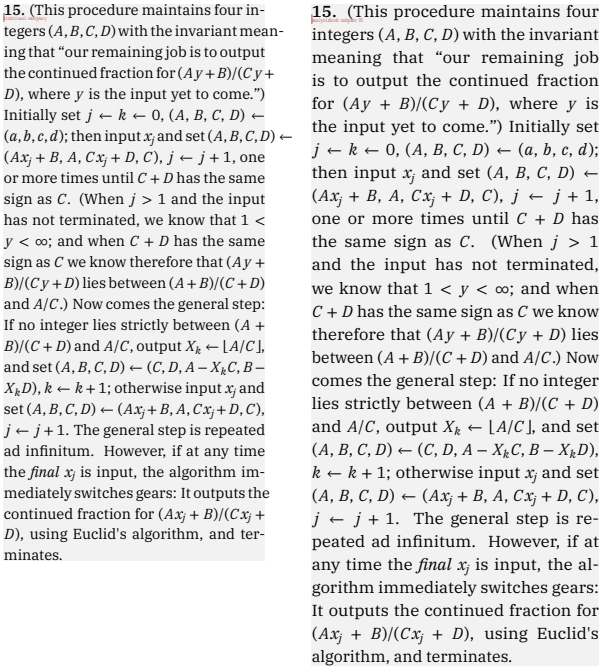


Figure 43: The test paragraph with `\hsize` = 180pt. Left: traditional TEX again has four overfull lines; right: par passes setup succeeds, with a broken formula and hyphens.

Externalization with Memoize

Sašo Živanović

Abstract

Memoize is a fast and flexible package for externalization of typesetting results, and for memoization of compilation results in general. By allowing the author to reuse the results of compilation of computationally intensive pieces of the document, it can radically cut down the compilation time of successive compilations.

In this article I introduce externalization and memoization, outline how they are implemented in Memoize, show how to use the package, and shamelessly try to entice package writers to support it in their packages.

1 Introduction

A long document containing a lot of computationally intensive content, such as Ti*k*Z pictures, can take a long time to compile even on modern machines. The wait can be frustrating, even more so because a document typically does not change much between successive compilations. We spend a lot of time waiting for all those pictures to be compiled over and over again.

One way of dealing with this is *externalization*. In this process, the results of typesetting of selected (computationally intensive) pieces of code are saved as separate PDF files. On subsequent compilations, these produced *externs* are included into the document PDF as images. As including an extern is typically much faster than compiling the code that produces it, externalization can cut down the compilation time rather radically.

Producing the externs, however, is not trivial. The issue is that TEX can only create a single PDF per compilation. Producing both the document and the externs in a single compilation is therefore impossible. To overcome this limitation, the venerable Ti*k*Z library `external` [8, §52] invokes a separate TEX process for each and every extern to be made. This approach is very natural, but has the downside that the document must be compiled as many times as there are externs to be created (plus one for the document itself). Even with the clever tricks employed by the `external` library, the time required to produce the externs may therefore greatly exceed the regular compilation time of the document — perhaps taking hours for a fresh compilation of a long book containing many Ti*k*Z pictures.

Memoize was developed specifically to overcome this issue. It is addressed with the following two-step externalization procedure, illustrated in Figure 1:

1. When compiling the main document, externs are stored into the main document PDF as separate pages. As all externs are produced in the same compilation, the document is compiled only once. Consequently, this step requires essentially the same amount of time as a regular compilation.
2. At the beginning of the next compilation of the document, the extern pages are *extracted* from the document PDF into extern PDFs. Memoize deploys an external extraction tool to extract all the externs in a single go, so extraction is almost instantaneous. Even for a long document with many externs, it takes only a couple of seconds.

You might wonder why Memoize is not called Externalize. The name of the package derives from term *memoization*, which Wikipedia defines as "an optimization technique used primarily to speed up computer programs by storing the results of expensive function calls to pure functions and returning the cached result when the same inputs occur again" [9]. Memoization is thus a broader concept than externalization. I use the latter term to refer to the subtype of memoization where it is specifically the results of typesetting which are stored for reuse. True to its name, Memoize can cache not only the typeset material (as externs) but the results of computation in general (as pieces of TEX code called *memos*). We'll see how this is useful in section 3.3.

We start gently by describing the usage of Memoize in section 2. Section 3 provides some details on how the package works, of interest to both the casual and advanced user. Section 4 goes into fairly TEXnical details about automemoization. Section 5 closes off by hinting how you can support Memoize in your own package.

2 Usage

2.1 Installation of auxiliary software

Before you can use Memoize, you have to install the PDF processing library used by the extern extraction script. The default Perl script requires the library `PDF::API2` [7] (or `PDF::Builder` [5]), and the alternative Python script requires `pdfrw2` [1] (or `pdfrw` [4]). The exact details on how to install the library depend on the system (see sections 1.1 and 3.2 of the manual for some hints), but in principle, assuming that Perl/Python is installed on the system, entering the following into the terminal should do the trick:

- `cpan PDF::API2` for the default Perl script
- `pip install pdfrw2` for the Python script

doi.org/10.47397/tb/46-1/tb142zivanovic-memoize

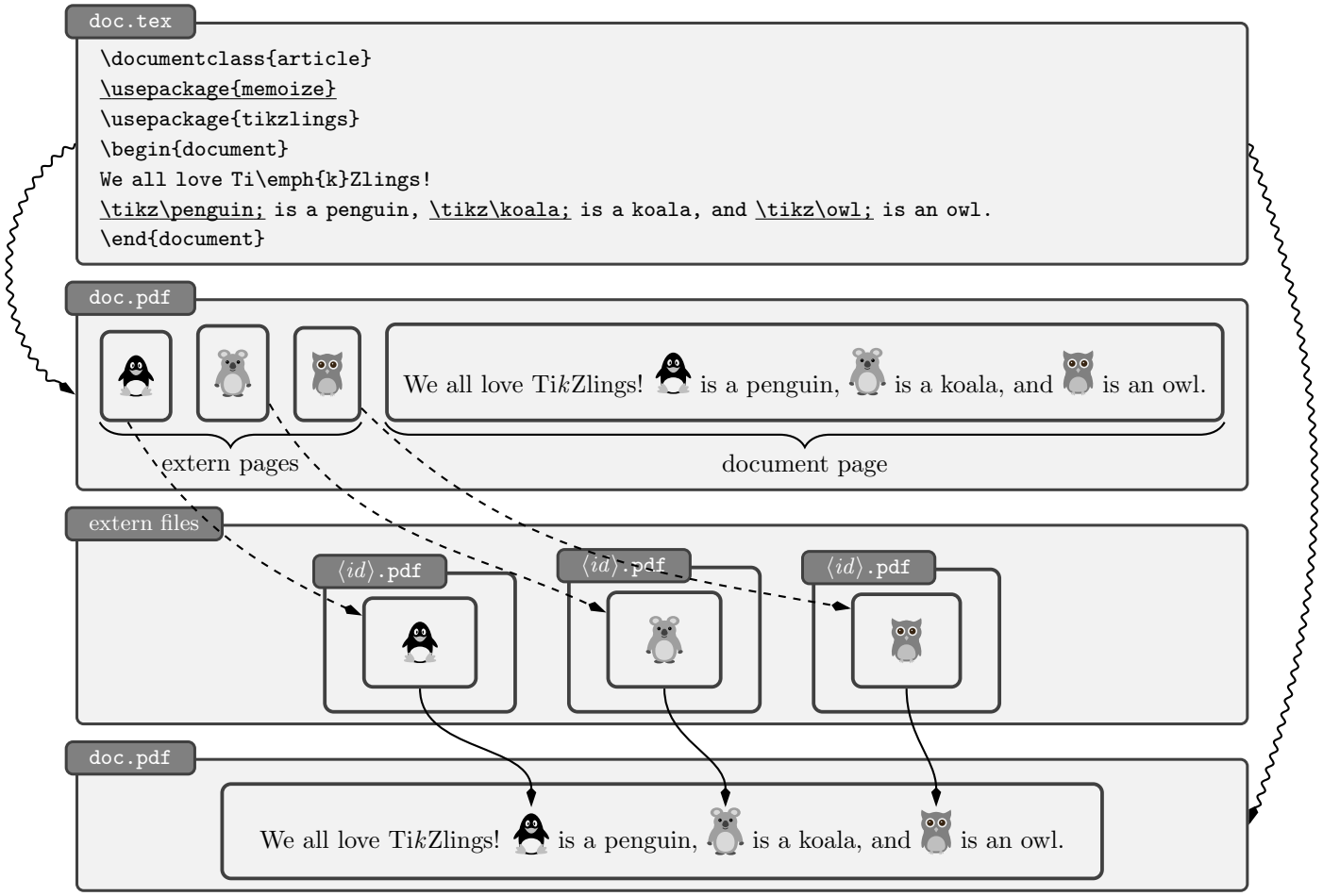


Figure 1: The externalization process in Memoize. The first compilation (the left wiggly arrow) of `doc.tex` produces `doc.pdf` containing the extern pages alongside the regular document pages. The extern pages are extracted (dashed arrows) from `doc.pdf` into extern files ⟨*id*⟩`.pdf`. On subsequent compilations (the right wiggly arrow), these extern files are included (full arrows) into `doc.pdf` as images.

2.2 Loading the package

Memoize automatically externalizes all supported content (at the time of writing this article, this amounts to Ti*k*Z pictures and Forest trees, which Memoize supports internally). Ideally, you therefore only have to load the package to use it:[1]

```
\usepackage{memoize}
```

Assuming that your document contains memoizable content, the first compilation with Memoize loaded will produce an unusual document. Each externalized piece of content will appear in the document twice, once on the page where you intend it to be, and once on a separate extern page preceding its normal location. But don't worry: the extern pages will disappear after the second compilation. The two-stage externalization process behind this behaviour, illustrated in Figure 1, was outlined in the introduction.

[1] Memoize also works with plain TEX and ConTEXt. Section 5.1 of the manual describes how to load it in these formats.

2.3 Configuration

Memoize can be configured via package options or using command `\mmzset`. Both accept a keylist, i.e., a comma-separated list of ⟨*key*⟩s and/or ⟨*key*⟩=⟨*value*⟩ pairs, which is processed using Ti*k*Z utility `pgfkeys` [8, 87]. For example, to recompile all externs and extract them using the Python script:

```
\usepackage[recompile,extract=python]{memoize}
```

To disable the functionality of the package for the duration of the TEX group:

```
\mmzset{disable}
```

Global or per-project configuration (using `\mmzset`) may be placed in a file `memoize.cfg`, which is input before processing package options.

2.4 Manual memoization

To memoize some code manually, give it as the argument of the command `\mmz`, or enclose it in an environment `memoize`. In either case, the memoized code should produce a box. Thus, you cannot (easily) externalize text which should be broken across lines or pages. Here is an example of each case:

```
\mmz{\qrcode{https://ctan.org/pkg/memoize}}

\begin{memoize}
  \begin{circuitikz}
    \draw (0,0) to[isource] (0,3) -- (2,3)
                        to[R] (2,0) -- (0,0);
  \end{circuitikz}
\end{memoize}
```

The only difference between the command and enviroment variants is cosmetic: the environment ignores spaces around the body. Both variants take an optional argument accepting the configuration keylist, which applies only to the given piece of memoized code.

2.5 Automemoization

Memoize can be instructed to automatically memoize all instances of a command or environment. To request this, specify the key `auto`, providing the name of the command/environment as the first argument and the key `memoize` as the second argument.

```
\mmzset{auto={circuitikz}{memoize}}
```

This suffices for automemoization of environments and of commands defined via `\NewDocumentCommand`. For commands defined by other means, e.g., with `\newcommand`, the second argument must also provide the argument specification of the command as understood by `\NewDocumentCommand` [6]. Below, we set `args=om` because `\qrcode` takes an optional argument (`o`) followed by a mandatory argument (`m`).

```
\mmzset{auto=\qrcode{memoize, args=om}}
```

The second argument of `auto` also accepts any regular configuration key, which will be in effect when (and only when) memoizing an instance of the submitted command. For example, to submit the standard LaTeX `verbatim` environment, which requires keys `verbatim` and `capture=vbox` (see section 2.6):

```
\mmzset{auto={verbatim}
              {memoize, verbatim, capture=vbox}}
```

Additional keys, called the *next options*, may be passed to an individual instance of an automemoized command by preceding it by a `\mmznext`, which accepts the same keys as `\mmzset`.

Automemoization of a submitted command can be switched on and off using keys `activate` and `deactivate`. For example, automemoization of Ti*k*Z pictures, which is active by default, may be suppressed by `deactivate=tikzpicture`.

2.6 Common configuration keys

The functionality of the package can be disabled using key `disable` (for the duration of the TeX group). I recommend disabling Memoize in the final version of the document, because the document will be smaller and because hyperlinks don't work in externs. Even better, I suggest loading the dummy package `nomemoize` instead of `memoize`. This package does nothing, but provides all Memoize's commands, so that Memoize can be fully "disabled" without changing the document.

Memoize can be used in one of three modes, selected by the following keys: `normal`, `recompile` and `readonly`. In the default normal mode of operation, Memoize uses any existing externs and creates any missing externs. In the recompile mode, it recreates all externs, even existing ones; this mode is useful if we change a macro used in the memoized code. In the `readonly` mode, Memoize uses any existing externs but does not create new ones; this is useful when working on a picture. Additional keys:

- The `verbatim` key allows memoizing code which needs to be read verbatim, or which performs other catcode magic.
- The key `capture=vbox` can be used to externalize material created for vertical mode.
- The `padding` key is useful for externalizing boxes whose content overflows the boundary of the box, e.g., as a consequence of using `overlay` in Ti*k*Z.
- The `per overlay` key is specific to Beamer, and allows one to externalize pictures which change from overlay to overlay.
- The keys `memo dir` and `prefix` control the location of extern and memo files. For example, they can be used to share externs between projects, or between a book and its independently compilable chapters.

2.7 Extraction

As already remarked, Memoize can perform extern extraction using a Perl script (the default) or a Python script, both requiring the installation of a PDF processing library. Memoize can also perform extraction using TeX itself. While this method has no external dependencies, it is slower, as it is again limited to

extracting a single extern at a time; nevertheless, the speed gain over the multiple-compilation method used the Ti*k*Z `external` library is still enormous.[2]

Out of the box, Memoize triggers extern extraction when it is loaded during the compilation of the document. However, extraction may also be triggered between the successive compilations of the document. This can be done either manually from the terminal, or in some automated way. Notably, recent versions of `latexmk` [2] support Memoize (consult the example rc file `memoize_latexmkrc`): they trigger extraction and perform as many compilations as necessary for the document to stabilize.

3 Auxiliary files

Out of the box, Memoize produces four kinds of auxiliary files: record files, c-memos, cc-memos, and externs.

3.1 Record files

The default record file is called ⟨*jobname*⟩`.mmz`. It resides next to the document PDF file, ⟨*jobname*⟩`.pdf`. It contains information about what memos and externs were used and created in the last compilation, and thereby drives the operation of the extraction and clean-up scripts. Once the externs are extracted out of the document PDF, the `.mmz` file can be safely deleted (although there is no need to do so).

3.2 Automatic recompilation (and not)

If a piece of memoized code is changed, Memoize will automatically recompile it. For example, changing '`\tikz\owl;`' from Figure 1 to '`\tikz\owl[hat];`' will recreate the owl extern. Memoize detects the change because it identifies an extern file by the MD5 checksum of the code which produced it. If it can find the extern with the right MD5 checksum, it will use it; if it cannot find it, it will create it. From Memoize's perspective, changing some memoized code produces a brand new piece of code with a brand new MD5 checksum, identifying a nonexistent extern, thereby triggering the creation of that extern.

If the memoized code deploys a command and the definition of that command changes, the memoized code and thus its MD5 checksum remain the same. Out of the box, changing the definition of a command does not trigger automatic recompilation (but see section 3.4). After the change, the author must trigger the recompilation manually. One way is to apply key `recompile` to the memoized code, for example by writing `\mmznext{recompile}` in front of it for one compilation. Another way is to delete the extern. This is handy because it is easy to identify an extern by its thumbnail in a file manager.

Memoize does not automatically delete stale externs (i.e., externs not used or created in the last compilation), mainly because what looks like a stale extern might not be stale after all. For example, the author might have merely temporarily commented out part of the document. To remove the pile of stale externs typically created over many compilations of the document, the author can use the cleanup script, `memoize-extract.pl`/`.py`. `latexmk`, when configured to support Memoize, automatically executes this script as a part of its clean-up routine.

3.3 Externs and cc-memos

We've already seen that an extern is a PDF file that stores the typeset result of memoization. Once created, the extern is included into the document on subsequent compilations. However, Memoize does not include an extern directly. It is the *cc-memo* file which contains the code including the extern into the document. Memoize merely `\input`s the cc-memo.

This seemingly roundabout path is useful because it makes it easy to record various information about the extern. The cc-memo records whether the extern should be included as a horizontal or a vertical box, and it also allows Memoize to set the baseline of this box appropriately by separately recording the height and the depth of an extern (in contrast, a PDF page only records the total height).

Furthermore, a cc-memo is essentially a free-form TEX file, and can therefore store anything expressible as TEX code. As one consequence, Memoize is not limited to precisely one extern per piece of memoized code. For example, it can be used to store the result of a complex mathematical computation. The resulting cc-memo would include no extern, but simply set some macro to contain the result of the computation. Conversely, a cc-memo may include several externs. This makes it possible to externalize a command with typeset output broken across pages.

These situations are not covered by Memoize out of the box, nor could they be. Since they are tied to specific usage cases, they require a specialized *memoization driver* — but writing one is less ominous than it sounds, and also fully documented.

One other notable usage of cc-memos is that they allow Memoize to support `\label`s and citations inside memoized code. Without a special provision, a `\label` occurring within memoized code would only be executed when creating the extern, but not

[2] An idea that is being actively investigated (see issue 38 on Memoize's GitHub repository) is whether one could perform extraction using `texlua`. Lua is part of every TEX distribution, so having a Lua script would eliminate the need for Perl or Python and the installation of PDF libraries before starting to use the package.

when including it in subsequent compilations. The auxiliary file created in those compilations therefore wouldn't contain the information needed by `\ref`, which would lead to an undefined cross-reference.

The cc-memo file provides a way out of this. Memoize hacks `\label` (using the key `auto`) to "replicate" itself in the cc-memo. In effect, a `\label` from the memoized code is reissued every time the cc-memo is used, and one can refer to it outside the memoized code.

3.4 Context and c-memos

In section 3.2, we saw that Memoize identifies an extern by the MD5 checksum of the code that produced it, and that this is how it automatically recreates an extern when the memoized code changes. While that is correct, it is not the whole truth. In fact, an extern is identified by MD5 checksum of the memoized code along with the MD5 checksum of the *context*. Thus, an extern is recreated upon the change of either the code or the context.

The context is the full expansion of the *context expression*, which contains arbitrary code set by the user using the `context` key or various shortcut keys. For example, the LaTeX macro keeping the current font size can be added to the context expression like this:

```
\mmzset{csname meaning to context=f@size}
```

Then any extern (say, a Ti*k*Z picture) in the scope of this command will be recompiled when we change the font size (either by choosing another document font size, or by placing e.g., `\small` in front of it).

Context underlies many features of Memoize (notably `beamer` and `biblatex` support), but it was developed specifically to support cross-references occurring inside memoized code, i.e., to have Memoize automatically recompile an extern containing a cross-reference, like a reference to a section, when that reference changes. This is achieved by hacking command `\ref` (again using the key `auto`) to use the `context` key to add its cross-reference key to the context — but to see how this works in full detail, we have to learn about c-memos.

The issue is that a `context` key hidden inside the hacked `\ref` *within* memoized code is only executed during memoization. By definition, the memoized code is not executed during utilization, i.e., when using the previously produced memos and externs. So how can the context computed at memoization (and thus containing the cross-reference) match the context computed at utilization (not containing the cross-reference)? Memoize gets out of this predicament by using *c-memos*, which store the additions to the context expression made during memoization.

Unlike cc-memos and externs, c-memos are identified solely by the MD5 checksum of the memoized code. The naming conventions listed below should explain the terms c-memo and cc-memo, which are abbreviations for *c*ode memo and *c*ode–*c*ontext memo.

- c-memo: ⟨*code md5*⟩`.memo`
- cc-memo: ⟨*code md5*⟩`-`⟨*context md5*⟩`.memo`
- extern: ⟨*code md5*⟩`-`⟨*context md5*⟩`.pdf`[3]

We have seen that Memoize cannot utilize a cc-memo directly, because the context expression at the start of utilization might not be complete. Therefore, Memoize first reads the c-memo, whose filename does not depend on the context MD5 checksum. The c-memo contains the part of the context expression added during memoization, so Memoize can update the context expression based on these additions. Once the context expression matches the context expression at the end of memoization, computing the MD5 checksum of its expansion will find the produced cc-memo precisely when the context did not change. Memoize then inputs the cc-memo, which typically includes the extern.

By default, c-memos also contain the memoized code for informational purposes. Therefore, one can identify the c-memo associated with some memoized code by searching c-memo files for that code, perhaps to trigger recompilation by deleting the memo (a piece of memoized code is recompiled if any of the associated auxiliary files is missing).

4 The auxiliary packages

Implementing robust and user-friendly automemoization was one of the major challenges in the development of Memoize. Eventually, most of the code was related to automemoization, but it was also clear that much of that code would be generic enough to warrant a separate package. At the end of the day, Memoize was shipped with two generic auxiliary packages, Advice and CollArgs: Memoize uses Advice, and Advice in turn uses CollArgs. Let's look at the challenges these packages were designed to address.

4.1 Advice

Package Advice, a namesake of Emacs's advice feature [3, §13.12], implements a generic framework for extending the functionality of selected commands and environments. Each *advised* command is assigned a piece of *advice*: a command which is executed instead of the advised command, and which may, or

[3] More precisely, if more than one extern is associated to a cc-memo, the name of a non-first extern is ⟨*code md5*⟩`-`⟨*context md5*⟩`-`⟨*N*⟩`.pdf`. The numbering starts at zero.

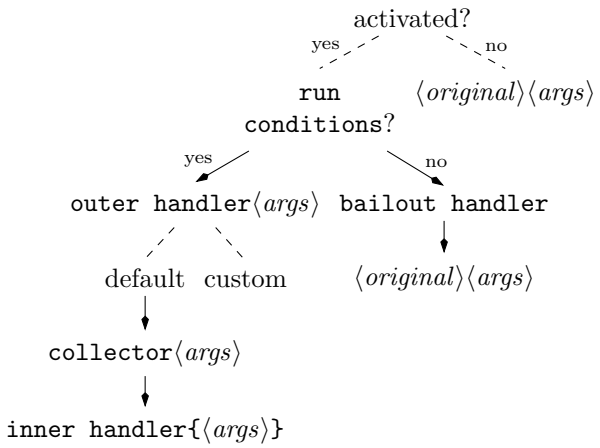


Figure 2: Advising a command

may not, invoke the original command during its execution. In other words, Advice offers a (reasonably safe and elegant) hacking mechanism. Writing `auto=\foo{...}` has an effect akin to:[4]

```
\let\original@foo\foo
\def\foo...{... maybe \original@foo ...}
```

From the author's perspective, the most appealing feature of Advice might be its user-friendly `pgfkeys`-based interface. First, once an advice is defined, it is easy to deactivate and (re)activate: the key `activate=\foo` (automatically executed by `auto`) redefines `\foo` into whatever advice was defined by the second argument of `auto`, and then its counterpart `deactivate=\foo` restores the original definition of `\foo`.

Moreover, an instance of Advice can be set up to allow advice to be easily created from various components set up by the programmer. Remember how we submitted `\qrcode` to automemoization in section 2.5:

```
\mmzset{auto=\qrcode{memoize, args=om}}
```

Quite a lot is happening behind the scenes here. An advice is composed of up to five components, illustrated in Figure 2, and the style `memoize` puts them together to perform the desired function.

Let's start with the final two components, which perform most of the real work. The *collector* receives the argument structure specified by `args`, collects the arguments given to the original command and passes them, in the form of a single braced argument, to the *inner handler*, which performs memoization.

The default collector is `\CollectArguments` of package CollArgs (see section 4.2), but custom collectors are also accepted. Advice defines one to deal with the idiosyncratic syntax of `\tikz`, whose mandatory argument can be either braced or delimited by a semicolon.

Furthermore, a handler may have associated *run conditions*; for example, the advice of an automemoized command is only executed when Memoize is enabled. If the run conditions are not met, Advice executes the *bailout handler* before yielding control to the original command; Memoize uses the bailout handler to throw away the `mmznext` keys.

One final component remains to be described: the *outer handler*. The outer handler performs no fixed function; it may do anything as long as, at the end of the day, all and only the arguments of the original command are consumed. By default, it does nothing but call the collector. In automemoization, it opens a group and applies the next options before calling the collector. But in several usage cases in Memoize, it is the outer handler which does the actual work, bypassing the collector and the inner handler completely.

For example, Memoize also uses Advice for making `\ref` add the cross-reference key into the context, and for aborting memoization upon executing certain commands (notably `\pdfsavepos`, which underlies Ti*k*Z's `remember picture` mechanism). These pieces of advice either expect to advise a command with a fixed argument structure (`\ref`) or do not care about the arguments of the advised command (abort). As they therefore have no need of a collector, they are implemented as the outer handler, in accord with the overall desire to have Advice incur as little overhead as possible.

4.2 CollArgs

To automatically memoize an occurrence of a command, Memoize has to determine its scope, i.e., it needs to figure out "how far the command extends". Suppose we have submitted the command `\qrcode`, which takes a single argument, to automemoization, and that we're about to read `\qrcode{text}{more text}`. Memoize has to grab precisely the command plus its argument, i.e., `\qrcode{text}`, and either run this piece of code inside an `\hbox` to produce an extern, or replace it by the code including an existing extern. Attempting to externalize merely `\qrcode` would produce an error, and externalizing `\qrcode{text}{more text}` would include "more text" in the extern.

Grabbing the arguments of a command is the job of `\CollectArguments` of the package CollArgs. But TEX being TEX, automatically determining the scope of a command is, *in general*, impossible. (Inspecting

[4] By default in the Advice package, the setup key is named `advice`, but a package using Advice can name this key whatever it wants, and Memoize names it `auto`.

the \meaning is not enough, because the "real" and the formal arguments of a command can, and often do, differ wildly.) This is why \CollectArguments must be informed what arguments to expect. As we have seen in section 2.5, the argument specification should be provided in the LaTeX3 format, i.e., the format understood by \NewDocumentCommand. For the user, this has the advantage of (possible) familiarity (no need to learn a new specification), and the bonus of not having to provide the argument structure for commands defined by \NewDocumentCommand, as it can be programmatically accessed through the macro \GetDocumentCommandArgSpec of package xparse.

CollArgs accepts all the argument types of the LaTeX3 specification, including the uncommon types only defined in package xparse. Further, any type of argument can be read either as-is or verbatim. One may even alternate regular and verbatim arguments. CollArgs takes care that the input following the arguments is tokenized under the original category code regime.

The package also honors all the modifiers of the specification, and introduces one new modifier (&) for configuration of individual arguments. Configuration can be given using pgfkeys key–value pairs, or via a faster, macro-based "programmer's interface".

Being a cross-format package, CollArgs automatically adapts the environment type b to the format, i.e., it respects the convention that environment foo is marked by \foo ... \endfoo in plain TeX and by \startfoo ... \stopfoo in ConTeXt.

Normally, the argument list is "returned" (by passing it to the given "next" macro as a single braced argument) exactly as it appeared in the input, but this is configurable. In particular, arguments can be post-processed. For example, an optional argument [opt] can be transformed into {opt}.

4.3 CollArgs and Advice in action

Suppose we want to develop a \listedhref command which uses \href of the package hyperref to typeset a hyperlink, but additionally add the URL to the list to be printed out at the end of the paper (using \printURLs). A naive approach might look like this:

```
\documentclass{article}
\usepackage{hyperref}
\newcommand\addURL[1]{%
  \gappto\URLs{\item\texttt{#1}}}
\newcommand\listedhref[2]{%
  \addURL{#1}\href{#1}{#2}}
\newcommand\printURLs{%
  \begin{itemize}\URLs\end{itemize}}
\begin{document}
\listedhref{https://www.tug.org/TUG}{TUG}
\printURLs
\end{document}
```

While this code works, the approach will fail for URLs containing special characters like & and %. One might try to address the issue by changing the definition of \listedhref to use \addURL{\detokenize{#1}}. This would work for & but not for %. The problem cannot be fully solved without playing with the category codes, and CollArgs can do that for us:

```
\usepackage{collargs}                                  1
\collargsSet{add URL/.style={                          2
  append preprocessor={\xappto\URLs{%                  3
    \noexpand\item                                     4
    \noexpand\texttt{\the\collargsArg}}}}}             5
\newcommand\listedhref{\CollectArguments               6
  [return=plain]                                       7
  {&{verb,add URL}m m}                                 8
  \href}                                               9
```

The key verb in line 8 selects the partial verbatim mode (all characters but braces are of category "other") for the first argument (the second argument is not affected). The style add URL defines a processor (because this is a *pre*processor, the argument has not yet been dressed up in braces) which appends the argument to the URL list. The key return=plain in line 7 prevents the collected argument list from being braced, so that it can be fed directly to \href (line 9), which expects two arguments.

Let us now see how we can wrap this idea into an advice to extend \href itself to append the given URLs to the list:

```
\usepackage{advice}
\pgfkeys{
  .install advice,
  advice=\href{
    collector options={return=plain},
    args=&{verb,add URL}m m,
    inner handler=\AdviceOriginal,
  },
}
```

We first install the advice mechanism into the root of the pgfkeys tree, and then advise \href. The values of the keys in the second argument of advice become the arguments of \CollectArguments, so the advised \href executes precisely \listedhref defined above. The macro \AdviceOriginal executes the original definition of the advised command.

5 Supporting Memoize in your package

As we have seen in section 2.5, some commands (e.g., the environment tikzpicture) may be submitted

to automemoization with a simple invocation of the `auto` key. What distinguishes these commands is that they produce a single box and let the paragraph or page building mechanism (or some other command) integrate it into the document as it sees fit. Other commands, like the standard floating environments `table` and `figure`, are (almost) all about integrating the given material into the document. It makes no sense to memoize these. In between, there are commands which both typeset some specific content and integrate it into the document. If the typesetting part is computationally intensive, it makes sense to (auto)memoize them. However, doing so requires dedicated support.

Consider a command which typesets some material and makes it float. A good example is environment `tcolorbox` with option `float`. Blindly memoizing this environment will fail with the error "Not in outer par mode". It is not hard to see why. On one hand, the floating mechanism can only operate in the external vertical mode (where material is being collected to be paginated). On the other hand, Memoize can only externalize the typeset material if it is collected into a box; in other words, memoization requires (and triggers) restricted horizontal mode (`capture=hbox`) or internal vertical mode (`capture=vbox`). The only solution I see is to typeset the material in a box, invoke Memoize to store the box as an extern page, and then float the box. However, this can be cleanly done only within `tcolorbox` itself. It is the specific implementation of this environment which determines the right moment to invoke Memoize; the code might even need to be refactored for such a moment to exist. Any attempt to implement the solution "from the outside" would surely count as a kludge.

Generally speaking, complex commands might be memoizable only if their implementation follows a certain design: it should be separated into a (computationally intensive) inner command, which typesets what should be memoized and is executed only during memoization, and an outer command, which integrates the typeset content into the document and is executed even when using an extern.

I hope the above example convinces you that for some commands, it only makes sense that the package implementing the command supports Memoize, rather then the other way around. To facilitate this direction of support, Memoize offers a fully documented memoization interface, plus a stub package `memoizable` which allows the supporting package to use Memoize's commands while leaving the loading of Memoize to the discretion of the author.

Furthermore, a large part of the Memoize manual is written specifically for the supportive package writer. Notably, section 3.5.2 illustrates the "memoizable design" outlined above on a dummy package, while section 4.4 explains the role of a memoization `driver` and shows how a specialized driver can be used to externalize content broken across pages, and how to implement memoizable design in a way that is transparent to the author.

To close off, let me point out that supporting Memoize makes sense even for packages implementing commands which are easy to (auto)memoize. If nothing else, preemptively submitting a command to automemoization is a courtesy to the author, because the author cannot be sure that automemoization is in fact easy. I therefore hope to see packages starting to support Memoize, and thank their authors in advance for their time and effort. If you have any questions about this, don't hesitate to write!

Acknowledgments

My biggest thanks go to Stefan Müller of Language Science Press, for the initial motivation to implement the package, Karl Berry of TEX Live, for getting the extraction scripts among the shell escape commands, and `cfr` from TEX StackExchange, for advertising and testing the package extensively. For the rest of the helpful people I'm grateful to, see the Acknowledgments section of the manual.

References

[1] S. Arnold. Pdfrw2. `pypi.org/project/pdfrw2`
[2] J. Collins. Latexmk. `ctan.org/pkg/latexmk`
[3] Free Software Foundation. *GNU Emacs Lisp Reference Manual*. `www.gnu.org/software/emacs/manual/html_node/elisp`
[4] P. Maupin. Pdfrw. `pypi.org/project/pdfrw`
[5] P.M. Perry. PDF::Builder. `metacpan.org/pod/PDF::Builder`
[6] LATEX Project Team. *LATEX for authors*. `www.latex-project.org/help/documentation/usrguide.pdf`
[7] S. Simms. PDF::API2. `metacpan.org/pod/PDF::API2`
[8] T. Tantau. *TikZ & PGF, Manual for Version 3.1.10*. `ctan.org/pkg/pgf`
[9] Wikipedia. Memoization. `en.wikipedia.org/wiki/Memoization`

◇ Sašo Živanović
University of Ljubljana
Slovenia
`saso dot zivanovic (at) guest dot arnes dot si`
`https://spj.ff.uni-lj.si/zivanovic`
ORCID 0009-0001-5099-2867

Processing data from CSV and JSON files with OpTEX

Petr Olšák

Introduction

When we want to do something with CSV or JSON files in LaTeX, we can load an appropriate package. In ConTeXt, we can use relevant commands directly. What can we do in OpTEX?

OpTEX [1] implements this feature by a set of OpTEX tricks [2]. They are grouped in the section "Databases". Short macro definitions are here. Users can copy them to their macro files and modify them to meet their specific requirements. Moreover, many OpTEX tricks are set as *autoloads*. This means that if you don't need to modify the presented macro code, then you can simply use the user-level macro (`\csvread`, `\dbloop`, `\jsonread` for example) and the corresponding bundle of macro code is loaded automatically at its first use. You need to do nothing more than use this command. No packages need to be declared in the preamble of the document.

This article shows the options for using CSV and JSON files with OpTEX, i. e. by the relevant OpTEX tricks.

Processing CSV files

CSV files are simple databases. They collect data in rows and columns of a table. The columns have their names declared in the first row. The data items in each row are optionally surrounded by special characters (typically `"`) and they are separated by another character, traditionally comma but semicolon, tab, etc., are also used. A CSV file might look like this:

```
id, "FirstName","LastName","Address"
013,"Petr",     "Olšák",   "Prague"
142,"Ferdinand","Mravenec","U lesa"
```

We can read a CSV file by the command

`\csvread {`⟨*filename*⟩`.csv}`

Once this is done, the macro

`\dbitem(`⟨*row*⟩`,`⟨*column*⟩`)`

expands to the corresponding data from the file ⟨*filename*⟩`.csv`. The macro `\colname` ⟨*name*⟩ can be used instead of the numeric ⟨*row*⟩. The number of rows and columns are saved in `\dbmaxrow` and `\dbmaxcol` macros. The column name can be retrieved with the macro `\dbhead(`⟨*column*⟩`)`.

Suppose that the CSV file shown above is saved in the `names.csv` file. Then we can do:

```
\csvread {names.csv}
   % reads CSV file and saves data to memory
\dbitem(1,2)
   % returns an item from 1-st row, 2-nd column,
   % i.e. Petr
\dbitem(1,\colname Address) % returns Prague
\dbhead(3)                  % returns LastName
\edef\foo{\dbitem(2,1)}     % does \def\foo{142}
\edef\address{\dbitem(2,\colname Address)}
   % does \def\address{U lesa}

\fornum 1..\dbmaxrow \do
   {\dbitem(#1,\colname LastName) }
   % prints all data in the column Lastname

\fornum 1..\dbmaxrow \do
   {\def\e##1{\dbitem(#1,\colname##1)}
    Here is \e{LastName}, \e{Address}, ...\par}
   % processes the whole database row by row
```

If a CSV file uses a separator other than comma, or a surrounding character other than `"`, you can declare `\csvdecl`⟨*o*⟩⟨*c*⟩⟨*s*⟩ before `\csvread`. The opening character is ⟨*o*⟩, the closing character is ⟨*c*⟩ and the separator is ⟨*s*⟩. By default `\csvdecl"",` is assumed.

You can read more CSV files and work with more databases in a single document when optional database name ⟨*dbname*⟩ is used. The syntax is:

`\csvread` ⟨*dbname*⟩`{`⟨*filename*⟩`.csv}`
`\dbitem` ⟨*dbname*⟩`(`⟨*row*⟩`,`⟨*column*⟩`)`
`\dbhead` ⟨*dbname*⟩`(`⟨*column*⟩`)`
`\dbmaxrow`⟨*dbname*⟩`, \dbmaxcol`⟨*dbname*⟩

The `\dbmaxrow`⟨*dbname*⟩, `\dbmaxcol`⟨*dbname*⟩ are single control sequences. If we did, for example, `\csvread xx{file.csv}` then the `\dbmaxrowxx` and `\dbmaxcolxx` macros are defined. They include the number of rows and columns in the `xx` database.

If we want to loop over database rows more comfortably than via `\fornum` (shown above), we can use the `\dbloop` macro with syntax

`\dbloop` ⟨*dbname*⟩`(`⟨*assign list*⟩`) {`⟨*text*⟩`}`

where ⟨*dbname*⟩ is the database name, ⟨*assign list*⟩ is a comma-separated list where local macros for selected columns are declared, and ⟨*text*⟩ will be repeated for each row. For example:

```
\csvread {names.csv}
\dbloop (\fname=FirstName,
         \lname=LastName,
         \address=Address)
   {%
      \lname\ \fname\ lives at
      the address: \address \par
   }
```


doi.org/10.47397/tb/46-1/tb142olsak-optex-db}user{/Subtype /Link /A << /S /URI /URI (https://doi.org/10.47397

Users must decide in what catcode regime the CSV file is to be read. It depends on the application. The data in the file can be strings that have nothing to do with TEX. Then we can deactivate special TEX characters used in data, for example:

```
{\catcode`\%=12 \catcode`\$=12 \catcode`\&=12
 \csvread {file.json}}
```

The `\csvread` can be used in the group because it defines all output macros globally. If you want to read the CSV file completely in verbatim mode, declare `\let\csvreadsetup=\_setverb`.

On the other hand, the CSV file may include TEX chunks, for example, math formulas. Then we can keep the catcodes unchanged during `\csvread`. The `\dbitem` macro expands (in `\edef` context) to the real data where possible macros are kept unexpanded.

Modifying and saving data in databases

Once a database is read by `\csvread`, we can make modifications of the data by

```
\dbinsert ⟨dbname⟩({data1}{data2}{data3})
   % inserts a new row to the database ⟨dbname⟩
\dbitemset ⟨dbname⟩(2,3){⟨data⟩}
   % replaces item(2,3) with ⟨data⟩
\dbheadset ⟨dbname⟩(3){⟨head⟩}
   % replaces header of the 3. column by ⟨head⟩
```

Also, you needn't read the database from CSV file with `\csvread`; you can create it from scratch with

```
\dbcreate ⟨dbname⟩({⟨head1⟩}{⟨head2⟩}..{⟨head n⟩})
```

and then use `\dbinsert` repeatedly.

If you want to add a new column to an existing database, then the following code can be used. We suppose in this example that the database name is `xx`.

```
\xdef\dbmaxcolxx{\the\numexpr\dbmaxcolxx+1}
\dbheadset xx(\dbmaxcolxx){New-head}
```

The following macros show database data in various ways:

```
\dbshow ⟨dbname⟩()
   % expands the whole database in format
   %    {data1}{data2}{data3}
\dbshow ⟨dbname⟩("",)
   % expands the whole database in format
   %    "data1","data2","data3"
\dbshowrow ⟨dbname⟩(3)("",)
   % expands row 3 only, in format
   %    "data1","data2","data3"
\dbshowhead ⟨dbname⟩("",)
   % expands headers in format
   %    "head1","head2","head3"
\wterm{\dbshow ⟨dbname⟩("",)}
   % prints database ⟨dbname⟩ on the terminal
```

And this macro writes database data to a CSV file:

```
\dbwrite ⟨dbname⟩("",){⟨filename⟩.csv}
   % writes database ⟨dbname⟩ to ⟨filename⟩.csv
```

Searching and sorting in databases

We want to search a row in the database (loaded by `\csvread`, for example) by a value in the given column. We can search it sequentially, but a better idea is to first prepare a hash table for the given column (especially if we want to do more searching in the same column). This can be done with `\dbhash(`⟨*column*⟩`)`. Then the row number with ⟨*data*⟩ in the given column is expanded by the macro `\dbwhere(`⟨*column*⟩`){`⟨*data*⟩`}`. Suppose our example above. Then:

```
\csvread {names.csv}
\dbhash (\colname LastName)
   % hash table for searching in LastName
   % column prepared
\dbitem (\dbwhere(\colname LastName){Olšák},
         \colname FirstName)
   % returns Petr
```

Databases with given names can be searched too:

```
\dbhash ⟨dbname⟩(⟨column⟩)
   % prepares the hash table for ⟨dbname⟩
\dbwhere ⟨dbname⟩(⟨column⟩){⟨data⟩}
   % returns row where ⟨data⟩ in ⟨column⟩
```

`\dbwhere` returns zero if a hash table for a given column isn't prepared by `\dbhash` or the ⟨*data*⟩ is not found.

You can prepare hash tables for more columns. For example

```
\fornum 1..\dbmaxcol \do {\dbhash(#1)}
```

prepares hash tables for all columns in the database.

If there are more equal data items in the column used for searching then the row number with the first one is returned by `\dbwhere`. But `\dbhash` prepares a set of macros `\dbrow:`⟨*dbname*⟩`(`⟨*column*⟩`)`⟨*data*⟩ which include the comma-separated list of row numbers where ⟨*data*⟩ is in the given ⟨*column*⟩s. We can use them together with the `\dbloop` macro (introduced in the first section) because of an additional feature of `\dbloop`: If the `\dblist:`⟨*dbname*⟩ macro is defined as a comma-separated list of row numbers then `\dbloop` executes ⟨*assign list*⟩ and ⟨*text*⟩ for these rows in the given order. Otherwise, it processes all rows in the database from the first to the last row.

We show this feature in the next example, where we use our testing file `names.csv` with additional lines. Now we have not only Olšák Petr, but also Olšák Miroslav and Olšák Radek in the file:

```
id, "FirstName","LastName","Address"
013,"Petr",     "Olšák",   "Prague"
142,"Ferdinand","Mravenec","U lesa"
014,"Miroslav", "Olšák",   "Cambridge"
015,"Radek",    "Olšák",   "Prague"
```

Then we can try:

```
\csvread {names.csv}
\dbhash (\colname LastName)
   % The \dbrow:(3)⟨data⟩ macros are created
\sxdef{dblist:}%
   {\trycs{dbrow:(\colname LastName)Olšák}{}}
\dbloop (\fn=FirstName) {\fn,}
   % expands to comma-separated list
   % Petr,Miroslav,Radek,
```

OpTEX is able to run a sorting algorithm internally. Primarily, it is used for sorting index entries. But the database entries can be sorted with the same algorithm too. We can use:

`\dbsort` ⟨*dbname*⟩`(`⟨*column*⟩`)`

This macro prepares a comma-separated list of row numbers saved in the macro `\dblist:`⟨*dbname*⟩. These numbers are ordered with respect to the ordered items in the given ⟨*column*⟩. Thus we can use `\dbloop` just after `\dbsort` and the rows are executed in sorted order. Example:

```
\csvread {names.csv}
\dbsort (\colname LastName)
   % data sorted alphabetically by LastName
\dbloop (\fn=FirstName,\ln=LastName,\ad=Address)
   {\ln\ \fn\ lives at the address: \ad \par}
   % prints data sorted by LastName
```

The macro `\dbsort` sorts all rows by default, but if `\db:sortlist` is defined as a comma-separated list of row numbers, it sorts only these rows. This means that we can do a search, then sort all matched rows, then print them:

```
\csvread {names.csv}
\dbhash (\colname LastName)
\sxdef{db:sortlist}%
   {\trycs{dbrow:(\colname LastName)Olšák}{}}
   % what we want to sort
\dbsort (\colname FirstName)
\dbloop (\fn=FirstName) {\fn,}
   % expands to comma-separated list
   % Miroslav,Petr,Radek,
```

The first names are printed in alphabetical order in this example. More options for working with simple databases can be found at the OpTEX tricks [2] web page.

JSON files and trees

A JSON file allows describing the tree data structure in human-readable form. To summarize the syntax, each `{...}` is a node, its children are listed in it and they have identifiers; each `[...]` is also a node, but its children have no identifiers; each `"name":"value"` is a leaf with an identifier; and a `"value"` (without `"name":`) is a leaf without an identifier. For further examples in this article, we use this `bib.json` file:

```
{
  "title": "Big Book",
  "isbn":  "123456789",
  "authors": [
  {
     "firstname": "John",
     "lastname": "Doe"
  },
  {
     "firstname": "Patrik",
     "lastname": "Black"
  }
  ]
}
```

We can read the data from a JSON file and save it to a `\macro` with

`\jsonread [`⟨*root-id*⟩`]\macro {`⟨*filename*⟩`.json}`

This command saves data to `\macro` in the tree-structure format declared in OpTEX trick 0153 [3]. All three data items are included in a single `\macro`. Depth-first processing over the tree can be done simply by executing `\macro`. For example, the data from the file `bib.json` is saved to `\macro` in the following structure:

```
\inode[root-id] {%
   \enode[title] {Big Book}%
   \enode[isbn] {123456789}%
   \inode[authors] {%
      \inode[] {%
         \enode[firstname] {John}%
         \enode[lastname] {Doe}}%
      \inode[] {%
         \enode[firstname] {Patrik}%
         \enode[lastname] {Black}}}}
```

The listing above was printed by the following code:

```
\jsonread[root-id]\macro{bib.json}
\wterm{\showtree\macro}
```

The `\showtree` macro used here prints the tree structure with correct indentation.

You can see that the tree-structure format is based on two sub-macros:

```
\inode[⟨id⟩] {⟨nodes⟩} % internal node
\enode[⟨id⟩] {⟨data⟩}  % external node (leaf)
```

The `\inode` macro expands to

`\startinode` ⟨*nodes*⟩ `\stopinode`

and the `\enode` macro expands to

`\execenode{`⟨*data*⟩`}`

Thus a macro programmer can declare macros `\startinode`, `\stopinode` and `\execenode#1`.

OpTEX trick 0155 [4] shows how to read data for invoices from a JSON file and print the invoices including the product list, prices and calculation of the summary price. The data is read by executing `\macro` as above, where a `\startinode`, `\stopinode` and `\execenode` are appropriately declared.

All child nodes of a selected node can be saved to a macro; let's call it `\cmacro`. Then the command `\execedges\cmacro{`⟨*text*⟩`}` does the following for each node in `\cmacro`: a) executes its subtree; then b) executes ⟨*text*⟩. It was used in the example with invoices. The node `items_sold` includes one or more nodes with products. They are executed by `\execedges`, i. e. data from their subtree is read and printed to the invoice table.

Next we show another approach. The tree-structure can be converted by a set of macros which include the data of external nodes. This can be done via `\treedata[`⟨*root-id*⟩`]\macro`. For example:

```
\jsonread\macro{bib.json} % read JSON to \macro
\tracingtreedata  % information on the terminal
\treedata[bib]\macro % does the conversion
```

The following message is printed on the terminal:

```
\treedata:
   \sdef {bib/title}{Big Book}
   \sdef {bib/isbn}{123456789}
   \sdef {bib/authors[1]/firstname}{John}
   \sdef {bib/authors[1]/lastname}{Doe}
   \sdef {bib/authors[2]/firstname}{Patrik}
   \sdef {bib/authors[2]/lastname}{Black}
```

We can see that macros `\bib/title`, `\bib/isbn`, etc., are defined by `\treedata` and user can use them for example by `\cs{bib/title}`, `\cs{bib/isbn}`, `\cs{bib/authors[1]/lastname}`, etc.

Suppose that we have more bibliography entries like the above in our JSON file in the format

```
[ { entry }, { entry },..., { entry } ]
```

Now, `\treedata` prints the following information on the terminal:

```
\treedata:
   \sdef {bib[1]/title}{Big Book}
   \sdef {bib[1]/isbn}{123456789}
   \sdef {bib[1]/authors[1]/firstname}{John}
   \sdef {bib[1]/authors[1]/lastname}{Doe}
   \sdef {bib[1]/authors[2]/firstname}{Patrik}
   \sdef {bib[1]/authors[2]/lastname}{Black}

   \sdef {bib[2]/title}{Nice Article}
   \sdef {bib[2]/isbn}{987654321}
   \sdef {bib[2]/authors[1]/firstname}{Helen}
   \sdef {bib[2]/authors[1]/lastname}{Green}
   ...
```

All nodes without identifiers are numbered (locally started from one). The number of numbered entries is saved in macros `\relevant/path[]`. For example `\cs{bib[1]/authors[]}` expands to 2 (two authors in the first entry) and `\cs{bib[]}` expands to the number of bibliography entries in the JSON file. You can use `\fornum 1..\cs{bib[]} \do{...}`.

Users can declare more node identifiers to be numbered in the control sequence names. For example, we might need to support a single author having more than one `firstname` value. Then declaring `\def\reptreenames{firstname}` before invoking `\treedata` causes the `firstname` nodes to be locally numbered too.

We can do more manipulation with trees as shown in OpTEX tricks. For example, the macro

`\treenodes[`⟨*id*⟩`]\macro`

converts the tree to a set of macros, each node is a single macro and they are linked together from parents to children and back. Then users can merge more trees into a single one, connect a tree as a subtree of another one, remove subtrees, change the order of edges of a given node, add new edges, replace data of selected leaves, run tree-processing algorithms, etc. Then the `\treenodes` format can be converted back to a single `\macro` tree-structure with `\inode`s and `\enode`s with

`\edef\macro{\treefromnodes{`⟨*root-node*⟩`}}`

References

1. OpTEX. `petr.olsak.net/optex/`
2. OpTEX — tips, tricks, howto.
   `https://petr.olsak.net/optex/optex-tricks.html`
3. OpTEX trick 0153: Working with tree data structures.
   `https://petr.olsak.net/optex/optex-tricks.html#showtree`
4. OpTEX trick 0155: Example of using the tree data structure.
   `https://petr.olsak.net/optex/optex-tricks.html#usetree`

◇ Petr Olšák
Czech Technical University
in Prague
`https://petr.olsak.net`

Output routines extending \plainoutput to avoid widow and other problematic lines

Udo Wermuth

Abstract

This article explains the implementation of three output routines — some with silent output paths — for plain TEX that address the reporting, prevention, or avoidance of widow lines on single pages and spreads. Club (or orphan) lines as well as hyphenated words at a page break are looked at too.

1 Introduction

In the article [8] I described a technique for output routines using *silent output paths* without destroying marks. With their help we are able to keep `\topmark` valid if we have to analyze `\box255` and put it back for reprocessing before we can ship it out. This follow-on article applies this technique to more realistic problems than shown in [8]. But I neither describe nor explain silent output paths again.

Terminology [5, p. 104]: In a paragraph that contains one or more displays we call every part that precedes a display a *partial paragraph*; the last part that only follows a display keeps the name of the whole: *paragraph*. The words "(partial) paragraph" mean "paragraph and/or partial paragraph".

Two problems. We want to detect pages that have either a *club line* (also called *orphan line*) or a *widow line*. The first line of a (partial) paragraph is named in *The TEXbook* a club line if it is the last line on a page. (Donald E. Knuth gives in the file `TEX82.DIF [TEX,DEK]` on `saildart.org` a reference: Geoffrey Glaister, *Glossary of the Book*, 1979 (2nd ed.), U. of Calif. Press.) The last line of a (partial) paragraph is called a widow line if it starts a page. Typographers [1, §2.4.8] and style guides [3, §3.11] recommend avoiding such lines although they sometimes differ in details: The typographer tolerates club lines but not widow lines; the style guide accepts widow lines if they occupy the full horizontal width, i.e., `\hsize`, and rejects club lines.

Rejection of widow lines is not limited to the typography of English texts. The German authors of [9, p. 196f.] allow club lines but reject widow lines except of width `\hsize`. Moreover, the text width in the last line on a spread's left page should exceed 0.25`\hsize`, i.e., the line should be $> 25\%$ filled, to keep the "aesthetic balance" between the pages.

The references [1, §2.4.9] and [3, §3.12] agree about how to fix the situation in single-column texts. First, try to change the line breaks of a paragraph either to create one more line or to save one. TEX provides the parameter `\looseness` [5, p. 103f.] to change the number of lines in a paragraph within the given parameters. Or one can add or remove white space, for example around a display or between paragraphs. Sometimes a typographer sets lines tighter than the specified minimum; with some effort this can be done in TEX too. And the author of the text can always rewrite it. If all available methods do not fix the situation, then, as a second step, change the length of pages to move lines from one page to another. But do that for the spread so that facing pages keep the same height.

On the other hand [9, p. 196] accepts different page heights in a spread if an additional line on one page is $< 25\%$ filled, but avoids such a line on the left page if the right ends in a short line too.

For single-sided documents the increase or decrease by one line for `\vsize` is always a solution — at least for straight texts, i.e., for documents that contain only single text lines and no unbreakable constructions with more than one line, like a multiline vbox, a display, or a line with a footnote.

Spreads need a more complex treatment as an addition of two lines, i.e., one line per page, keeps, for example, a widow line if the paragraphs that are split by the page breaks for the odd-numbered, i.e., right, page consist of two lines each.

TEX's solution. Of course, Donald E. Knuth is aware of these problems. In the very first meeting of TUG he was asked about TEX78 as a general typesetting tool. The protocol states (*TUGboat* **1**:1, p. 13): "The line breaking and paragraph building mechanism is particularly successful, producing text with astonishingly few hyphenations. Widow avoidance is less successful: in mathematics, adding lead to a display is feasible to defeat widows, but in straight text non-uniform interline spacing is frowned upon." (That is, `\baselineskip` should be a dimension.)

The current TEX is better than TEX78. For example, it now uses three penalty parameters instead of one for page breaks that split one line from a (partial) paragraph. TEX uses `\clubpenalty` between the first and second line of a (partial) paragraph. It inserts `\widowpenalty` or `\displaywidowpenalty` between the last two lines of a paragraph or a partial paragraph, respectively [5, p. 104].

An author might change them to 10000, preventing a page break at these points, i.e., TEX never breaks a page leaving a club or widow line unless the user inserts their own penalty or a break after a display leaves only one line. But if a page doesn't have enough stretch- or shrinkability, TEX issues a warn-

doi.org/10.47397/tb/46-1/tb142wermuth-widow

ing message. Thus, the value 10000 might work with documents that contain a lot of vertical glue but not with, say, a novel, i.e., straight text.

A third problem. A hyphenated word at the end of a page is not only an aesthetic problem. A reader must remember the first part of the word longer than with a continuation of the word on the next line of the page. On a right-hand page it's not only a longer move for the reader's eyes, as the turning of the page stops any reading. Robert Bringhurst mentions in [1, §2.4.10] that style guides don't like the last word on a page to be hyphenated. But he accepts it as he finds the turning of a page less distracting than, for example, a photograph directly after a broken word. Hans Peter Willberg and Friedrich Forssman state in [9, p. 239] that hyphenated words at the end of a page, especially on right-hand pages, are always disliked. They recommend avoiding irritating hyphenations but tolerate those that can be easily completed.

TEX's penalty for this problem. TEX applies `\brokenpenalty` in a (partial) paragraph between pairs of lines if the pair's first line ends with an explicit or implicit hyphen or a dash.

Contents. Section 2 uses only penalties that TEX inserts into the processed text to report club and widow lines as well as hyphenated words (or dashes) at page breaks. The defined output routine needs no silent output paths. As this output routine is very close to `\plainoutput` we reuse it in the other sections.

Sections 3 and 4 implement output routines to automatically prevent or try to avoid widow lines. Section 3 presents a solution to prevent club and widow lines in one-sided output with single-column pages by changing the `\vsize` by ±1 line. Section 4 extends this approach to spreads and tries to avoid widow lines, hyphenated words at a page break, and club lines if possible. Both sections use silent output paths.

Section 5 collects macros to switch between the presented output routines. We also implement macros to support manual `\vsize` changes for single pages and ranges when using the output routine of section 2. The last section contains a summary.

Related work. Of course, other authors have written about the above problems too. As a simple solution for single pages one finds, for example, an imperfect code snippet, named a "skeleton", in *TEX by Topic* by Victor Eijkhout; see §28.4.4. Paul Isambert [4] uses the idea of the skeleton for pairs of single pages; the code assigns zero to `\brokenpenalty` and `\clubpenalty` to fix the skeleton's flaw. Frank Mittelbach implements for LATEX in [7] the reporting of problematic lines; the user must then apply manual fixes. Max Chernoff provides a lot of automation in [2] using the LuaTEX engine. And there are CTAN packages that address the problems; for example, see the package `lua-typo` by Daniel Flipo.

2 Reporting club and widow lines

TEX includes in total seven vertical-mode penalties that it might insert to make a page break in a (partial) paragraph less desirable.

1. `\interlinepenalty` (default: 0) is inserted between all pairs of lines in every (partial) paragraph, i.e., only displays are excluded;
2. `\brokenpenalty` (100) is added for every pair of lines whose first ends in a discretionary break;
3. `\clubpenalty` (150) is added for every first pair;
4. `\widowpenalty` (150) is added for the last pair in every paragraph;
5. `\displaywidowpenalty` (50) is added for the last pair in every partial paragraph;
6. `\predisplaypenalty` (10000) is inserted before a display;
7. `\postdisplaypenalty` (0) is inserted after a display.

With the exception of `\displaywidowpenalty` and `\widowpenalty`, which exclude each other, the penalties in numbers 1–5 might occur at the same place and then their values are added. The value of `\interlinepenalty` is always added to the other four, but as its default value is zero it is usually not seen in a trace. For example, the penalty between the lines of a two-line paragraph whose first line ends in a hyphen becomes $0+100+150+150=400$, i.e., the sum of `\interlinepenalty`, `\brokenpenalty`, `\clubpenalty`, `\widowpenalty`. Note: TEX only inserts a nonzero penalty value or a nonzero sum of penalty values [6, §890].

The last two penalties are always inserted [6, §§1203/1205/1206] and occur in isolation, that is, they are not combined with the others. This creates a problem with the default values. TEX might break a page after a display as `\postdisplaypenalty` is zero. But `\widowpenalty` is not added to this penalty even if there is only one line in the next paragraph. Thus we can't detect all widow lines for sure.

A penalty given by the user, for example, via `\vadjust{\penalty10}`, is completely independent of TEX's inserted penalties and appears in a trace as a second penalty. The smallest penalty is picked by TEX to penalize a page break if the break occurs at this place [5, ex. 14.10].

Reminder: as explained in section 1, a club line appears on the current page, a widow or display widow line on the next page.

The report. We keep three lists in token registers. They collect the pages for which we see a penalty at the page break that involves a penalty for a club or widow line or for a possibly hyphenated word, i.e., the penalties that we are interested in. We need some macros — names are prefixed with `PX` — that add the page number to the corresponding list.

```
%%% control sequences for club lines
\newtoks\PXclublines % stores their page numbers
\newif\ifPXfoundclubline % true: >= 1 entry
\def\PXpagehasclubline{\edef\next{%   no \global
  \ifPXfoundclubline \the\PXclublines,
   \the\pageno\else\the\pageno\fi}%
 \global\PXfoundclublinetrue
 \global\PXclublines=\expandafter{\next}}
%%% and for widow and displaywidow lines
\newtoks\PXwidowlines \newif\ifPXfoundwidowline
\def\PXpagehaswidowline#1{% #1: 0, 1, or 2
% 0: widow, 1: displaywidow, 2: postdisplay
 \edef\next{%               \global not required
  \ifPXfoundwidowline \the\PXwidowlines,
   % displaywidow and postdisplay in parentheses
   \ifnum#1=0 \the\pageno \else (\the\pageno)\fi
  \else\ifnum#1=0 \the\pageno\else(\the\pageno)%
  \fi\fi \ifnum#1=2 ?\fi}% '?' marks postdisplay
 \global\PXfoundwidowlinetrue
 \global\PXwidowlines=\expandafter{\next}}
%%% and for broken lines
\newtoks\PXbrokenlines\newif\ifPXfoundbrokenline
\def\PXpagehasbrokenline{\edef\next{%
  \ifPXfoundbrokenline \the\PXbrokenlines,
   \the\pageno\else\the\pageno\fi}%
 \global\PXfoundbrokenlinetrue
 \global\PXbrokenlines=\expandafter{\next}}
```

Display widow lines and widow lines are stored in one list. Parentheses around page numbers for display widow lines distinguish the cases. An additional question mark signals a possible widow line after a display.

At the end of the TEX run the lists must be output. We redefine `\bye`, protected for file reload, so that no other macro need be inserted by the user.

```
\newhelp\PXnoreload{The reload redeclares a few
 registers; already stored information is lost.}
\def\PXreport{% report problematic line breaks
 \ifnum\PXprint=1 \par   % exercise page builder
  \write16{Widow lines on the following pages:
   \ifPXfoundwidowline \the\PXwidowlines
   \else none found\fi.}
  \write16{Club lines on the following pages:
   \ifPXfoundclubline \the\PXclublines
   \else none found\fi.}
  \write16{Hyphen at page break on the following
   pages: \ifPXfoundbrokenline
    \the\PXbrokenlines\else none found\fi.}
 \fi}
\expandafter\ifx\csname PXprint\endcsname\relax
 \def\PXprint{1 }\expandafter\let\csname PXbye%
 \expandafter\endcsname\csname bye\endcsname
\else\errhelp=\PXnoreload \errmessage{Don't
 input this file more than once}\fi
\outer\def\bye{\PXreport\csname PXbye\endcsname}
```

The reporting can be suppressed if `\PXprint` gets, for example, the replacement text `0`. It is also used to prevent an infinite loop if the file with this code is loaded twice; see [5, p. 382f.]. But `\PXprint` must not be defined before this code's initial load.

Values for the penalties. To identify if a page break occurs, for example, between the first and second line of a paragraph, we must have unique values for all sums containing `\clubpenalty`. That is, all sums without `\clubpenalty` must differ from those involving `\clubpenalty`. As `\clubpenalty` equals `\widowpenalty` by default, this condition fails. We need to assign `\clubpenalty` or `\widowpenalty` a new unique value. I decided to give the value 151 to `\widowpenalty`, 51 to `\displaywidowpenalty`, and 101 to `\brokenpenalty`; `\postdisplaypenalty` is set to 2 so that we can detect it although we do not know how many text lines follow. The choice of the values for these penalties determines all the penalty sums that are of interest to us.

```
% if \interlinepenalty<>0 add its value to \defs
\def\PXpenB{101 }\brokenpenalty=101 % change: +1
\def\PXpenC{150 }\clubpenalty=150 %            0
\def\PXpenD{51 }\displaywidowpenalty=51 %     +1
\def\PXpenW{151 }\widowpenalty=151 %          +1
%%% combinations (sums) of two or more penalties
% ... with \widowpenalty
\def\PXpenBW{252 }
\def\PXpenCW{301 }
\def\PXpenBCW{402 }
% ... with \clubpenalty, no \widowpenalty
\def\PXpenCD{201 }
\def\PXpenBC{251 }
\def\PXpenBCD{302 }
% ... with \displaywidowpenalty, no \clubpenalty
\def\PXpenBD{152 }
% check pages for ``postdisplay widow line''
\postdisplaypenalty=2 %              change = +2
%%% IMPORTANT: all values must be unique
```

In the above set, all the values must not only be different, but also must differ from any other vertical penalty used in `plain`, for example, `\proclaim` uses 55 and `\endinsert` 100. (The macros that mark good break points have negative values, for example, `\smallbreak` uses −50.) Moreover, a vertical penalty entered by the user should not use any

of the above values. Thus, if unsure, use only penalties that end with the digits 3 to 9 as long as `\interlinepenalty` ends with 0.

One should set `\postdisplaypenalty` back to the value 0 inside the double dollar signs if you have checked that the text after the display needs more than one line. This establishes plain TEX's default value and the page is no longer reported.

The output routine. TEX assigns the penalty at a page break to `\outputpenalty`; it uses 10000 if it finds none [5, p. 125]. Five different flags, used only locally, are set if we detect that one of our penalties contributes to `\outputpenalty`. They are set in the twelve cases listed below if the penalty of the case matches the problem that a flag represents.

```
\newif\ifPXpagewithclub % true: page has a club
\newif\ifPXpagewithwidow      % or a widow or a
\newif\ifPXpagewithdisplaywidow % display widow
\newif\ifPXpagewithhyphen % or ends with hyphen
\newif\ifPXpagewithpostdisplaytext % check page
\def\PXoutput{% report problematic lines
 \ifnum\outputpenalty=\PXpenW
  \PXpagewithwidowtrue
 \else\ifnum\outputpenalty=\PXpenBW
  \PXpagewithhyphentrue
  \PXpagewithwidowtrue
 \else\ifnum\outputpenalty=\PXpenCW
  \PXpagewithclubtrue
  \PXpagewithwidowtrue
 \else\ifnum\outputpenalty=\PXpenBCW
  \PXpagewithhyphentrue
  \PXpagewithclubtrue
  \PXpagewithwidowtrue
 \else\ifnum\outputpenalty=\PXpenC
  \PXpagewithclubtrue
 \else\ifnum\outputpenalty=\PXpenBC
  \PXpagewithhyphentrue
  \PXpagewithclubtrue
 \else\ifnum\outputpenalty=\PXpenCD
  \PXpagewithclubtrue
  \PXpagewithdisplaywidowtrue
 \else\ifnum\outputpenalty=\PXpenBCD
  \PXpagewithhyphentrue
  \PXpagewithclubtrue
  \PXpagewithdisplaywidowtrue
 \else\ifnum\outputpenalty=\PXpenB
  \PXpagewithhyphentrue
 \else\ifnum\outputpenalty=\PXpenBD
  \PXpagewithhyphentrue
  \PXpagewithdisplaywidowtrue
 \else\ifnum\outputpenalty=\PXpenD
  \PXpagewithdisplaywidowtrue
 \else\ifnum\outputpenalty=\postdisplaypenalty
  \PXpagewithpostdisplaytexttrue
 \fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi
 \PXpreplain \plainoutput \PXpostplain
 \PXpostoutput}
\output={\PXoutput}\let\PXpostoutput=\relax
```

The macro `\PXpostoutput` allows us to switch the output routine after `\PXoutput` finished a page.

The page number is stored in one of the three lists if the corresponding flag is found true in the macros around `\plainoutput`. The page number of club lines and of hyphens must be stored before `\plainoutput` starts; the page number for (display) widow lines after its execution. (See [5, p. 255].)

```
\def\PXpreplain{% todos before \plainoutput
 \ifPXpagewithclub \PXpagehasclubline \fi
 \ifPXpagewithhyphen \PXpagehasbrokenline \fi}
\def\PXpostplain{% todos after \plainoutput
 \ifPXpagewithwidow \PXpagehaswidowline0\fi
 \ifPXpagewithdisplaywidow
  \PXpagehaswidowline1\fi
 \ifPXpagewithpostdisplaytext
  \PXpagehaswidowline2\fi}
```

3 Preventing club and widow lines

We now look at single pages and try to prevent club and widow lines automatically. Such automation should be used with caution; it is often not a good decision for an author to lose control over the page breaks. Moreover, the code here does not consider aesthetic criteria. For example, we might accept a widow line if it is completely filled, or keep a club line if the previous line is very short.

The reason to focus on club and widow lines is that we know how to fix them. A display widow line cannot be fixed by making the previous page a line longer as `\predisplaypenalty` is 10000. And a new club line might occur if the page length is decreased.

Theory. We have to deal with four situations depending on the penalty that we find at the page break. Either there is a club line on the current page and no widow line, or no club line but a widow line on the next page, or both a club line on the current and a widow line on the next page, or neither a club nor a widow line. I use the following symbols to describe the four situations: c w $\frac{\text{c}}{\text{w}}$ $\frac{\not\text{c}}{\text{w}}$.

We look at three consecutive lines as we want to change the `\vsize` by at most one line. With four situations this gives 64 three-line combinations. But of course, most of them are impossible. There are constraints that must be obeyed.

Let's look at one example in detail, the case $\frac{\not\text{c}}{\text{w}}$. It can be followed by any other symbol as we are either (a) between two paragraphs or (b) between line 2 and the penultimate line inside a paragraph. If the second line is c, i.e., we have case (a) and see now a new first line, then the third can be either before a penultimate line (case $\frac{\not\text{c}}{\text{w}}$) or before a last line

(case w). But if the second line is a widow, i.e., we started with case (b), the third must be $\not{\text{w}}$ again as we return to case (a). If the second line has type $\overset{\text{c}}{\text{w}}$, i.e., we start with (a) and the next paragraph has exactly two lines, then the third finishes the paragraph and, again, we see $\not{\text{w}}$. The last case for the second line is $\not{\text{w}}$. If we are in case (b) either the paragraph continues, so we get another $\not{\text{w}}$, or, in front of the penultimate line, we have as the third line w. Otherwise it's case (a) and a one-line paragraph follows (again $\not{\text{w}}$) or a new paragraph starts, i.e., cases c or $\overset{\text{c}}{\text{w}}$.

- $\not{\text{w}}$ It can be followed by c, w, $\overset{\text{c}}{\text{w}}$, or $\not{\text{w}}$. $(\not{\text{w}}, \text{w})$ and $(\not{\text{w}}, \overset{\text{c}}{\text{w}})$ must be followed by $\not{\text{w}}$, $(\not{\text{w}}, \text{c})$ by w or $\not{\text{w}}$, and $(\not{\text{w}}, \not{\text{w}})$ allows any symbol.
- c It must be followed by either $\not{\text{w}}$ or w. Then $(\text{c}, \not{\text{w}})$ must be followed by w or $\not{\text{w}}$ and (c, w) by $\not{\text{w}}$.
- w It must be followed by $\not{\text{w}}$ and this must be followed by c, $\overset{\text{c}}{\text{w}}$, or $\not{\text{w}}$.
- $\overset{\text{c}}{\text{w}}$ It must be followed by $\not{\text{w}}$ and then by c, $\overset{\text{c}}{\text{w}}$, or $\not{\text{w}}$.

These rules leave us with just seventeen cases. The next table shows them together with the change of the `\vsize` in the first column.

−1 :	$\not{\text{w}}$	$\not{\text{w}}$	$\not{\text{w}}$	$\not{\text{w}}$	$\not{\text{w}}$	$\not{\text{w}}$	$\not{\text{w}}$	$\not{\text{w}}$	c	c	c	w	w	w	$\overset{\text{c}}{\text{w}}$	$\overset{\text{c}}{\text{w}}$	$\overset{\text{c}}{\text{w}}$
0 :	$\not{\text{w}}$	$\not{\text{w}}$	$\not{\text{w}}$	$\not{\text{w}}$	c	c	w	$\overset{\text{c}}{\text{w}}$	$\not{\text{w}}$	$\not{\text{w}}$	w	$\not{\text{w}}$	$\not{\text{w}}$	$\not{\text{w}}$	$\not{\text{w}}$	$\not{\text{w}}$	$\not{\text{w}}$
+1 :	$\not{\text{w}}$	c	w	$\overset{\text{c}}{\text{w}}$	$\not{\text{w}}$	w	$\not{\text{w}}$	$\not{\text{w}}$	$\not{\text{w}}$	w	$\not{\text{w}}$	$\not{\text{w}}$	c	$\overset{\text{c}}{\text{w}}$	$\not{\text{w}}$	c	$\overset{\text{c}}{\text{w}}$

This table proves that the reduction by one line gives in all but one case a result without club or widow lines if the page with an unchanged `\vsize` ends in line 0 with a club line or generates a widow line. The exception is the pair (c, w). Then the addition of a line is the solution. Simpler is to add a line if we find a widow line and to remove one if it's a club line but not a club/widow line. I implement the simple case.

Non-straight texts might create problems, for example, if a line is added that refers to a footnote. In essence we then add two lines and the skip in front of the footnote text. This material does not fit on the page and TEX warns about a bad vbox.

The code to keep `\topmark` valid needs to save and restore the `\botmark` of the previously shipped out page. We keep the names assigned in [8].

```
\holdinginserts=1 % change its default value
\newbox\PrevPageBotMark % box to store \botmark
\def\SaveCurrentBotMark{% fill \PrevPageBotMark
 \global\setbox\PrevPageBotMark % must be global
  =\vbox{Restore\mark{\botmark}\vfil}}
\SaveCurrentBotMark % initialize the box
\def\RestoreBotMark{-"C0DA }% unique < -10000
```

The setting of `\holdinginserts` must be changed as the output routine is called several times. It must be zero only during the activation when the output routine ships out a page.

The code for the problem. We reuse the assignments and the definitions for the different penalties that contain `\widowpenalty` or `\clubpenalty` as a summand. Our solution needs only a few new macros: Two to change the `\vsize` and one to reset it to its former value. Such a change also moves the footline up or down. But we want to keep it always at the same position. Thus, we change the macro `\makefootline` of `plain.tex` and replace the fixed value of 24 pt by the macro `\footlinedist` with the value 30 pt, i.e., we change plain TEX's default.

```
\def\footlinedist{30pt }%plain TeX's value: 24pt
\def\makefootline{\baselineskip=\footlinedist
 \lineskiplimit=0pt\line{\the\footline}}
\begingroup \dimen255=\footlinedist
 \gdef\PXnormaldist{\gdef\footlinedist{30 pt}}
 \advance\dimen255 by +1\baselineskip
 \xdef\PXplusdist{\noexpand\gdef\noexpand
  \footlinedist{\the\dimen255 }}
 \advance\dimen255 by -2\baselineskip
 \xdef\PXminusdist{\noexpand\gdef\noexpand
  \footlinedist{\the\dimen255 }}\endgroup
\def\PXincVsize{% increase \vsize by one line
 \global\advance\vsize by +1\baselineskip
 \PXminusdist\wlog{SINGLE: \vsize + 1 line}}
\def\PXdecVsize{% decrease \vsize by one line
 \global\advance\vsize by -1\baselineskip
 \PXplusdist\wlog{SINGLE: \vsize - 1 line}}
\def\PXresetVsize{% reset \vsize to former value
 \ifdim\vsize=\PXsaveVsize \else \PXnormaldist
  \global\vsize=\PXsaveVsize \fi}
```

Of course, we must know `\vsize`'s former value and therefore we store it in another macro after it is set.

```
\def\PXstorevsize{\edef\PXsaveVsize{\the\vsize}}
\PXstorevsize % call this if \vsize is changed
```

A user must call `\PXstorevsize` if the `\vsize` is manually changed in the document. Otherwise the output routine returns to the previously saved value.

The output routine. The final step is to code `\output`. Here we do not add a new penalty after `\unvbox255` as TEX's inserted `\penalty10000` ([5, p. 125]) does no harm: We do not want a page break after a club line or in front of a widow line. (Moreover, no one wants repeated calls to `\PXincVsize` and `\PXdecVsize`.) In the case of a non-straight text it prevents an unwanted page break at the risk of getting a warning for an under- or overfull vbox.

```
\newif\ifPXshowoutputroutine % true: show method
\def\PXlog#1{% #1: string; flag-protected \wlog
 \ifPXshowoutputroutine\wlog{#1}\fi}
\def\PXsinglepage{% prevent club and widow lines
 \ifnum\outputpenalty=\RestoreBotMark
  \setbox0=\vbox{\unvbox255 }%   ignore contents
 \else\ifnum\outputpenalty=\PXpenW
  \PXincVsize \unvbox255 %    keep \penalty10000
```

```
 \else\ifnum\outputpenalty=\PXpenBW
  \PXincVsize \unvbox255
 \else\ifnum\outputpenalty=\PXpenCW
  \PXincVsize \unvbox255
 \else\ifnum\outputpenalty=\PXpenBCW
  \PXincVsize \unvbox255
 \else\ifnum\outputpenalty=\PXpenC
  \PXdecVsize \unvbox255
 \else\ifnum\outputpenalty=\PXpenCD
  \PXdecVsize \unvbox255
 \else\ifnum\outputpenalty=\PXpenBC
  \PXdecVsize \unvbox255
 \else\ifnum\outputpenalty=\PXpenBCD
  \PXdecVsize \unvbox255
 \else\ifnum\holdinginserts=1 % switch its value
  \global\holdinginserts=0
  \unvbox\PrevPageBotMark  % restore \botmark to
  \penalty\RestoreBotMark % get a valid \topmark
  \unvbox255 %          next, reprocess material
  \ifnum\outputpenalty<10000 % including penalty
   \penalty\outputpenalty \fi% keep \TeX's 10000
 \else \SaveCurrentBotMark   % for next \topmark
  \PXlog{OUTPUT: single }\PXoutput \PXresetVsize
  \global\holdinginserts=1 \PXpostsingle
 \fi\fi\fi\fi\fi\fi\fi\fi\fi\fi}
\output={\PXsinglepage}
```

As mentioned earlier, instead of `\plainoutput` the output routine of section 2, `\PXoutput`, is used. After it finishes its work we must reset `\vsize` if it was changed. And here is the place to change the output routine if the user so desires. As `\PXsinglepage` needs several activations to do its work, it is dangerous to switch the output routine between these activations (see also section 5).

In this section we stay with the above output routine, `\PXsinglepage`, and thus we set:

```
\let\PXpostsingle=\relax
```

4 Working with spreads

In this section we apply the method of the previous section to spreads. This implies that we need more than one output routine to solve the problem. This should be no surprise as we want to analyze spreads but we still want to ship out single pages.

We concentrate on spreads for straight texts and try to avoid widow lines. As stated in section 1 this is not always possible. We keep things simpler if we don't immediately consider club lines; as mentioned above, typographers accept them. And as another, secondary, criterion we try to avoid hyphenated words at the break of the spread.

When we work with spreads in the described way, several complications occur. For example, a single `\eject` in vertical mode throws out the current page, but we certainly do not want to throw out the current spread. But when the user accepts different heights for the two pages of a spread, they can be shipped out independently, for example, with the output routine of section 3. This problem also occurs at the end of the document as the `\supereject` has the same effect as an `\eject`. Another obvious complication: The first page is not a part of a spread; it should be output as a single page. As stated above, an output routine similar to the one of the previous section is required here too.

Theory. For the left page of a spread the theory of the single page stays valid. But the right page is different. When `\vsize` is changed by one line it changes the shown contents by two lines. One line less for `\vsize` means that one line from the left page is moved to the right and therefore the right page loses not only its previous last line but also its previous penultimate line.

Therefore we take the table shown in section 3, remove its middle line, and are left with twelve cases for one change in the height of the spread. The second change, again by two lines, increases this to 37 cases. I keep the labels for the `\vsize` change per page and use the new symbol '$*$' if all four symbols can occur at a certain place to keep the number of entries in the table smaller.

$$
\begin{array}{rl}
-1: & \frac{\not c}{w}\ \frac{\not c}{w}\ \frac{\not c}{w}\ \frac{\not c}{w}\ \frac{\not c}{w}\ \frac{\not c}{w}\ \frac{\not c}{w}\ \frac{\not c}{w}\ \frac{\not c}{w}\ \frac{\not c}{w}\ \ c\ \ c\ \ c\ \ c\ \ w\ \ w\ \ w\ \ w\ \ w\ \ w\ \frac{c}{w}\ \frac{c}{w}\ \frac{c}{w}\ \frac{c}{w}\ \frac{c}{w}\ \frac{c}{w} \\
0: & \frac{\not c}{w}\ \ c\ \ c\ \ w\ \ w\ \ w\ \frac{c}{w}\ \frac{c}{w}\ \frac{c}{w}\ \frac{\not c}{w}\ \ w\ \ w\ \ w\ \ w\ \frac{\not c}{w}\ \ c\ \ c\ \frac{c}{w}\ \frac{c}{w}\ \frac{c}{w}\ \frac{\not c}{w}\ \ c\ \ c\ \frac{c}{w}\ \frac{c}{w}\ \frac{c}{w} \\
+1: & *\ \frac{\not c}{w}\ w\ \frac{\not c}{w}\ c\ \frac{c}{w}\ \frac{\not c}{w}\ c\ \frac{c}{w}\ *\ \frac{\not c}{w}\ c\ \frac{c}{w}\ *\ \frac{\not c}{w}\ w\ \frac{\not c}{w}\ c\ \frac{c}{w}\ *\ \frac{\not c}{w}\ w\ \frac{\not c}{w}\ c\ \frac{c}{w}
\end{array}
$$

When we look at the last column of the above table, we see that in a sequence of several two-line paragraphs a change of `\vsize` cannot avoid the occurrence of a widow line. Another case is $(w, \frac{c}{w}, \frac{c}{w})$.

Reuse. In this section we reuse the two macros for a silent output path from section 3: the macros `\SaveCurrentBotMark` and `\RestoreBotMark` as we apply this method here too. As mentioned earlier we reuse `\PXoutput`, the output routine of section 2. And we also reuse the output routine of section 3, for example, to output the first page which does not belong to a spread. But `\PXpostsingle` needs to be changed because we must switch to another output routine. Later we also call the output routine of section 3 for two pages if they do not need to be balanced. Then we also have to return to the main output routine for spreads.

```
\newif\ifPXprinttwo %true:do \PXsinglepage twice
\def\PXuseSpread{% initialize: work with spreads
 \ifPXprinttwo %print with \PXsinglepage 2 pages
  \global\PXprinttwofalse    % first of two done
 \else \PXactivateSpread % printing done; switch
 \fi}
\let\PXpostsingle=\PXuseSpread
```

How to find the best page breaks? I decided to use a point system and choose the \vsize that minimizes the points for the current spread. A widow line gets 16 points, a hyphenated last word 4 points, and a club line 1 point. For the right-hand page the values are doubled. A total of 1 point, i.e., only a club line left, is ignored for the normal \vsize to reduce changes of \vsize.

The points for several analyzed page heights are collected in three count registers. The macro \PXpts identifies which register has to be changed. A flag signals which page break, left or right, is checked.

```
\newcount\PXppts % sum of points for +1
\newcount\PXzpts % sum of points for  0
\newcount\PXmpts % sum of points for -1
\newif\ifPXrightPage % true: check spread break
\def\PXactivateSpread{% switch output routine
 % initialize 3 registers, the flag, and \PXpts
 % that stores the 1st letter of a register name
 \gdef\PXpts{p}\global\PXppts=0 \global\PXzpts=0
 \global\PXmpts=0 \global\PXrightPagetrue
 \PXmaxVsize % start with 2\vsize+2\baselineskip
 \global\output={\PXspread}}
```

Let's name the current \vsize v and the current \baselineskip b. First, we do three checks for the right-hand page with $2v + 2b$, $2v$, and $2v - 2b$. Then we do three checks with $v + b$, v, and $v - b$. Thus, all possible page breaks are looked at and we sum the points for related breaks.

Here is the code to add points to the registers and to evaluate the smallest value among them. We use \PXpts to keep track of the current \vsize status and apply its replacement text to select the corresponding register. The evaluation macro shows that for equal values the preference is to stay with v and otherwise to decrease it by one line.

```
\def\PXbrkPts #1.{% #1: points; add to register
 \expandafter\global\expandafter\advance
  \csname PX\PXpts pts\endcsname by #1
 \ifPXrightPage % double them for the right page
  \expandafter\global\expandafter\advance
   \csname PX\PXpts pts\endcsname by #1 \fi}
\def\PXlogPts{\PXlog{Points: \the\PXmpts,
 \the\PXzpts, \the\PXppts}}
\def\PXevalPtsSetVsize{\PXresetVsize
 % \PXlogPts % to see the points, uncomment line
 \ifnum\PXzpts>1 % ignore club line on left page
  \ifnum\PXzpts>\PXppts \ifnum\PXmpts>\PXppts
     \global\advance\vsize by 1\baselineskip
     \PXminusdist\wlog{SPREAD: \vsize + 1 line}%
   \else\global\advance\vsize by -1\baselineskip
    \PXplusdist\wlog{SPREAD: \vsize - 1 line}\fi
  \else\ifnum\PXzpts>\PXmpts
   \global\advance\vsize by -1\baselineskip
   \PXplusdist\wlog{SPREAD: \vsize - 1 line}%
 \fi\fi\fi}
```

Next comes the macro that sets the start value of \vsize for each of the two test sequences with three checks. The method to reduce this value for each check is coded in a second macro.

```
\def\PXmaxVsize{%use max \vsize for current page
 \PXresetVsize %start with saved value of \vsize
 \global\advance\vsize by +1\baselineskip
 \ifPXrightPage   % double the (\vsize + 1 line)
  \global\advance\vsize by \vsize
  % second \topskip becomes a \baselineskip
  \global\advance\vsize by -1\topskip
  \global\advance\vsize by +1\baselineskip \fi}
\def\PXdecVsizeChgReg{% change \vsize & register
 \if\PXpts m\else   % change \vsize if > minimum
  \global\advance\vsize by -1\baselineskip
  \ifPXrightPage    % change value for two lines
   \global\advance\vsize by -1\baselineskip
 \fi\fi  % indicate current register with \PXpts
 \xdef\PXpts{\if\PXpts pz%   cycle chars: p -> z
  \else\if\PXpts zm\else p\fi\fi}}%  z -> m -> p
```

Output routines for spreads. In the above code the macro \PXactivateSpread was called that prepares for the switch to the spread output routine, \PXspread. This output routine never outputs anything. It just checks the six different page breaks mentioned above and collects their points.

```
\def\PXspread{% main \output for spreads
 \ifnum\outputpenalty     =\PXpenW  \PXbrkPts16.
 \else\ifnum\outputpenalty=\PXpenBW \PXbrkPts20.
 \else\ifnum\outputpenalty=\PXpenCW \PXbrkPts17.
 \else\ifnum\outputpenalty=\PXpenBCW\PXbrkPts21.
 \else\ifnum\outputpenalty=\PXpenB  \PXbrkPts 4.
 \else\ifnum\outputpenalty=\PXpenBC \PXbrkPts 5.
 \else\ifnum\outputpenalty=\PXpenBD \PXbrkPts 4.
 \else\ifnum\outputpenalty=\PXpenBCD\PXbrkPts 5.
 \else\ifnum\outputpenalty=\PXpenC  \PXbrkPts 1.
 \else\ifnum\outputpenalty=\PXpenCD \PXbrkPts 1.
 \else\ifnum\outputpenalty<-9999 %  forced break
   % switch \output except for 2\vsize + 2 lines
   \if\PXpts p\PXbrkPts32. % wait, assign 64 pts
   \else \let\PXnextStep=\PXprepOutputOfPage \fi
 \fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi
 \if\PXpts m\ifPXrightPage     % go to left page
   \global\PXrightPagefalse \PXmaxVsize
  \else \let\PXnextStep=\PXprepOutputOfPage \fi
 \fi \PXnextStep}
```

Penalties that receive 0 points, like \PXpenD, aren't listed in this output routine.

As mentioned above the output routine of section 3 becomes active if there is a forced break. But the code makes sure that the forced break does not occur *after* two pages of the given \vsize. That is, when we see a forced page break for the spread with size $2v + 2b$ the break gets many points so that it is never chosen by the evaluation. Of course, the following page might have only one or two lines but

that can also happen with \PXsinglepage. It also keeps a balanced spread at the end of the document in certain situations.

What is \PXnextStep? The standard behavior is to reduce the current \vsize inside a block and to put the material of \box255 back for reprocessing. When we want to call another output routine for printing we locally assign \PXprepOutputOfPage to \PXnextStep. This macro makes sure that the input needed to get a correct \topmark is placed in front of the returned material. Next, it switches to one of the output routines that are able to ship out pages. As explained above, this is either the one of section 3 or a new one for two pages with the same \vsize.

```
\def\PXnextStep{% default action without switch
 \PXdecVsizeChgReg %prepare next \vsize & \PXpts
 \unvbox255 \ifnum\outputpenalty<10000
  \penalty\outputpenalty \else \penalty0 \fi}
\def\PXprepOutputOfPage{\PXevalPtsSetVsize
 \ifnum\outputpenalty<-9999 %     a forced break
  \unvbox255 \penalty\outputpenalty
  \global\PXprinttwotrue  % print 2 single pages
  \global\output={\PXsinglepage}%
 \else \global\holdinginserts=0
  \unvbox\PrevPageBotMark % restore old \botmark
  \penalty\RestoreBotMark \unvbox255
  \ifnum\outputpenalty<10000
   \penalty\outputpenalty \else \penalty0 \fi
  \global\output={\PXsinglespreadpages}\fi}
```

We use \penalty0 when we return the processed material and we see an \outputpenalty of 10000. This annihilates TEX's inserted penalty.

The new output routine is rather simple. Besides the code that we need to keep \topmark valid it contains two calls to section 2's \PXoutput for the two pages of the spread. And we identify the output routine if a user sets \PXshowoutputroutinetrue.

```
\def\PXsinglespreadpages{% print pages of spread
 \ifnum\outputpenalty=\RestoreBotMark
  \setbox0=\vbox{\unvbox255 }%   ignore contents
 \else\ifPXrightPage \SaveCurrentBotMark
  \PXlog{(right) spread }\PXoutput \PXresetVsize
  \global\holdinginserts=1 \PXpostspread
 \else \PXlog{OUTPUT: (left) }\PXoutput
  \global\PXrightPagetrue
 \fi\fi}
```

As with the output routine of section 3 a switch to another output routine should not be made while \PXspread has not finished its work and the pages were not yet shipped out with the calls of \PXoutput in \PXsinglespreadpages. Therefore, we define the macro \PXpostspread that can be used to activate another output routine. The default is to keep working with spreads.

```
\let\PXpostspread=\PXactivateSpread
```

5 Manual interventions

After loading the macros of section 2, the output routine \PXoutput is active. Loading the macros of sections 2 and 3 leads to \PXsinglepage, and loading all macros from sections 2, 3, and 4 initializes the output routines needed for spreads. But how do we get then, say, \PXoutput, when we don't want to work with spreads? We can easily switch to other output routines if nothing has been processed yet.

```
\def\startwithReport{\holdinginserts=0
 \output={\PXoutput}}
\def\startwithSingle{\let\PXpostsingle=\relax
 \output={\PXsinglepage}}
\def\startwithPlain{\holdinginserts=0
 \displaywidowpenalty=50 \widowpenalty=150
 \brokenpenalty=100 \postdisplaypenalty=0
 \def\footlinedist{24pt }\def\PXprint{0 }%
 \output={\plainoutput}\wlog{use plainoutput}}
```

The following code allows switching between the output routines if a user wants to do that for any reason. When one wants to stop the automation, i.e., the output routines of sections 3 or 4, after the processing has started, the output routine of section 2 and not \plainoutput should be called.

```
\def\fromSingletoReport{%
 \def\PXpostsingle{\global\holdinginserts=0
  \global\output={\PXoutput}}}
\def\fromSpreadtoReport{%
 \def\PXpostspread{\global\holdinginserts=0
  \global\output={\PXoutput}}}
```

The activation of the automation from the output routine of section 2 needs more code.

```
\def\fromReporttoSingle{%
 \SaveCurrentBotMark \global\holdinginserts=1
 \global\let\PXpostoutput=\relax
 \global\let\PXpostsingle=\relax
 \global\output={\PXsinglepage}}
\def\PXswitchtoSpread{%
 \global\let\PXpostoutput=\relax
 \global\let\PXpostsingle=\PXuseSpread
 \global\let\PXpostspread=\PXactivateSpread
 \PXactivateSpread}
\def\fromReporttoSpread{% sometimes delayed
 \ifodd\pageno \gdef\PXpostoutput{%
   \SaveCurrentBotMark \global\holdinginserts=1
   \PXswitchtoSpread}%
 \else \SaveCurrentBotMark
  \global\holdinginserts=1 \PXswitchtoSpread
 \fi}
```

Of course, one can also switch between the output routines from sections 3 and 4.

```
\def\fromSingletoSpread{% sometimes delayed
 \def\PXpostsingle{\ifodd\pageno
   \global\let\PXpostsingle=\PXswitchtoSpread
  \else \PXswitchtoSpread \fi}}
```

```
\def\fromSpreadtoSingle{\def\PXpostspread{%
  \global\let\PXpostsingle=\relax
  \global\output{\PXsinglepage}}}
```

To keep the overview about the output routine used, set the flag `\ifPXshowoutputroutine` to true.

Change `\vsize` with `\PXoutput`. To change the page height by hand use the following macros inside `\PXpostoutput`. For example,

```
\def\PXpostoutput{\PXminus(14) \PXstop(16)
 \PXplus(18) \PXminus(20) \PXstop(24)}
```

reduces the `\vsize` for pages 14 and 15, increases it for pages 18 and 19, and reduces it again for pages 20, 21, 22, and 23. A user can define `\PXpostoutput` several times if the page ranges don't overlap. To clear the hook, use `\let\PXpostoutput=\relax`.

The code uses `\PXresetVsize`, `\PXincVsize`, and `\PXdecVsize` and their dependencies from section 3. Also, `\PXsaveVsize` must be defined. That is, the user should execute `\PXstorevsize` every time the `\vsize` is manually changed.

```
\def\PXplus(#1){% #1: page number; now +1 line
 \ifnum#1=\pageno \PXresetVsize \PXincVsize
 \fi \ignorespaces}
\def\PXminus(#1){% #1: page number; now -1 line
 \ifnum#1=\pageno \PXresetVsize \PXdecVsize
 \fi \ignorespaces}
\def\PXstop(#1){% #1: page number; now no change
 \ifnum#1=\pageno \PXresetVsize
  \wlog{SINGLE: \vsize}\fi \ignorespaces}
```

6 Summary

Section 2 uses more or less `\plainoutput` with additional reporting. Compared to plain TEX's output routine only small changes to four internal parameters are made (plus `\footlinedist`; reset it to 24 pt if you use only this output routine). It is expected that the new output routine finds the same page breaks as before. Thus it is a perfect replacement for `\plainoutput`. All problematic lines discussed in section 1 are reported. They must be fixed manually by the author who stays in complete control.

Section 3 presents an automatic solution to prevent club and widow lines, but not display widow lines. It might change the page breaks from those of plain TEX's output routine. As the specified `\vsize` is sometimes increased and sometimes decreased, always by one line, facing pages might have different page heights. This output routine is ideal for single-sided documents, letters, memos, notifications, etc.

Section 4 shows another automatic solution but this time mainly for widow lines. As a secondary criterion the output routine tries to avoid a hyphenated word at a page break, and as a tertiary criterion, club lines are considered. The main difference from the output routine of section 3 is that it tries to keep equal heights for facing pages. It works best with straight text; otherwise, as with section 3's output routine, under- and overfull pages might sometimes occur. Nevertheless, under certain circumstances widow lines on a spread with equal page heights cannot be prevented. But as section 2's output routine is used to ship out pages, a user gets the page numbers of all pages with widow lines, club lines, and hyphenated last words.

Instead of spreads two-column pages can be created with `\PXspread` (plus new initialization) and a replacement for `\plainoutput` [5, p. 257]. Spreads of two-column pages need more output routines.

Section 5 demonstrates how a user can switch between the output routines if a document requires different methods to achieve the best result. It also contains code to change `\vsize` manually while using the output routine of section 2.

References

[1] Robert Bringhurst, *The Elements of Typographic Style*, 4th edition, version 4.2, Seattle, Washington: Hartley & Marks, 2016.

[2] Max Chernoff, "Automatically removing widows and orphans with lua-widow-control", *TUGboat* **43**:1, 2022, 28–39. (See also **43**:3, 2022, 340–342.) tug.org/TUGboat/tb43-1/tb133chernoff-widows.pdf

[3] *The Chicago Manual of Style*, 15th edition, Chicago, Illinois: University of Chicago Press, 2003.

[4] Paul Isambert, "Strategies against widows", *TUGboat* **31**:1 (2010), 12–17. tug.org/TUGboat/tb31-1/tb97isambert.pdf

[5] Donald E. Knuth, *The TEXbook*, Volume A of *Computers & Typesetting*, Boston, Massachusetts: Addison-Wesley, 1984.

[6] Donald E. Knuth, *TEX: The Program*, Volume B of *Computers & Typesetting*, Boston, Massachusetts: Addison-Wesley, 1986.

[7] Frank Mittelbach, "The widows-and-orphans package", *TUGboat* **39**:3 (2018), 252–262. tug.org/TUGboat/tb39-3/tb123mitt-widows-code.pdf

[8] Udo Wermuth, "`\topmark` in output routines without `\shipout`", *TUGboat* **45**:3 (2024), 382–387. tug.org/TUGboat/tb45-3/tb141wermuth-silent.pdf

[9] Hans Peter Willberg, Friedrich Forssman, *Lesetypographie*, 2nd printing, Mainz, Germany: Verlag Hermann Schmidt, 2005.

◇ Udo Wermuth
Dietzenbach, Germany
u dot wermuth (at) icloud dot com

Is a given input a valid TEX ⟨*glue*⟩/⟨*muglue*⟩?

Udo Wermuth

Abstract

This article discusses the question of how one can determine if a given string of characters represents a valid glue or muglue value for TEX. A macro that looks and behaves like a Boolean conditional is implemented to answer the question.

1 Introduction

This text is a follow-on article to [3] and [4], which explain how to decide if a given string of characters (without control words and TEX's special characters) is either a valid number or a valid dimension for TEX. Two macros are implemented with the respective names `\ifisint` and `\ifisdim`. In the current article we look at the problem to decide if a given input is a valid glue specification for TEX.

The phrase "glue specification" represents two syntactic quantities of TEX: ⟨*glue*⟩ and ⟨*muglue*⟩. The first uses the quantities ⟨*dimen*⟩ and ⟨*fil dimen*⟩; the second ⟨*mudimen*⟩ and ⟨*fil dimen*⟩ [1, p. 270f.]. The article [4] only discusses the quantity ⟨*dimen*⟩ and does not handle ⟨*mudimen*⟩ or ⟨*fil dimen*⟩ as they cannot be assigned to `\dimen` registers.

This article explains the implementation of a macro named `\ifisglue` with the structure known from `\ifisint` and `\ifisdim`; see [3, section 2]. For example, we let `\Boolend` equal `\iffalse` and use it as delimiter for `\ifisglue`'s argument.

Contents. Section 2 lists facts about TEX's ⟨*glue*⟩ and ⟨*muglue*⟩ that are important to understand the macro `\ifisglue`. Section 3 contains the code for this macro.

2 About glue and muglue

As section 1 points out, TEX knows two kinds of glue, ⟨*glue*⟩ and ⟨*muglue*⟩. The first is used in horizontal and vertical mode and the second is applied in the math modes. The first type must be assigned to `\skip` registers, the second to `\muskip` registers.

Both types can have up to three components. First comes the required *natural width*. The other two components are optional. The second component is the *stretchability*, and the third the *shrinkability*, if all three occur in the glue item. The natural width is either a ⟨*dimen*⟩ or a ⟨*mudimen*⟩. With a ⟨*dimen*⟩ the second and third components must be either a ⟨*dimen*⟩ or a ⟨*fil dimen*⟩. If the natural width is a ⟨*mudimen*⟩ then the other components must be either a ⟨*mudimen*⟩ or a ⟨*fil dimen*⟩.

The keyword `plus` separates the natural width and the stretchability. The separator in front of the shrinkability is the keyword `minus`. TEX allows keywords to be written with either lower- or uppercase characters of either category 11 or 12 [1, p. 61].

A ⟨*mudimen*⟩ shares the same numerical value range as a dimension with the unit `pt` (see [4]) but, of course, it uses a different unit: `mu`. The numerical range of a ⟨*fil dimen*⟩ also equals that of a dimension with the unit `pt`; the unit must be one of `fil`, `fill`, or `filll`. Note that `fil` and `l` are keywords for TEX and thus, as TEX ignores spaces in front of keywords [1, p. 268], `fil l` is the same as `fill`.

3 The code for `\ifisglue`

The program TEX implements the scanning of glue and muglue values in one procedure: *scan_glue* [2, §§461–462]. It calls *scan_dimen* (see [4]).

An error occurs if one of the components contains an invalid dimension. The possible errors in connection with ⟨*dimen*⟩ are analyzed in [4]; these familiar errors also occur with ⟨*mudimen*⟩ and ⟨*fil dimen*⟩. The keywords between the components do not throw errors. TEX accepts, for example, typos in `plus` or `minus` without raising an error [2, §407]. It does not recognize such words as keywords and ends the glue specification earlier than intended, leaving the badly spelled word in the input. But this is not a problem as we know how to handle a specification that contains additional data (see [3]). There is only one new error message in *scan_glue*: "Incompatible glue units" [2, §408]. It is given if TEX finds a mix of ⟨*mudimen*⟩ and ⟨*dimen*⟩ in one glue specification.

Analysis. We proceed as in the previous articles and list all possible scenarios that appear when TEX reads a glue specification.

1. TEX reads a valid glue.
2. TEX reads a valid muglue.
3. TEX reads a valid glue; more input is available.
4. TEX reads a valid muglue; more input is available.
5. The natural width is neither a ⟨*dimen*⟩ nor a ⟨*mudimen*⟩.
6. The stretchability is neither a ⟨*dimen*⟩ nor a ⟨*mudimen*⟩ nor a ⟨*fil dimen*⟩.
7. The shrinkability is neither a ⟨*dimen*⟩ nor a ⟨*mudimen*⟩ nor a ⟨*fil dimen*⟩.
8. TEX finds a ⟨*dimen*⟩ and a ⟨*mudimen*⟩ in the specification.

Only cases 1 and 2 are valid glue specifications for TEX.

Cases 3 and 4 include the occurrence of invalid keywords like `plvs`, `pl us`, `p{l}us`, etc. To detect

doi.org/10.47397/tb/46-1/tb142wermuth-isglue

the additional data, a final test applies the technique that we used in [3] and [4]: a box-width test. An assignment of the input to `\skip` or `\muskip` is made inside an hbox. The width of this box is 0 pt if and only if the input is a valid glue specification or the erroneous input is fixed by TEX to become a valid glue specification. Our task is to detect the second case before we do the box-width test.

For cases 5–7 we can reuse `\ifisdim` if we replace the units `mu`, `fil`, `fill`, and `filll` by `pt`. As the numerical ranges agree the change only makes a difference for the validity of the component if the keyword `true` was used. With the unit `pt` TEX sees a valid dimension, but with `mu`, `fil`, `fill`, and `filll` TEX reports an invalid construction. But that is not a problem as TEX fixes this error situation in a way that data remains in the input and the final validity check, i.e., the box-width test, fails.

We need to split the input to do individual tests for the components. And that is possible only if we can identify the keywords `plus` and `minus`. In `\ifisdim` the keywords were changed to lowercase letters to eliminate uppercase or category 12 characters. Here we do that for the complete specification, changing alphabetic constants, except the hexadecimal digit 'F' must be kept. The box-width test detects the errors that occur through any side-effects.

Case 8 remains. TEX's fix for this error leaves data in the input and the final test fails if we can identify muglue. Thus, we count how often `mu` and a ⟨*fil dimen*⟩ are replaced by `pt` and if stretchability or shrinkability is missing. If we count to three, the input tries to represent muglue.

Thus we need to realize the following procedure.

Step 1: Convert the characters of the keywords to lowercase letters; use uppercase for 'f'.

Step 2: Split at keywords `minus` and `plus` and store the components in `\II@nw`, `\II@st`, `\II@sh`.

Step 3: 1) Replace in `\II@nw` `mu` by `pt`. 2) Replace in `\II@st` `mu`, `Fil`, `Fill`, and `Filll` by `pt`. 3) Do 2) for `\II@sh`. 4) Count replacements and absences; if = 3 set `\ifIImugluetrue`.

Step 4: 1) Check `\II@nw`. 2) If okay check `\II@st`. 3) If okay check `\II@sh`. 4) If okay perform the box-width test using `\ifIImuglue`. 5) If okay return true; otherwise return false.

This procedure works with intentional errors that TEX reports while `\batchmode` is active. As TEX limits the number of errors in a single paragraph to 100 [2, §76] the macro shouldn't be applied, for example, in a loop.

My implementation. The code reuses `\Boolend`, `\II@font`, `\IIcurrentmode`, `\II@cnt`, and the macro `\ifisdim` of [4]; see the file `ifisdimension.tex` for their declarations or definitions. The delimiter `\Boolend` is required to make `\ifisglue` skippable.

The code takes care that the content of the macros `\II@nw`, `\II@st`, `\II@sh`, and `\II@skip` always end in at most one space. The macros `\II@sto` and `\II@sho` indicate the order of the ⟨*fil dimen*⟩ for stretchability or shrinkability, respectively.

```
\input ifisdimension.tex % reuse \ifisdim
\catcode`\@=11
\newif\ifIImuglue % true: input might be muglue
\newif\ifII@sofar % true: glue/muglue OK so far
%% helper macros
\def\II@kwlc #1{% 'plus/minus/fil' to letters
 \if#1fF\else\if#1FF\else % keep hex. digit 'F'
 \if#1pp\else\if#1Pp\else % otherwise lowercase
 \if#1ll\else\if#1Ll\else
 \if#1uu\else\if#1Uu\else
 \if#1ss\else\if#1Ss\else
 \if#1mm\else\if#1Mm\else
 \if#1ii\else\if#1Ii\else
 \if#1nn\else\if#1Nn\else\if#1W \II@endkwlc
 \else #1\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi
 \fi\fi\fi\fi\II@kwlc}% the 'W' stops with a ' '
\def\II@endkwlc #1\II@kwlc{\fi\fi\fi\fi\fi\fi\fi
 \fi\fi\fi\fi\fi\fi\fi\fi\fi\fi}
% split the glue into its components
\def\II@kwminus #1minus#2 #3\II@end{% get shrink
 \def\II@X{#3}\ifx\II@X\empty \def\II@skipx{#1}%
 \else \def\II@skipx{#1 }\fi \def\II@sh{#2 }}
\def\II@kwplus #1plus#2 #3\II@end{% get stretch
 \def\II@st{#2 }\ifx\II@st\space \def\II@nw{#1}%
 \else \def\II@nw{#1 }\fi}
% replace 'Fil', 'Fill', 'Filll', 'mu' by 'pt'
\def\II@uchg{\advance\II@cnt by 1 }%S3.4's count
\def\II@fil #1Fi#2 #3\II@end#4#5{% get order of
% #4's value, extracted from '#1fi#2 #3', in #5
% and replace a <fil dimen> in #4 with unit pt
 \def#5{#2}\ifx#5\II@l \def#4{#1pt }\II@uchg
 \else\ifx#5\II@ll \def#4{#1pt }\II@uchg
 \else\ifx#5\II@lll \def#4{#1pt }\II@uchg
 \else\def#5{}\fi\fi\fi}
\def\II@l{l}\def\II@ll{ll}\def\II@lll{lll}
\def\II@mu #1mu#2\II@end#3{% change mu to pt for
 \def\II@X{#2}%      #3, extracted from '#1mu#2'
 \ifx\II@X\empty\else \def#3{#1pt }\II@uchg\fi}
%% main macro
\def\ifisglue #1\Boolend{\II@sofarfalse\II@cnt=0
 \edef\II@skip{\II@kwlc #1W}%                 S1
 \edef\II@skip{\expandafter\II@kwlc\II@skip W}%
 \expandafter\II@kwminus\II@skip minus \II@end
 \expandafter\II@kwplus\II@skipx plus \II@end%S2
 \expandafter\II@mu\II@nw mu\II@end\II@nw % S3.1
 \def\II@sto{}\ifx\II@st\space \II@uchg   % S3.2
 \else \expandafter\II@mu\II@st mu\II@end\II@st
  \ifx\II@X\empty \expandafter
   \II@fil\II@st Fi \II@end\II@st\II@sto \fi\fi
 \def\II@sho{}\ifx\II@sh\space \II@uchg   % S3.3
```

```
\else \expandafter\II@mu\II@sh mu\II@end\II@sh
 \ifx\II@X\empty \expandafter
  \II@fil\II@sh Fi \II@end\II@sh\II@sho \fi\fi
\ifnum\II@cnt=3 \IImugluetrue            % S3.4
\else \IImugluefalse \fi
\expandafter\ifisdim \II@nw\Boolend      % S4.1
 \II@sofartrue \fi
\ifII@sofar \ifx\II@st\space             % S4.2
 \else\expandafter\ifisdim \II@st\Boolend
  \else \II@sofarfalse \fi\fi\fi
\ifII@sofar \ifx\II@sh\space             % S4.3
 \else\expandafter\ifisdim \II@sh\Boolend
  \else \II@sofarfalse \fi\fi\fi
\ifII@sofar                              % S4.4
 % use a different pattern for the macro's wlog
 \wlog{=*= start ignore}\batchmode\begingroup
 \setbox0=\hbox{\II@font \ifIImuglue \muskip255
  \else \skip255 \fi=#1}\xdef\II@wd{\the\wd0}%
 \endgroup\IIcurrentmode\wlog{=*= stop ignore}%
 \ifdim\II@wd=0pt \else \II@sofarfalse   % S4.5
\fi\fi \ifII@sofar}
\catcode`\@=12
```

Remark. The macros `\II@sto` and `\II@sho` are defined but not used. I accept this overhead as the glue is split into all components and `\ifIImuglue` is set. Except for the values of some alphabetic constants we know everything of the glue if the orders of the dimensions in the stretch and shrink components are available too. If `\ifisglue` returns true then the number of letter 'l's in the above macros corresponds to the number of '`l`'s after '`Fi`'; i.e., 'l' stands for `Fil`, 'll' for `Fill`, and 'lll' for `Filll`. An empty replacement text signals a finite dimension.

References

[1] Donald E. Knuth, *The TEXbook*, Volume A of *Computers & Typesetting*, Boston, Massachusetts: Addison-Wesley, 1984.

[2] Donald E. Knuth, *TEX: The Program*, Volume B of *Computers & Typesetting*, Boston, Massachusetts: Addison-Wesley, 1986.

[3] Udo Wermuth, "Is a given input a valid TEX ⟨*number*⟩?", *TUGboat* **45**:1 (2024), 106–109. `tug.org/TUGboat/tb45-1/tb139wermuth-isint.pdf`
See file `ifisinteger.tex` in package `ifis-macros`. `ctan.org/pkg/ifis-macros`

[4] Udo Wermuth, "Is a given input a valid TEX ⟨*dimen*⟩?", *TUGboat* **45**:1 (2024), 109–112. `tug.org/TUGboat/tb45-1/tb139wermuth-isdim.pdf`
See file `ifisdimension.tex` in the package of [3].

◇ Udo Wermuth
Dietzenbach, Germany
`u dot wermuth (at) icloud dot com`

Comparisons for glue and muglue

Udo Wermuth

Abstract

This article develops macros for the original TEX program that behave like Boolean conditionals to compare corresponding components of two glue or two muglue specifications.

1 Introduction

It is well known that TEX doesn't provide a built-in conditional for two of its basic data structures — token lists and glue specifications.

Well, token lists are quite complex so any test other than equality is hardly useful. Such a test is easily accomplished when the two token lists are placed in separate macro definitions using `\edef` and `\the`; these macros are then tested by `\ifx`.

But skips are, like dimensions, in some sense numeric and one might expect a few conditionals for them. Sure, the width component, i.e., the first component, of two skips can be compared with `\ifdim`. In this case TEX coerces the glue to its first dimension, i.e., it ignores the stretch and shrink components [1, p. 118]. We don't have a direct access to stretch or shrink components for a test. (ε-TEX provides commands to extract the components as dimensions and their orders as integers.)

For muskips `\ifdim` doesn't work; TEX doesn't list `mu` as a valid unit for a dimension. The article [3] briefly discusses that ⟨*mudimen*⟩ and ⟨*fil dimen*⟩ aren't dimensions that can be assigned to TEX's `\dimen` registers; see also [1, p. 270f.].

I'm not aware of any documented complaints that there is no conditional for token lists. And it seems that only two authors have stated in *TUGboat* articles that they miss an `\ifskip`: D. Salomon on page 214 in issue **11**:2 and P. Huszár on page 199 in issue **13**:2. A detailed look at the problems that they want to solve with `\ifskip` shows that the `\edef` & `\ifx` technique can be used here too.

There are good reasons why TEX does not provide the conditional `\ifskip`. Although numeric in some sense, a skip has three components and it isn't obvious what "≤" (less than or equal) means as a relation between two skips. For example, is `1pt plus 100pt minus 2pt` larger or smaller than `1pt plus 100pt minus 1pt`? `\ifdim` compares the natural widths and gives the answer: The dimensions are equal. But the skips clearly aren't equal. A *lexicographical order* can be established if we compare two skips by their components with the relation ≤.

doi.org/10.47397/tb/46-1/tb142wermuth-ifglue

Let the tuple (g°, g^+, g^-) represent the three components of a glue item: natural width, stretchability, shrinkability. Then the lexicographic order is: $(g_1^\circ, g_1^+, g_1^-) \le (g_2^\circ, g_2^+, g_2^-) \iff (g_1^\circ < g_2^\circ) \vee (g_1^\circ = g_2^\circ \wedge g_1^+ < g_2^+) \vee (g_1^\circ = g_2^\circ \wedge g_1^+ = g_2^+ \wedge g_1^- \le g_2^-)$. In the example above, the second glue value is less than or equal to the first. But do we want that a larger shrink value means that a skip is larger than another if natural widths and stretch values are equal?

This little note does not code the above or any other *partial order* for glue and muglue. It only shows how the macro `\ifisglue` of [3] can be used to implement a structure that helps to do all kinds of componentwise tests for two glue specifications.

Contents. Section 2 introduces the aforementioned structure that will simplify the coding of the macros. Section 3 presents macros `\ifnatwd`, `\ifstretch`, and `\ifshrink`. They compare the corresponding components of two glue or two muglue values. Section 4 explains their usage.

2 Tsgm: TEX split glue macro

There are many ways to split a glue specification into its components. I use the macro `\ifisglue` although its main task is to check the validity of a glue specification. But we need error checking anyway if the glue specification is entered by a user.

We must store the glue specification with the ability to easily access the data of each component. Of course, several macros and/or registers could be used to store the individual components. But I prefer to have all information in one macro. In this macro the access to each component is realized with an `\ifcase`. The cases are:

0 type: 0 represents glue and 1 muglue
1 natural width as a dimension; i.e., the unit `mu` is replaced by `pt`
2 stretchability as a dimension; i.e., the units `mu`, `fil`, `fill`, and `filll` are replaced by `pt`
3 order of the stretch component with range 0–3: 0 for a finite dimension or a ⟨*mudimen*⟩, 1–3 for `fil`, `fill`, and `filll`, resp.
4 shrinkability as in no. 2
5 order of the shrink component as in no. 3
6 the original glue specification in TEX's canonical form
7 314.271 (as an identification)
else cm (forms with case 7 a dimension)

We call a macro definition that stores glue in this form a *TEX Split Glue Macro* or *Tsgm* for short.

Let's start with a simple task to see an example. We want to create a Tsgm for the glue value 0 pt.

```
\def\II@setzero #1{% #1: Tsgm name for zero skip
 \def#1##1{\ifcase##1 0 \or 0pt % type and natwd
  \or 0pt \or 0 \or 0pt \or 0 % stretch & shrink
  \or 0.0pt \or 314.271\else cm\fi}}%glue and id
```

The macro receives a control word as argument and defines it to store the zero-skip Tsgm. (Error checking is done before we call the macro; see below.)

It is easy to create a Tsgm when `\ifisglue` is used. This macro splits the given glue specification and provides us with all details of the components. Only the order of stretchability and shrinkability must be decoded: `\ifisglue` represents the *order of infinity* by the number of 'l's in the macros `\II@sto` and `\II@sho`, resp.; see [3]. The macro has no error checking as it is only used in a safe way.

```
\def\II@getorder #1{% #1: \II@sto or \II@sho
 \ifx#1\empty 0 \else\ifx#1\II@l 1
 \else\ifx#1\II@ll 2 \else 3 \fi\fi\fi}
```

For the next step the file `ifisglue.tex` must be loaded; it implements `\ifisglue`.

```
\def\makeTsgm #1!#2{% #1: glue; #2: name of Tsgm
 \ifcat\relax\noexpand#2%can #2 be a macro name?
  \expandafter\ifisglue #1\Boolend %must be true
   {\ifIImuglue\muskip255\else\skip255\fi=#1%
    \xdef\II@glue{\ifIImuglue\the\muskip255
      \else\the\skip255\fi}}% TeX canonical form
   \expandafter\ifisglue\II@glue\Boolend
    \edef#2##1{\noexpand\ifcase##1 % create Tsgm
     \ifIImuglue1 \else0 \fi\noexpand\or %0:type
     \II@nw\noexpand\or       % 1: natural width
     \ifx\II@st\space 0pt \else\II@st\fi % 2 and
     \noexpand\or \II@getorder\II@sto %3:stretch
     \noexpand\or
     \ifx\II@sh\space 0pt \else\II@sh\fi % 4 and
     \noexpand\or \II@getorder\II@sho %5: shrink
     \noexpand\or \II@glue\noexpand\or %6: value
     314.271\noexpand\else cm\noexpand\fi}\fi%id
  \else\errhelp{A glue specification should have
    been here; I replaced your input by '0pt' to
    build the Tsgm.}\II@setzero #2%
   \errmessage{Missing glue, treated as zero}\fi
 \else \errhelp{The argument after '!' must be a
   control sequence that I can define/redefine.
   Proceed; I'll ignore the \makeTsgm I just
   read.}%
  \errmessage{Invalid name for new macro}\fi}
```

The next macros extract the data of a Tsgm. Again, there is no error checking as these macros are only used internally with valid Tsgms.

```
\def\II@gluetype #1{#10 }%   the argument #1 is
\def\II@natwd #1{#11 }%      in all these macros
\def\II@stretchdim #1{#12 }% a valid Tsgm, i.e.,
\def\II@stretchord #1{#13 }% it's a control word
\def\II@shrinkdim #1{#14 }%  filled by the macro
\def\II@shrinkord #1{#15 }%  \makeTsgm
```

To verify that a macro is a Tsgm, we need to rebuild the Tsgm from case 6, if it exists, and compare all cases. This is very slow so a faster but fallible macro is coded that combines case 7 and the `\else` case. It catches the most common errors and cannot be fooled by a single-valued replacement text.

```
\def\ifIIisTsgm #1\Boolend{%compare to \ifisglue
 \wlog{=+= start ignore}\batchmode\begingroup
  \setbox0=\hbox{\dimen255=0#17cm
   \ifdim\dimen255=314.271#1{-1}\else 0\fi}%
  \xdef\II@wd{\the\wd0 }\endgroup\IIcurrentmode
 \wlog{=+= stop ignore}\ifdim\II@wd=0pt }
```

We need the delimiter `\Boolend`, which was used for `\ifisglue`, to make the macro skippable [1, p. 211].

3 Macros for comparisons

Before we implement the macros that do the comparison of Tsgms, let's list the possible error situations. The macros have three arguments like `\ifnum` or `\ifdim`. The first and the third must be tested for being Tsgms. The second is the relation, that is, we expect '<', '=', or '>'; otherwise we imitate TEX's error message used for `\ifnum` and `\ifdim` [2, §503].

As we allow comparisons only if the types of the glue specifications agree, i.e., both are glue or both are muglue, we also imitate TEX's corresponding error message shown for `\ifdim` [2, §408].

```
\def\II@noTsgm #1{% #1: 1 or 2 to select 1st/3rd
 \errhelp{I expected a macro with a special
  structure. But the received argument does not
  obey this structure. I cannot perform the test
  and return false.}\errmessage{Not a TeX split
  glue macro (\ifodd#1 1st \else 3rd \fi arg.)}}
\def\II@chkrel #1#2{% #1: character; #2: caller
 \def\II@rel{#1}%              save the relation
 \if#1<\else\if#1=\else\if#1>\else
  \errhelp{I was expecting to see `<', `=', or
   `>'. Didn't.}\def\II@rel{=}% replace relation
  \errmessage{Missing = inserted for #2}%
 \fi\fi\fi}
\def\II@badtypes{\errhelp{I'm going to assume
  that 1mu=1pt when they're mixed.}%
 \errmessage{Incompatible glue units}}
```

After two glue or muglue values are transformed into Tsgms, for example, via `\makeTsgm<1st val>!\one` and `\makeTsgm<2nd val>!\two`, the components of the Tsgms `\one` and `\two` can be compared.

First, let's code the macro to compare the natural widths of glue or muglue. One usage is, for example, `\ifnatwd\one>\two\Boolend` ⟨*true branch*⟩ `\else` ⟨*false branch*⟩`\fi`. As mentioned above, the natural widths of two glue values can also simply be compared with `\ifdim` if it is the only comparison that is needed. But now you can also compare the natural widths of two muglue values without error.

```
% macro arguments #1,#3: Tsgm; #2: relation
\def\ifnatwd #1#2#3\Boolend{\II@sofarfalse %init
 \ifIIisTsgm#1\Boolend \ifIIisTsgm#3\Boolend
  \ifnum\II@gluetype#1=\II@gluetype#3%test types
  \else \II@badtypes \fi  % used muglue and glue
  \II@chkrel#2{\string\ifnatwd}%  check relation
  \ifdim\II@natwd#1\II@rel\II@natwd#3%
   \II@sofartrue \fi
 \else\II@noTsgm2\fi \else\II@noTsgm1 % bad Tsgm
 \fi \ifII@sofar}
```

Note, the flag `\ifII@sofar` is declared in the file `ifisglue.tex`.

The other two macros are similar. The only new aspect is that we must include the order of the stretch- or shrinkability in the comparison.

```
\def\ifstretch #1#2#3\Boolend{\II@sofarfalse
 \ifIIisTsgm#1\Boolend \ifIIisTsgm#3\Boolend
  \ifnum\II@gluetype#1=\II@gluetype#3%test types
  \else \II@badtypes \fi  % used muglue and glue
  \II@chkrel#2{\string\ifstretch}% test relation
  \ifnum\II@stretchord#1=\II@stretchord#3\relax
   \ifdim\II@stretchdim#1\II@rel % compare dimen
    \II@stretchdim#3\relax \II@sofartrue \fi
  \else\ifnum\II@stretchord#1\II@rel
    \II@stretchord#3\relax \II@sofartrue \fi\fi
 \else\II@noTsgm2\fi \else\II@noTsgm1 % bad Tsgm
 \fi \ifII@sofar}

\def\ifshrink #1#2#3\Boolend{\II@sofarfalse
 \ifIIisTsgm#1\Boolend \ifIIisTsgm#3\Boolend
  \ifnum\II@gluetype#1=\II@gluetype#3%test types
  \else \II@badtypes \fi  % used muglue and glue
  \II@chkrel#2{\string\ifshrink}%  test relation
  \ifnum\II@shrinkord#1=\II@shrinkord#3\relax
   \ifdim\II@shrinkdim#1\II@rel  % compare dimen
    \II@shrinkdim#3\relax \II@sofartrue \fi
  \else\ifnum\II@shrinkord#1\II@rel
    \II@shrinkord#3\relax \II@sofartrue \fi\fi
 \else\II@noTsgm2\fi\else \II@noTsgm1 % bad Tsgm
 \fi \ifII@sofar}
```

4 Usage

Let's become more familiar with the macros of section 3 and the data structure of section 2 by studying some examples. This is the initial set-up: Register `\muskip255` contains the muglue specification `3mu plus 2mu minus 1mu` and we want to compare its components to a user entered specification: `3.1mu plus 2mu minus 2mu`.

Step 1. We must create Tsgms from the given glue specifications. Thus we define the two control words `\mgone` and `\mgtwo`.

```
\muskip255=3mu plus 2mu minus 1mu
\expandafter\makeTsgm \the\muskip255 !\mgone

\makeTsgm 3.1mu plus 2mu minus 2mu!\mgtwo
```

TEX shows the contents of these macros if `\show` is placed in front of each one. (I edited the output a little bit to make it more readable.)

```
> \mgone=macro:
#1->\ifcase #1 1 \or 3.0pt
    \or 2.0pt \or 0 \or 1.0pt \or 0
    \or 3.0mu plus 2.0mu minus 1.0mu
    \or 314.271\else cm\fi .

> \mgtwo=macro:
#1->\ifcase #1 1 \or 3.1pt
    \or 2.0pt \or 0 \or 2.0pt \or 0
    \or 3.1mu plus 2.0mu minus 2.0mu
    \or 314.271\else cm\fi .
```

As explained in section 2 all dimensions are given in `pt` as the unit `mu` is replaced. That we deal with muglue is stated by the `1` after `#1`. We know that in both Tsgms the stretch and shrink components — shown in the second lines — are given in `mu` too as the cases for their orders are, in both macros, zero.

The third line in both Tsgms contains the input stated by the user in TEX's canonical form that we get if the contents of a register is output with `\the`. For example, integers receive a "`.0`" and all units are in `pt`, `mu`, or a ⟨*fil dimen*⟩. All letters in this representation have the category code 12 [1, p. 214].

Step 2. First, we want to compare the natural widths of the two Tsgms. Let's play with different scenarios.

```
\ifnatwd\mgone<\mgtwo\Boolend
  \message{1st natural width smaller}%
\else \message{1st nat. wd. not smaller}\fi

\ifnatwd\mgone=\mgtwo\Boolend
  \message{Natural widths are equal}%
\else \message{Different natural widths}\fi

\ifnatwd\mgone>\mgtwo\Boolend
  \message{1st natural width larger}%
\else \message{1st natural width not larger}\fi

\ifnatwd\mgone-\mgtwo\Boolend % a typo
  \message{Natural widths are equal}%
\else \message{Different natural widths}\fi
```

The messages that are shown are the expected ones: "1st natural width smaller", "Different natural widths", "1st natural width not larger", an error message "Missing = inserted for `\ifnatwd`" followed by another "Different natural widths".

Step 3. The other comparisons are used in the same way. Thus, a single test should suffice to explain their usage.

```
\ifstretch\mgone<\mgtwo\Boolend
  \message{1st stretchability smaller}%
\else\message{1st stretchability not smaller}\fi
\ifshrink\mgone<\mgtwo\Boolend
  \message{1st shrinkability smaller}%
\else \message{1st shrinkability not smaller}\fi
```

These tests show "1st stretchability not smaller" and "1st shrinkability smaller".

Step 4. Now we redefine the second Tsgm.

```
\makeTsgm 1mu plus 1fil!\mgtwo
```

With `\show\mgtwo` we get (again edited):

```
> \mgtwo=macro:
#1->\ifcase #1 1 \or 1.0pt
    \or 1.0pt \or 1 \or 0pt \or 0
    \or 1.0mu plus 1.0fil
    \or 314.271\else cm\fi .
```

The missing shrink component is coded as the pair (0 pt, 0); the order of the stretch component becomes one. The tests of step 3 now give "1st stretchability smaller" and "1st shrinkability not smaller".

Step 5. The Tsgm `\mgtwo` is redefined again. But this time we don't obey the structure of a Tsgm.

```
\def\mgtwo{314.271cm }
\ifshrink\mgone=\mgtwo\Boolend
  \message{Same shrinkability}\fi
```

TEX responds with the error message "Not a TeX split glue macro (3rd arg.)" with the help text "I expected a macro with a special structure. But the received argument does not obey this structure. I cannot perform the test and return false." That is, the `\ifshrink` doesn't show a message.

Step 6. The next redefinition of `\mgtwo` proves that the check for a valid Tsgm might fail. In this case the user gets the error message "Missing number, treated as zero".

```
\def\mgtwo#1{\ifnum#1>0 314.271 \else cm\fi}
\ifshrink\mgone=\mgtwo\Boolend
  \message{Same shrinkability}\fi
```

The effort required to fool the check of Tsgms seems to be high enough to exclude it occurring by accident.

References

[1] Donald E. Knuth, *The TEXbook*, Volume A of *Computers & Typesetting*, Boston, Massachusetts: Addison-Wesley, 1984.

[2] Donald E. Knuth, *TEX: The Program*, Volume B of *Computers & Typesetting*, Boston, Massachusetts: Addison-Wesley, 1986.

[3] Udo Wermuth, "Is a given input a valid TEX ⟨*glue*⟩/⟨*muglue*⟩?", *TUGboat* **46**:1 (2024), 145–147. `tug.org/TUGboat/tb46-1/tb142wermuth-isglue.pdf`

◇ Udo Wermuth
Dietzenbach, Germany
`u dot wermuth (at) icloud dot com`

Cookie cutters: An example workflow of how to generate simple 3D-printable objects from LaTeX graphics

samcarter, Ulrike Fischer

Abstract

This article will describe a step-by-step workflow for converting LaTeX vector graphics into simple 3D models. As a specific example, we will show how to create a 3D-printable model for a cookie cutter from a TikZ drawing using the open source program OpenSCAD.

1 Preparing a LaTeX graphic

As our first step, we need to prepare a vector graphic in LaTeX. This can be done in various ways. For the workflow shown here, we are going to use the squirrel from the `tikzlings` package[1] as an example, but other methods to produce a vector image will work the same.

To create a usable cookie cutter, we need to make some adjustments to the LaTeX graphic. If possible, we want to avoid openings within the cookie cutter. They would require additional support structures to hold them in place within the outline of the cookie cutter. The squirrel in this example has a small gap between the body and its tail. We can cover this up by adding a circle over the gap. This patch does not need to look nice or match the opening exactly, it just needs to be confined within the outline of the figure. The resulting code and image can be seen in Code 1.

Code 1: Squirrel

```
\documentclass{standalone}
\usepackage{tikzlings}

\begin{document}

\begin{tikzpicture}
  \squirrel
  \path (-0.8,-0.1) -- (1.4,2.4);
  \fill (0.5,0.9) circle [radius=0.25cm];
\end{tikzpicture}

\end{document}
```



While it is technically possible to 3D print cookie cutters with small and detailed elements, they are unlikely to be suitable for actually making cookies. The cookie dough might get stuck in tight spaces or thin elements might break off after baking. To avoid this, we want to increase the thickness of very thin elements while preparing our LaTeX graphic. The bee, shown in Code 2, has very thin antennae, which would be problematic for a cookie cutter. Thus we enhance their thickness by adding thicker lines.

For the workflow shown in this article, it is also helpful to avoid clipping paths in TikZ, as not all the tools we will subsequently use in this workflow will respect the clipped paths. It might be tempting to draw a half circle by clipping a full circle, but the following conversion into a 3D object will be easier if the half circle is constructed from an arc.

2 Converting to SVG silhouette

In order to create a cookie cutter, we only need the outline of our image without all the details. An easy way to create such a silhouette is to first generate an SVG file. There are several

[1] `ctan.org/pkg/tikzlings`

doi.org/10.47397/tb/46-1/tb142samcarter-cookies

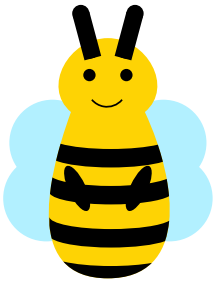
Code 2: Bee

```
\documentclass{standalone}
\usepackage{tikzlings}

\begin{document}

\begin{tikzpicture}
\bee
\fill[rotate around={15:(-0.175,2.115)}]
    (-0.24,1.93) rectangle (-0.1,2.3);
\fill[rotate around={-15:(0.175,2.115)}]
    (0.24,1.93) rectangle (0.1,2.3);
\end{tikzpicture}

\end{document}
```

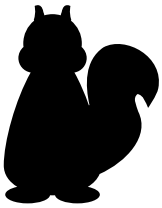


possible approaches to get an SVG from LaTeX. For this example we are going to first compile a PDF as normal and then use `pdftocairo` from the poppler library[2] to convert it to SVG with `pdftocairo -svg example.pdf example.svg`.

The resulting SVG still contains all the details of your LaTeX graphic. To create a silhouette, we can utilise the vector drawing program Inkscape,[3] either via its GUI or the command line. Code 3 shows a command line call of Inkscape, which first selects all objects and converts them to path if necessary (meaning that things like lines will become a rectangular path), then merges all paths to create a silhouette and finally saves the result as a new SVG file.

Code 3: Inkscape

```
inkscape -g --actions="\
  select-all; object-to-path;\
  select-all; path-union;\
  export-filename:example_merged.svg;\
  export-do; quit-immediate;"\
  "example.svg"
```



3 OpenSCAD

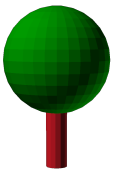
In the next step, we will use the open-source program OpenSCAD[4] to further process the merged SVG file. OpenSCAD is a text-based computer-aided design (CAD) program. Being text-based should make it feel more familiar to users with a TeX background than other more GUI-oriented alternatives. For users new to OpenSCAD its website has a tutorial[5] and many examples. To give a short impression of what the language used by OpenSCAD looks like, Code 4 shows an example of how to draw a simple tree by combining a cylinder as trunk and a sphere for the foliage. This isn't too different from, for example, drawing a similar tree in Ti*k*Z, maybe with a brown rectangle for the trunk and a green circle above it for the foliage.

[2] poppler.freedesktop.org
[3] inkscape.org
[4] openscad.org
[5] openscad.org/documentation.html#tutorial

Code 4: OpenSCAD Tree

```
color("brown")
  cylinder(h=30, r=4);

translate([0, 0, 40])
  color("green")
    sphere(20);
```



Giving a full overview of all the available functions of OpenSCAD would go beyond the scope of this article,[6] but the main features we need to create our cookie cutters are:

`import("...")`: imports files, e.g. our SVG silhouette

`offset(...)`: increases the size of the following shapes by a specific length

`difference()`: subtracts the second shape/object (and all further ones) from the first one

`linear_extrude(height=...)`: generates a 3D object from a 2D shape

Using these features, we can start building our cookie cutter by importing our SVG silhouette to get a 2D shape; see Figure 1a. With `offset()` we can import a second, slightly larger, copy and subtract the first shape from the larger one. This gives us a thin two-dimensional rim around our cookie cutter as can be seen in Figure 1b. Figure 1c shows how the shape looks once `linear_extrude()` has created a 3D object from the two-dimensional shape. In the final image, Figure 1d, we used the same approach to add a thicker edge to the cookie cutter, which will make it more stable and will also be gentler on one's hands while pressing the cookie cutter into the dough.

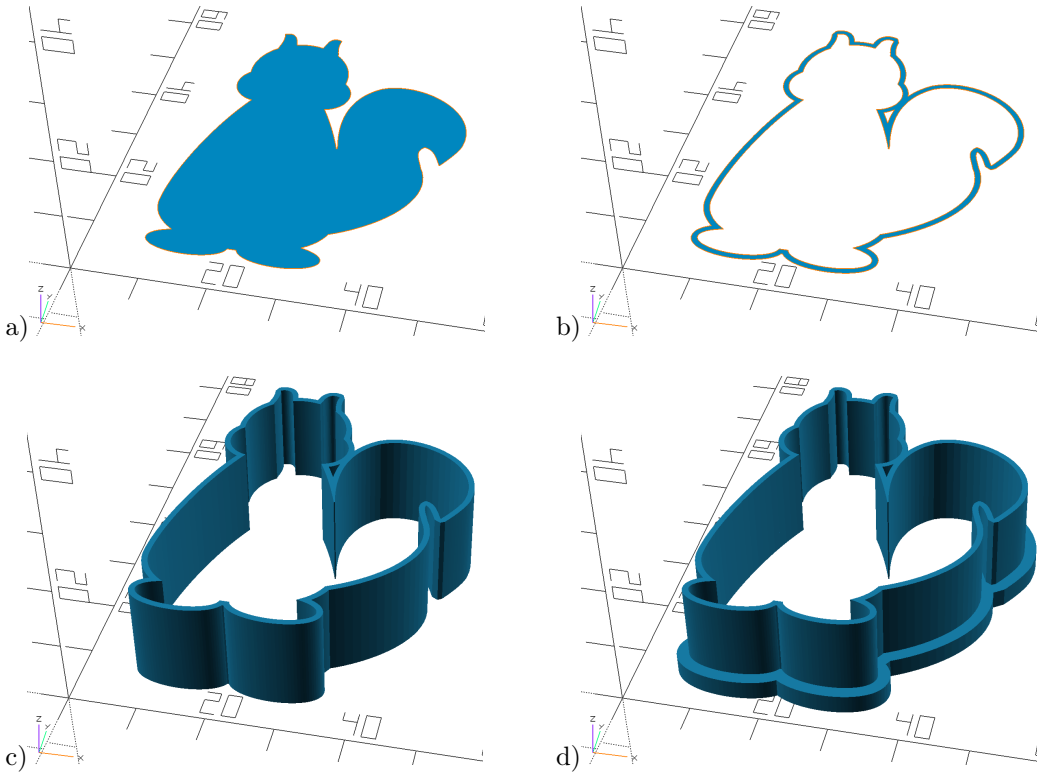


Figure 1: Step-by-step conversion of an SVG silhouette into the 3D model of a cookie cutter.

[6] See, e.g. `openscad.org/cheatsheet` for an overview.

Code 5 lists a complete[7] script for creating our cookie cutter. The measurements are given in millimetres. The printed cookie cutter might have slightly different dimensions, depending on the 3D printer used.

Code 5: Cookie Cutter

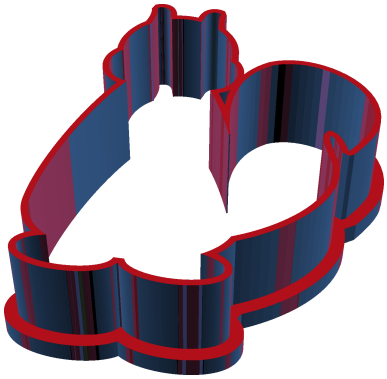
```
filepath = "example_merged.svg";

wall_thickness = 0.9;
wall_height = 15;

rim_thickness = 2.7;
rim_height = 3;

// thin wall
linear_extrude(height=wall_height)
difference(){
  offset(wall_thickness) import(filepath);
  import(filepath);
}

// broader rim
linear_extrude(height=rim_height)
difference(){
  offset(rim_thickness) import(filepath);
  import(filepath);
}
```



The finished 3D model can be saved from OpenSCAD as an STL file, which can be imported into all the major 3D printing programs. A collection of various STL files based on the `tikzlings` and `tikzducks` packages can be downloaded from `github.com/TikZlings/cookiecutter`. Figure 2 shows a couple of cookie cutters printed with PLA filament.



Figure 2: Examples of 3D printed cookie cutters.

[7] Slightly abbreviated for readability; the actual script will also rotate the object for proper positioning on the printing bed, etc. The full version is available at `github.com/TikZlings/cookiecutter`.

4 Cookie recipe

Now that we can create cookie cutters from all of our favourite LaTeX graphics, the only thing missing is a good cookie recipe:

"Mürbeteig" (German shortbread)

Ingredients:

For about three trays of cookies. You can double the recipe if enough children are available to help with decorating and eating.

- 300 g flour (2 1/3 cups)
- 200 g cold butter (7/8 cup)
- 100 g sugar (1/2 cup)
- 1 egg or 1 egg yolk
- a bit of cold/icy water
- a bit of salt

Instructions:

1. Cut the butter in dice, put everything in an electric mixer and knead (not for too long — it is ok if some butter can still be seen) into a dough. Make a disk out of the dough, put it into a plastic bag and place it in the refrigerator for a few hours (or until the next day).
2. To bake the cookies: Roll out part of the dough on a floured surface (and perhaps between sheets of plastic), cut out cookies and place them on baking paper.
3. Continue with the next part of the dough. Work quickly; the dough becomes soft and difficult to manage if it becomes too warm. A cold marble slab can help, if available. Bake every tray at around 180 °C (356 °F) for about 10 minutes until the cookies are light brown. Let them cool.
4. Get food colouring and optionally other decoration elements like pearls and flowers (children like those). Also prepare enough kitchen brushes (not too large), bowls and teaspoons (one for each colour). Syringes (without needles) are useful too.
5. In each bowl, mix powdered sugar with lemon juice (not too much at first, one needs less than one thinks) and colour the mixture with the food colouring. Put a spoon and a brush in every bowl. Then start decorating.
6. Let everything dry before storing the cookies in an airtight container.



Figure 3: Photo from a previous Christmas of decorated cookies (the cookies in this photo were not [yet] made with the Ti*k*Zlings cookie cutters described in this article)

◇ samcarter

◇ Ulrike Fischer
LaTeX project team
Bonn, Germany
ulrike.fischer (at)
latex-project.org

Styling Microsoft Power BI dashboards with TEX

Travis Stenborg

Abstract

Microsoft Power BI dashboards can be styled with TEX. Such business intelligence dashboards can be displayed online via the Power BI Service, or offline in the Power BI Desktop application. This versatility can be implemented via Mathtext, a lightweight TEX parser embedded in Python's Matplotlib library. Power BI's TEX presentation capabilities are explored here with an example dashboard. Links to supporting file resources are supplied.

1 TEX and LATEX in Matplotlib

Power BI has many data visualization tools. In addition to an extensive set of native charts, it has a dedicated Python plotting tool with built-in script editor. Python support, in turn, allows use of the TEX and LATEX-friendly Matplotlib plotting library.

Matplotlib offers multiple ways to incorporate TEX and LATEX into plots. Plotting with Matplotlib and LATEX has appeared previously in *TUGboat* [9]. For users with a LATEX installation, Matplotlib provides a pathway to LATEX-styling visuals for Power BI Desktop.

Power BI dashboards however, run on browser-accessible Power BI Service servers without LATEX installed. For this scenario, Mathtext, a lightweight TEX parser embedded in Matplotlib [10, 12], can be used instead. This approach alleviates the need for LATEX installed on cloud resources.

2 Dashboard data

To provide business intelligence, a dashboard needs data. A real-world gestational diabetes dataset [4, 5] from the Kaggle data science platform was imported here. Judicious data transformation was performed, e.g., column renaming, data type conversion and new calculated columns created via Data Analysis Expressions (DAX).

3 Dashboard design

A dashboard of visuals (cards, interactive key influencers, Python), static elements and a custom theme was composed in Power BI Desktop. Guides to building Power BI applications abound [1, 8, 11], including an easy-to-digest *For Dummies* book [3].

The Python visual was refined via Power BI's built-in script editor. The editor assumes a Python installation external to Power BI. Python 3.13.2 on Windows 11 was used here. The main Python executable was listed on the system's `PATH` environment variable. In Power BI Desktop, the Python home directory should be set from File → Options and settings → Options → Global, Python scripting.

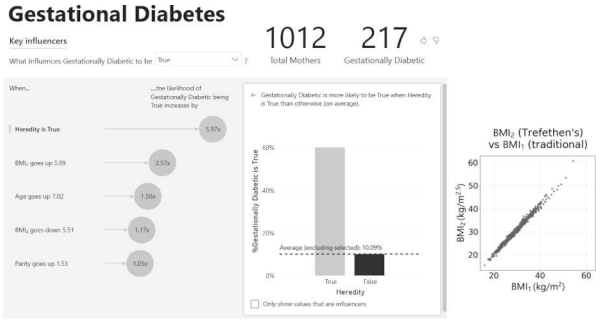


Figure 1: Power BI dashboard for a gestational diabetes dataset.

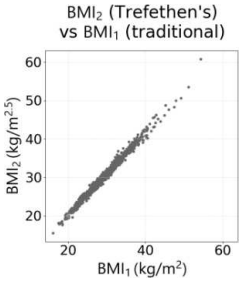


Figure 2: Power BI dashboard scatterplot. Subscripts and superscripts in the title and axis labels were generated via the Mathtext TEX parser in Matplotlib.

Matplotlib was used to visualize correlation of traditional body mass index with Trefethen's formulation [2] (calculated here with DAX). Subscripts and superscripts in the title and axis labels were specified with Mathtext. A Segoe UI plot typeface was used for consistency with Power BI Desktop defaults [7].

A neutral, black-gray-white dashboard palette was adopted. That choice was partly aesthetic, but primarily for accessibility to the colorblind [6].

4 Online publishing

After completing the example dashboard, it was published to the Power BI Service for online, browser-based access. A *Publish* button is available from the Home section of the Power BI Desktop ribbon to do that. The dashboard, as viewed with Microsoft Edge version 134.0.3124.72, is shown in Figure 1. The TEX-styled scatterplot is given enlarged in Figure 2.

The Power BI source (`.pbix` file), theme and diabetes data are available on GitHub at: `github.com/tstenborg/TeX-Styling-Power-BI`.

Travis Stenborg

doi.org/10.47397/tb/46-1/tb142stenborg-powerbi

References

[1] J. Arnold. *Learning Microsoft Power BI*. O'Reilly, 2022.

[2] H. Blackburn, D. Jacobs, Jr. Commentary: Origins and evolution of body mass index (BMI): continuing saga. *Int. J. Epidemiol.*, 43(3):665–669, 2014. doi.org/10.1093/ije/dyu061

[3] J.A. Hyman. *Microsoft Power BI for dummies*. Wiley, 2022.

[4] R. Jader. Gestational diabetes. Kaggle, 2024. doi.org/10.34740/KAGGLE/DSV/8301853

[5] R. Jader, S. Aminifar, R. Ejbali. Predictive model for diagnosis of gestational diabetes in the Kurdistan region by a combination of clustering and classification algorithms: An ensemble approach. *Appl. Comput. Intell. Soft Comput.*, 2022:1–11, 2022. doi.org/10.1155/2022/9749579

[6] A. Katsnelson. Colour me better: fixing figures for colour blindness. *Nature*, 598(7879):224–225, 2021. doi.org/10.1038/d41586-021-02696-z

[7] A. Kolokolov, M. Zelensky. *Data Visualization with Microsoft Power BI*. O'Reilly, 2024.

[8] A. Loth. *Teach Yourself Visually Power BI*. Wiley, 2023.

[9] A. Mertz, W. Slough. A gentle introduction to PythonTEX. *TUGboat* 34(3):302–312, 2013. tug.org/TUGboat/tb34-3/tb108mertz.pdf

[10] D.J. Pine. *Introduction to Python for Science and Engineering*. CRC Press, 2019.

[11] R. Rad. *Pro Power BI Architecture*. Apress, 2nd ed., 2023.

[12] S. Tosi. *Matplotlib for Python Developers*. Packt, 2009.

◇ Travis Stenborg
Sydney, Australia
ORCID 0000-0002-2693-9628

Abstracts

Die TEXnische Komödie 4/2024–1/2025

Die TEXnische Komödie is the journal of DANTE e.V., the German-language TEX user group (dante.de).

Die TEXnische Komödie 4/2024

Martin Sievers, Grußwort [Greeting]; pp. 4–5
Introductory words from the DANTE president.

Martin Sievers and Torsten-Karl Strempel, Einladung zur Frühjahrstagung 2025 und 67. Mitgliederversammlung von DANTE e.V. an der Hochschule Darmstadt [Invitation to the Spring Conference 2025 and 67th General Meeting of DANTE e.V. at the Darmstadt University of Applied Sciences]; pp. 6–7
We invite you to the 2025 spring conference from 3 to 5 April 2025 in Darmstadt.

Martin Sievers and Torsten-Karl Strempel, Beiträge gesucht [Call for presentations]; p. 8
Call for presentations at the conference.

Volker RW Schaa, Bericht von der TUG 2024 in Prag – Tag 1 [Report from TUG 2024 in Prague — Day 1]; pp. 9–17
The day 1 report from TUG 2024 in Prague.

Keno Wehr, LATEX und Schulphysik 8: Atom- und Kernphysik [LATEX and School Physics 8: Atomic and nuclear physics]; pp. 18–34
The eighth part of the series of articles on school physics deals with notations and graphical representations of atomic and nuclear physics. It presents commands from the packages bohr, chemfig, chemformula, chemmacros, elements, pgf-periodictable, pgf-spectra and textgreek.

Alexander Willand, Variable Fonts [Variable fonts]; pp. 34–41
There is a new toy in the font world: variable fonts. Fortunately, the LATEX3 team has created the necessary support so that variable fonts can be used with LATEX without any problems.

Luzia Dietsche, Aufgeschnappt [Picked up]; pp. 41–42
Every now and then, while surfing the Internet or reading various mailing lists, I come across information that might also be of interest to others.

Henning Hraban Ramm, ConTEXt kurz notiert [ConTEXt short notes]; pp. 42–48
Some news from the ConTEXt world.

Uwe Ziegenhagen, ASN-Labels mit LaTeX [ASN labels with LaTeX]; pp. 48–51

"Digitization of document storage" is a topic that is becoming increasingly relevant for me. In this article I describe how I create the useful ASN labels with LaTeX.

Frank Mittelbach, LaTeX News — Issue 39, June 2024; pp. 52–65

Published in *TUGboat* 45:2 and online at `latex-project.org/news`.

Jürgen Fenn, Neue Pakete auf CTAN [New packages on CTAN]; pp. 66–70

The list of new packages on CTAN.

Die TeXnische Komödie 1/2025

Martin Sievers, Grußwort [Greeting]; pp. 4–6

Introductory words from the DANTE president.

Martin Sievers, Uwe Ziegenhagen, Doris Behrendt, Was macht eigentlich der Vorstand? [What does the board actually do?]; pp. 6–9

The authors of this article decided not to run again for a board seat in 2025, so we want to invite other DANTE e.V. members to join.

Martin Sievers and Torsten-Karl Strempel, Einladung zur Frühjahrstagung 2025 und 67. Mitgliederversammlung von DANTE e.V. an der Hochschule Darmstadt [Invitation to the Spring Conference 2025 and 67th General Meeting of DANTE e.V. at the Darmstadt University of Applied Sciences]; pp. 9–10

We invite you to the 2025 Spring Conference from 3 to 5 April 2025 in Darmstadt.

Nadia Kwast and Marei Peischl, Bericht der Rechnungsprüfer zum Vereinsjahr 2022 – Korrektur und Erweiterung [Auditors' report for the DANTE year 2022 — Correction and extension]; pp. 11–17

Klaus Höppner, Bericht von der TUG 2024 in Prag – Tag 2 [Report from TUG 2024 in Prague — Day 2]; pp. 17–21

Marcel Kapfer, BayTeX 2024 [BayTeX 2024]; pp. 21–26

The report from BayTeX 2024.

Adelheid Bonnetsmüller, Jetzt schlägt's 12:00 Uhr – das Paket `uhrzeit` [Now it's 12:00 — the `uhrzeit` package]; pp. 27–28

For the lecture series "Having Fun With LaTeX", which I have been presenting at the DANTE e.V. conferences in Vienna since 2009, I closely follow what's happening on CTAN and often search there in the evenings or when I want to relax for packages that might fit the series. And I always find what I'm looking for — be it newer packages or ones that have been there for a long time (presumably less used). One of the newer ones is `uhrzeit` by Olaf Melzer from 2016, for printing times using German conventions.

Adelheid Bonnetsmüller, Playing Games with LaTeX – Schiffe versenken [Playing games with LaTeX — sinking ships]; pp. 29–34

As of summer 2024, there are 93 packages on CTAN that deal with games or puzzles. Of these, 19 deal with chess, 6 with Go, and 11 with bridge, role-playing games, and crossword puzzles. In addition, under the "Games" category, there are 14 that are not directly concerned with games, but rather with gadgets, such as the "thirteen" package or "coffeestains", which depicts coffee stains. These packages make up approximately 50% of the game packages; the other 50% deal with all sorts of card, board, logic, computer games, or game theory. One of these is the well-known game Battleship: The associated `battleship` package and its capabilities are explained here.

Henning Hraban Ramm, ConTeXt kurz notiert [ConTeXt short notes]; pp. 35–36

Some news from the ConTeXt world.

Frank Mittelbach, LaTeX News — Issue 40, November 2024; pp. 37–48

Published in this issue (pp. 35–39) and at `latex-project.org/news`.

Jürgen Fenn, Neue Pakete auf CTAN [New packages on CTAN]; pp. 49–52

Jerzy Ludwichowski, The 30th GUST TeX conference — TeX in education; pp. 53–54

Invitation to BachoTeX 2025.

[Received from Uwe Ziegenhagen.]

Zpravodaj 2024/1–4

Zpravodaj is the journal of $\mathcal{CS}$TUG, the TEX user group oriented mainly but not entirely to the Czech and Slovak languages. The full issue can be downloaded at `cstug.cz/bulletin`.

Vít Starý Novotný, Michal Hoftich, Jaromír Kuben, $\mathcal{CS}$TUG na konferenci TUG 2024 [$\mathcal{CS}$TUG at the TUG 2024 conference]; pp. 1–6

A report on the participation of $\mathcal{CS}$TUG members at TUG 2024 in Prague.

Vít Starý Novotný, Příprava videozáznamu české premiéry multimediálního díla „Fantasia Apocalyptica“ [Recording the Czech premiere of the “Fantasia Apocalyptica” multimedia work]; pp. 7–27

On October 11, 2019, the Czech premiere of the “Fantasia Apocalyptica” multimedia work was held for the 25th anniversary of Masaryk University’s Faculty of Informatics, featuring its author, Donald Knuth. A video recording of the performance was taken and published at the YouTube channel of the Faculty of Informatics. (`www.youtube.com/watch?v=wk7dEKMPP68`)

In this article, I describe how the recording was prepared from processing the raw footage through replicating the panels that accompanied the performance to the final composition. Apart from super-resolution imaging, the editing of the intro and the wrap-up, and the compression of sound, only free open-source tools like TEX, Audacity, Aegisub, FFmpeg, MLT, PDFtk, and ImageMagick were used. In the article, I describe how readers can use these tools for their own recordings.

Michal Hoftich, Responzivní design a automatická sazba s LuaLaTeXem [Responsive design and automatic typesetting with LuaLaTeX]; pp. 28–38

[An extended version of the article published in English in *TUGboat* 45:2.]

Michal Stano, Vít Starý Novotný, Overleaf Community Edition: Postup inštalácie a rozdiely oproti Overleaf Professional [Overleaf Community Edition: Installation and comparison with Overleaf Professional]; pp. 39–43

The Overleaf web editor enables convenient writing of documents using LaTeX. A significant advantage of Overleaf is that it enables collaboration among multiple users in real-time. In this article, we describe the process of installing the free version of Overleaf Community Edition on your own device and compare it with the paid version of Overleaf Professional.

Martin Ruckert, Profiling TEX input files; pp. 44–58

[Published in *TUGboat* 45:2.]

Marei Peischl, Marcel Krüger, Oliver Kopp, Vít Starý Novotný, Příprava (La)TeXových dokumentů v cloudu [Creation of LaTeX documents using a cloud-based pipeline]; pp. 59–72

A Czech translation of the article published in *TUGboat* 45:2. Translation by Vít Starý Novotný.

[Received from Vít Starý Novotný.]

La Lettre GUTenberg 54, 2024

La Lettre GUTenberg is a publication of GUTenberg, the French-language TEX user group (`gutenberg-asso.fr`)

Published November 2, 2024: A special issue about the GUTenberg Day 2024. The association has just been officially recognised as being in the public interest. `doi.org/10.60028/lettre.vi54`

Patrick Bideault, Éditorial [Editorial]; pp. 1–2

Journée GUTenberg, le 16 novembre 2024, à Paris [GUTenberg Day, November 16, 2024, in Paris]; pp. 2–4

Program for the GUTenberg Day and General Assemblies 2024.

Maxime Chupin, Rapport d’activité : novembre 2023 – novembre 2024 [Activity report: November 2023–November 2024]; pp. 4–9

About GUTenberg’s achievements this year.

François Druel, Rapport de gestion, rapport financier 2024, budget 2025, proposition de cotisations [Financial report]; pp. 9–13

François Druel, Convocation à l’assemblée générale extraordinaire [Notice of extraordinary general meeting]; pp. 14–20

Patrick Bideault, Bastien Dumont & Yvon Henel, Et maintenant, une bonne *vieille* veille technologique ! [Technology watch]; pp. 21–23

October 2024: 15 new CTAN packages, including three from French-speaking countries.

Patrick Bideault, La fonte de ce numéro : Overlock [This issue’s font: Overlock]; pp. 24–27

Prochaines rencontres [The next TEX-related meetings]; pp. 27–28

Yvon Henel, Rébus [Solution to the puzzle in the previous issue]; p. 28

[Received from Patrick Bideault.]



The Treasure Chest

These are the new packages posted to CTAN (`ctan.org`) from October 2024–April 2025. Descriptions are based on the announcements, edited for extreme brevity.

Entries are listed alphabetically within CTAN directories. More information about any package can be found at `ctan.org/pkg/`*pkgname*.

◇ Karl Berry
`https://tug.org/TUGboat/Chest`
`https://ctan.org/topic`

fonts

`gentium-otf` in `fonts`
Support for the Gentium TrueType fonts.

`gentium-sil` in `fonts`
Greek font with Latin and Cyrillic, in TrueType.

`hittype` in `fonts`
Hittite cuneiform.

`luciole` in `fonts`
OpenType font family, including math, developed for visually-impaired people.

`jetbrainsmono-otf` in `fonts`
Support for the JetBrainsMono OpenType font.

graphics

`callouts-box` in `graphics`
Callout boxes based on `tcolorbox`.

graphics/metapost/c/macros

`mp-neuralnetwork` in `graphics/metapost/c/macros`
Artificial neural networks with MetaObj.

graphics/pgf/contrib

`euclidean-lattice` in `graphics/pgf/contrib`
Two-dimensional Euclidean lattices with Ti*k*Z.

`luatikz` in `graphics/pgf/contrib`
2D graphics library to draw Ti*k*Z graphics in Lua.

`maritime` in `graphics/pgf/contrib`
International maritime signal flags using Ti*k*Z.

`pgf-go` in `graphics/pgf/contrib`
Diagram and comment on Go games, in PGF.

`prisma-flow-diagram` in `graphics/pgf/contrib`
Prisma 2009 flow charts using Ti*k*Z.

`qworld` in `graphics/pgf/contrib`
Graphical calculus in monoidal categories and quantum theory with Ti*k*Z.

`sunpath` in `graphics/pgf/contrib`
Sun path charts with Ti*k*Z.

`tikz-bpmn` in `graphics/pgf/contrib`
Ti*k*Z library for business processing models.

`tikz-cookingsymbols` in `graphics/pgf/contrib`
Cooking symbols with Ti*k*Z.

`tikz-triminos` in `graphics/pgf/contrib`
Triminos (triangular dominoes) with Ti*k*Z.

`tikzcalendarnotes` in `graphics/pgf/contrib`
Highlighting, marking and annotating dates in a Ti*k*Z calendar.

`timechart` in `graphics/pgf/contrib`
Extensive chronological charts with Ti*k*Z (see article in this issue, pp. 40–48).

macros/generic

`tuple` in `macros/generic`
Expandable operations for tuples of numbers.

macros/latex/contrib

`anima` in `macros/latex/contrib`
Slideshows with simple animations.

`atableau` in `macros/latex/contrib`
Symmetric group combinatorics.

`beamertools` in `macros/latex/contrib`
Programming tools for `beamer`.

`bxcoloremoji` in `macros/latex/contrib`
Use color emojis conveniently; all engines supported.

`calculatoritems` in `macros/latex/contrib`
Insert menus and items for classic calculators.

`chronos` in `macros/latex/contrib`
Ti*k*Z-based timelines with support for styles and color schemes.

`chuushaku` in `macros/latex/contrib`
Flexible book notes in Japanese.

`complexpolylongdiv` in `macros/latex/contrib`
Typesetting (complex) polynomial long division.

`cora-macs` in `macros/latex/contrib`
Macros for continuous reachability analysis.

`convert-jpfonts` in `macros/latex/contrib`
Typeset half-width Japanese as full-width.

`datatool-english` in `macros/latex/contrib`
English language support for `datatool`.

`datatool-regions` in `macros/latex/contrib`
Region (numeric) support for `datatool` (see article in this issue, pp. 74–78).

`dlrg-templates` in `macros/latex/contrib`
German Lifesaving Association (DLRG) templates.

`enumext` in `macros/latex/contrib`
Enumerated exercises, supporting tagged PDF.

`eqnlines` in `macros/latex/contrib`
Extended single- and multiline equation support.

`fillpages` in `macros/latex/contrib`
Insert blank or predefined pages to match a given page count.

`fragoli` in `macros/latex/contrib`
Complex semantic derivations for linguistics.

`gb4e-next` in `macros/latex/contrib`
Relative example reference commands for `gb4e`.

`gratzer-color-scheme` in `macros/latex/contrib`
Colors for definitions, theorems, corollaries, lemmas, and propositions.

doi.org/10.47397/tb/46-1/tb142chest

`indextra` in `macros/latex/contrib`
Continuation texts and mark access for indexes.

`inlinegraphicx` in `macros/latex/contrib`
Insert inline images, with automatic size and positioning.

`liederbuch` in `macros/latex/contrib`
Manage content (songs and more), with metadata.

`listparskip` in `macros/latex/contrib`
Adjust vertical space around lists.

`math-operator` in `macros/latex/contrib`
Predefined (about 150) and new math operators.

`neoschool` in `macros/latex/contrib`
Class for secondary school teachers.

`nodepthtext` in `macros/latex/contrib`
Automatic raising and scaling to remove depth of letters, for small texts.

`nstc-proposal` in `macros/latex/contrib`
Grant proposals for the Taiwan National Science and Technology Council.

`outoruby` in `macros/latex/contrib`
Ruby with line break support for Japanese text.

`packdoc` in `macros/latex/contrib`
Document LaTeX packages consistently.

`pats-resume` in `macros/latex/contrib`
Resume template.

`pegmatch` in `macros/latex/contrib`
Parsing Expression Grammars (PEG) for LaTeX.

`physics-patch` in `macros/latex/contrib`
Patches for the `physics` package, and integration with `siunitx`.

`pronunciation` in `macros/latex/contrib`
Typeset phonetics of the American English pronunciation of about 20K words.

`rebuttal` in `macros/latex/contrib`
Markup for structured rebuttals to papers.

`scaletextbullet` in `macros/latex/contrib`
Resize `\textbullet` without changing its vertical center.

`sshrc-insight` in `macros/latex/contrib`
SSHRC Insight Grant (Canada) proposals.

`stocksize` in `macros/latex/contrib`
Change paper (stock) size in the middle of a document.

`tabularray-abnt` in `macros/latex/contrib`
ABNT (Brazilian standard) theme for `tabularray`.

`temporal-logic` in `macros/latex/contrib`
Symbols for temporal logic operators.

`timeop` in `macros/latex/contrib`
Arithmetic operations with durations.

`tuzuk` in `macros/latex/contrib`
Turkish bylaws and regulations.

`ualberta` in `macros/latex/contrib`
Theses and dissertations at the University of Alberta.

m/l/c/beamer-contrib/themes

`beamerthemecelestia` in `m/l/c/b-c/themes`
Theme for academic/professional presentations.

`beamertheme-mirage` in `m/l/c/b-c/themes`
Provides dark and light color schemes.

`novabeamer-theme` in `m/l/c/b-c/themes`
Customizable colors, title page, design, and more.

macros/latex/contrib/biblatex-contrib

`biblatex-accursius` in `m/l/c/biblatex-contrib`
Citations for Italian legal scholarship.

`bibbreeze` in `m/l/c/biblatex-contrib`
Automatically reconstructing bibliography data (see article in this issue, pp. 86–93).

`biblatex-cse` in `m/l/c/biblatex-contrib`
Council of Science Editors reference style.

macros/luatex/generic

`unibidi-lua` in `macros/luatex/generic`
Unicode bidi algorithm implementation for several LuaTeX formats.

macros/luatex/latex

`lecturenotes` in `macros/luatex/latex`
Notes for a seminar, talk, or complete course.

`luatbls` in `macros/luatex/latex`
Lua tables made accessible to LaTeX.

`marginalia` in `macros/luatex/latex`
Non-floating marginal content with automatic placement for LuaLaTeX(see article in this issue, pp. 49–53).

macros/pdftex/plain

`pdftoolbox` in `macros/pdftex/plain`
Data structures, document layout, color and illustration support.

macros/unicodetex/latex

`nxuthesis` in `macros/unicodetex/latex`
Thesis template for Ningxia University.

`omgtudoc-asoiu` in `macros/unicodetex/latex`
Documents for the ASOIU department at Omsk State Technical University.

support

`expltools` in `support`
Static analysis of `expl3` code.

`extractbb` in `support`
Lua replacement for the `ebb`/`extractbb` program previously distributed with `dvipdfmx`.

systems

`apltex` in `systems`
Small distribution built on Web2C and LuaTeX.

TUG 2025 election report

Nominations for TUG President and the Board of Directors in 2025 have been received and validated. Because there is a single nomination for the office of President and because there are not more nominations for the Board of Directors than there are open seats, there is no requirement for a ballot this election.

For President, Arthur Rosendahl was nominated. As there were no other nominees, he is duly elected and will serve for a two-year term.

For the Board of Directors, the following individuals were nominated:

Doris Behrendt, Karl Berry, Johannes Braams, Kaja Christiansen, Klaus Höppner, Frank Mittelbach, Erik Nijenhuis.

As there were not more nominations than open positions, all the nominees are duly elected to a four-year term. Thanks to all for their willingness to serve. New terms for both President and members of the Board of Directors will begin at the Annual Meeting.

Board member Ross Moore has decided to step down. Ross was first elected to the Board in 1997 to fill a position that remained open after the election. His dedication and service to the community for these many years are gratefully acknowledged.

Election statements by all candidates are given below. They are also available online, along with announcements and results of previous elections.

◇ Barbara Beeton
for the Elections Committee
`tug.org/election`

Arthur Rosendahl (né Reutenauer)



(Candidate for TUG President.)

TEX biography: I seek a second term as TUG president, having served on the board since 2013.

I started using TEX as a mathematics student over 25 years ago and have been involved in the development and maintenance of several packages, including `polyglossia` and `hyph-utf8`; but although I co-created and am still actively working on the latter, my involvement in TEX affairs today is mostly on the organisational side.

I have been part of the organisation committee of the three online TUG conferences from 2020 to 2022 and am also active in other user groups of the TEX world, as a founding member of the ConTEXt Group, and a board member of GUTenberg, the French-language TEX users group.

Last year, I represented the TEX world in a unusual setting, acting as an expert witness on LATEX in a court of law. For full disclosure, I received a fee for my services.

Statement: If I am reelected as TUG president, I will advocate for TUG to continue doing what it does best, publish TUGboat and organise conferences, and support individuals and organisations active in the TEX world, in particular the TEX Live team. When most new users come to TEX through Overleaf, it may seem like a non-profit organisation with a dwindling membership is out of date, but if we don't keep doing the above, who will? We should keep acting as the group of all TEX users.

Doris Behrendt



(Candidate for TUG Board of Directors.)

Biography: Since 2023, I have been the main manager of the CrypTool project [1]. This project is based at Research Institute CODE at the University of the German Federal Armed Forces in Munich [2], where we do research and teaching in the field of cryptography, cyber defense and smart data.

From 2005 to 2023, I worked as a maths and physics teacher, raised two children, and contributed to the TEX community by serving (since 2016) as the treasurer of Dante e.V., the German-speaking TEX user group.

I earned a PhD in non-Euclidean geometry in the early 2000s and have been using LATEX since the mid-1990s.

Statement: I support the open-source idea in general. LATEX is not the only project to which I commit my free time or money — for example, also noyb [3], CCC [4] or SageMath [5].

There is a lot of change management to be done in many open-source-related communities, especially in organizing generational change. That's why, for example, I decided to make room for a younger Dante treasurer.

I am not a developer, but I am ready to engage in securing funding, traveling and networking, scouting for younger TEX talents, and trying to get them involved.

Accessibility is very important in my opinion.

I also find topics where AI and TEX intersect quite interesting.

[1]: `https://www.cryptool.org/en/`
[2]: `https://www.unibw.de/code-en`

doi.org/10.47397/tb/46-1/tb142elec

[3]: https://noyb.eu/en
[4]: https://www.ccc.de/en/
[5]: https://www.sagemath.org

Karl Berry



(Candidate for TUG Board of Directors.)

TEX biography: I served as TUG president from 2003–2011 and was a board member both before and after being president. I am running again for a position on the board.

I'm one of the primary system administrators and webmasters for the TUG servers, and the production manager for our journal *TUGboat*. I co-sponsored the creation of the TEX Development Fund in 2002.

On the development side, I'm currently the editor of TEX Live, the largest free software TEX distribution, and thus coordinate with many other TEX projects around the world, such as CTAN, LATEX, and pdfTEX. I developed and still (co-)maintain Web2c (Unix TEX) and its basic library Kpathsea, Eplain (a macro package extending plain TEX), and other projects. I am also a co-author of *TEX for the Impatient*, an early comprehensive book on plain TEX, now freely available. I first encountered and installed TEX in 1982, as a college undergraduate.

Statement of intent: I believe TUG can best serve its members and the general TEX community by working in partnership with the other TEX user groups worldwide, and sponsoring projects and conferences that will increase interest in and use of TEX. I've been fortunate to be able to work pro bono on TUG and TEX activities the past several years, and plan to continue doing so if re-elected.

Johannes Braams



(Candidate for TUG Board of Directors.)

Biography: I encountered TEX and friends sometime around 1985 when it was installed on our research VAX. It didn't take long for me to get hooked on LATEX and I started to think about and work on multilingual support, later to be known as *babel*. Besides that I have been active on quite a number of activities:

- For TEX and the TEX User Group
  - Co-founder and board member of the NTG, the Dutch-speaking TEX User group
  - As chairman of NTG I have served on the board of directors in 1994/1995 as special director for the NTG
  - designer of a couple of PhD-theses
  - Original author of the *babel* language support system
  - Member of the LATEX3 team, author of one of the chapters in the *LATEX Companion*
  - author, co-author or maintainer of a couple of publicly available LATEX packages and classes
- Professionally
  - System administrator for VAX/VMS and Unix systems
  - Manager of a team of System administrators
  - Functional administrator of one of the large administrative systems of KPN (PTT Telecom back then)
  - Project manager for various IT-projects within KPN for about 18 years
  - Consultant in the area of cyber security in Industrial Control Systems since 2015

Statement: In the last years I have been around again in the TEX-community. I would very much like to keep serving this wonderful community by continuing my role in the board of directors of TUG. I think TEX should and will be alive and well for many years to come, as the quality of typesetting that can be achieved by using TEX is still unsurpassed.

Kaja Christiansen



(Candidate for TUG Board of Directors.)

I live in the city of Aarhus, Denmark. I heard about TEX for the first time in the fall of 1979. In Palo Alto at the time, I wanted to audit courses at Stanford and my top priority was lectures by Prof. Donald Knuth. That, I was told, was not possible as Prof. Knuth was on leave due to work on a text processing project... This project was TEX! Back home, it didn't take long till we had a runnable TEX system in Denmark.

Statement: I have served at the TUG Board since 1997. Over the years, I worked (with Mimi Burbank) on TUG's web pages, co-sponsored the creation of the TeX Development Fund, served as the chair of the Technical Council and Development Fund, hosted and shared system administrator's responsibilities for the TUG server (which access to the Internet was faciliteted for several years by the

University of Aarhus), worked with the first three editions of TL and have been on various committees. I have been the Board member of Danish TEX Users Group and the president of DK-TUG during the years 2002–2011.

My special interest are the projects of immediate value to the TEX community. TEX and friends are the only software I know of that, since its young years, have been the best and highly sophisticated tool for typesetting beautiful books.

Klaus Höppner



(Candidate for TUG Board of Directors.)

Biography: I got a PhD in Physics in 1997. My first contact to LATEX was in 1991, using it frequently since then.

I have been preparing the CTAN snapshot on CD, distributed to the members of many user groups, from 1999 until 2002. I was the local organizer of TUG2015 and was heavily involved in the organization of several DANTE conferences and EuroTEX 2005. I've been a member of the TUG board since 2005, serving some time as treasurer and as secretary, and I am member of the DANTE board until 2016, including terms acting as president, vice president, and treasurer.

Statement: As in the past, I want to be the voice of European users, in particular those who need characters with funny accents.

Frank Mittelbach



(Candidate for TUG Board of Directors.)

I came in contact with TEX in the mid-eighties and over the years TEX, LATEX and typography in general became a very important part of my life. In 1990 I took over the maintenance and further development of LATEX from Leslie Lamport and together with a small number of people (most notably David Carlisle, Chris Rowley and Rainer Schöpf at that time) we designed and implemented what became LATEX 2_ε in 1994 — the LATEX you still essentially use today (even though it has undergone smaller modifications and improvements through by now 25 further releases).

Despite all predictions made during the last decades, TEX and LATEX are alive and kicking as proven by their (still?) strong use in various ways around the world.

Nevertheless the world has changed and is changing further and in that changing world user groups like TUG need to find their place and possibly reinvent themselves by redefining and reshaping their role. With my work on the TUG board I would like to help in that process and ensure a future for high quality typography as provided by TEX.

Erik Nijenhuis



(Candidate for TUG Board of Directors.)

TEX biography: My LATEX journey began in 2016 during my studies in Software Engineering (major) and Embedded Systems and Automation (minor) at Windesheim University of Applied Sciences. During this time, I used LATEX to write technical reports and datasheets and conducted research into the applicability of Git and LATEX for technical learning resources.

In 2021, I started my own company with a strong commitment to Free Software principles, selecting LATEX as the core documentation tool. I began combining application programming with LATEX code for invoice generation, and by 2022, I was designing contract templates for my legal documents.

After two years of working with LATEX, I decided to package and share the concepts I had developed, leading to the creation of the `gitinfo-lua`, `lua-placeholders`, and `regulatory` packages. This also marked my first encounter with CTAN and TUG.

My interaction with these communities was deeply moving, and I realized I had underestimated the social impact of collaborating with them compared to solely practicing the principles of Free Software on my own.

Statement: TEX offers immense potential for producing documents within systems and applications. Apparently, society needed AI to finally acknowledge that — after more than 40 years...

On a personal level, I am committed to getting to know other TUG members and understanding their motivations and requirements for using TEX. I aim to use this insight to help shape a strategic direction that is fair and beneficial to all members.

Beyond the community aspect, I am dedicated to advancing TUG's current goals, such as tagged-PDF development, and advocating for improved integration solutions that address both software needs and free entrepreneurial requirements.

I hope to contribute both to the technical and human sides of TUG, ensuring the group continues to thrive and innovate in the years ahead.

TUG financial statements for 2024

Karl Berry, TUG treasurer

The financial statements for 2024 have been reviewed by the TUG board but have not been audited. The totals may vary slightly due to rounding. As a US tax-exempt organization, TUG's annual information returns are publicly available on our web site, below.

Revenue (income) highlights

Membership dues revenue was slightly up in 2024 compared to 2023; we ended the year with 1,098 paid members, 46 fewer than in 2023.

The 2024 conference in Prague had a noticeable gain, thanks to the organizers keeping expenses down and good attendance. Overall, 2024 income was up almost 5%.

Other highlights; the bottom line

Costs held more or less steady in 2024. We no longer produce and mail the physical software DVD, so that expense is gone. Our bottom line for 2024 was slightly positive: $1,881.

Balance sheet highlights

TUG's end-of-year asset total increased slightly, following that gain.

Committed Funds are reserved for designated projects: LaTeX, CTAN, MacTeX, the TeX development fund, and others (`tug.org/donate`). TUG charges no overhead to administer these funds.

The Prepaid Member Income category is member dues that were paid in earlier years for the current year, and beyond. The 2024 portion of this liability was converted into regular Membership Dues in January of 2024. The payroll liabilities are for 2024 state and federal taxes due in January, 2025.

Notes for 2025

We held membership fees the same in 2025, and added two new patron categories (`tug.org/join`) for those who wish to support TUG in an even more (financially) substantial way. Three institutions have already chosen to become patrons, which we greatly appreciate.

Worldwide support from members, along with donations, are what allow us to continue, so thank you all! As always, we welcome ideas for new TUG benefits or activities.

◇ Karl Berry, TUG treasurer
`https://tug.org/tax-exempt`

TUG 12/31/2024 (vs. 2023) Revenue, Expense

	Dec 31, 24	Dec 31, 23
ORDINARY INCOME/EXPENSE		
Income		
Membership Dues	76,385	75,928
Product Sales	3,349	3,655
Contributions Income	12,924	12,597
Annual Conference	5,700	(989)
Interest Income	4,899	4,144
Advertising Income	230	340
Reimbursed Expenses	(5,000)	(1,600)
Total Income	98,487	94,075
Cost of Goods Sold		
Member Renewal	(544)	(639)
Lucida Sales to B&H	(1,381)	(1,495)
Members Postage/Delivery	(2,633)	(2,480)
Software Prod/Mailing		(3,120)
TUGboat Crossref	(738)	(490)
TUGboat Prod/Mailing	(28,207)	(29,540)
Total COGS	(33,532)	(37,753)
Gross Profit	64,955	56,322
Expense		
Office Overhead	(13,060)	(15,783)
Payroll Expense	(49,573)	(60,496)
Professional Fees		(41)
Interest Expense		(6)
Total Expense	(62,643)	(76,776)
Net Ordinary Income	2,312	(20,457)
OTHER INCOME/EXPENSE		
Prior year adjustment	(431)	(5,703)
NET INCOME	1,881	(26,160)

TUG 12/31/2024 (vs. 2023) Balance Sheet

	Dec 31, 24	Dec 31, 23
ASSETS		
Current Assets		
Total Checking/Savings	170,739	168,569
Accounts Receivable	5	10
Total Current Assets	170,744	168,579
LIABILITIES & EQUITY		
Current Liabilities		
Committed Funds	49,175	51,538
Prepaid Member Income	15,795	13,290
Payroll Liabilities	1,122	978
Total Current Liabilities	66,092	65,806
Equity		
Unrestricted	102,764	128,934
Retained Earnings	10	0
Net Income	1,879	(26,159)
Total Equity	104,653	102,775
TOTAL LIABILITIES & EQUITY	170,745	168,581

doi.org/10.47397/tb/46-1/tb142treas

TEX Consultants

The information here comes from the consultants themselves. We do not include information we know to be false, but we cannot check out any of the information; we are transmitting it to you as it was given to us and do not promise it is correct. Also, this is not an official endorsement of the people listed here. We provide this list to enable you to contact service providers and decide for yourself whether to hire one.

TUG also provides an online list of consultants at `tug.org/consultants`. If you'd like to be listed, please visit that page.

Boris Veytsman Consulting
132 Warbler Ln.
Brisbane, CA 94005
+1 703-915-2406
Email: `borisv@lk.net`
Web: `www.borisv.lk.net`

TEX and LATEX consulting, training, typesetting and seminars. Integration with databases, automated document preparation, custom LATEX packages, conversions (Word, OpenOffice etc.) and much more.

About two decades of experience in TEX and three decades of experience in teaching & training. More than forty packages on CTAN as well as Perl packages on CPAN and R packages on CRAN, published papers in TEX-related journals, and conducted several workshops on TEX and related subjects. Among my customers have been Amnesty International, Annals of Mathematics, ACM, FAO UN, Google, Israel Journal of Mathematics, No Starch Press, Philosophers' Imprint, Res Philosophica, US Army Corps of Engineers, US Treasury, and many others.

We recently expanded our staff and operations to provide copy-editing, cleaning and troubleshooting of TEX manuscripts as well as typesetting of books, papers & journals, including multilingual copy with non-Latin scripts, and more.

Dangerous Curve
Email: `khargreaves@gmail.com`

Typesetting for over 40 years, we have experience in production typography, graphic design, font design, and computer science, to name a few things. One DC co-owner co-authored, designed, and illustrated a TEX book (*TEX for the Impatient*).

We can: ▪ convert your documents to LATEX from just about anything ▪ type up your handwritten pages ▪ proofread, copyedit, and structure documents in English ▪ apply publishers' specs ▪ write custom packages and documentation ▪ resize and edit your images for a better aesthetic effect ▪ make your mathematics beautiful ▪ produce commercial-quality tables with optimal column widths for headers and wrapped paragraphs ▪ modify bibliography styles ▪ make images using TEX-related graphic programs ▪ design programmable fonts using METAFONT ▪ and more! (Just ask.)

Our clients include high-end branding and advertising agencies, academics at top universities, leading publishers. We are a member of TUG, and have supported the GNU Project for decades (including working for them). All quote work is complimentary.

Amy Hendrickson (TEXnology Inc)
100 Centre Street #420
Brookline, MA 02446
+1 617-738-8029
Email: `amyh@texnology.com`
Web: `www.texnology.com`

Full time LATEX consultant for more than 30 years; have worked for major publishing companies, leading universities, and scientific journals. Our macro packages are distributed on-line and used by thousands of authors. See our site for many examples: `texnology.com`.

▪ LATEX Macro Writing: Packages for books, journals, slides, posters, e-publishing and more; Sophisticated documentation for users.

▪ Data Visualization: database publishing.

▪ Innovative uses for LATEX: creative solutions our speciality.

▪ LATEX Training: customized to your needs, on-site or via Zoom. See `texnology.com/train.htm` for sample of course notes.

▪ Author of newly completed 300+ page e-book *Understanding LATEX: A Visual Guide for Beginners*.

Call or send email, no project too big or too small—I'll be glad to discuss your project with you.

LATEX Typesetting
Boston, MA
Email: `enquiries@latextypesetting.com`
Web: `www.latextypesetting.com`

LATEX Typesetting has been in business since 2013 and is run by Vel, the developer behind `LaTeXTemplates.com`. The primary focus of the service is on creating high quality LATEX templates and typesetting for business purposes, but individual clients are welcome too.

I pride myself on a strong attention to detail, friendly communication, high code quality with extensive commenting and an understanding of your business needs. I can also help you with automated document production using LATEX. I'm a scientist, designer and software developer, so no matter your field, I've got you covered.

I invite you to review the extensive collection of past work at the showcase on my web site. Submit an enquiry for a free quote!

Monsurate, Rajiv
Web: www.rajivmonsurate.com
latexwithstyle.com

I offer: design of books and journals for print and online layouts with LaTeX and CSS; production of books and journals for any layout with publish-ready PDF, HTML and XML from LaTeX (bypassing any publishers' processes); custom development of LaTeX packages with documentation; copyediting and proofreading for English; training in LaTeX for authors, publishers and typesetters.

I have over two decades of experience in academic publishing, helping authors, publishers and typesetters use LaTeX. I've built typesetting and conversion systems with LaTeX and provided TeX support for a major publisher.

Warde, Jake
90 Resaca Ave. / Box 452
Forest Knolls, CA 94933
+1 650-468-1393
Email: jwarde@wardepub.com
Web: myprojectnotebook.com

I have been in academic publishing for 30+ years. I was a linguistics major at Stanford in the mid-1970s, then started a publishing career. I knew about TeX from editors at Addison-Wesley who were using it to publish beautifully set math and computer science books.

Long story short, I started using LaTeX for exploratory projects (see website above). I have completed typesetting projects for several journal articles. I have also explored the use of multiple languages in documents extensively. I have a strong developmental editing background in STEM subjects. If you need assistance getting your manuscript set in TeX, I can help. And if I cannot help I'll let you know right away.

Science is what we understand well enough to explain to a computer. Art is everything else we do.

— Donald E. Knuth



*stm*DOCS

the confluence of art and science of text processing in the cloud!

- empowering authors to self-publish
- assisted authoring
- TeXFolio — the complete journal production in the cloud
- NEPTUNE — proofing framework for TeX authors

STM DOCUMENT ENGINEERING PVT LTD
Trivandrum • India 695571 • www.stmdocs.in • info@stmdocs.in

Calendar

2024

Apr 22 – 26 Association Typographique Internationale, ATypI Copenhagen 2025, Copenhagen, Denmark. `atypi.org/conferences-events/atypi-copenhagen-2025`

Apr 29 – May 4 BachoTEX 2025, 30th BachoTEX Conference, Bachotek, Poland. `www.gust.org.pl/bachotex/2025-en`

Jun 1 – 6 Book History Workshop, Texas A & M University, College Station, Texas. `library.tamu.edu/book-history`

Jun 25 – 27 Twenty-third International Conference on New Directions in the Humanities, "Oceanic Journeys: Multicultural Approaches in the Humanities", University of Hawaii, Hilo, Hawaii (and online). `thehumanities.com/2025-conference`

Jul 14 – 18 Digital Humanities 2025, Alliance of Digital Humanities Organizations, "Building access and accessibility, open science to all citizens", Universidade NOVA de Lisboa, Lisbon, Portugal. `dh2025.adho.org`

Jul 17 – 20 Ladies of Letterpress, J. Willard Marriott Library, University of Utah, Salt Lake City, Utah. `ladiesofletterpress.com/conference`

TUG 2025 Trivandrum, Kerala, India

Jul 18 – 20 The 46th annual meeting of the TEX Users Group. `tug.org/tug2025`

Jul 27 *TUGboat* **46**:2, proceedings issue, submission deadline.

Aug 7 – 9 TypeCon 2024, Revolution Hall, Portland, Oregon. `typecon.com`

Aug 10 – 14 SIGGRAPH 2025, Vancouver, BC, Canada. `s2025.siggraph.org`

Aug 22 – 29 19th International ConTEXt Meeting, "What if . . . ", Ośrodek Wypoczynkowy KREFTA, Chmielno, Poland. `meeting.contextgarden.net/2025`

Sep 2 – 5 25th ACM Symposium on Document Engineering, University of Nottingham, Nottingham, UK. `doceng.org/doceng2025`

Sep 5 The Updike Prize for Student Type Design, application deadline, 5:00 p.m. EST. Providence Public Library, Providence, Rhode Island. `prov.pub/updikeprize`

Sep 14 – 19 XML Summer School, St Edmund Hall, Oxford University, Oxford, UK. `xmlsummerschool.com`

Oct 3 *TUGboat* **46**:3, submission deadline.

2026

Mar 20 *TUGboat* **47**:1, submission deadline.

Status as of 1 April 2025

For additional information on TUG-sponsored events listed here, contact the TUG office by email: `office@tug.org`). For events sponsored by other organizations, please use the contact address provided.

User group meeting announcements are posted at `tug.org/meetings`. Interested users can subscribe and/or post to the related mailing list, and are encouraged to do so.

Other calendars of typographic interest are linked from `tug.org/calendar`.

Reports and notices

2 Institutional members
157 From other TEX journals: *Die TEXnische Komödie* 4/2024–1/2025; *Zpravodaj* 2024/1–4; *La Lettre GUTenberg* 54 (2024)
162 *TUG Elections committee* / 2025 TEX Users Group election
165 *Karl Berry* / TUG financial statements for 2025
166 TEX consulting and production services
167 Advertisement: TEXFolio
168 Calendar

TUGBOAT Volume 46 (2025), No. 1

Introductory

4 *Barbara Beeton* / Editorial comments

10 *Peter Flynn* / Typographers' Inn
• Style and fashion; LaTeX processors; Afterthought

3 *Arthur Rosendahl* / From the president
• conference musings and invitations

7 *Boris Veytsman* / Face/Interface 2025 conference: Global type design and human-computer interaction
• summary of talks at this multi-disciplinary conference

Intermediate

160 *Karl Berry* / The treasure chest
• new CTAN packages, October 2024–April 2025

12 *Šimon Konečný, Petr Olšák* / OpBible — a tool for creating study Bibles
• adding notes, maps, other material to multiple biblical texts

35 *LaTeX Project Team* / LaTeX news, issue 40, November 2024
• thirty years of LaTeX2ε; tagged PDF news; changes to the kernel, `amsmath`, `tools`, `l3build`

19 *Carla Maggi* / The DuckBoat — Beginners' Pond: A pilates . . . oops, PyLaTeX course
• writing Python to generate LaTeX, with discussion of escaping, customizing, and more

101 *Paul Murrell* / A DVI driver for R
• converting DVI files into R objects, supporting LaTeX fragments as labels and annotations

156 *Travis Stenborg* / Styling Microsoft Power BI dashboards with TeX
• using Python and Mathtext in Matplotlib to style business intelligence displays

28 *Nicola Talbot* / The `glossaries` package: Getting started
• introduction to creating glossaries and lists of abbreviations

Intermediate Plus

71 *John E. Beach* / `dtxmacrofont`: Resizable `macrocode` in `doc` implementations
• using different font sizes for `\MacroFont` and `\AltMacroFont`

40 *Alan J. Cain* / Experiences in typesetting a complex book: Consistency, graphics, languages, notes
• some generalizable techniques from typesetting the author's book *Form & Number*

49 *Alan J. Cain* / `marginalia` at work: Running heads, float captions, citations, and small figures in the margins
• configurable LuaLaTeX package for automatic adjustments, optional and required marginal elements, etc.

79 *Walter Daems, Paul Levrie* / SpeLaTeX — Speech-enabled LaTeX
• generating PDFs with linked high-quality audio readings, including math

132 *Petr Olšák* / Processing data from CSV and JSON files with OpTeX
• database operations and tree structures

54 *Geoffrey M. Poore* / Typesetting code with syntax highlighting: Introducing `minted` version 3
• improving security, usability, debugging, and enhancements via a dedicated `latexminted` executable

151 *samcarter, Ulrike Fischer* / Cookie cutters: An example workflow of how to generate simple 3D-printable objects from LaTeX graphics
• step-by-step workflow to convert vector graphics into 3D models, using OpenSCAD

69 *Vít Starý Novotný, Marei Peischl* / Piping YAML files into LaTeX3 key–values
• using YAML to set document metadata and configure LaTeX packages

Advanced

104 *Hans Hagen, Mikael P. Sundqvist* / A new take on paragraphs
• adding more passes and flexibility to the Knuth–Plass algorithm for breaking paragraphs into lines

60 *Geoffrey M. Poore* / New tools for LaTeX package authors: Buffering, hashing, caching, serializing, and executing
• new packages and features to help with verbatim processing, calling external programs, and more

94 *Martin Ruckert* / Efficient input file processing with TeX: A case study
• profiling and reimplementing critical input parsing with a finite state machine

86 *Amir Mohammad Tahsiri* / BibBreeze: A LaTeX package to automate bibliography data
• reconstructing references from inaccurate source files using `expl3` and regular expressions

74 *Nicola Talbot* / `datatool` v3: Performance, localisation, databases and more
• overview of new features in this package for handling CSV data and much more

136 *Udo Wermuth* / Output routines extending `\plainoutput`
• reporting and automatic prevention of widow lines, and some help for club lines and hyphenations

145 *Udo Wermuth* / Is a given input a valid TeX ⟨*glue*⟩/⟨*muglue*⟩?
• plain TeX conditional-like macro to test for a syntactic (mu)glue

147 *Udo Wermuth* / Comparisons for glue and muglue
• plain TeX macros for conditional tests of (mu)glue

124 *Sašo Živanović* / Externalization with Memoize
• memoization of typesetting, including TikZ, and other results to avoid recompilation

Reports and notices

2 Institutional members
157 From other TEX journals: *Die TEXnische Komödie* 4/2024–1/2025; *Zpravodaj* 2024/1–4; *La Lettre GUTenberg* 54 (2024)
162 *TUG Elections committee* / 2025 TEX Users Group election
165 *Karl Berry* / TUG financial statements for 2025
166 TEX consulting and production services
167 Advertisement: TEXFolio
168 Calendar